PARKRADAR ULTRA

Modern solution for obstacle detection

1. Introduction

1.1. Administrative Information

« Microcontrôleurs et systèmes numériques EE-208 » course project.

Auteurs :
Hudz Denys / Burenok Yehor

Document dated by 24/05/2026.

1.2. Main idea

Create a simple yet effective system for scanning the immediate surroundings at close range to detect obstacles. The core principle is to combine ultrasonic distance measurement with mechanical scanning capability, enabling comprehensive proximity awareness. This approach ensures reliable detection while maintaining minimal system complexity and cost-effectiveness.

1.3. Functionality and use

Object detection at distances of up to 2.5 meters using ultrasound. The use of a servo motor allows measurements to be taken from any side of the sensor. The buzzer emits distance-dependent audio feedback, while the display provides real-time status and sensor measurements. The device is controlled using a remote, all the buttons are laid out in a logical and user-friendly. This multi-sensory feedback approach—combining audio, visual, and mechanical rotation—provides intuitive real-time awareness of the surrounding environment.

1.4. Application

The system can be used as a parking sensor, a radar, or simply a distance meter, depending on how the user chooses to use it. To do this, you can switch between the appropriate modes; the key function is to measure the distance and display the measurement on the screen. You can easily move the sensor manually as well, to make measurements wherever you want. The portability and ease of reconfiguration make it suitable for both permanent installation and temporary deployment scenarios.

2. User Guide and Technical Architecture

2.1. User Manual

The system always initializes in parking sensor mode. The servo motor holds the sensor in a stationary position and measures the distance in front of it. The maximum measurement range is 250 centimeters.

The beeping sound from the buzzer varies depending on how close an object is to the sensor and begins when an object is within 50 centimeters of the sensor. The beep frequency changes when the distance is less than 25 and 10 centimeters, respectively.

Buzzer beeping frequency:

- 255 – 51 cm: no beeping
- 50 – 25 cm: rare beeping
- 25 – 10 cm: frequent beeping
- 25 – 10 cm: almost constant beeping

If necessary, the user can mute the sound at any time using the “mute” button.

When the “guide” button is pressed, the system switches to radar mode. The servo motor will rotate the sensor 360 degrees clockwise and counterclockwise, continuously taking measurements. The sound will also change in this mode and will resemble the beeping of a radar. To switch to parking sensor mode, press the “guide” button again. The servo motor will stop, and the sensor will measure in a fixed direction.

When the “**+**”, “**-**”, “  ” or “  ” buttons are pressed, the user can switch the system to manual control mode regardless of its current state. Essentially, this is simply a dynamic parking sensor mode in which the user can do the following:

- “**+**” set the sensor rotation speed counterclockwise from 1 to 5 (slowest to fastest);
- “**-**” set the sensor rotation speed clockwise from -1 to -5 (slowest to fastest);
- “  ” a quick counterclockwise motion of the sensor;
- “  ”a quick clockwise motion of the sensor;

The user can also hold down the “  ” / “  ” buttons to adjust the position and release them at the right moment to set the sensor to the desired location.

The power button turns our system on and off. Consequently, the other buttons do not respond when the system is off. When the system is turned back on, a message will appear on the display and the system will initialize into parking sensor mode.

During this entire time, the user will see the following information on the display:

- distance “Dist:” in centimeters from the sensor to the nearest obstacle;
- speed as key letter “s”, visible in a Manual mode from -5 to 5;
- chosen mode itself – Manual / Radar / Park (stands for Parking Sensor)

Quick button guide

Name	Function
GUIDE	Mode switch between “Park” and “Radar”
+	Add counterclockwise speed in Manual mode
-	Add clockwise speed in Manual mode
	Short counterclockwise rotation
	Short clockwise rotation
POWER	Turn OFF / Turn ON the system
Mute	Turn OFF / Turn ON sound

2.2. Hardware Architecture

In addition to the ATmega128 MCU, the following components were used to implement the project. All parts below are powered up by MCU through GND and VCC pins (5V).

Name	Function	Port / Pin	Interface	Timer/ Interrupt
HC-SR04	Ultrasonic distance measurement (Trig pulse → Echo response timing)	Trig = **PD0** (output), Echo = **PD1** (input).	Digital I/O	-
Servo SG90	Rotates ultrasonic sensor	**PE3** (OC3A)	PWM output	Timer 3
Buzzer (M2)	Distance-proportional audio feedback	**PE2**	Digital output	-
IR Receiver (M2)	Remote control input.	**PE7** (INT7)	Digital output	INT7
HD44780U 2x16 LCD	Display of system data and system status.	**Port A, Port C**	8-bit parallel via external memory interface	-
Remote Vivanco UR Z2	Transmitting the command in the RC5 data format.	-	Wireless IR (RC5)	-

2.3. Top – down logic explanation

The program is entered via reset which initialises the LCD, HC-SR04, SG-90, clears all shared SRAM variables and enables the INT7 interrupt for the IR receiver before entering the main loop.

Each iteration of the main loop decrements the rotation iterator (stopping the small rotation caused by MANUAL mode) , ticks the debounce counter by calling *deb_tick* (the program waits some time before enabling INT7 after the button was pushed) and then checks the power state of the system (disables everything if the power is off). After that it dispatches to the current mode (PARK/RADAR/MANUAL). In PARK and MANUAL modes the program does the sequence of commands: set servo speed (for MANUAL) -> fire ultrasonic sensor and get the distance (*measure_and_filter*)-> beep or stay silent based on the distance (*buzz_tick*)-> update the LCD with current information. In RADAR mode: set the direction and the speed of rotation (direction changes every 15 iterations of main) -> same sequence as PARK. INT7 Interrupt for the IR Receiver is fired every time the remote button is pressed (when the interruptions are enabled). INT7 sub-routine decodes the RC-5 frame and updates shared SRAM varibles such as mode, *servo_level, power_state, buzzer_on, step_timer* that the main loop reads every iteration. At the end INT7 is disabled (because RC-5 sends the same frame over and over again), the debounce counter is loaded (to again enable INT7 interruptions in main loop)

2.4. Software modules description

We should begin with our sensor module **HC-SR04.asm**, which is the eyes of our system. This module is the software interface to the HC-SR04 ultrasonic sensor that we use. It exposes a single function that returns the measured distance in centimeters, hiding the trigger/echo timing details from the rest of the program. The module also contains the two acoustic feedback from buzzer routines used by the parking and radar modes. It was made to simplify the architecture because the work of the buzzer really depends on sensor measurements.

Lets proceed to **servo.asm** module driver, which is our software interface to the SG90 servo motor that holds our sensor. It configurates the Timer 3 **PWM** output and exposes simple subroutines to set servo to a desired or programmed position.

Module **ir_rc5.asm** contains code to the IR receiver and the RC-5 remote. It contains the interrupt service routine that decodes incoming frames and translates each button into a state change in shared memory so the rest of the program can react to remote commands without dealing with the protocol.

Module named **lcd.asm** allows us access our display. It contains details of communicating with the display and allows the rest of the program write text numbers and letters in a readable way using format specified in side module printf.asm.

The **main.asm** is our top-level module. It operates the state of the system, defines the initialisation sequences, runs the main loop and dispatches between all three operating modes.

In addition to the above, modules **definitions.asm** and **macros.asm** should be mentioned. The first one defines pins, registers etc and the other one is a library of useful macros.

3. Hardware communication description

3.1 Access to peripheral modules

HC-SR04

Our sensor is driven by software polling. **HC_init** configures **PD0** as an output (TRIG) and **PD1** as an input (ECHO). When the application calls **HC_measure**, the routine generates a 10 µs trigger pulse, then waits up to about 30 ms for ECHO to rise. The HC-SR04 keeps its ECHO pin high for the round-trip time of the ultrasonic pulse; since sound travels at 343 m/s, this means 1 cm of distance corresponds to 58.14 µs of echo. The driver measures that time by counting passes of a hand-tuned 7-cycle polling loop (1.75 µs per pass at 4 MHz) and divides the result by 33 (= 58.14 / 1.75) to obtain the distance in centimetres, returned in register r21. Only one byte is used to store the distance, so the maximum displayed value is 254 cm — the value 0xFF is reserved as the "no-echo" when no reflection is detected. No interrupt is used, the routine is called synchronously from the main loop to guarantee that a measurement is always made on every pass. **HC_measure** is invoked synchronously from two places: once per main-loop pass in **PARK** and **MANUAL** mode, and inside the **RADAR** sweep. The result is then passed to the glitch filter (**measure_and_filter**), which discards sudden jumps greater than 20 cm to avoid spurious readings.

SG-90 servo motor

The servo is driven by Timer3 in Fast PWM mode 14. First, Servo_init selects Fast PWM mode 14 (**TCCR3A** = (**1<<COM3A1**)|(**1<<WGM31**), **TCCR3B** = (**1<<WGM33**)|(**1<<WGM32**)|(**1<<CS31**)), sets the period through **ICR3** = 10000 (= 20 ms at 4 MHz with /8 prescaler → 50 Hz), and the initial pulse width through **OCR3A** = 760 (= 1.52 ms, neutral). From then on, the application changes rotation speed and direction simply by writing a new value to **OCR3A**; the timer continuously regenerates the corresponding pulse on **PE3** without further CPU intervention. In **RADAR** mode only two fixed **OCR3A** values are used — one for the right sweep and one for the left.

Buzzer (M2 from our kit)

The buzzer unlike our servo is not driven by any time, sound is generated by our soft if specific conditions are met. To make the buzzer beep, the program rapidly switches PE2 between high and low, which creates a square wave. The pin is set high with *sbi* **PORTE,** SPEAKER and low with *cbi* **PORTE**, SPEAKER, with a short calibrated delay (`WAIT_US`) between each toggle. The duration of that delay determines the frequency of the tone (shorter delay → higher pitch), and the number of toggles determines the length of the beep. We have 2 different tones:

- **`Park_tone`** — a fixed ~1 kHz beep that lasts about 60 ms. It toggles PE2 sixty times with producing a short clean tick used in **PARK** and **MANUAL** modes.
- **`Radar_tone`** — a frequency-sweeping beep used in **RADAR** mode. The delay between toggles changes on each cycle, producing the characteristic "radar ping" effect.

IR receiver (M2 from our kit)
IR receiver outputs a demodulated digital signal on **PE7**. Each command from the remote is encoded with the **RC-5 protocol**: a 14-bit Manchester-encoded frame with a fixed bit period of **T1 = 1778 µs**. Because the timing of every bit is measured from the falling edge of the start bit, the receiver cannot be polled from the main loop — it has to be read at the exact moment a frame begins. For that reason, **PE7** is wired to **INT7**, configured to trigger on the falling edge. The full decoding logic lives in the interrupt service routine **INT7_handler** (see Section 2), which samples the line 14 times at the correct intervals, builds the command byte and stores it in the shared variable *last_cmd* so that the main loop can react to the press on its next pass.

Vivanco UR Z2 (Remote control)
Remote is configured to transmit commands using the RC-5 protocol. When a button is pressed, the remote drives its IR LED modulated by the 14-bit RC-5 frame, which the receiver on the board demodulates and feeds to **PE7**. Each frame carries the address of the target device and the code of the pressed button; the system only looks at the **8-bit command field** (the lower bits of the frame) and recognises seven keys: **POWER**, **MUTE**, **GUIDE**, **VOL+**, **VOL-**, **UP** and **DOWN**.

HD44780 display

Display wired into the external *SRAM* bus of the ATmega128, not to ordinary GPIO pins. *LCD_init* sets bits SRE (SRAM Enable) and SRW10 in the MCUCR register, which routes **PORTA**, **PORTC** and **PG0–PG2** to the external memory interface. From that point the LCD's two internal registers appear as ordinary memory locations: the instruction register at address 0x8000 and the data register at address *0xC000*. Writing a byte to either register is then as simple as a regular sts instruction, and reading the busy flag uses lds — the helpers *LCD_wr_ir* and *LCD_wr_dr* wrap these accesses, polling the busy flag before each write so the controller is never overrun.

---

*Datasheets of the used peripherals can be found in the attachments**

3.2 Interrupt usage

We used 1 single hardware interrupt – **INT7** dedicated to the IR receiver on **PE7**. No need for the other interrupters or timers as our system needs to react on a simple trigger – remote button press.

INT7 is configured in reset (main.asm) with three writes: ***EICRB*** = *(1<<**ISC71**)|(0<<**ISC70**)* selects the falling-edge trigger (matching the active-low output of the IR demodulator), ***EIMSK*** = *(1<<**INT7**)* unmasks the interrupt and *sei* globally enables interrupts. The interrupt vector at address *0x0012* jumps to the handler *INT7_handler* located in *ir_rc5.asm*.

On a falling edge, *INT7_handler* saves the working registers and SREG, then decodes one 14-bit RC-5 frame: the first bit is sampled µs after the edge to land on the centre of the start bit, and the

next 13 bits are read every T1 = 1778 µs. The lower 8 bits form the command byte, which is matched against the codes POWER, MUTE, GUIDE, VOL+, VOL-, UP, DOWN. The matched label updates the shared SRAM state (mode, servo_level etc.) so the main loop reacts on its next pass.

RC-5 remotes retransmit the active command every ~114 ms while the button is held, which would otherwise process each press many times. To suppress this, the ISR disables itself before returning (**EIMSK** = 0) and loads *deb_counter* with 3 (VOL±/MUTE) or 5 (POWER/GUIDE); the main loop decrements it via *deb_tick* and re-arms INT7 at zero. UP and DOWN are an exception: they skip *deb_lock* on purpose, so holding the key keeps reloading *step_timer* with STEP_KICK_PASSES. When the button is released, *step_timer* runs out and *servo_level* is cleared automatically, stopping the motor.

3.3 Memory Allocation

ATmega128`s flash memory was used as a single *.cseg* part with the reset vector at *0x0000 (rjmp reset*) and the **INT7** vector at *0x0012*. Data segment *.dseg* starts at *0x0200* to leave lower I/O space untouched. The shared state of the application is allocated statically by the assembler as a series of *.byte* directives in main.asm. Table of SRAM distribution is presented below.

Address	Symbol	Size	Purpose
0x0200	mode	1 B	Active operating mode (0=PARK, 1=RADAR, 2=MANUAL)
0x0201	servo_level	1 B	Current servo level
0x0202	last_cmd	1 B	Last decoded IR command code
0x0203	power_state	1 B	0=on, 1=off, 2=turning_on
0x0204	buzzer_on	1 B	Buzzer enable flag (1=enabled, 0=muted)
0x0205	deb_counter	1 B	IR stadebounce countdown
0x0206	step_timer	1 B	UP/DOWN momentary kick countdown
0x0207	distance_cm	1 B	Raw distance from HC-SR04
0x0208	dist_prev	1 B	Previous raw sample (for the glitch filter)
0x0209	dist_display	1 B	Filtered distance shown on LCD
0x020A	buzz_counter	1 B	Non-blocking buzzer cadence counter
0x020B-C	tone_fn	2 B	Word address of active tone routine

Also, should be mentioned that LCD occupies 2 registers, the addresses *0x8000* (instruction register) and *0xC000* (data register). Although these are accessed by the CPU through ordinary memory instructions, they are not part of the internal 4 KB SRAM — they are decoded externally on the STK-300 board from the address lines driven by **PORTA** and **PORTC**.

*As reference files we have only our peripheric modules documentation presented below**

Data Formats for IR Remote Control

In most remote control transmission systems, only small data rates are required for transmitting the control functions of home entertainment equipment. The reliability of the transmission is essential as an incorrect interpretation of a transmitted code is not permissible. Corrupted signals must be ignored. In most coding schemes, commands are repeated until the remote controlled device reacts as desired. The operator can directly observe the result of pressing a key by means of visual feedback.

Because IR signals are confined within a room and because there is only a short period of data transmission with each key press, there are no legal restrictions for IR transmission in the frequency band between 30 kHz and 56 kHz.

Several methods of modulation have become well established. A reliable and power saving transmission method in which bursts of the carrier frequency are transmitted is called “Pulse Code Modulation” (PCM). There are three commonly used representations of one bit in remote control systems which are described in the following diagrams.

The “Bi Phase Coding” has one rising or falling edge in the center of each time slot (Fig. 1). In the “Pulse Distance Coding”, all bursts have the same length but the time between the bursts is different depending on the value of the bit (Fig. 2). In the “Pulse Length Code”, there are two kinds of burst lengths depending on the bit value (Fig. 3).

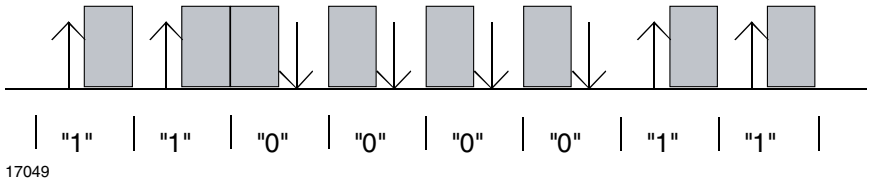


Fig. 1 - BI Phase Coding (a rising edge within a time window is equivalent to a “1“, a falling edge represents a “0“)

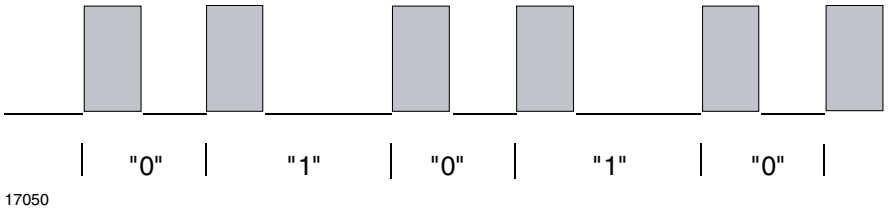


Fig. 2 - Pulse Distance Coding

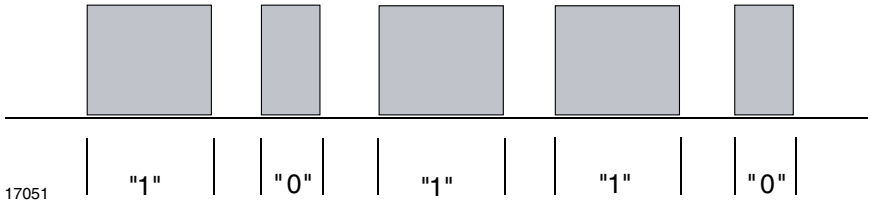


Fig. 3 - Pulse Length Coding

The Vishay IR receiver modules were developed and optimised for use in all such carrier frequency burst transmission systems. Standard types are available for the frequencies **30 kHz, 33 kHz, 36 kHz, 38 kHz, 40 kHz, and 56 kHz**.

In addition to different kinds of coding and different carrier frequencies, there are further variations in the data formats; with and without pre-burst, with different numbers of bits in a command, and with different bit lengths.

Almost all codes have address bits and data bits. For reliability reasons, some codes send the data twice, once inverted and once non-inverted. Usually the data command is repeatedly sent as long as the key is being pressed. There are different ways to distinguish between a multiple key press and an interruption of the transmission link (e.g. to avoid the TV selecting channel “11” when channel “1” was intended). Some codes use a toggle bit, which changes its value at each key-press. Some codes send a pre- or post-burst at the beginning and/or at the end of each key press. And some codes send the data only once for each key-press.

Two common data formats, the RC5 code and the NEC code, are described in more detail here.

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

THE RC 5 CODE

The RC 5 standard uses a bi-phase coding (see Fig. 4) the carrier frequency fixed at 36 kHz.

The transmission of a data word begins with two start bits followed by a toggle bit. The toggle bit changes its value at each new key-press. The five address bits represent the address of the device to be controlled. The six command bits contain the information to be transmitted.

Each bit in the data word consists of half a bit period with no transmission and half a bit period with a burst of 32 pulses at 36 kHz. The timing is shown in the pulse diagrams.

The most suitable IR receivers for receiving the RC5 code are those with “AGC6” in Cyllene (TSOP9...) and Mneme (TSOP1..) receivers as well as “AGC4” in Aether (TSOP3...) and Methone (TSOP5...) receivers. For RC5 a band pass frequency of 36 kHz is recommended. Some examples are: TSOP13636, TSOP18636, TSOP93636, TSOP98636, TSOP95636, TSOP96636.

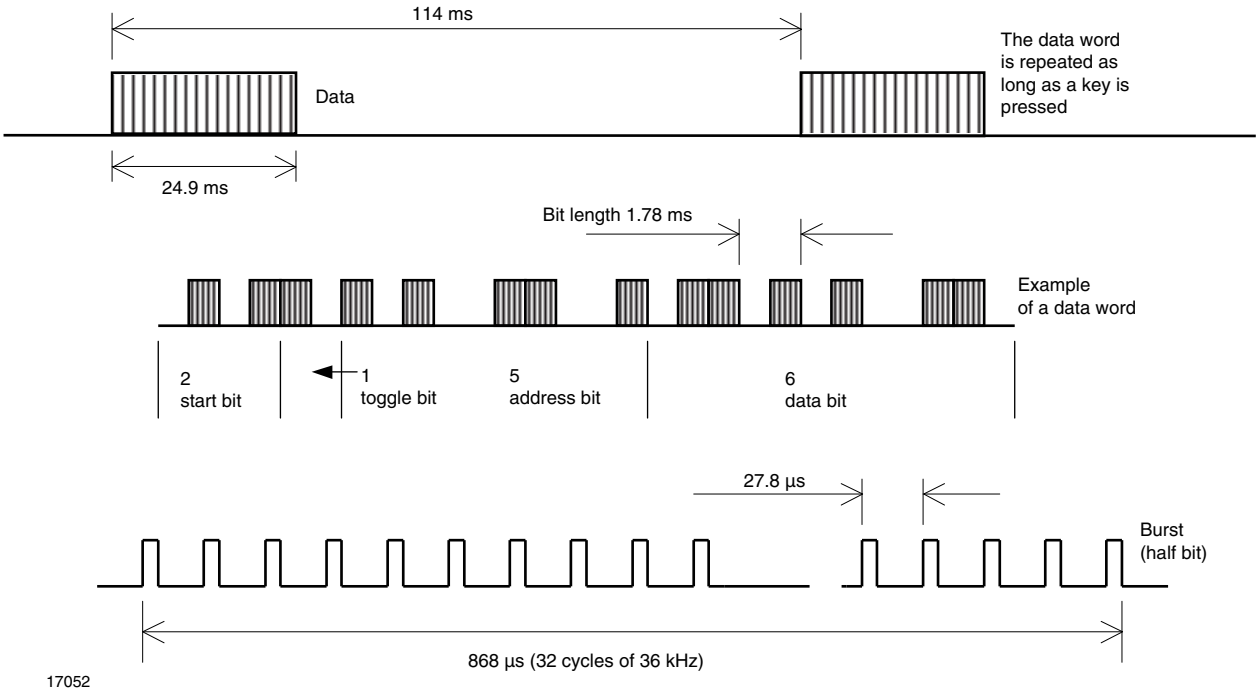


Fig. 4 - RC 5 Transmission Code

THE NEC CODE

The NEC code uses bursts at a carrier frequency of 38 kHz. All Vishay receiver modules operate well with this coding scheme, but the best noise suppression with this format is gained with “AGC6” for Cyllene series and “AGC4” for all other TSOP series (e.g. TSOP13438, TSOP18438, TSOP93638, TSOP95638).

The NEC code starts the transmission using a so called leader code, a burst with a length of 9 ms, followed by a pause of 4.5 ms and then the data word. The original purpose of this leader code was to let the internal control loops in the receiver modules settle. But such a pre-burst is not necessary for the Vishay receivers to function correctly.

After transmitting the data word, only the leader code and a single bit are transmitted repeatedly for as long as a key is pressed. A special property of this code is a constant word length in combination with pulse distance modulation. Both the address and the data bits are transmitted twice, first as a normal byte followed by an inverted byte. This is shown in Fig. 5. The half period burst portion of each bit contains 22 pulses, each with a width of 8.77 µs and a period of 26.3 µs. A “0” is represented by a pulse distance of 1.125 ms and a “1” by a pulse distance of 2.25 ms. 8 address bits are used to identify the device to be controlled. A further 8 bits are used for the transmission of the command data. As mentioned above, the words are always followed, without a pause, by the inverted words. E.g., the transmission of the address word “00110111” and the command data word “00011010” is performed by sending the bits:

“00110111'11001000'00011010'11100101”

In a special version of the NEC code, the pre-burst, including all of the address and data bits, is repeated in each 108 ms time slot for as long as the key is pressed.

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

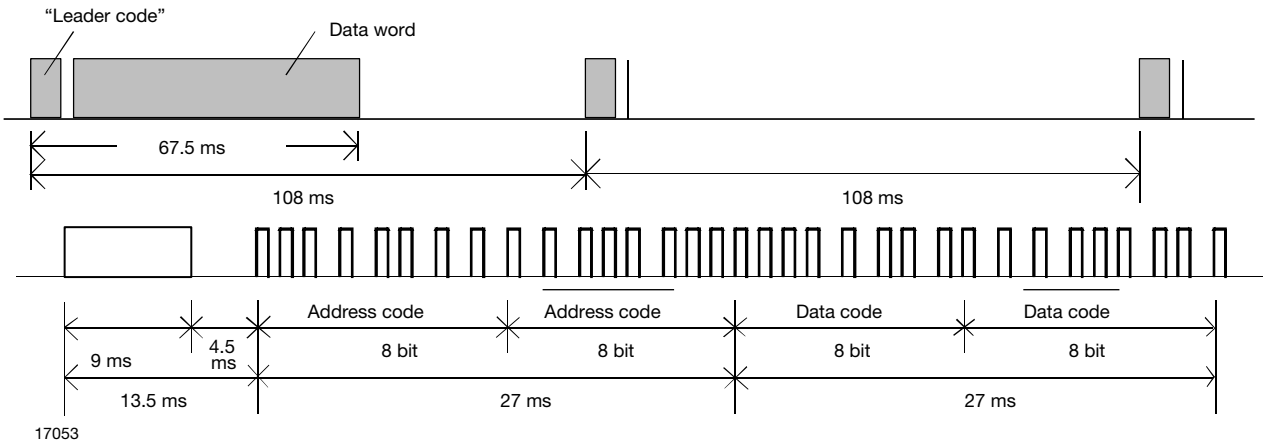


Fig. 5 - NEC Transmission Code

DATA TRANSMISSION WITH THE TSDP RECEIVER MODULES

Although IR receiver modules are mainly developed and used for IR remote control, they can also serve for continuous data transmission.

For this purpose, Vishay designed the TSDP series receivers for data transmission, with enhanced output pulse with accuracy.

The AGC 1 setting for example in TSDP34138 is adapted to long burst length supporting the transmission of successive zero bits in RS232.

If high data rates up to 9600 bps are required the new Cyllene series receiver TSOP93156 is the most suitable solution.

Three proposals for such continuous data encoding are shown below in Fig. 6, Fig. 7, and Fig. 8:

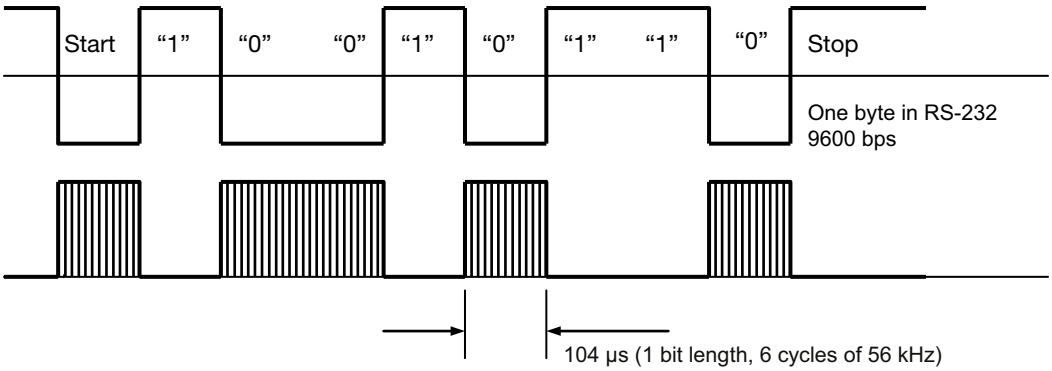


Fig. 6 - Example of a Data Transmission at 9600 bps With TSOP93156

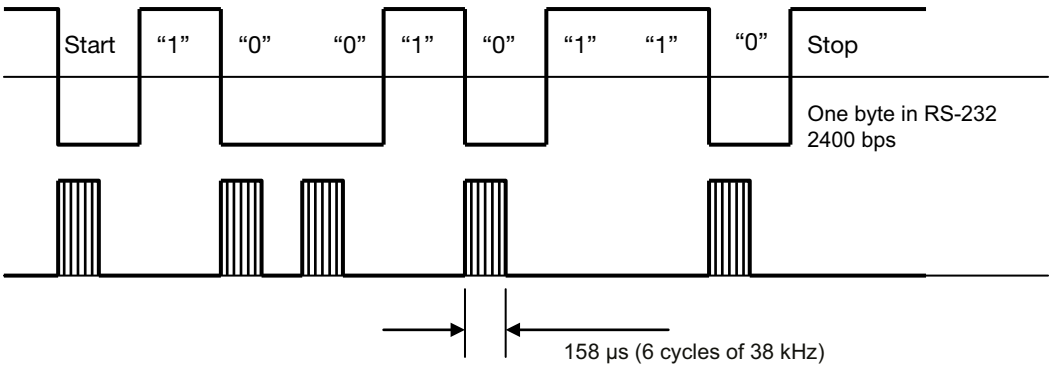


Fig. 7 - Example of a Data Transmission at 2400 bps with TSDP34138

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

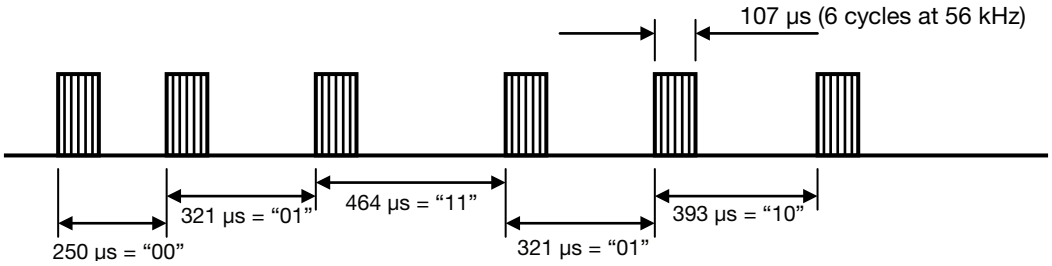


Fig. 8 - Example of a Data Transmission With the TSOP93156

COMPATIBILITY OF THE TSOP RECEIVER MODULES WITH DATA FORMATS

Vishay offers a variety of IR receiver series in order to supply an optimised solution for each application. Guidelines for selecting the best part for each data format is given here.

Basically there are six categories of IR receiver settings regarding noise suppression and data format compatibility. The summary of the features of these AGC types is listed here:

- AGC1 is compatible with any coding scheme, it is optimized for continuous data transmission
- AGC2 is optimized for most common remote control standard applications with typical long burst data formats
- AGC3 is optimized for short burst data formats in noisy environments
- AGC4 is optimized for most common remote control standard applications in very noisy environments (including dimmed LCD backlightings)
- AGC5 is optimized for short burst data formats in very noisy environments
- AGC6 has best sensitivity for NEC and RC5 code in Cyllene series receivers and offers high robustness against smart phone interference in Mneme series receivers
- AGC-S and AGC-C are optimized for Sony code and Cisco code respectively. These AGCs are exclusively designed for Aether series receivers

The tables below provide an overview of which IR receiver type can be used for the various data formats.

CYLLENE IC (TSOP9...) - COMPATIBILITY FOR DATA FORMATS							
	AGC1	AGC2	AGC3	AGC4	AGC5	AGC6	BEST CHOICE
NEC	yes	yes	yes	yes	yes	yes	AGC6
RC5 code	yes	yes	yes	yes	yes	yes	AGC6
RC6 mode 0	yes	yes	yes	yes	yes	yes	AGC4
RCMM	yes	no	yes	no	yes	no	AGC3
RECS-80 code	yes	no	yes	no	yes	no	AGC3
R-2000 code (33 kHz)	yes	yes	yes	yes	yes	yes	AGC6
Mitsubishi code 38 kHz	yes	yes	yes	yes	yes	yes	AGC6
Sony code SIRCS 12 bit	yes	yes	no	no	no	yes	AGC2
Sony code SIRCS 15 bit	yes	yes	no	no	no	yes	AGC2
Sony code SIRCS 20 bit	yes	yes	no	no	no	no	AGC2
r-map data format 38 kHz	yes	no	yes	no	yes	no	AGC5
r-step data format 38 kHz	yes	no	yes	no	yes	no	AGC3
r-step data format for KB 56 kHz	yes	no	yes	yes	yes	yes	AGC4
XMP code	yes	no	yes	no	yes	no	AGC3
Cisco format 57kHz	yes	yes	no	no	no	yes	AGC2
Cisco format 37kHz	yes	yes	no	yes	yes	yes	AGC6
Low latency protocol - worst case frame 16 bit	yes	yes	yes	yes	yes	yes	AGC6
Low latency protocol - extended frame 24 bit	yes	yes	no	yes	yes	yes	AGC6
Sejin 4PPM format (38 kHz or 56 kHz)	yes	no	yes	yes	yes	no	AGC4
MCIR code keyboard package timing	yes	no	yes	no	yes	no	AGC3
MCIR code pointing device timing	yes	no	yes	no	yes	no	AGC3
MCIR code remote control timing	yes	no	yes	no	yes	no	AGC3
Konka TV data format 2004	yes	yes	yes	yes	yes	yes	AGC6
RCA code 56 kHz	yes	yes	no	yes	yes	yes	AGC4
Panasonic 36.7 kHz	yes	yes	yes	yes	yes	yes	AGC6
Sharp 38 kHz	yes	no	yes	yes	yes	yes	AGC6

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

MNEME IC (TSOP1...) - COMPATIBILITY FOR DATA FORMATS							
	AGC1	AGC2	AGC3	AGC4	AGC5	AGC6	BEST CHOICE
NEC	yes	yes	yes	yes	yes	yes	AGC4
RC5 code	yes	yes	yes	yes	yes	yes	AGC6
RC6 mode 0	yes	yes	yes	yes	yes	no	AGC4
RCMM	yes	no	yes	no	yes	no	AGC3
RECS-80 code	yes	no	yes	no	yes	no	AGC3
R-2000 code (33 kHz)	yes	yes	yes	yes	yes	yes	AGC4
Mitsubishi code 38 kHz	yes	yes	yes	yes	yes	no	AGC4
Sony code SIRCS 12 bit	yes	yes	yes	no	no	yes	AGC2
Sony code SIRCS 15 bit	yes	yes	yes	no	no	yes	AGC2
Sony code SIRCS 20 bit	yes	yes	no	no	no	no	AGC2
r-map data format 38 kHz	yes	no	yes	no	yes	no	AGC3
r-step data format 38 kHz	yes	yes	yes	yes	yes	no	AGC4
r-step data format for KB 56 kHz	yes	yes	yes	yes	yes	no	AGC4
XMP code	yes	no	yes	no	yes	no	AGC3
Cisco format 57kHz	yes	yes	no	no	no	yes	AGC2
Cisco format 37kHz	yes	yes	yes	yes	yes	yes	AGC6
Low latency protocol - worst case frame 16 bit	yes	yes	yes	yes	yes	yes	AGC6
Low latency protocol - extended frame 24 bit	yes	yes	yes	yes	yes	yes	AGC6
Sejin 4PPM format (38 kHz or 56 kHz)	yes	yes	yes	yes	yes	no	AGC4
MCIR code keyboard package timing	yes	yes	yes	yes	yes	no	AGC4
MCIR code pointing device timing	yes	yes	yes	yes	yes	no	AGC4
MCIR code remote control timing	yes	yes	yes	yes	yes	no	AGC4
Konka TV data format 2004	yes	yes	yes	yes	yes	yes	AGC4
RCA code 56 kHz	yes	yes	yes	yes	no	yes	AGC6
Panasonic 36.7 kHz	yes	no	yes	yes	yes	no	AGC4
Sharp 38 kHz	yes	no	yes	yes	yes	no	AGC4

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

AETHER IC (TSOP3...) - COMPATIBILITY FOR DATA FORMATS						
	AGC1	AGC2	AGC3	AGC4	AGC5	BEST CHOICE
NEC	yes	yes	yes	yes	yes	AGC4
RC5 code	yes	yes	yes	yes	yes	AGC4
RC6 mode 0	yes	yes	yes	yes	yes	AGC4
RCMM	yes	no	yes	no	yes	AGC3
RECS-80 code	yes	no	yes	no	yes	AGC3
R-2000 code (33 kHz)	yes	yes	yes	yes	yes	AGC4
Mitsubishi code 38 kHz	yes	yes	yes	no	yes	AGC3
Sony code SIRCS 12 bit	yes	yes	yes	no	no	AGC-S
Sony code SIRCS 15 bit	yes	yes	no	no	no	AGC-S
Sony code SIRCS 20 bit	yes	yes	no	no	no	AGC-S
r-map data format 38 kHz	yes	no	yes	no	yes	AGC3
r-step data format 38 kHz	yes	yes	yes	yes	yes	AGC4
r-step data format for KB 56 kHz	yes	yes	yes	yes	yes	AGC4
XMP code	yes	no	yes	no	yes	AGC3
Cisco format 57kHz	yes	yes	no	no	no	AGC-C
Cisco format 37kHz	yes	yes	yes	yes	no	AGC-C
Low latency protocol - worst case frame 16 bit	yes	yes	yes	no	no	AGC3
Low latency protocol - extended frame 24 bit	yes	yes	yes	no	no	AGC3
Sejin 4PPM format (38 kHz or 56 kHz)	yes	yes	yes	yes	yes	AGC4
MCIR code keyboard package timing	yes	yes	yes	no	yes	AGC3
MCIR code pointing device timing	yes	yes	yes	no	yes	AGC3
MCIR code remote control timing	yes	yes	yes	yes	yes	AGC3
Konka TV data format 2004	yes	yes	yes	yes	yes	AGC4
RCA code 56 kHz	yes	yes	yes	yes	no	AGC4
Panasonic 36.7 kHz	yes	yes	yes	yes	yes	AGC4
Sharp 38 kHz	yes	yes	yes	yes	yes	AGC4

THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

METHONE IC (TSOP5...) - COMPATIBILITY FOR DATA FORMATS					
	AGC1	AGC2	AGC3	AGC4	BEST CHOICE
NEC	yes	yes	yes	yes	AGC4
RC5 code	yes	yes	yes	yes	AGC4
RC6 mode 0	yes	yes	yes	yes	AGC4
RCMM	yes	no	yes	no	AGC3
RECS-80 code	yes	no	yes	no	AGC3
R-2000 code (33 kHz)	yes	yes	yes	yes	AGC4
Mitsubishi code 38 kHz	yes	yes	yes	yes	AGC3
Sony code SIRCS 12 bit	yes	yes	yes	yes	AGC2
Sony code SIRCS 15 bit	yes	yes	yes	no	AGC2
Sony code SIRCS 20 bit	yes	yes	no	no	AGC2
r-map data format 38 kHz	yes	no	yes	no	AGC3
r-step data format 38 kHz	yes	yes	yes	yes	AGC4
r-step data format for KB 56 kHz	yes	yes	yes	yes	AGC4
XMP code	yes	no	yes	no	AGC3
Cisco format 57kHz	yes	yes	no	no	AGC2
Cisco format 37kHz	yes	yes	yes	yes	AGC2
Low latency protocol - worst case frame 16 bit	yes	yes	yes	yes	AGC4
Low latency protocol - extended frame 24 bit	yes	yes	yes	yes	AGC4
Sejin 4PPM format (38 kHz or 56 kHz)	yes	yes	yes	yes	AGC4
MCIR code keyboard package timing	yes	yes	yes	no	AGC3
MCIR code pointing device timing	yes	yes	yes	no	AGC3
MCIR code remote control timing	yes	yes	yes	yes	AGC3
Konka TV data format 2004	yes	yes	yes	yes	AGC4
RCA code 56 kHz	yes	yes	yes	yes	AGC4
Panasonic 36.7 kHz	yes	yes	yes	yes	AGC4
Sharp 38 kHz	yes	yes	yes	yes	AGC4

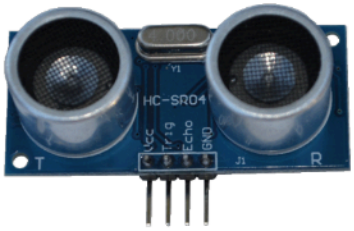
THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE. THE PRODUCTS DESCRIBED HEREIN AND THIS DOCUMENT ARE SUBJECT TO SPECIFIC DISCLAIMERS, SET FORTH AT www.vishay.com/doc?91000

HC-SR04 Ultrasonic Sensor

Elijah J. Morgan

Nov. 16 2014

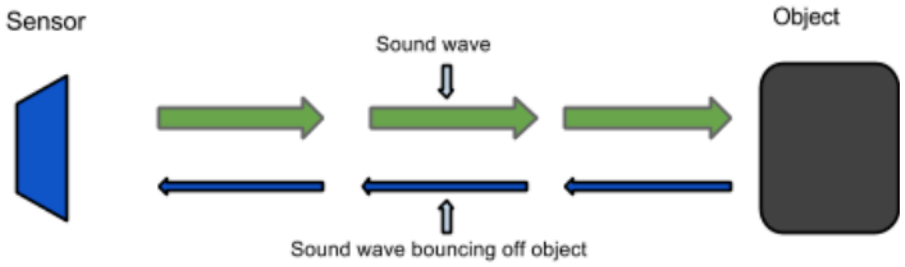
The purpose of this file is to explain how the HC-SR04 works. It will give a brief explanation of how ultrasonic sensors work in general. It will also explain how to wire the sensor up to a microcontroller and how to take/interpret readings. It will also discuss some sources of errors and bad readings.



1. **How Ultrasonic Sensors Work**
2. **HC-SR04 Specifications**
3. **Timing chart, Pin explanations and Taking Distance Measurements**
4. **Wiring HC-SR04 with a microcontroller**
5. **Errors and Bad Readings**

1. How Ultrasonic Sensors Work

Ultrasonic sensors use sound to determine the distance between the sensor and the closest object in its path. How do ultrasonic sensors do this? Ultrasonic sensors are essentially sound sensors, but they operate at a frequency above human hearing.



The sensor sends out a sound wave at a specific frequency. It then listens for that specific sound wave to bounce off of an object and come back (Figure 1). The sensor keeps track of the time between sending the sound wave and the sound wave returning. If you know how fast something is going and how long it is traveling you can find the distance traveled with equation 1.

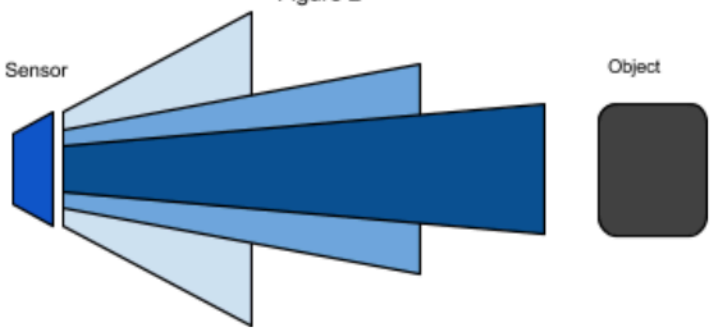
Equation 1. $d = v \times t$

The speed of sound can be calculated based on the a variety of atmospheric conditions, including temperature, humidity and pressure. Actually calculating the distance will be shown later on in this document.

It should be noted that ultrasonic sensors have a cone of detection, the angle of this cone varies with distance, Figure 2 show this relation. The ability of a sensor to

detect an object also depends on the objects orientation to the sensor. If an object doesn't present a flat surface to the sensor then it is possible the sound wave will bounce off the object in a way that it does not return to the sensor.

Figure 2



2. HC-SR04 Specifications

The sensor chosen for the Firefighting Drone Project was the HC-SR04. This section contains the specifications and why they are important to the sensor module. The sensor modules requirements are as follows.

- Cost
- Weight
- Community of hobbyists and support
- Accuracy of object detection
- Probability of working in a smoky environment
- Ease of use

The HC-SR04 Specifications are listed below. These specifications are from the Cytron Technologies HC-SR04 User's Manual (source 1).

- Power Supply: +5V DC
- Quiescent Current: <2mA
- Working current: 15mA
- Effectual Angle: <15º
- Ranging Distance: 2-400 cm
- Resolution: 0.3 cm
- Measuring Angle: 30º
- Trigger Input Pulse width: 10uS
- Dimension: 45mm x 20mm x 15mm
- Weight: approx. 10 g

The HC-SR04's best selling point is its price; it can be purchased at around $2 per unit.

3. Timing Chart and Pin Explanations

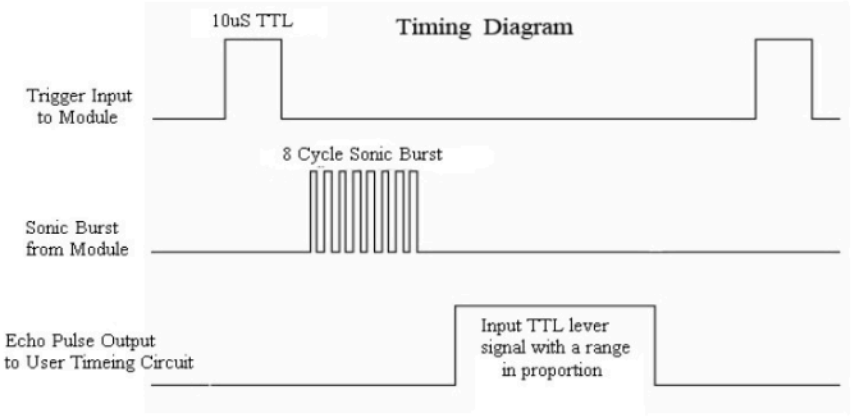
The HC-SR04 has four pins, VCC, GND, TRIG and ECHO; these pins all have different functions. The VCC and GND pins are the simplest -- they power the HC-SR04. These pins need to be attached to a +5 volt source and ground respectively. There is a single control pin: the TRIG pin. The TRIG pin is responsible for sending the ultrasonic burst. This pin should be set to HIGH for 10 µs, at which point the HC-SR04 will send out an eight cycle sonic burst at 40 kHZ. After a sonic burst has been sent the ECHO pin will go HIGH. The ECHO pin is the data pin -- it is used in taking distance measurements. After an ultrasonic burst is sent the pin will go HIGH, it will stay high until an ultrasonic burst is detected back, at which point it will go LOW.

Taking Distance Measurements

The HC-SR04 can be triggered to send out an ultrasonic burst by setting the TRIG pin to HIGH. Once the burst is sent the ECHO pin will automatically go HIGH. This pin will remain HIGH until the the burst hits the sensor again. You can calculate the distance to the object by keeping track of how long the ECHO pin stays HIGH. The time ECHO stays HIGH is the time the burst spent traveling. Using this measurement in equation 1 along with the speed of sound will yield the distance travelled. A summary of this is listed below, along with a visual representation in Figure 2.

1. Set TRIG to HIGH
2. Set a timer when ECHO goes to HIGH
3. Keep the timer running until ECHO goes to LOW
4. Save that time
5. Use equation 1 to determine the distance travelled

Figure 3
Source 2



Source 2

To interpret the time reading into a distance you need to change equation 1. The clock on the device you are using will probably count in microseconds or smaller. To use equation 1 the speed of sound needs to determined,which is 343 meters per second at standard temperature and pressure. To convert this into more useful form use equation 2 to change from meters per second to microseconds per centimeter. Then equation 3 can be used to easily compute the distance in centimeters.

Equation 2. $Distance = \frac{Speed}{170.15\ m} \times \frac{Meters}{100\ cm} \times \frac{1e6\ \mu S}{170.15\ m} \times \frac{58.772\ \mu S}{cm}$

Equation 3. $Distance = \frac{time}{58} = \frac{\mu s}{\mu s/cm} = cm$

4. Wiring the HC-SR04 to a Microcontroller

This section only covers the hardware side. For information on how to integrate the software side, look at one of the links below or look into the specific microcontroller you are using.

The HC-SR04 has 4 pins: VCC, GND, TRIG and ECHO.

1. VCC is a 5v power supply. This should come from the microcontroller
2. GND is a ground pin. Attach to ground on the microcontroller.
3. TRIG should be attached to a GPIO pin that can be set to HIGH
4. ECHO is a little more difficult. The HC-SR04 outputs 5v, which could destroy many microcontroller GPIO pins (the maximum allowed voltage varies). In order to step down the voltage use a single resistor or a voltage divider circuit. Once again this depends on the specific microcontroller you are using, you will need to find out its GPIO maximum voltage and make sure you are below that.

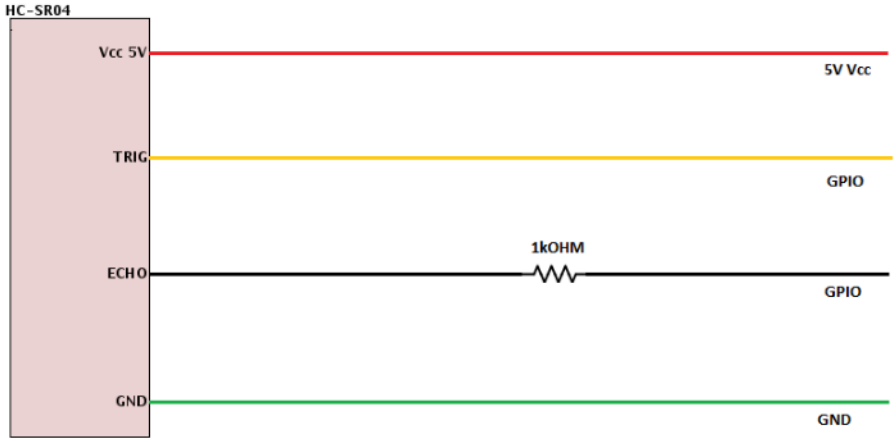
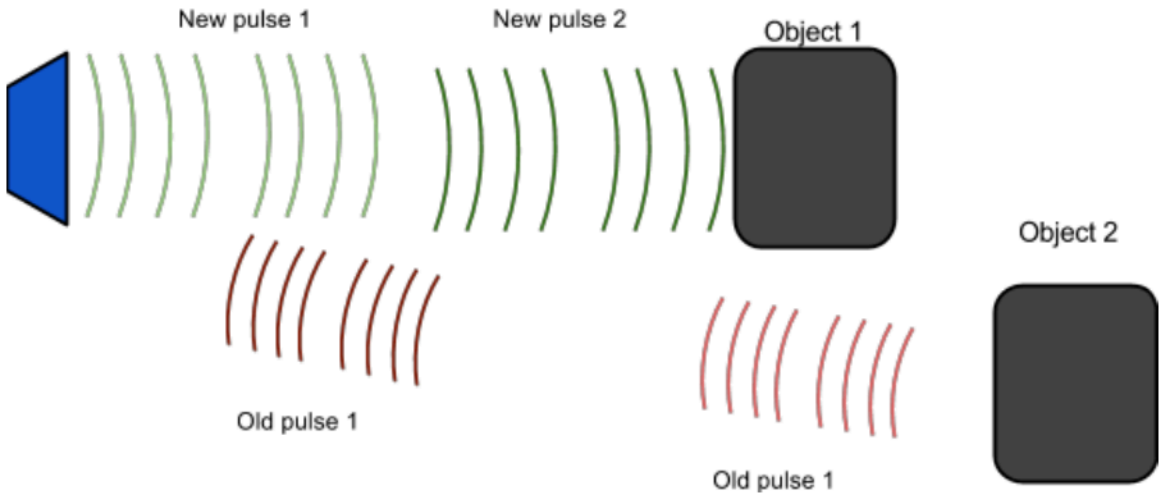


Figure 4

5. Errors and Bad Readings

Ultrasonic sensors are great sensors -- they work well for many applications where other types of sensors fall short. Unfortunately, they do have weaknesses. These weaknesses can be mitigated and worked around, but first they must be understood. The

first weakness is that they use sound. There is a limit to how fast ultrasonic sensors can get distance measurements. The longer the distance, the slower they are at reporting the distance. The second weakness comes from the way sound bounces off of objects. In enclosed spaces it is possible, if not probable that there will be unintended echos. The echos can very easily cause false short readings. In Figure 2 a pulse was sent out. It bounced off of object 1 and returned to the sensor. The distance was recorded and then a new pulse was sent. There was another object farther away, so that when the new pulse reaches object 1, the first signal will reach the sensor. This will cause the sensor to think that there is an object closer than is actually true. The old pulse is smaller than the new pulse because it has grown weaker. The longer the pulse exists the weaker it grows until it is negligible. If multiple sensors are being used, the number of echos will increase along with the number of errors. There are two main ways to reduce the number of errors. The first is to provide shielding around the sensor. This prevents echos coming in from angle outside what the sensor should actually pick up. The second is to reduce the frequency at which pulses are sent out. This gives more time for the echos to dissipate.



Works Cited

Source 1.
"HC-SR04 User's_Manual." *docs.google.* Cytron Technologies, May 2013 Web. 5 Dec. 2009.
<https://docs.google.com/document/d/1Y-yZnNhMYy7rwhAgyL_pfa39RsB-x2qR4vP8saG73rE/edit>

Source 2.
"Attiny2313 Ultrasonic distance (HR-SR04) example." *CircuitDB*. n.a. 7 Sept. 2014 Web. 5 Dec. 2014. <http://www.circuitdb.com/?p=1162>

Links

These are not formatted; you will need to copy and paste them into your web browser.

Want to learn about Ultrasonic Sensors in general?
http://www.sensorsmag.com/sensors/acoustic-ultrasound/choosing-ultrasonic-sensor-proximity-or-distance-measurement-825

All about the HC-SR04

- http://www.circuitdb.com/?p=1162
- http://www.micropik.com/PDF/HCSR04.pdf
- http://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/
- http://www.ezdenki.com/ultrasonic.php
  (^fantastic tutorial, explains a lot of stuff)
- http://www.elecrow.com/hcsr04-ultrasonic-ranging-sensor-p-316.html
  (^ this one has some cool charts)

HD44780U (LCD-II)

(Dot Matrix Liquid Crystal Display Controller/Driver)

HITACHI

ADE-207-272(Z)
'99.9
Rev. 0.0

Description

The HD44780U dot-matrix liquid crystal display controller and driver LSI displays alphanumerics, Japanese kana characters, and symbols. It can be configured to drive a dot-matrix liquid crystal display under the control of a 4- or 8-bit microprocessor. Since all the functions such as display RAM, character generator, and liquid crystal driver, required for driving a dot-matrix liquid crystal display are internally provided on one chip, a minimal system can be interfaced with this controller/driver.

A single HD44780U can display up to one 8-character line or two 8-character lines.

The HD44780U has pin function compatibility with the HD44780S which allows the user to easily replace an LCD-II with an HD44780U. The HD44780U character generator ROM is extended to generate 208 5 × 8 dot character fonts and 32 5 × 10 dot character fonts for a total of 240 different character fonts.

The low power supply (2.7V to 5.5V) of the HD44780U is suitable for any portable battery-driven product requiring low power dissipation.

Features

- 5 × 8 and 5 × 10 dot matrix possible
- Low power operation support:
  — 2.7 to 5.5V
- Wide range of liquid crystal display driver power
  — 3.0 to 11V
- Liquid crystal drive waveform
  — A (One line frequency AC waveform)
- Correspond to high speed MPU bus interface
  — 2 MHz (when V_{CC} = 5V)
- 4-bit or 8-bit MPU interface enabled
- 80 × 8-bit display RAM (80 characters max.)
- 9,920-bit character generator ROM for a total of 240 character fonts
  — 208 character fonts (5 × 8 dot)
  — 32 character fonts (5 × 10 dot)

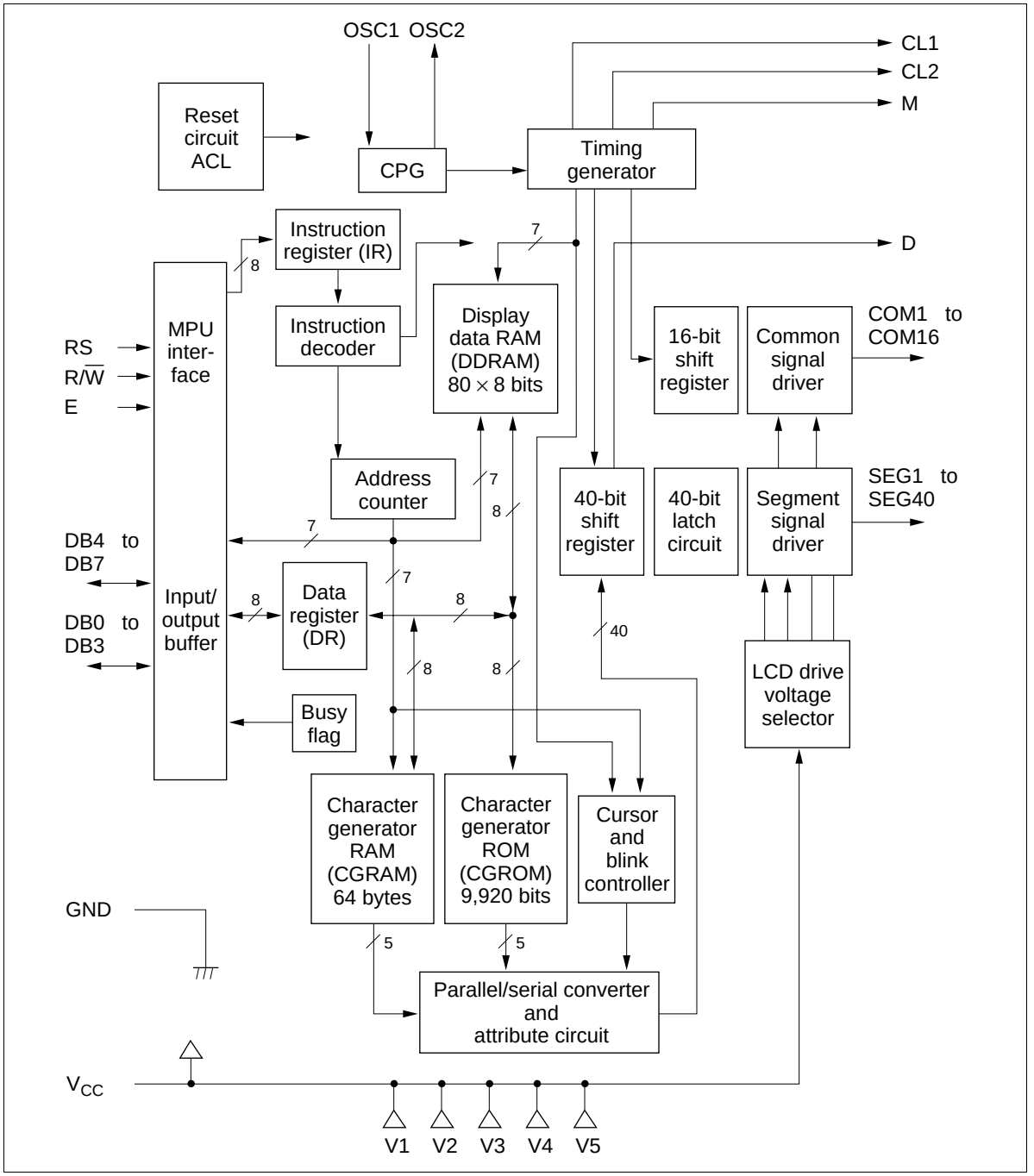
- 64 × 8-bit character generator RAM
  - — 8 character fonts (5 × 8 dot)
  - — 4 character fonts (5 × 10 dot)
- 16-common × 40-segment liquid crystal display driver
- Programmable duty cycles
  - — 1/8 for one line of 5 × 8 dots with cursor
  - — 1/11 for one line of 5 × 10 dots with cursor
  - — 1/16 for two lines of 5 × 8 dots with cursor
- Wide range of instruction functions:
  - — Display clear, cursor home, display on/off, cursor on/off, display character blink, cursor shift, display shift
- Pin function compatibility with HD44780S
- Automatic reset circuit that initializes the controller/driver after power on
- Internal oscillator with external resistors
- Low power consumption

Ordering Information

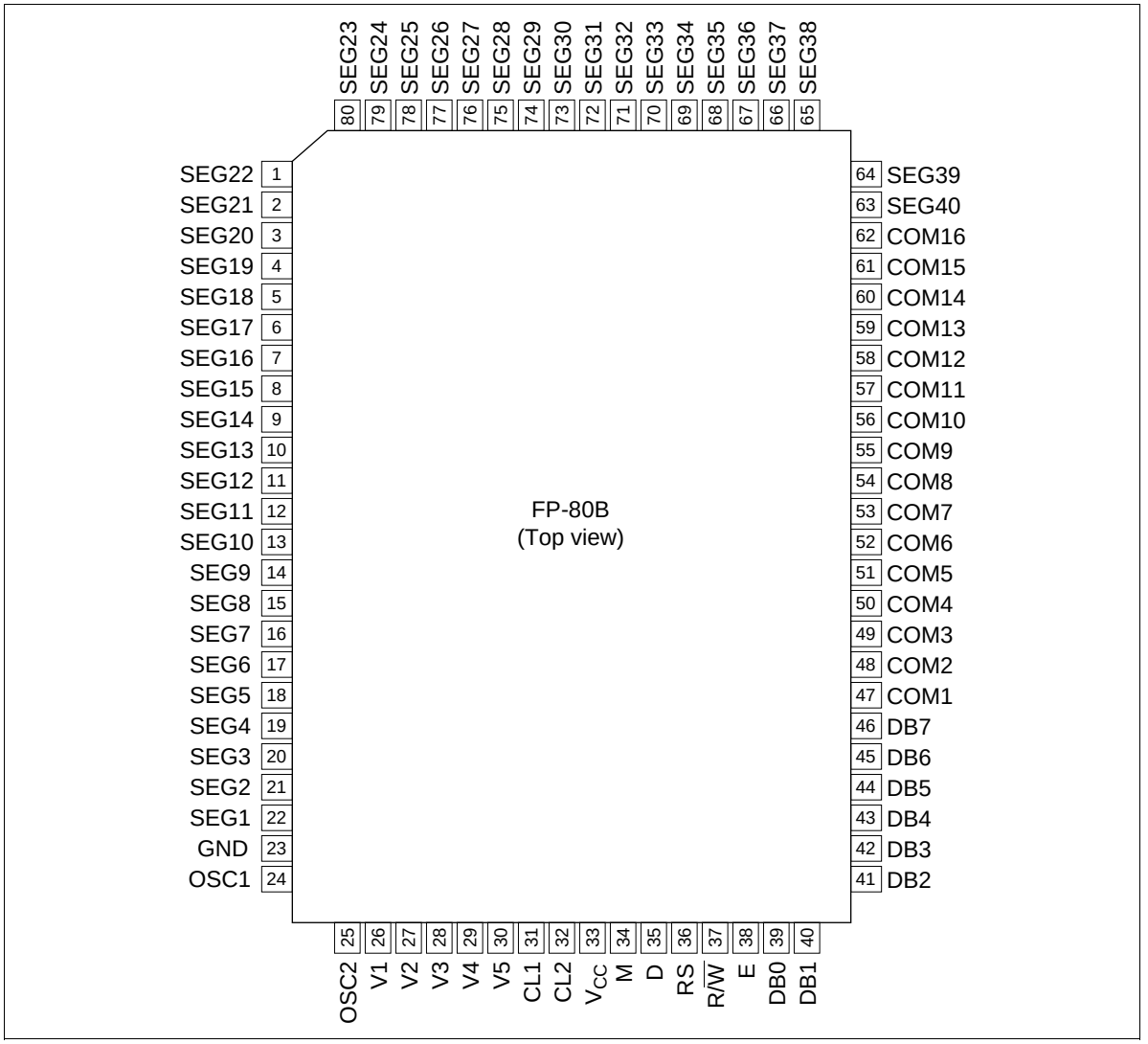
Type No.	Package	CGROM
HD44780UA00FS	FP-80B	Japanese standard font
HCD44780UA00	Chip	
HD44780UA00TF	TFP-80F	
HD44780UA02FS	FP-80B	European standard font
HCD44780UA02	Chip	
HD44780UA02TF	TFP-80F	
HD44780UBxxFS	FP-80B	Custom font
HCD44780UBxx	Chip	
HD44780UBxxTF	TFP-80F	

Note: xx: ROM code No.

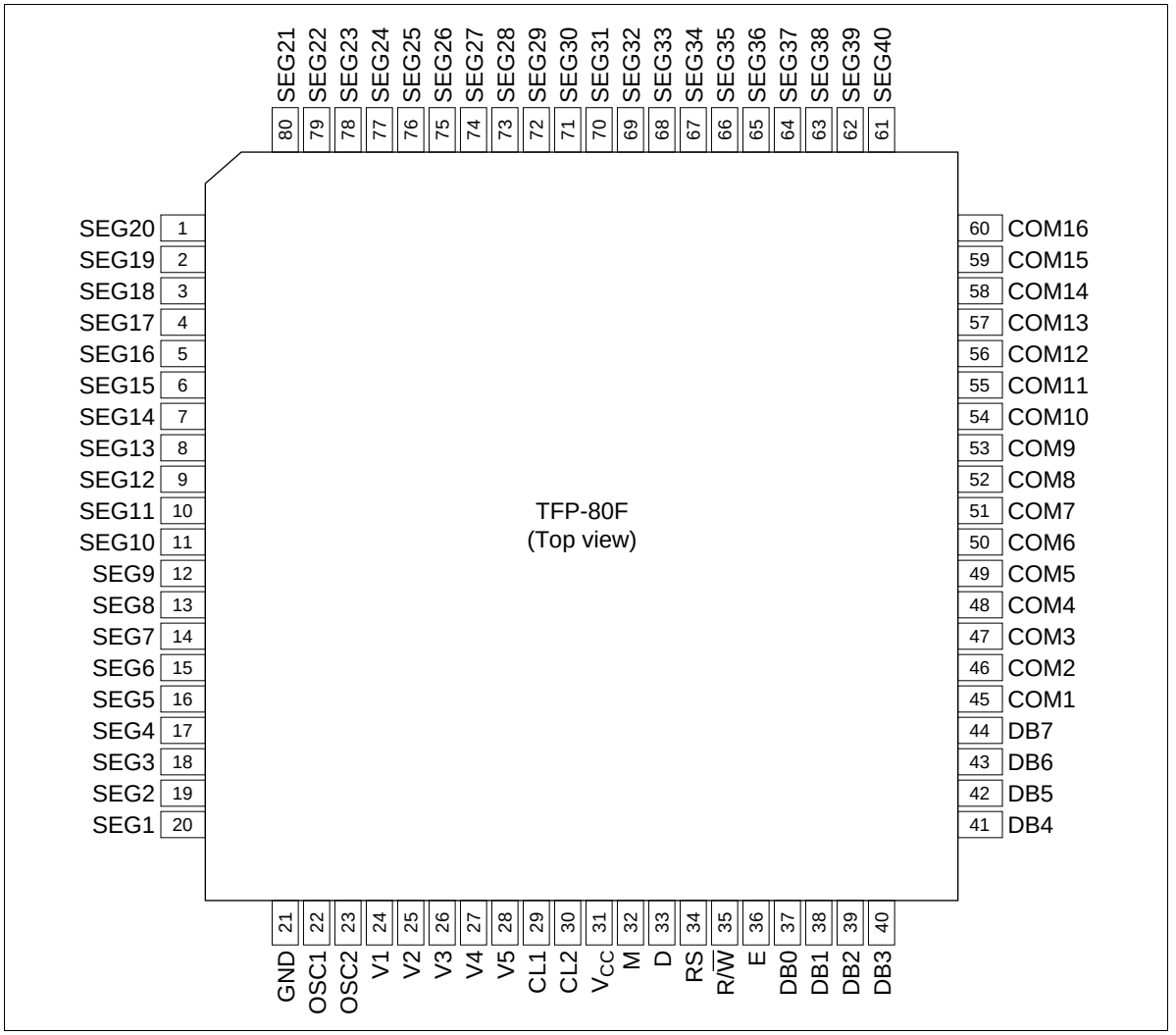
HD44780U Block Diagram



HD44780U Pin Arrangement (FP-80B)

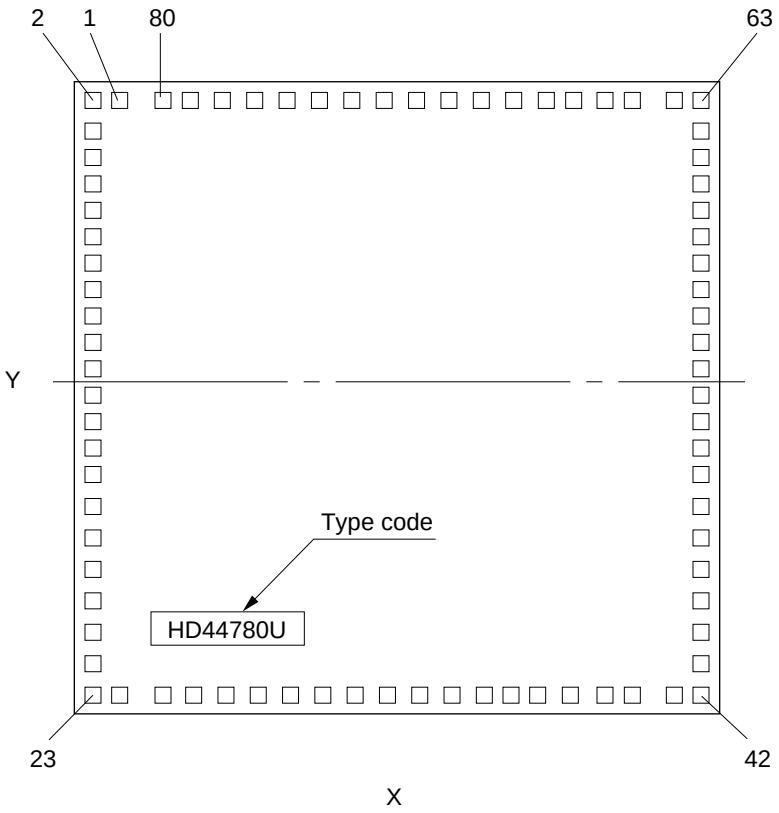


HD44780U Pin Arrangement (TFP-80F)



HD44780U Pad Arrangement

Chip size: 4.90×4.90 mm^2
Coordinate: Pad center (μm)
Origin: Chip center
Pad size: 114×114 μm^2



HCD44780U Pad Location Coordinates

		Coordinate				Coordinate	
Pad No.	Function	X (um)	Y (um)	Pad No.	Function	X (um)	Y (um)
1	SEG22	–2100	2313	41	DB2	2070	–2290
2	SEG21	–2280	2313	42	DB3	2260	–2290
3	SEG20	–2313	2089	43	DB4	2290	–2099
4	SEG19	–2313	1833	44	DB5	2290	–1883
5	SEG18	–2313	1617	45	DB6	2290	–1667
6	SEG17	–2313	1401	46	DB7	2290	–1452
7	SEG16	–2313	1186	47	COM1	2313	–1186
8	SEG15	–2313	970	48	COM2	2313	–970
9	SEG14	–2313	755	49	COM3	2313	–755
10	SEG13	–2313	539	50	COM4	2313	–539
11	SEG12	–2313	323	51	COM5	2313	–323
12	SEG11	–2313	108	52	COM6	2313	–108
13	SEG10	–2313	–108	53	COM7	2313	108
14	SEG9	–2313	–323	54	COM8	2313	323
15	SEG8	–2313	–539	55	COM9	2313	539
16	SEG7	–2313	–755	56	COM10	2313	755
17	SEG6	–2313	–970	57	COM11	2313	970
18	SEG5	–2313	–1186	58	COM12	2313	1186
19	SEG4	–2313	–1401	59	COM13	2313	1401
20	SEG3	–2313	–1617	60	COM14	2313	1617
21	SEG2	–2313	–1833	61	COM15	2313	1833
22	SEG1	–2313	–2073	62	COM16	2313	2095
23	GND	–2280	–2290	63	SEG40	2296	2313
24	OSC1	–2080	–2290	64	SEG39	2100	2313
25	OSC2	–1749	–2290	65	SEG38	1617	2313
26	V1	–1550	–2290	66	SEG37	1401	2313
27	V2	–1268	–2290	67	SEG36	1186	2313
28	V3	–941	–2290	68	SEG35	970	2313
29	V4	–623	–2290	69	SEG34	755	2313
30	V5	–304	–2290	70	SEG33	539	2313
31	CL1	–48	–2290	71	SEG32	323	2313
32	CL2	142	–2290	72	SEG31	108	2313
33	V_{CC}	309	–2290	73	SEG30	–108	2313
34	M	475	–2290	74	SEG29	–323	2313
35	D	665	–2290	75	SEG28	–539	2313
36	RS	832	–2290	76	SEG27	–755	2313
37	R/$\overline{W}$	1022	–2290	77	SEG26	–970	2313
38	E	1204	–2290	78	SEG25	–1186	2313
39	DB0	1454	–2290	79	SEG24	–1401	2313
40	DB1	1684	–2290	80	SEG23	–1617	2313

Pin Functions

Signal	No. of Lines	I/O	Device Interfaced with	Function
RS	1	I	MPU	Selects registers. 0: Instruction register (for write) Busy flag: address counter (for read) 1: Data register (for write and read)
R/$\overline{W}$	1	I	MPU	Selects read or write. 0: Write 1: Read
E	1	I	MPU	Starts data read/write.
DB4 to DB7	4	I/O	MPU	Four high order bidirectional tristate data bus pins. Used for data transfer and receive between the MPU and the HD44780U. DB7 can be used as a busy flag.
DB0 to DB3	4	I/O	MPU	Four low order bidirectional tristate data bus pins. Used for data transfer and receive between the MPU and the HD44780U. These pins are not used during 4-bit operation.
CL1	1	O	Extension driver	Clock to latch serial data D sent to the extension driver
CL2	1	O	Extension driver	Clock to shift serial data D
M	1	O	Extension driver	Switch signal for converting the liquid crystal drive waveform to AC
D	1	O	Extension driver	Character pattern data corresponding to each segment signal
COM1 to COM16	16	O	LCD	Common signals that are not used are changed to non-selection waveforms. COM9 to COM16 are non-selection waveforms at 1/8 duty factor and COM12 to COM16 are non-selection waveforms at 1/11 duty factor.
SEG1 to SEG40	40	O	LCD	Segment signals
V1 to V5	5	—	Power supply	Power supply for LCD drive V_{CC} –V5 = 11 V (max)
V_{CC}, GND	2	—	Power supply	V_{CC}: 2.7V to 5.5V, GND: 0V
OSC1, OSC2	2	—	Oscillation resistor clock	When crystal oscillation is performed, a resistor must be connected externally. When the pin input is an external clock, it must be input to OSC1.

Function Description

Registers

The HD44780U has two 8-bit registers, an instruction register (IR) and a data register (DR).

The IR stores instruction codes, such as display clear and cursor shift, and address information for display data RAM (DDRAM) and character generator RAM (CGRAM). The IR can only be written from the MPU.

The DR temporarily stores data to be written into DDRAM or CGRAM and temporarily stores data to be read from DDRAM or CGRAM. Data written into the DR from the MPU is automatically written into DDRAM or CGRAM by an internal operation. The DR is also used for data storage when reading data from DDRAM or CGRAM. When address information is written into the IR, data is read and then stored into the DR from DDRAM or CGRAM by an internal operation. Data transfer between the MPU is then completed when the MPU reads the DR. After the read, data in DDRAM or CGRAM at the next address is sent to the DR for the next read from the MPU. By the register selector (RS) signal, these two registers can be selected (Table 1).

Busy Flag (BF)

When the busy flag is 1, the HD44780U is in the internal operation mode, and the next instruction will not be accepted. When RS = 0 and R/$\overline{W}$ = 1 (Table 1), the busy flag is output to DB7. The next instruction must be written after ensuring that the busy flag is 0.

Address Counter (AC)

The address counter (AC) assigns addresses to both DDRAM and CGRAM. When an address of an instruction is written into the IR, the address information is sent from the IR to the AC. Selection of either DDRAM or CGRAM is also determined concurrently by the instruction.

After writing into (reading from) DDRAM or CGRAM, the AC is automatically incremented by 1 (decremented by 1). The AC contents are then output to DB0 to DB6 when RS = 0 and R/$\overline{W}$ = 1 (Table 1).

Table 1 Register Selection

RS	R/$\overline{W}$	Operation
0	0	IR write as an internal operation (display clear, etc.)
0	1	Read busy flag (DB7) and address counter (DB0 to DB6)
1	0	DR write as an internal operation (DR to DDRAM or CGRAM)
1	1	DR read as an internal operation (DDRAM or CGRAM to DR)

Display Data RAM (DDRAM)

Display data RAM (DDRAM) stores display data represented in 8-bit character codes. Its extended capacity is 80 × 8 bits, or 80 characters. The area in display data RAM (DDRAM) that is not used for display can be used as general data RAM. See Figure 1 for the relationships between DDRAM addresses and positions on the liquid crystal display.

The DDRAM address (A_{DD}) is set in the address counter (AC) as hexadecimal.

- 1-line display (N = 0) (Figure 2)
  - When there are fewer than 80 display characters, the display begins at the head position. For example, if using only the HD44780, 8 characters are displayed. See Figure 3.

    When the display shift operation is performed, the DDRAM address shifts. See Figure 3.

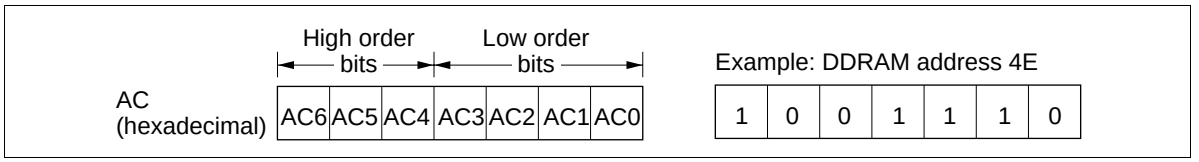


Figure 1 DDRAM Address

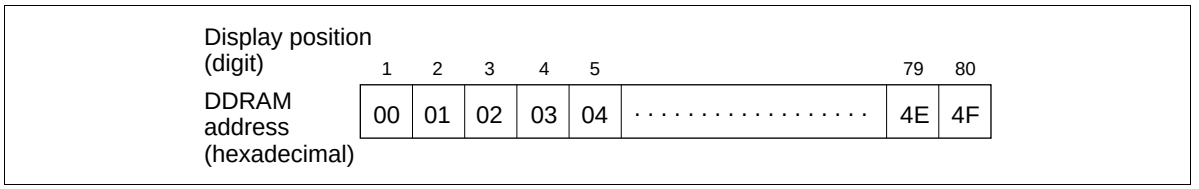


Figure 2 1-Line Display

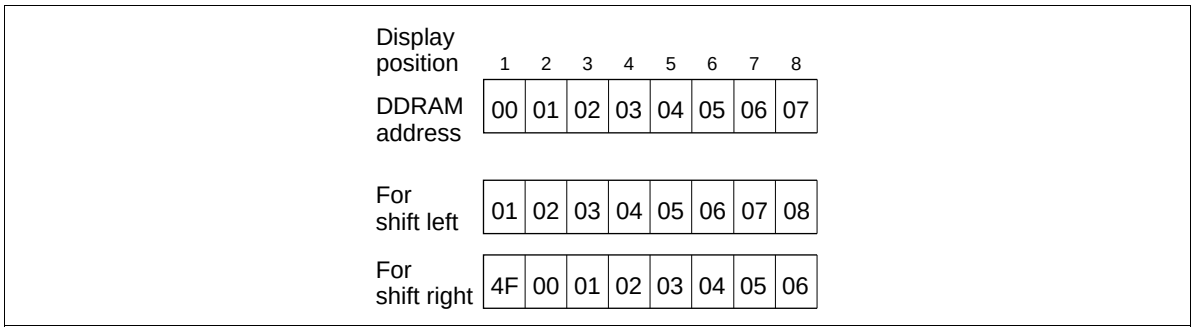


Figure 3 1-Line by 8-Character Display Example

- 2-line display (N = 1) (Figure 4)

  — Case 1: When the number of display characters is less than 40 × 2 lines, the two lines are displayed from the head. Note that the first line end address and the second line start address are not consecutive. For example, when just the HD44780 is used, 8 characters × 2 lines are displayed. See Figure 5.

  When display shift operation is performed, the DDRAM address shifts. See Figure 5.

Display position	1	2	3	4	5		39	40
DDRAM address (hexadecimal)	00	01	02	03	04	· · · · · · · · · · · · · · · · · ·	26	27
	40	41	42	43	44	· · · · · · · · · · · · · · · · · ·	66	67

Figure 4 2-Line Display

Display position	1	2	3	4	5	6	7	8
DDRAM address	00	01	02	03	04	05	06	07
	40	41	42	43	44	45	46	47
For shift left	01	02	03	04	05	06	07	08
	41	42	43	44	45	46	47	48
For shift right	27	00	01	02	03	04	05	06
	67	40	41	42	43	44	45	46

Figure 5 2-Line by 8-Character Display Example

— Case 2: For a 16-character × 2-line display, the HD44780 can be extended using one 40-output extension driver. See Figure 6.

When display shift operation is performed, the DDRAM address shifts. See Figure 6.

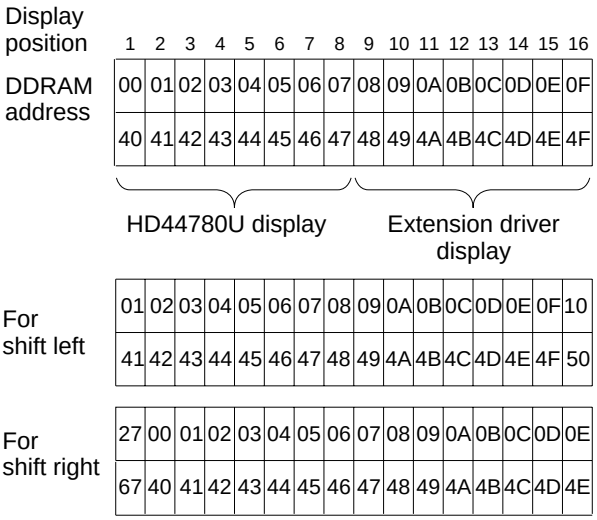


Figure 6 2-Line by 16-Character Display Example

Character Generator ROM (CGROM)

The character generator ROM generates 5 × 8 dot or 5 × 10 dot character patterns from 8-bit character codes (Table 4). It can generate 208 5 × 8 dot character patterns and 32 5 × 10 dot character patterns. User-defined character patterns are also available by mask-programmed ROM.

Character Generator RAM (CGRAM)

In the character generator RAM, the user can rewrite character patterns by program. For 5 × 8 dots, eight character patterns can be written, and for 5 × 10 dots, four character patterns can be written.

Write into DDRAM the character codes at the addresses shown as the left column of Table 4 to show the character patterns stored in CGRAM.

See Table 5 for the relationship between CGRAM addresses and data and display patterns.

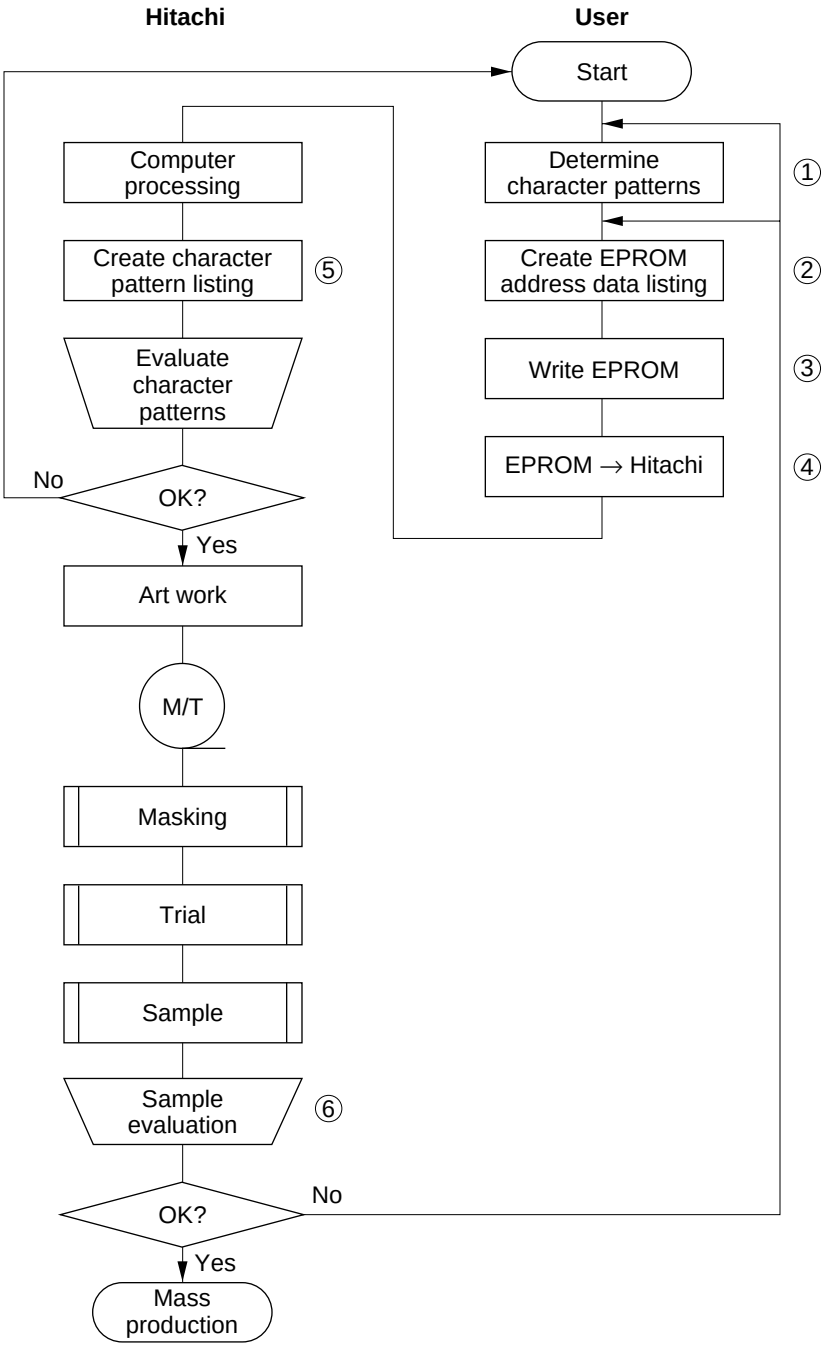
Areas that are not used for display can be used as general data RAM.

Modifying Character Patterns

- Character pattern development procedure

The following operations correspond to the numbers listed in Figure 7:

1. Determine the correspondence between character codes and character patterns.
2. Create a listing indicating the correspondence between EPROM addresses and data.
3. Program the character patterns into the EPROM.
4. Send the EPROM to Hitachi.
5. Computer processing on the EPROM is performed at Hitachi to create a character pattern listing, which is sent to the user.
6. If there are no problems within the character pattern listing, a trial LSI is created at Hitachi and samples are sent to the user for evaluation. When it is confirmed by the user that the character patterns are correctly written, mass production of the LSI proceeds at Hitachi.



Note: For a description of the numbers used in this figure, refer to the preceding page.

Figure 7 Character Pattern Development Procedure

- Programming character patterns

  This section explains the correspondence between addresses and data used to program character patterns in EPROM. The HD44780U character generator ROM can generate 208 5 × 8 dot character patterns and 32 5 × 10 dot character patterns for a total of 240 different character patterns.

  — Character patterns

  EPROM address data and character pattern data correspond with each other to form a 5 × 8 or 5 × 10 dot character pattern (Tables 2 and 3).

Table 2 Example of Correspondence between EPROM Address Data and Character Pattern (5 × 8 Dots)

EPROM Address												Data					
A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	O4	O3	O2	O1	LSB O0	
								0	0	0	0	1	0	0	0	0	
								0	0	0	1	1	0	0	0	0	
								0	0	1	0	1	0	1	1	0	
								0	0	1	1	1	1	0	0	1	
								0	1	0	0	1	0	0	0	1	
								0	1	0	1	1	0	0	0	1	
								0	1	1	0	1	1	1	1	0	
0	1	1	0	0	0	1	0	0	1	1	1	0	0	0	0	0	← Cursor position
								1	0	0	0	0	0	0	0	0	
								1	0	0	1	0	0	0	0	0	
								1	0	1	0	0	0	0	0	0	
								1	0	1	1	0	0	0	0	0	
								1	1	0	0	0	0	0	0	0	
								1	1	0	1	0	0	0	0	0	
								1	1	1	0	0	0	0	0	0	
								1	1	1	1	0	0	0	0	0	

Character code — Line position

Notes:
1. EPROM addresses A11 to A4 correspond to a character code.
2. EPROM addresses A3 to A0 specify a line position of the character pattern.
3. EPROM data O4 to O0 correspond to character pattern data.
4. EPROM data O5 to O7 must be specified as 0.
5. A lit display position (black) corresponds to a 1.
6. Line 9 and the following lines must be blanked with 0s for a 5 × 8 dot character fonts.

— Handling unused character patterns

1. EPROM data outside the character pattern area: Always input 0s.
2. EPROM data in CGRAM area: Always input 0s. (Input 0s to EPROM addresses 00H to FFH.)
3. EPROM data used when the user does not use any HD44780U character pattern: According to the user application, handled in one of the two ways listed as follows.
   a. When unused character patterns are not programmed: If an unused character code is written into DDRAM, all its dots are lit. By not programing a character pattern, all of its bits become lit. (This is due to the EPROM being filled with 1s after it is erased.)
   b. When unused character patterns are programmed as 0s: Nothing is displayed even if unused character codes are written into DDRAM. (This is equivalent to a space.)

Table 3 Example of Correspondence between EPROM Address Data and Character Pattern (5 × 10 Dots)

EPROM Address												Data					
A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	O4	O3	O2	O1	LSB O0	
								0	0	0	0	0	0	0	0	0	
								0	0	0	1	0	0	0	0	0	
								0	0	1	0	0	1	1	0	1	
								0	0	1	1	1	0	0	1	1	
								0	1	0	0	1	0	0	0	1	
								0	1	0	1	1	0	0	0	1	
								0	1	1	0	0	1	1	1	1	
0	1	0	1	0	0	1	0	0	1	1	1	0	0	0	0	1	
								1	0	0	0	0	0	0	0	1	
								1	0	0	1	0	0	0	0	1	
								1	0	1	0	0	0	0	0	0	← Cursor position
								1	0	1	1	0	0	0	0	0	
								1	1	0	0	0	0	0	0	0	
								1	1	0	1	0	0	0	0	0	
								1	1	1	0	0	0	0	0	0	
								1	1	1	1	0	0	0	0	0	

Character code — Line position

Notes: 1. EPROM addresses A11 to A3 correspond to a character code.
2. EPROM addresses A3 to A0 specify a line position of the character pattern.
3. EPROM data O4 to O0 correspond to character pattern data.
4. EPROM data O5 to O7 must be specified as 0.
5. A lit display position (black) corresponds to a 1.
6. Line 11 and the following lines must be blanked with 0s for a 5 × 10 dot character fonts.

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A00)

Upper 4 Bits / Lower 4 Bits	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000	CG RAM (1)															
xxxx0001	(2)															
xxxx0010	(3)															
xxxx0011	(4)															
xxxx0100	(5)															
xxxx0101	(6)															
xxxx0110	(7)															
xxxx0111	(8)															
xxxx1000	(1)															
xxxx1001	(2)															
xxxx1010	(3)															
xxxx1011	(4)															
xxxx1100	(5)															
xxxx1101	(6)															
xxxx1110	(7)															
xxxx1111	(8)															

Note: The user can specify any pattern for character-generator RAM.

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A02)

Upper 4 Bits / Lower 4 Bits	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000	CG RAM (1)														
xxxx0001	(2)															
xxxx0010	(3)															
xxxx0011	(4)															
xxxx0100	(5)															
xxxx0101	(6)															
xxxx0110	(7)															
xxxx0111	(8)															
xxxx1000	(1)															
xxxx1001	(2)															
xxxx1010	(3)															
xxxx1011	(4)															
xxxx1100	(5)															
xxxx1101	(6)															
xxxx1110	(7)															
xxxx1111	(8)															

Table 5 Relationship between CGRAM Addresses, Character Codes (DDRAM) and Character Patterns (CGRAM Data)

For 5 × 8 dot character patterns

Character Codes (DDRAM data)	CGRAM Address	Character Patterns (CGRAM data)	
7 6 5 4 3 2 1 0 High Low	5 4 3 2 1 0 High Low	7 6 5 4 3 2 1 0 High Low	
0 0 0 0 * 0 0 0	0 0 0	0 0 0 * * * 1 1 1 1 0 0 0 1 * * * 1 0 0 0 1 0 1 0 * * * 1 0 0 0 1 0 1 1 * * * 1 1 1 1 0 1 0 0 * * * 1 0 1 0 0 1 0 1 * * * 1 0 0 1 0 1 1 0 * * * 1 0 0 0 1 1 1 1 * * * 0 0 0 0 0	Character pattern (1) Cursor position
0 0 0 0 * 0 0 1	0 0 1	0 0 0 * * * 1 0 0 0 1 0 0 1 * * * 0 1 0 1 0 0 1 0 * * * 1 1 1 1 1 0 1 1 * * * 0 0 1 0 0 1 0 0 * * * 1 1 1 1 1 1 0 1 * * * 0 0 1 0 0 1 1 0 * * * 0 0 1 0 0 1 1 1 * * * 0 0 0 0 0	Character pattern (2) Cursor position
		0 0 0 * * *	
		0 0 1 * * *	
0 0 0 0 * 1 1 1	1 1 1	1 0 0 * * *	
		1 0 1 * * *	
		1 1 0 * * *	
		1 1 1 * * *	

Notes: 1. Character code bits 0 to 2 correspond to CGRAM address bits 3 to 5 (3 bits: 8 types).

2. CGRAM address bits 0 to 2 designate the character pattern line position. The 8th line is the cursor position and its display is formed by a logical OR with the cursor.

   Maintain the 8th line data, corresponding to the cursor display position, at 0 as the cursor display.

   If the 8th line data is 1, 1 bits will light up the 8th line regardless of the cursor presence.

3. Character pattern row positions correspond to CGRAM data bits 0 to 4 (bit 4 being at the left).

4. As shown Table 5, CGRAM character patterns are selected when character code bits 4 to 7 are all 0. However, since character code bit 3 has no effect, the R display example above can be selected by either character code 00H or 08H.

5. 1 for CGRAM data corresponds to display selection and 0 to non-selection.

* Indicates no effect.

Table 5 Relationship between CGRAM Addresses, Character Codes (DDRAM) and Character Patterns (CGRAM Data) (cont)

For 5 × 10 dot character patterns

Character Codes (DDRAM data)	CGRAM Address	Character Patterns (CGRAM data)	
7 6 5 4 3 2 1 0 High Low	5 4 3 2 1 0 High Low	7 6 5 4 3 2 1 0 High Low	
0 0 0 0 * 0 0 *	0 0	0 0 0 0 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 0 1 0 1 0 1 1 0 0 1 1 1 1 0 0 0 1 0 0 1 1 0 1 0	Character pattern Cursor position
		1 0 1 1 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1	
0 0 0 0 * 1 1 *	1 1	0 0 0 0 0 0 0 1 1 0 0 1 1 0 1 0	
		1 0 1 1 1 1 0 0 1 1 0 1 1 1 1 0 1 1 1 1	

Notes: 1. Character code bits 1 and 2 correspond to CGRAM address bits 4 and 5 (2 bits: 4 types).

2. CGRAM address bits 0 to 3 designate the character pattern line position. The 11th line is the cursor position and its display is formed by a logical OR with the cursor.

   Maintain the 11th line data corresponding to the cursor display positon at 0 as the cursor display.

   If the 11th line data is “1”, “1” bits will light up the 11th line regardless of the cursor presence.

   Since lines 12 to 16 are not used for display, they can be used for general data RAM.

3. Character pattern row positions are the same as 5 × 8 dot character pattern positions.

4. CGRAM character patterns are selected when character code bits 4 to 7 are all 0.

   However, since character code bits 0 and 3 have no effect, the P display example above can be selected by character codes 00H, 01H, 08H, and 09H.

5. 1 for CGRAM data corresponds to display selection and 0 to non-selection.

* Indicates no effect.

Timing Generation Circuit

The timing generation circuit generates timing signals for the operation of internal circuits such as DDRAM, CGROM and CGRAM. RAM read timing for display and internal operation timing by MPU access are generated separately to avoid interfering with each other. Therefore, when writing data to DDRAM, for example, there will be no undesirable interferences, such as flickering, in areas other than the display area.

Liquid Crystal Display Driver Circuit

The liquid crystal display driver circuit consists of 16 common signal drivers and 40 segment signal drivers. When the character font and number of lines are selected by a program, the required common signal drivers automatically output drive waveforms, while the other common signal drivers continue to output non-selection waveforms.

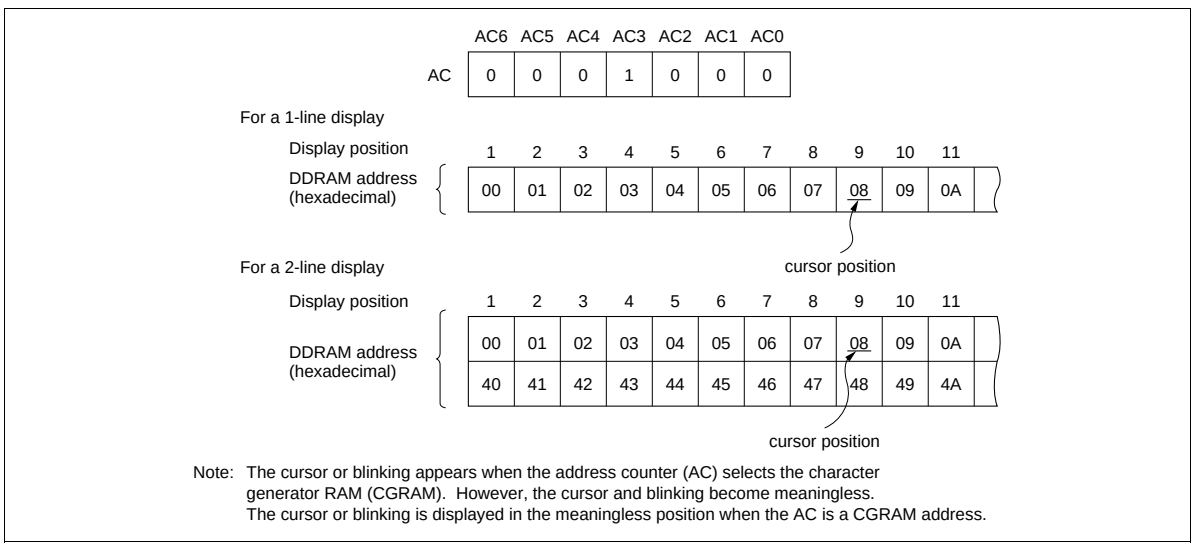
Sending serial data always starts at the display data character pattern corresponding to the last address of the display data RAM (DDRAM).

Since serial data is latched when the display data character pattern corresponding to the starting address enters the internal shift register, the HD44780U drives from the head display.

Cursor/Blink Control Circuit

The cursor/blink control circuit generates the cursor or character blinking. The cursor or the blinking will appear with the digit located at the display data RAM (DDRAM) address set in the address counter (AC).

For example (Figure 8), when the address counter is 08H, the cursor position is displayed at DDRAM address 08H.



Note: The cursor or blinking appears when the address counter (AC) selects the character generator RAM (CGRAM). However, the cursor and blinking become meaningless. The cursor or blinking is displayed in the meaningless position when the AC is a CGRAM address.

Figure 8 Cursor/Blink Display Example

Interfacing to the MPU

The HD44780U can send data in either two 4-bit operations or one 8-bit operation, thus allowing interfacing with 4- or 8-bit MPUs.

- For 4-bit interface data, only four bus lines (DB4 to DB7) are used for transfer. Bus lines DB0 to DB3 are disabled. The data transfer between the HD44780U and the MPU is completed after the 4-bit data has been transferred twice. As for the order of data transfer, the four high order bits (for 8-bit operation, DB4 to DB7) are transferred before the four low order bits (for 8-bit operation, DB0 to DB3).

  The busy flag must be checked (one instruction) after the 4-bit data has been transferred twice. Two more 4-bit operations then transfer the busy flag and address counter data.
- For 8-bit interface data, all eight bus lines (DB0 to DB7) are used.

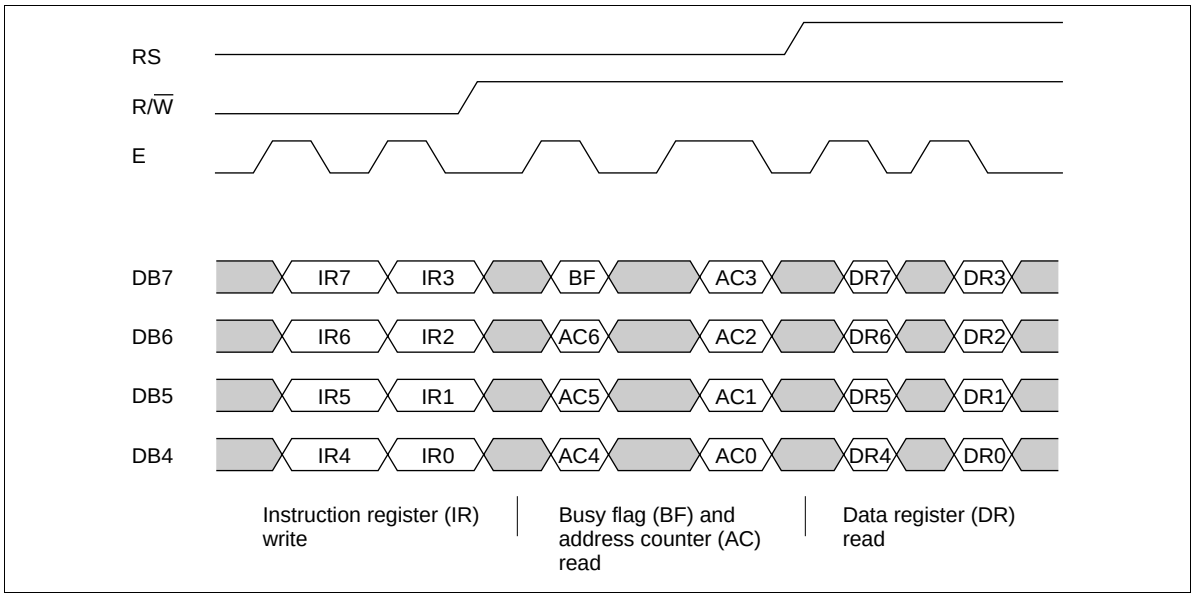


Figure 9 4-Bit Transfer Example

Reset Function

Initializing by Internal Reset Circuit

An internal reset circuit automatically initializes the HD44780U when the power is turned on. The following instructions are executed during the initialization. The busy flag (BF) is kept in the busy state until the initialization ends (BF = 1). The busy state lasts for 10 ms after V_{CC} rises to 4.5 V.

1. Display clear
2. Function set:
   DL = 1; 8-bit interface data
   N = 0; 1-line display
   F = 0; 5 × 8 dot character font
3. Display on/off control:
   D = 0; Display off
   C = 0; Cursor off
   B = 0; Blinking off
4. Entry mode set:
   I/D = 1; Increment by 1
   S = 0; No shift

Note: If the electrical characteristics conditions listed under the table Power Supply Conditions Using Internal Reset Circuit are not met, the internal reset circuit will not operate normally and will fail to initialize the HD44780U. For such a case, initial-ization must be performed by the MPU as explained in the section, Initializing by Instruction.

Instructions

Outline

Only the instruction register (IR) and the data register (DR) of the HD44780U can be controlled by the MPU. Before starting the internal operation of the HD44780U, control information is temporarily stored into these registers to allow interfacing with various MPUs, which operate at different speeds, or various peripheral control devices. The internal operation of the HD44780U is determined by signals sent from the MPU. These signals, which include register selection signal (RS), read/

write signal (R/$\overline{W}$), and the data bus (DB0 to DB7), make up the HD44780U instructions (Table 6). There are four categories of instructions that:

- Designate HD44780U functions, such as display format, data length, etc.
- Set internal RAM addresses
- Perform data transfer with internal RAM
- Perform miscellaneous functions

Normally, instructions that perform data transfer with internal RAM are used the most. However, auto-incrementation by 1 (or auto-decrementation by 1) of internal HD44780U RAM addresses after each data write can lighten the program load of the MPU. Since the display shift instruction (Table 11) can perform concurrently with display data write, the user can minimize system development time with maximum programming efficiency.

When an instruction is being executed for internal operation, no instruction other than the busy flag/address read instruction can be executed.

Because the busy flag is set to 1 while an instruction is being executed, check it to make sure it is 0 before sending another instruction from the MPU.

Note: Be sure the HD44780U is not in the busy state (BF = 0) before sending an instruction from the MPU to the HD44780U. If an instruction is sent without checking the busy flag, the time between the first instruction and next instruction will take much longer than the instruction time itself. Refer to Table 6 for the list of each instruc-tion execution time.

Table 6 Instructions

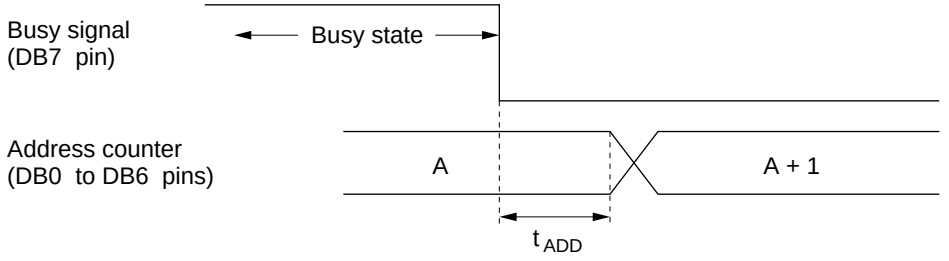
	Code											Execution Time (max) (when f_{cp} or f_{osc} is 270 kHz)
Instruction	**RS**	**R/$\overline{W}$**	**DB7**	**DB6**	**DB5**	**DB4**	**DB3**	**DB2**	**DB1**	**DB0**	**Description**	
Clear display	0	0	0	0	0	0	0	0	0	1	Clears entire display and sets DDRAM address 0 in address counter.	
Return home	0	0	0	0	0	0	0	0	1	—	Sets DDRAM address 0 in address counter. Also returns display from being shifted to original position. DDRAM contents remain unchanged.	1.52 ms
Entry mode set	0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction and specifies display shift. These operations are performed during data write and read.	37 µs
Display on/off control	0	0	0	0	0	0	1	D	C	B	Sets entire display (D) on/off, cursor on/off (C), and blinking of cursor position character (B).	37 µs
Cursor or display shift	0	0	0	0	0	1	S/C	R/L	—	—	Moves cursor and shifts display without changing DDRAM contents.	37 µs
Function set	0	0	0	0	1	DL	N	F	—	—	Sets interface data length (DL), number of display lines (N), and character font (F).	37 µs
Set CGRAM address	0	0	0	1	ACG	ACG	ACG	ACG	ACG	ACG	Sets CGRAM address. CGRAM data is sent and received after this setting.	37 µs
Set DDRAM address	0	0	1	ADD	ADD	ADD	ADD	ADD	ADD	ADD	Sets DDRAM address. DDRAM data is sent and received after this setting.	37 µs
Read busy flag & address	0	1	BF	AC	AC	AC	AC	AC	AC	AC	Reads busy flag (BF) indicating internal operation is being performed and reads address counter contents.	0 µs

Table 6 Instructions (cont)

Instruction	Code RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Description	Execution Time (max) (when f_{cp} or f_{OSC} is 270 kHz)
Write data to CG or DDRAM	1	0	Write data								Writes data into DDRAM or CGRAM.	37 µs t_{ADD} = 4 µs*
Read data from CG or DDRAM	1	1	Read data								Reads data from DDRAM or CGRAM.	37 µs t_{ADD} = 4 µs*
	I/D = 1: Increment I/D = 0: Decrement S = 1: Accompanies display shift S/C = 1: Display shift S/C = 0: Cursor move R/L = 1: Shift to the right R/L = 0: Shift to the left DL = 1: 8 bits, DL = 0: 4 bits N = 1: 2 lines, N = 0: 1 line F = 1: 5 × 10 dots, F = 0: 5 × 8 dots BF = 1: Internally operating BF = 0: Instructions acceptable										DDRAM: Display data RAM CGRAM: Character generator RAM ACG: CGRAM address ADD: DDRAM address (corresponds to cursor address) AC: Address counter used for both DD and CGRAM addresses	Execution time changes when frequency changes Example: When f_{cp} or f_{OSC} is 250 kHz, $37\ \mu s \times \frac{270}{250} = 40\ \mu s$

Note: — indicates no effect.

* After execution of the CGRAM/DDRAM data write or read instruction, the RAM address counter is incremented or decremented by 1. The RAM address counter is updated after the busy flag turns off. In Figure 10, t_{ADD} is the time elapsed after the busy flag turns off until the address counter is updated.



Note: t_{ADD} depends on the operation frequency
t_{ADD} = 1.5/(f_{cp} or f_{OSC}) seconds

Figure 10 Address Counter Update

Instruction Description

Clear Display

Clear display writes space code 20H (character pattern for character code 20H must be a blank pattern) into all DDRAM addresses. It then sets DDRAM address 0 into the address counter, and returns the display to its original status if it was shifted. In other words, the display disappears and the cursor or blinking goes to the left edge of the display (in the first line if 2 lines are displayed). It also sets I/D to 1 (increment mode) in entry mode. S of entry mode does not change.

Return Home

Return home sets DDRAM address 0 into the address counter, and returns the display to its original status if it was shifted. The DDRAM contents do not change.

The cursor or blinking go to the left edge of the display (in the first line if 2 lines are displayed).

Entry Mode Set

I/D: Increments (I/D = 1) or decrements (I/D = 0) the DDRAM address by 1 when a character code is written into or read from DDRAM.

The cursor or blinking moves to the right when incremented by 1 and to the left when decremented by 1. The same applies to writing and reading of CGRAM.

S: Shifts the entire display either to the right (I/D = 0) or to the left (I/D = 1) when S is 1. The display does not shift if S is 0.

If S is 1, it will seem as if the cursor does not move but the display does. The display does not shift when reading from DDRAM. Also, writing into or reading out from CGRAM does not shift the display.

Display On/Off Control

D: The display is on when D is 1 and off when D is 0. When off, the display data remains in DDRAM, but can be displayed instantly by setting D to 1.

C: The cursor is displayed when C is 1 and not displayed when C is 0. Even if the cursor disappears, the function of I/D or other specifications will not change during display data write. The cursor is displayed using 5 dots in the 8th line for 5 × 8 dot character font selection and in the 11th line for the 5 × 10 dot character font selection (Figure 13).

B: The character indicated by the cursor blinks when B is 1 (Figure 13). The blinking is displayed as switching between all blank dots and displayed characters at a speed of 409.6-ms intervals when f_{cp} or f_{OSC} is 250 kHz. The cursor and blinking can be set to display simultaneously. (The blinking frequency changes according to f_{OSC} or the reciprocal of f_{cp}. For example, when f_{cp} is 270 kHz, $409.6 \times 250/270 = 379.2$ ms.)

Cursor or Display Shift

Cursor or display shift shifts the cursor position or display to the right or left without writing or reading display data (Table 7). This function is used to correct or search the display. In a 2-line display, the cursor moves to the second line when it passes the 40th digit of the first line. Note that the first and second line displays will shift at the same time.

When the displayed data is shifted repeatedly each line moves only horizontally. The second line display does not shift into the first line position.

The address counter (AC) contents will not change if the only action performed is a display shift.

Function Set

DL: Sets the interface data length. Data is sent or received in 8-bit lengths (DB7 to DB0) when DL is 1, and in 4-bit lengths (DB7 to DB4) when DL is 0.When 4-bit length is selected, data must be sent or received twice.

N: Sets the number of display lines.

F: Sets the character font.

Note: Perform the function at the head of the program before executing any instructions (except for the read busy flag and address instruction). From this point, the function set instruction cannot be executed unless the interface data length is changed.

Set CGRAM Address

Set CGRAM address sets the CGRAM address binary AAAAAA into the address counter.

Data is then written to or read from the MPU for CGRAM.

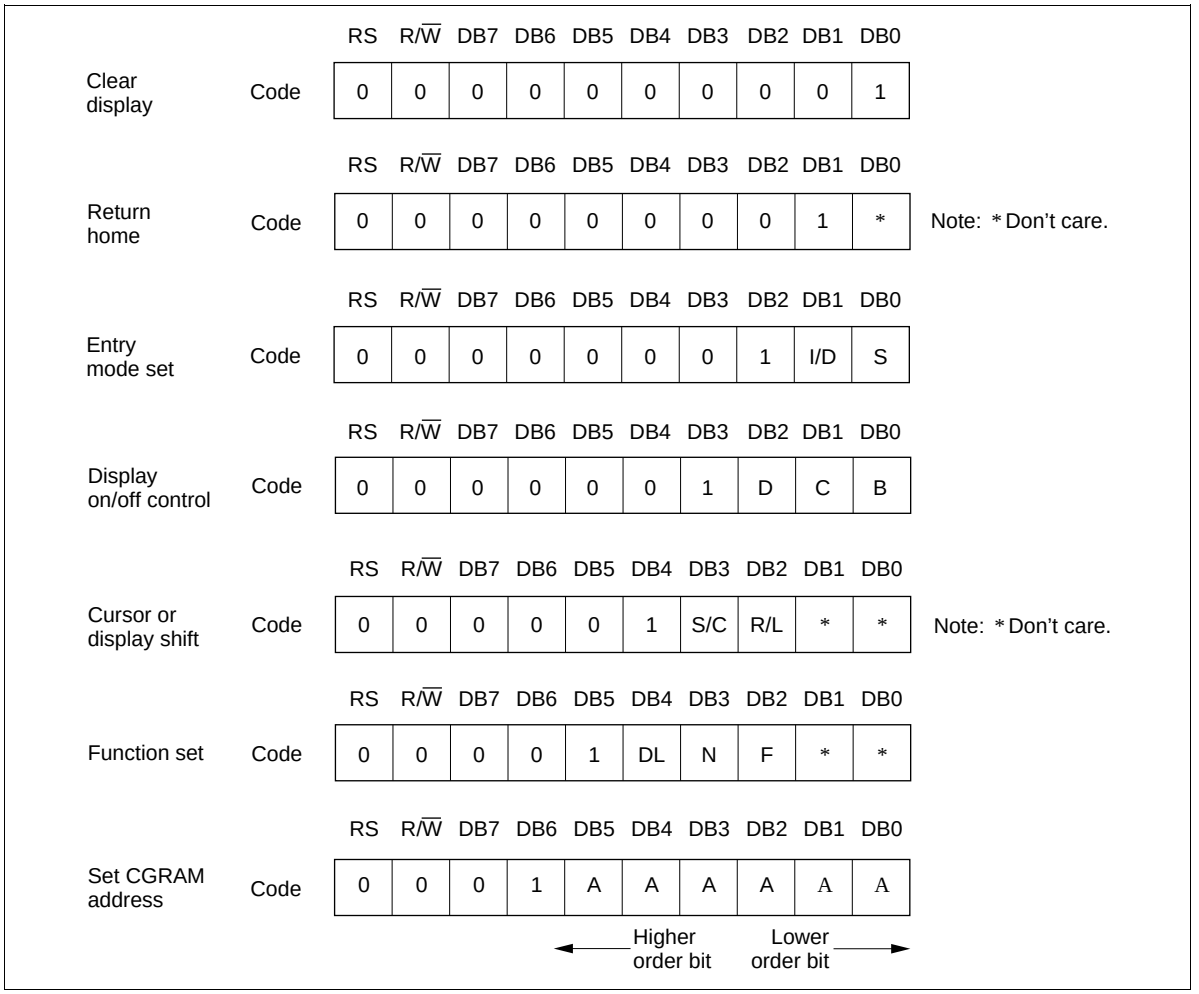


Figure 11 Instruction (1)

Set DDRAM Address

Set DDRAM address sets the DDRAM address binary AAAAAAA into the address counter.

Data is then written to or read from the MPU for DDRAM.

However, when N is 0 (1-line display), AAAAAAA can be 00H to 4FH. When N is 1 (2-line display), AAAAAAA can be 00H to 27H for the first line, and 40H to 67H for the second line.

Read Busy Flag and Address

Read busy flag and address reads the busy flag (BF) indicating that the system is now internally operating on a previously received instruction. If BF is 1, the internal operation is in progress. The next instruction will not be accepted until BF is reset to 0. Check the BF status before the next write operation. At the same time, the value of the address counter in binary AAAAAAA is read out. This address counter is used by both CG and DDRAM addresses, and its value is determined by the previous instruction. The address contents are the same as for instructions set CGRAM address and set DDRAM address.

Table 7 Shift Function

S/C	R/L	
0	0	Shifts the cursor position to the left. (AC is decremented by one.)
0	1	Shifts the cursor position to the right. (AC is incremented by one.)
1	0	Shifts the entire display to the left. The cursor follows the display shift.
1	1	Shifts the entire display to the right. The cursor follows the display shift.

Table 8 Function Set

N	F	No. of Display Lines	Character Font	Duty Factor	Remarks
0	0	1	5 × 8 dots	1/8	
0	1	1	5 × 10 dots	1/11	
1	*	2	5 × 8 dots	1/16	Cannot display two lines for 5 × 10 dot character font

Note: * Indicates don't care.

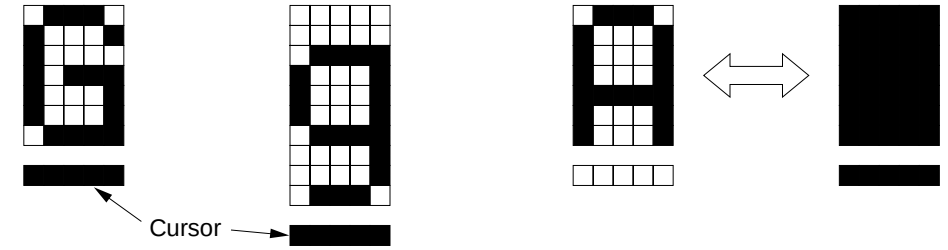


Figure 12 Cursor and Blinking

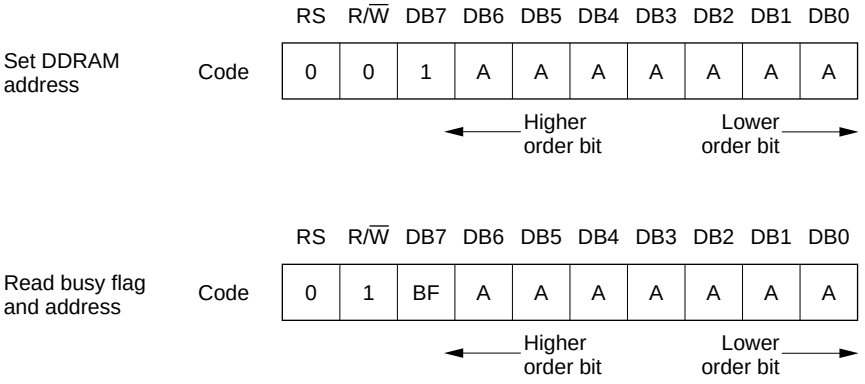


Figure 13 Instruction (2)

Write Data to CG or DDRAM

Write data to CG or DDRAM writes 8-bit binary data DDDDDDDD to CG or DDRAM.

To write into CG or DDRAM is determined by the previous specification of the CGRAM or DDRAM address setting. After a write, the address is automatically incremented or decremented by 1 according to the entry mode. The entry mode also determines the display shift.

Read Data from CG or DDRAM

Read data from CG or DDRAM reads 8-bit binary data DDDDDDDD from CG or DDRAM.

The previous designation determines whether CG or DDRAM is to be read. Before entering this read instruction, either CGRAM or DDRAM address set instruction must be executed. If not executed, the first read data will be invalid. When serially executing read instructions, the next address data is normally read from the second read. The address set instructions need not be executed just before this read instruction when shifting the cursor by the cursor shift instruction (when reading out DDRAM). The operation of the cursor shift instruction is the same as the set DDRAM address instruction.

After a read, the entry mode automatically increases or decreases the address by 1. However, display shift is not executed regardless of the entry mode.

Note: The address counter (AC) is automatically incremented or decremented by 1 after the write instructions to CGRAM or DDRAM are executed. The RAM data selected by the AC cannot be read out at this time even if read instructions are executed. Therefore, to correctly read data, execute either the address set instruction or cursor shift instruction (only with DDRAM), then just before reading the desired data, execute the read instruction from the second time the read instruction is sent.

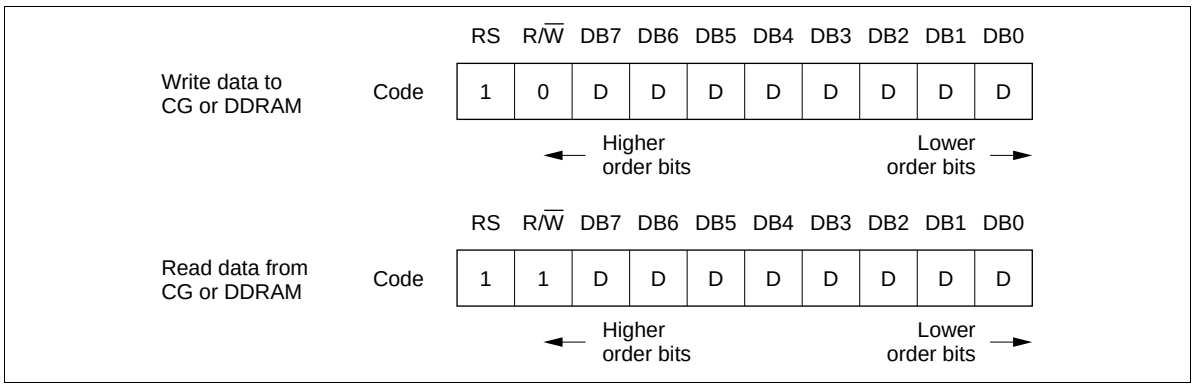


Figure 14 Instruction (3)

Interfacing the HD44780U

Interface to MPUs

- Interfacing to an 8-bit MPU

  See Figure 16 for an example of using a I/O port (for a single-chip microcomputer) as an interface device.

  In this example, P30 to P37 are connected to the data bus DB0 to DB7, and P75 to P77 are connected to E, R/$\overline{W}$, and RS, respectively.

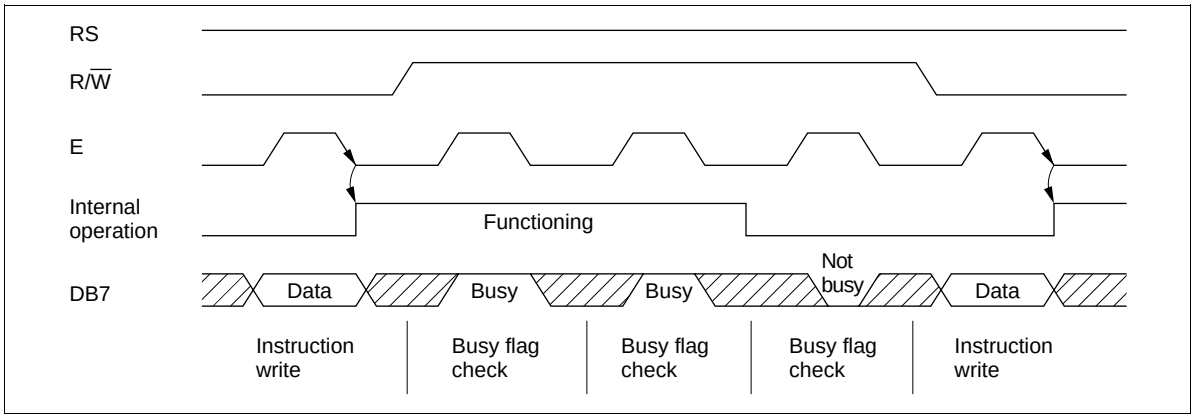


Figure 15 Example of Busy Flag Check Timing Sequence

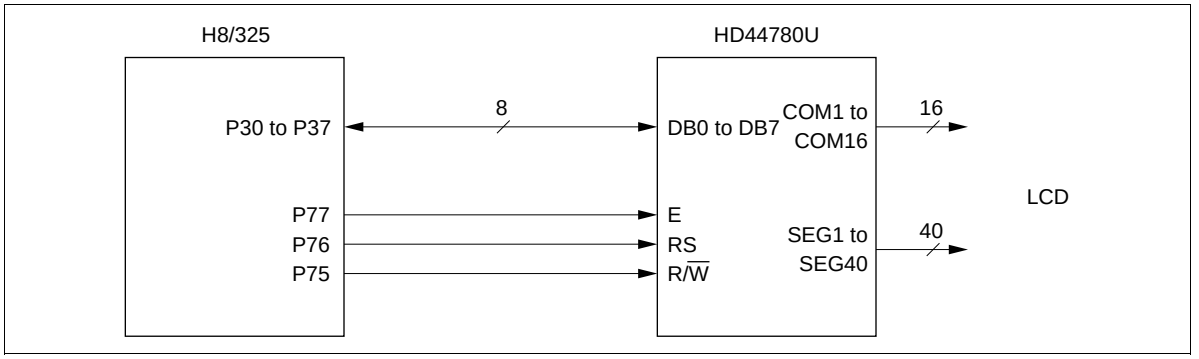


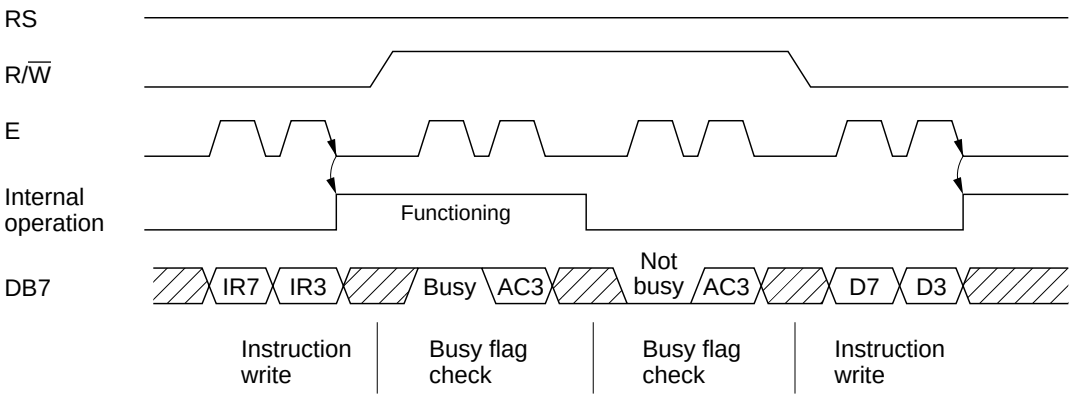
Figure 16 H8/325 Interface (Single-Chip Mode)

- Interfacing to a 4-bit MPU

  The HD44780U can be connected to the I/O port of a 4-bit MPU. If the I/O port has enough bits, 8-bit data can be transferred. Otherwise, one data transfer must be made in two operations for 4-bit data. In this case, the timing sequence becomes somewhat complex. (See Figure 17.)

  See Figure 18 for an interface example to the HMCS4019R.

  Note that two cycles are needed for the busy flag check as well as for the data transfer. The 4-bit operation is selected by the program.



Note: IR7 , IR3 are the 7th and 3rd bits of the instruction.
AC3 is the 3rd bit of the address counter.

Figure 17 Example of 4-Bit Data Transfer Timing Sequence

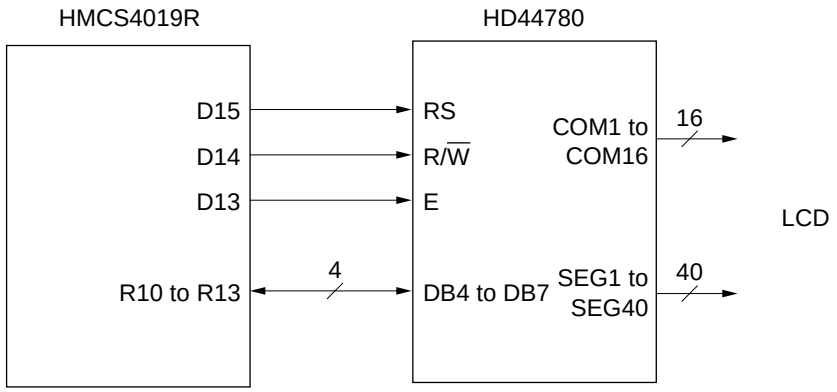


Figure 18 Example of Interface to HMCS4019R

Interface to Liquid Crystal Display

Character Font and Number of Lines: The HD44780U can perform two types of displays, 5 × 8 dot and 5 × 10 dot character fonts, each with a cursor.

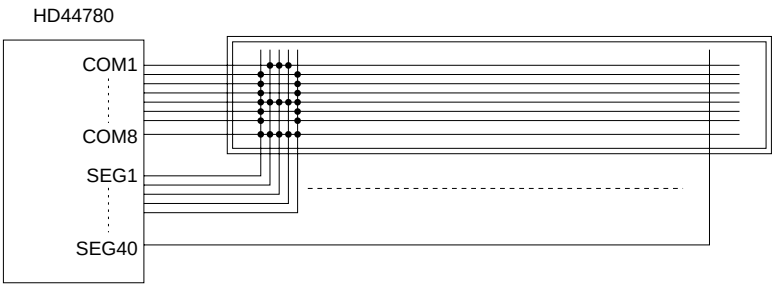
Up to two lines are displayed for 5 × 8 dots and one line for 5 × 10 dots. Therefore, a total of three types of common signals are available (Table 9).

The number of lines and font types can be selected by the program. (See Table 6, Instructions.)

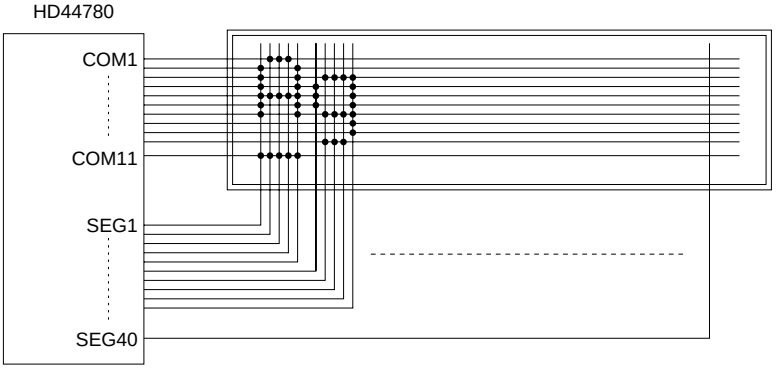
Connection to HD44780 and Liquid Crystal Display: See Figure 19 for the connection examples.

Table 9 Common Signals

Number of Lines	Character Font	Number of Common Signals	Duty Factor
1	5 × 8 dots + cursor	8	1/8
1	5 × 10 dots + cursor	11	1/11
2	5 × 8 dots + cursor	16	1/16



Example of a 5 × 8 dot, 8-character × 1-line display (1/4 bias, 1/8 duty cycle)

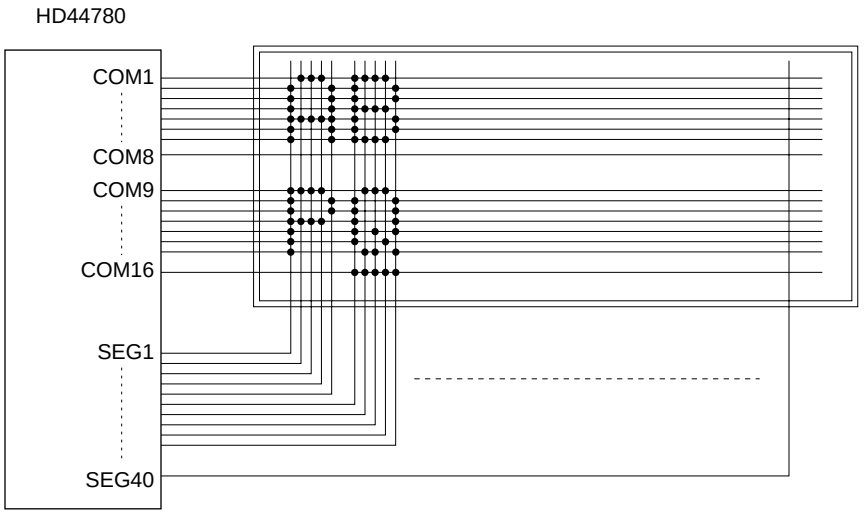


Example of a 5 × 10 dot, 8-character × 1-line display (1/4 bias, 1/11 duty cycle)

Figure 19 Liquid Crystal Display and HD44780 Connections

Since five segment signal lines can display one digit, one HD44780U can display up to 8 digits for a 1-line display and 16 digits for a 2-line display.

The examples in Figure 19 have unused common signal pins, which always output non-selection waveforms. When the liquid crystal display panel has unused extra scanning lines, connect the extra scanning lines to these common signal pins to avoid any undesirable effects due to crosstalk during the floating state.



Example of a 5 × 8 dot, 8-character × 2-line display (1/5 bias, 1/16 duty cycle)

Figure 19 Liquid Crystal Display and HD44780 Connections (cont)

Connection of Changed Matrix Layout: In the preceding examples, the number of lines correspond to the scanning lines. However, the following display examples (Figure 20) are made possible by altering the matrix layout of the liquid crystal display panel. In either case, the only change is the layout. The display characteristics and the number of liquid crystal display characters depend on the number of common signals or on duty factor. Note that the display data RAM (DDRAM) addresses for 4 characters × 2 lines and for 16 characters × 1 line are the same as in Figure 19.

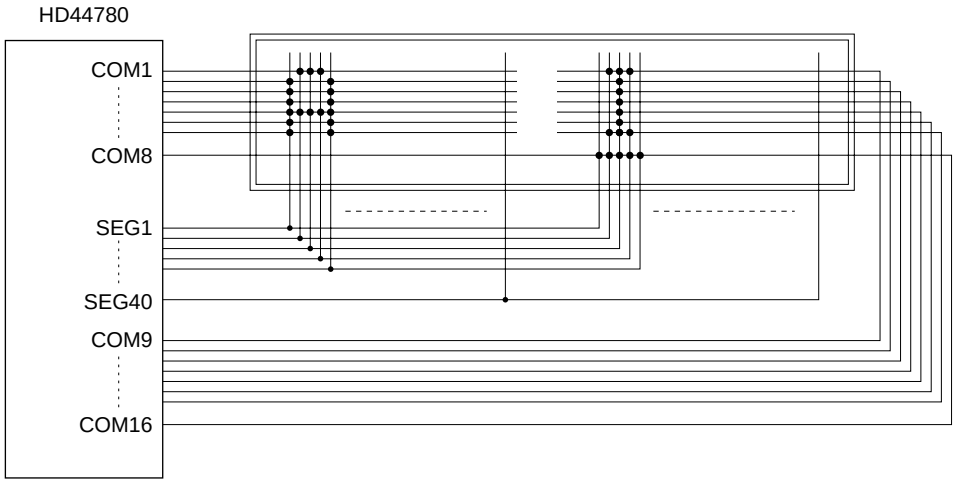


Figure 20 Changed Matrix Layout Displays

Power Supply for Liquid Crystal Display Drive

Various voltage levels must be applied to pins V1 to V5 of the HD44780U to obtain the liquid crystal display drive waveforms. The voltages must be changed according to the duty factor (Table 10).

VLCD is the peak value for the liquid crystal display drive waveforms, and resistance dividing provides voltages V1 to V5 (Figure 21).

Table 10 Duty Factor and Power Supply for Liquid Crystal Display Drive

	Duty Factor	
	1/8, 1/11	1/16
		Bias
Power Supply	1/4	1/5
V1	V_{cc}–1/4 VLCD	V_{cc}–1/5 VLCD
V2	V_{cc}–1/2 VLCD	V_{cc}–2/5 VLCD
V3	V_{cc}–1/2 VLCD	V_{cc}–3/5 VLCD
V4	V_{cc}–3/4 VLCD	V_{cc}–4/5 VLCD
V5	V_{cc}–VLCD	V_{cc}–VLCD

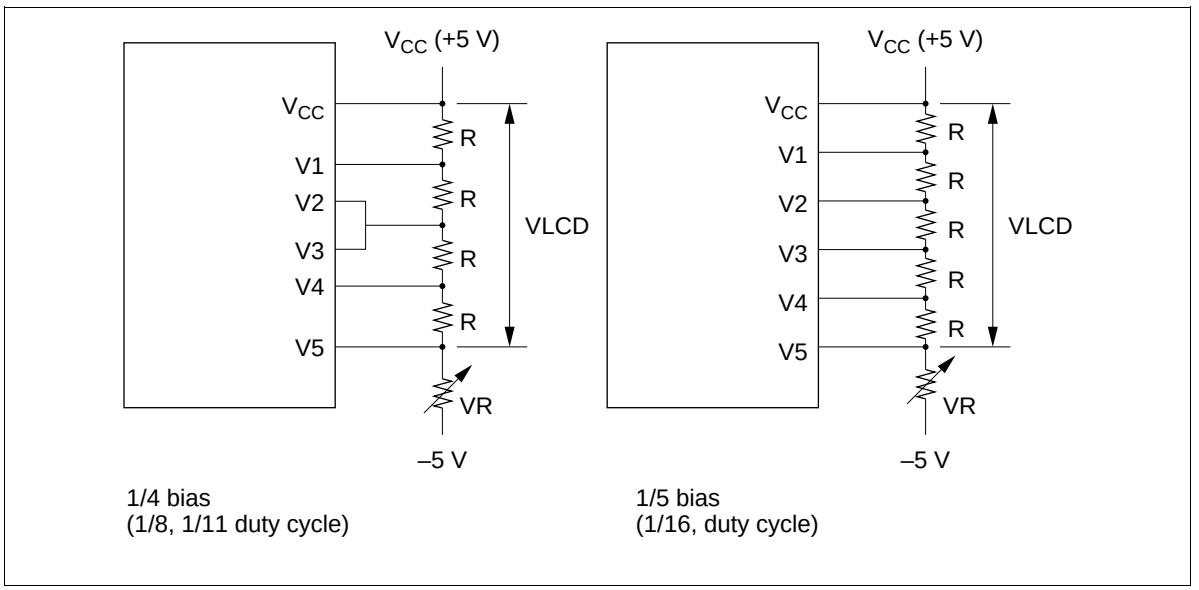


Figure 21 Drive Voltage Supply Example

Relationship between Oscillation Frequency and Liquid Crystal Display Frame Frequency

The liquid crystal display frame frequencies of Figure 22 apply only when the oscillation frequency is 270 kHz (one clock pulse of 3.7 μs).

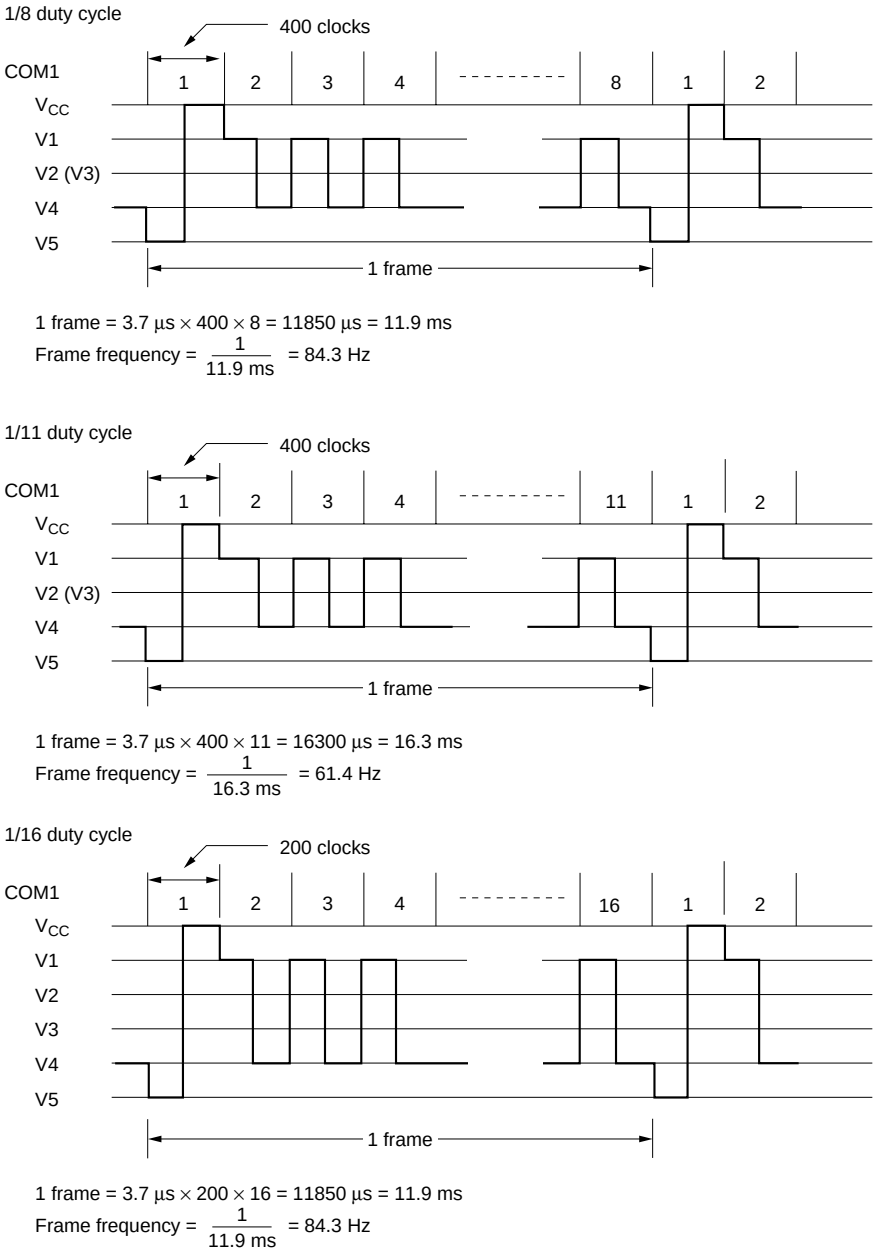


Figure 22 Frame Frequency

Instruction and Display Correspondence

- 8-bit operation, 8-digit × 1-line display with internal reset

  Refer to Table 11 for an example of an 8-digit × 1-line display in 8-bit operation. The HD44780U functions must be set by the function set instruction prior to the display. Since the display data RAM can store data for 80 characters, as explained before, the RAM can be used for displays such as for advertising when combined with the display shift operation.

  Since the display shift operation changes only the display position with DDRAM contents unchanged, the first display data entered into DDRAM can be output when the return home operation is performed.

- 4-bit operation, 8-digit × 1-line display with internal reset

  The program must set all functions prior to the 4-bit operation (Table 12). When the power is turned on, 8-bit operation is automatically selected and the first write is performed as an 8-bit operation. Since DB0 to DB3 are not connected, a rewrite is then required. However, since one operation is completed in two accesses for 4-bit operation, a rewrite is needed to set the functions (see Table 12). Thus, DB4 to DB7 of the function set instruction is written twice.

- 8-bit operation, 8-digit × 2-line display

  For a 2-line display, the cursor automatically moves from the first to the second line after the 40th digit of the first line has been written. Thus, if there are only 8 characters in the first line, the DDRAM address must be again set after the 8th character is completed. (See Table 13.) Note that the display shift operation is performed for the first and second lines. In the example of Table 13, the display shift is performed when the cursor is on the second line. However, if the shift operation is performed when the cursor is on the first line, both the first and second lines move together. If the shift is repeated, the display of the second line will not move to the first line. The same display will only shift within its own line for the number of times the shift is repeated.

Note: When using the internal reset, the electrical characteristics in the Power Supply Conditions Using Internal Reset Circuit table must be satisfied. If not, the HD44780U must be initialized by instructions. See the section, Initializing by Instruction.

Table 11 8-Bit Operation, 8-Digit × 1-Line Display Example with Internal Reset

Step No.	Instruction RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Display	Operation
1	Power supply on (the HD44780U is initialized by the internal reset circuit)											Initialized. No display.
2	Function set											Sets to 8-bit operation and selects 1-line display and 5 × 8 dot character font. (Number of display lines and character fonts cannot be changed after step #2.)
	0	0	0	0	1	1	0	0	*	*		
3	Display on/off control											Turns on display and cursor. Entire display is in space mode because of initialization.
	0	0	0	0	0	0	1	1	1	0	_	
4	Entry mode set											Sets mode to increment the address by one and to shift the cursor to the right at the time of write to the DD/CGRAM. Display is not shifted.
	0	0	0	0	0	0	0	1	1	0	_	
5	Write data to CGRAM/DDRAM											Writes H. DDRAM has already been selected by initialization when the power was turned on. The cursor is incremented by one and shifted to the right.
	1	0	0	1	0	0	1	0	0	0	H_	
6	Write data to CGRAM/DDRAM											Writes I.
	1	0	0	1	0	0	1	0	0	1	HI_	
7	.				.						.	
	.				.						.	
	.				.						.	
	.				.						.	
	.				.						.	
8	Write data to CGRAM/DDRAM											Writes I.
	1	0	0	1	0	0	1	0	0	1	HITACHI_	
9	Entry mode set											Sets mode to shift display at the time of write.
	0	0	0	0	0	0	0	1	1	1	HITACHI_	
10	Write data to CGRAM/DDRAM											Writes a space.
	1	0	0	0	1	0	0	0	0	0	ITACHI _	

Table 11 8-Bit Operation, 8-Digit × 1-Line Display Example with Internal Reset (cont)

Step	Instruction											
No.	RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Display	Operation
11	Write data to CGRAM/DDRAM										TACHI M_	Writes M.
	1	0	0	1	0	0	1	1	0	1		
12						.					.	
						.					.	
						.					.	
						.					.	
						.					.	
13	Write data to CGRAM/DDRAM										MICROKO_	Writes O.
	1	0	0	1	0	0	1	1	1	1		
14	Cursor or display shift										MICROKO	Shifts only the cursor position to the left.
	0	0	0	0	0	1	0	0	*	*		
15	Cursor or display shift										MICROKO	Shifts only the cursor position to the left.
	0	0	0	0	0	1	0	0	*	*		
16	Write data to CGRAM/DDRAM										ICROCO	Writes C over K. The display moves to the left.
	1	0	0	1	0	0	0	0	1	1		
17	Cursor or display shift										MICROCO	Shifts the display and cursor position to the right.
	0	0	0	0	0	1	1	1	*	*		
18	Cursor or display shift										MICROCO_	Shifts the display and cursor position to the right.
	0	0	0	0	0	1	0	1	*	*		
19	Write data to CGRAM/DDRAM										ICROCOM_	Writes M.
	1	0	0	1	0	0	1	1	0	1		
20						.					.	
						.					.	
						.					.	
						.					.	
						.					.	
21	Return home										HITACHI	Returns both display and cursor to the original position (address 0).
	0	0	0	0	0	0	0	0	1	0		

Table 12 4-Bit Operation, 8-Digit × 1-Line Display Example with Internal Reset

Step No.	Instruction RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	Display	Operation
1	Power supply on (the HD44780U is initialized by the internal reset circuit)							Initialized. No display.
2	Function set							Sets to 4-bit operation. In this case, operation is handled as 8 bits by initialization, and only this instruction completes with one write.
	0	0	0	0	1	0		
3	Function set							Sets 4-bit operation and selects 1-line display and 5 × 8 dot character font. 4-bit operation starts from this step and resetting is necessary. (Number of display lines and character fonts cannot be changed after step #3.)
	0	0	0	0	1	0		
	0	0	0	0	*	*		
4	Display on/off control							Turns on display and cursor. Entire display is in space mode because of initialization.
	0	0	0	0	0	0		
	0	0	1	1	1	0		
5	Entry mode set							Sets mode to increment the address by one and to shift the cursor to the right at the time of write to the DD/CGRAM. Display is not shifted.
	0	0	0	0	0	0		
	0	0	0	1	1	0		
6	Write data to CGRAM/DDRAM						H_	Writes H. The cursor is incremented by one and shifts to the right.
	1	0	0	1	0	0		
	1	0	1	0	0	0		

Note: The control is the same as for 8-bit operation beyond step #6.

Table 13 8-Bit Operation, 8-Digit × 2-Line Display Example with Internal Reset

Step No.	Instruction RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Display	Operation
1	Power supply on (the HD44780U is initialized by the internal reset circuit)											Initialized. No display.
2	Function set											Sets to 8-bit operation and selects 2-line display and 5 × 8 dot character font.
	0	0	0	0	1	1	1	0	*	*		
3	Display on/off control											Turns on display and cursor. All display is in space mode because of initialization.
	0	0	0	0	0	0	1	1	1	0	_	
4	Entry mode set											Sets mode to increment the address by one and to shift the cursor to the right at the time of write to the DD/CGRAM. Display is not shifted.
	0	0	0	0	0	0	0	1	1	0	_	
5	Write data to CGRAM/DDRAM											Writes H. DDRAM has already been selected by initialization when the power was turned on. The cursor is incremented by one and shifted to the right.
	1	0	0	1	0	0	1	0	0	0	H_	
6	. . . . .					. . . . .					. . . . .	
7	Write data to CGRAM/DDRAM											Writes I.
	1	0	0	1	0	0	1	0	0	1	HITACHI_	
8	Set DDRAM address											Sets DDRAM address so that the cursor is positioned at the head of the second line.
	0	0	1	1	0	0	0	0	0	0	HITACHI _	

Table 13 8-Bit Operation, 8-Digit × 2-Line Display Example with Internal Reset (cont)

Step No.	Instruction RS	R/$\overline{W}$	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	Display	Operation
9	Write data to CGRAM/DDRAM 1	0	0	1	0	0	1	1	0	1	HITACHI M_	Writes M.
10	.	.	.	.	.	.	.	.	.	.	.	
11	Write data to CGRAM/DDRAM 1	0	0	1	0	0	1	1	1	1	HITACHI MICROCO_	Writes O.
12	Entry mode set 0	0	0	0	0	0	0	1	1	1	HITACHI MICROCO_	Sets mode to shift display at the time of write.
13	Write data to CGRAM/DDRAM 1	0	0	1	0	0	1	1	0	1	ITACHI ICROCOM_	Writes M. Display is shifted to the left. The first and second lines both shift at the same time.
14	.	.	.	.	.	.	.	.	.	.	.	
15	Return home 0	0	0	0	0	0	0	0	1	0	HITACHI MICROCOM	Returns both display and cursor to the original position (address 0).

Initializing by Instruction

If the power supply conditions for correctly operating the internal reset circuit are not met, initialization by instructions becomes necessary.

Refer to Figures 23 and 24 for the procedures on 8-bit and 4-bit initializations, respectively.

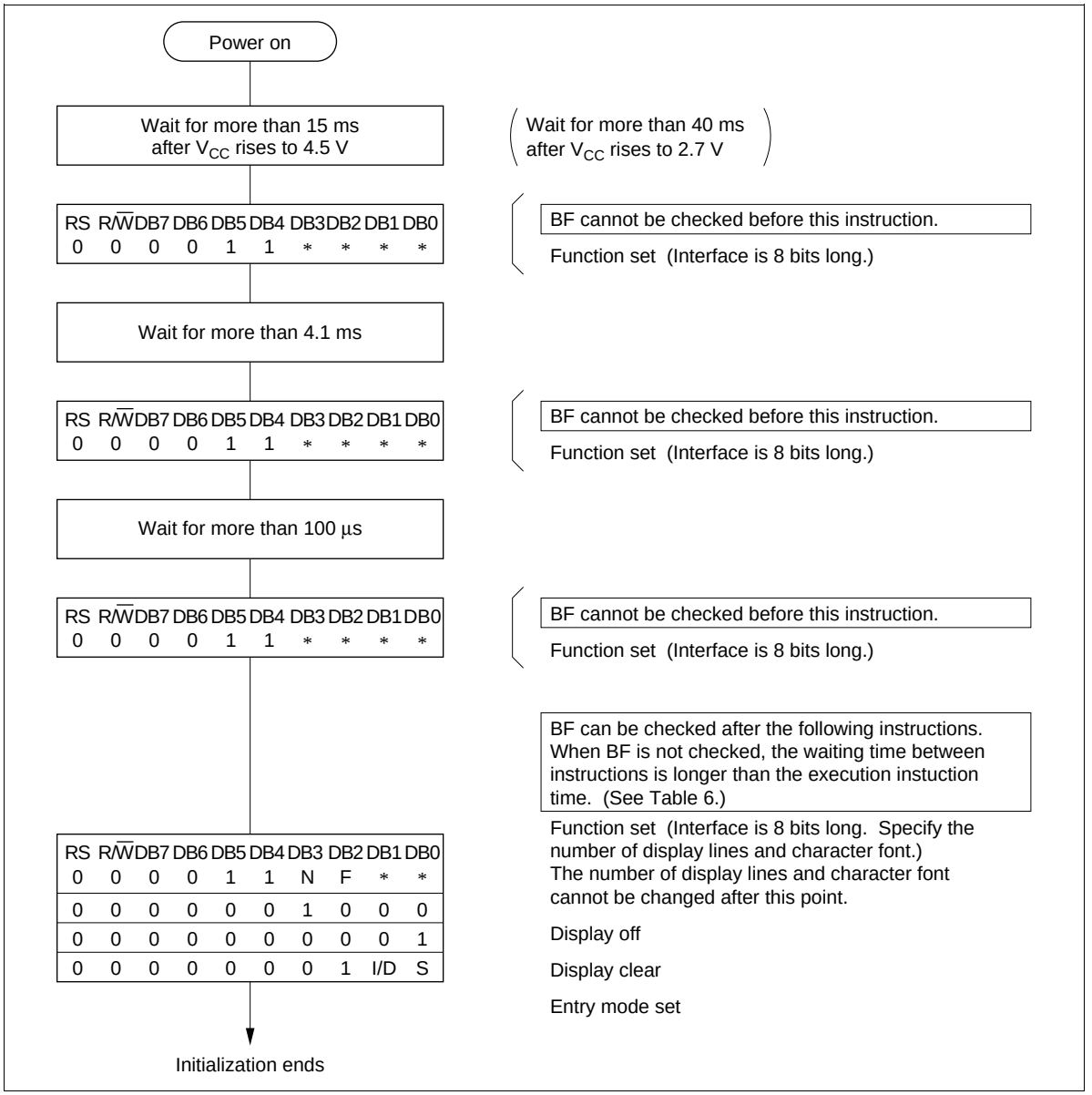


Figure 23 8-Bit Interface

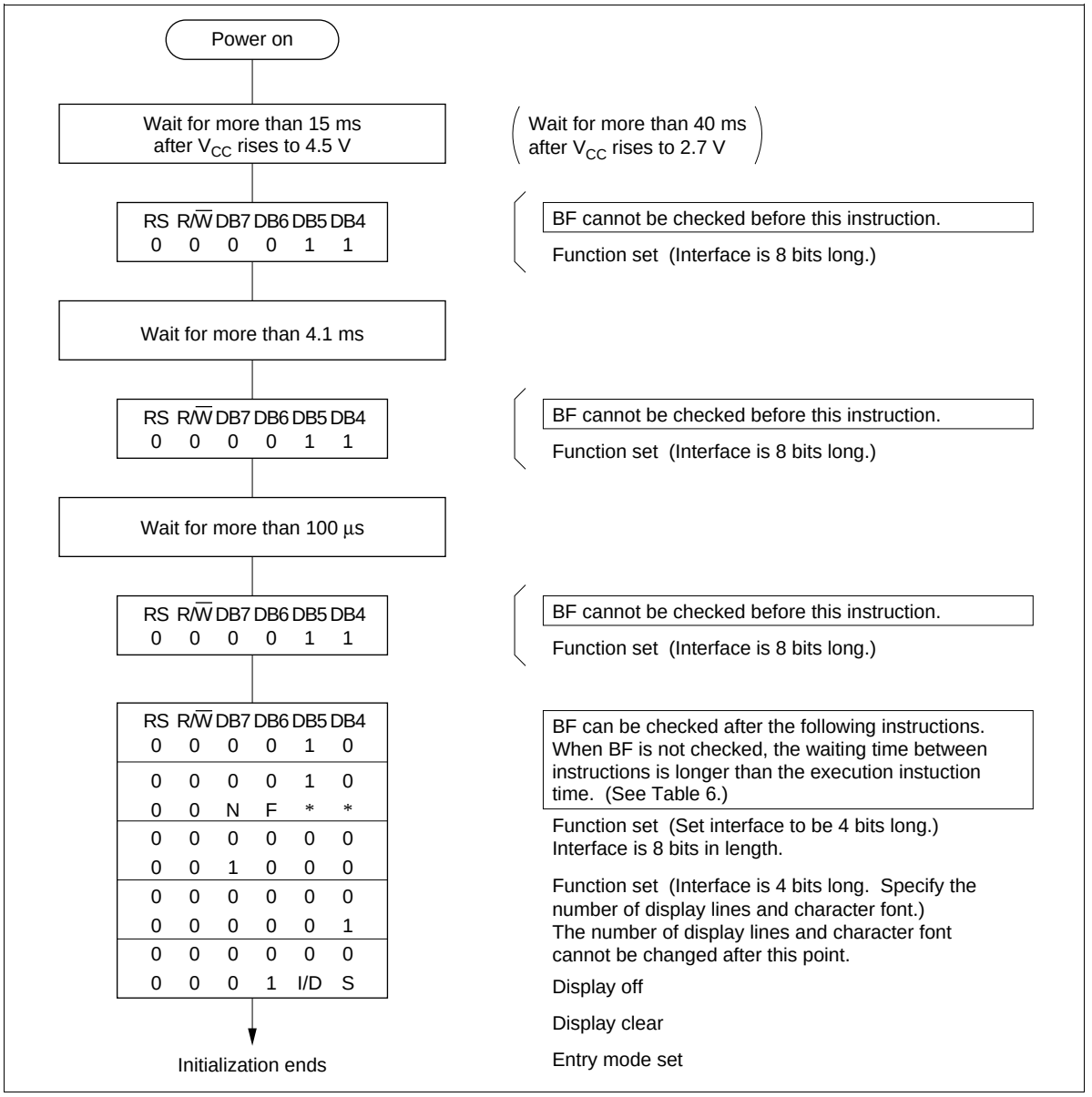


Figure 24 4-Bit Interface

Absolute Maximum Ratings*

Item	Symbol	Value	Unit	Notes
Power supply voltage (1)	V_{CC}–GND	–0.3 to +7.0	V	1
Power supply voltage (2)	V_{CC}–V5	–0.3 to +13.0	V	1, 2
Input voltage	Vt	–0.3 to V_{CC} +0.3	V	1
Operating temperature	T_{opr}	–30 to +75	°C	
Storage temperature	T_{stg}	–55 to +125	°C	4

Note: * If the LSI is used above these absolute maximum ratings, it may become permanently damaged. Using the LSI within the following electrical characteristic limits is strongly recommended for normal operation. If these electrical characteristic conditions are also exceeded, the LSI will malfunction and cause poor reliability.

DC Characteristics (V_{CC} = 2.7 to 4.5 V, T_a = –30 to +75°C*[3])

Item	Symbol	Min	Typ	Max	Unit	Test Condition	Notes*
Input high voltage (1) (except OSC1)	VIH1	$0.7V_{CC}$	—	V_{CC}	V		6
Input low voltage (1) (except OSC1)	VIL1	–0.3	—	0.55	V		6
Input high voltage (2) (OSC1)	VIH2	$0.7V_{CC}$	—	V_{CC}	V		15
Input low voltage (2) (OSC1)	VIL2	—	—	$0.2V_{CC}$	V		15
Output high voltage (1) (DB0–DB7)	VOH1	$0.75V_{CC}$	—	—	V	$–I_{OH}$ = 0.1 mA	7
Output low voltage (1) (DB0–DB7)	VOL1	—	—	$0.2V_{CC}$	V	I_{OL} = 0.1 mA	7
Output high voltage (2) (except DB0–DB7)	VOH2	$0.8V_{CC}$	—	—	V	$–I_{OH}$ = 0.04 mA	8
Output low voltage (2) (except DB0–DB7)	VOL2	—	—	$0.2V_{CC}$	V	I_{OL} = 0.04 mA	8
Driver on resistance (COM)	R_{COM}	—	2	20	kΩ	±Id = 0.05 mA, VLCD = 4 V	13
Driver on resistance (SEG)	R_{SEG}	—	2	30	kΩ	±Id = 0.05 mA, VLCD = 4 V	13
Input leakage current	I_{LI}	–1	—	1	µA	VIN = 0 to V_{CC}	9
Pull-up MOS current (DB0–DB7, RS, R/$\overline{W}$)	$–I_p$	10	50	120	µA	V_{CC} = 3 V	
Power supply current	I_{CC}	—	150	300	µA	R_f oscillation, external clock V_{CC} = 3 V, f_{OSC} = 270 kHz	10, 14
LCD voltage	VLCD1	3.0	—	11.0	V	V_{CC}–V5, 1/5 bias	16
	VLCD2	3.0	—	11.0	V	V_{CC}–V5, 1/4 bias	16

Note: * Refer to the Electrical Characteristics Notes section following these tables.

AC Characteristics (V_{CC} = 2.7 to 4.5 V, T_a = –30 to +75°C*[3])

Clock Characteristics

Item		Symbol	Min	Typ	Max	Unit	Test Condition	Note*
External clock operation	External clock frequency	f_{cp}	125	250	350	kHz		11
	External clock duty	Duty	45	50	55	%		
	External clock rise time	t_{rcp}	—	—	0.2	µs		
	External clock fall time	t_{fcp}	—	—	0.2	µs		
R_f oscillation	Clock oscillation frequency	f_{OSC}	190	270	350	kHz	R_f = 75 kΩ, V_{CC} = 3 V	12

Note: * Refer to the Electrical Characteristics Notes section following these tables.

Bus Timing Characteristics

Write Operation

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Enable cycle time	t_{cycE}	1000	—	—	ns	Figure 25
Enable pulse width (high level)	PW_{EH}	450	—	—		
Enable rise/fall time	t_{Er}, t_{Ef}	—	—	25		
Address set-up time (RS, R/$\overline{W}$ to E)	t_{AS}	60	—	—		
Address hold time	t_{AH}	20	—	—		
Data set-up time	t_{DSW}	195	—	—		
Data hold time	t_H	10	—	—		

Read Operation

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Enable cycle time	t_{cycE}	1000	—	—	ns	Figure 26
Enable pulse width (high level)	PW_{EH}	450	—	—		
Enable rise/fall time	t_{Er}, t_{Ef}	—	—	25		
Address set-up time (RS, R/$\overline{W}$ to E)	t_{AS}	60	—	—		
Address hold time	t_{AH}	20	—	—		
Data delay time	t_{DDR}	—	—	360		
Data hold time	t_{DHR}	5	—	—		

Interface Timing Characteristics with External Driver

Item		Symbol	Min	Typ	Max	Unit	Test Condition
Clock pulse width	High level	t_{CWH}	800	—	—	ns	Figure 27
	Low level	t_{CWL}	800	—	—		
Clock set-up time		t_{CSU}	500	—	—		
Data set-up time		t_{SU}	300	—	—		
Data hold time		t_{DH}	300	—	—		
M delay time		t_{DM}	–1000	—	1000		
Clock rise/fall time		t_{ct}	—	—	200		

Power Supply Conditions Using Internal Reset Circuit

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Power supply rise time	t_{rCC}	0.1	—	10	ms	Figure 28
Power supply off time	t_{OFF}	1	—	—		

DC Characteristics (V_{CC} = 4.5 to 5.5 V, T_a = –30 to +75°C*[3])

Item	Symbol	Min	Typ	Max	Unit	Test Condition	Notes*
Input high voltage (1) (except OSC1)	VIH1	2.2	—	V_{CC}	V		6
Input low voltage (1) (except OSC1)	VIL1	–0.3	—	0.6	V		6
Input high voltage (2) (OSC1)	VIH2	V_{CC}–1.0	—	V_{CC}	V		15
Input low voltage (2) (OSC1)	VIL2	—	—	1.0	V		15
Output high voltage (1) (DB0–DB7)	VOH1	2.4	—	—	V	$–I_{OH}$ = 0.205 mA	7
Output low voltage (1) (DB0–DB7)	VOL1	—	—	0.4	V	I_{OL} = 1.2 mA	7
Output high voltage (2) (except DB0–DB7)	VOH2	0.9 V_{CC}	—	—	V	$–I_{OH}$ = 0.04 mA	8
Output low voltage (2) (except DB0–DB7)	VOL2	—	—	0.1 V_{CC}	V	I_{OL} = 0.04 mA	8
Driver on resistance (COM)	RCOM	—	2	20	kΩ	±Id = 0.05 mA, VLCD = 4 V	13
Driver on resistance (SEG)	RSEG	—	2	30	kΩ	±Id = 0.05 mA, VLCD = 4 V	13
Input leakage current	I_{LI}	–1	—	1	µA	VIN = 0 to V_{CC}	9
Pull-up MOS current (DB0–DB7, RS, R/$\overline{W}$)	$–I_p$	50	125	250	µA	V_{CC} = 5 V	
Power supply current	I_{CC}	—	350	600	µA	R_f oscillation, external clock V_{CC} = 5 V, f_{OSC} = 270 kHz	10, 14
LCD voltage	VLCD1	3.0	—	11.0	V	V_{CC}–V5, 1/5 bias	16
	VLCD2	3.0	—	11.0	V	V_{CC}–V5, 1/4 bias	16

Note: * Refer to the Electrical Characteristics Notes section following these tables.

AC Characteristics (V_{CC} = 4.5 to 5.5 V, T_a = –30 to +75°C*[3])

Clock Characteristics

Item		Symbol	Min	Typ	Max	Unit	Test Condition	Notes*
External clock operation	External clock frequency	f_{cp}	125	250	350	kHz		11
	External clock duty	Duty	45	50	55	%		11
	External clock rise time	t_{rcp}	—	—	0.2	µs		11
	External clock fall time	t_{fcp}	—	—	0.2	µs		11
R_f oscillation	Clock oscillation frequency	f_{OSC}	190	270	350	kHz	R_f = 91 kΩ V_{CC} = 5.0 V	12

Note: * Refer to the Electrical Characteristics Notes section following these tables.

Bus Timing Characteristics

Write Operation

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Enable cycle time	t_{cycE}	500	—	—	ns	Figure 25
Enable pulse width (high level)	PW_{EH}	230	—	—		
Enable rise/fall time	t_{Er}, t_{Ef}	—	—	20		
Address set-up time (RS, R/$\overline{W}$ to E)	t_{AS}	40	—	—		
Address hold time	t_{AH}	10	—	—		
Data set-up time	t_{DSW}	80	—	—		
Data hold time	t_H	10	—	—		

Read Operation

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Enable cycle time	t_{cycE}	500	—	—	ns	Figure 26
Enable pulse width (high level)	PW_{EH}	230	—	—		
Enable rise/fall time	t_{Er}, t_{Ef}	—	—	20		
Address set-up time (RS, R/$\overline{W}$ to E)	t_{AS}	40	—	—		
Address hold time	t_{AH}	10	—	—		
Data delay time	t_{DDR}	—	—	160		
Data hold time	t_{DHR}	5	—	—		

Interface Timing Characteristics with External Driver

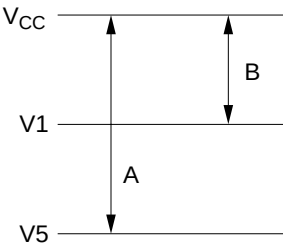
Item		Symbol	Min	Typ	Max	Unit	Test Condition
Clock pulse width	High level	t_{CWH}	800	—	—	ns	Figure 27
	Low level	t_{CWL}	800	—	—		
Clock set-up time		t_{CSU}	500	—	—		
Data set-up time		t_{SU}	300	—	—		
Data hold time		t_{DH}	300	—	—		
M delay time		t_{DM}	–1000	—	1000		
Clock rise/fall time		t_{ct}	—	—	100		

Power Supply Conditions Using Internal Reset Circuit

Item	Symbol	Min	Typ	Max	Unit	Test Condition
Power supply rise time	t_{rCC}	0.1	—	10	ms	Figure 28
Power supply off time	t_{OFF}	1	—	—		

Electrical Characteristics Notes

1. All voltage values are referred to GND = 0 V.

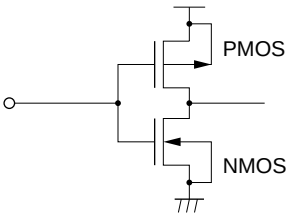


$A = V_{CC} - V5$
$B = V_{CC} - V1$
$A \geq 1.5$ V
$B \leq 0.25 \times A$

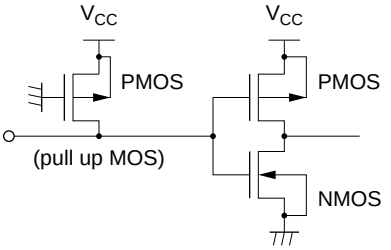
The conditions of V1 and V5 voltages are for proper operation of the LSI and not for the LCD output level. The LCD drive voltage condition for the LCD output level is specified as LCD voltage VLCD.

2. $V_{CC} \geq V1 \geq V2 \geq V3 \geq V4 \geq V5$ must be maintained.
3. For die products, specified at 75°C.
4. For die products, specified by the die shipment specification.
5. The following four circuits are I/O pin configurations except for liquid crystal display output.

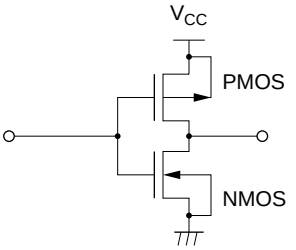
Input pin
Pin: E (MOS without pull-up)



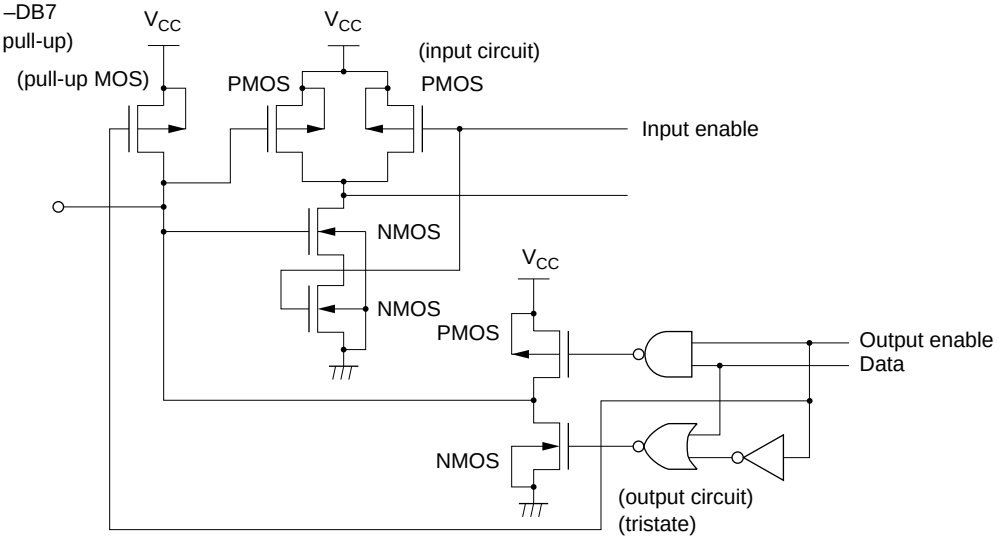
Pins: RS, R/$\overline{W}$ (MOS with pull-up)



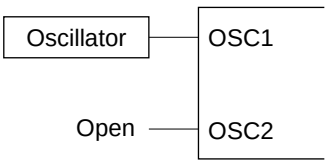
Output pin
Pins: CL1, CL2, M, D

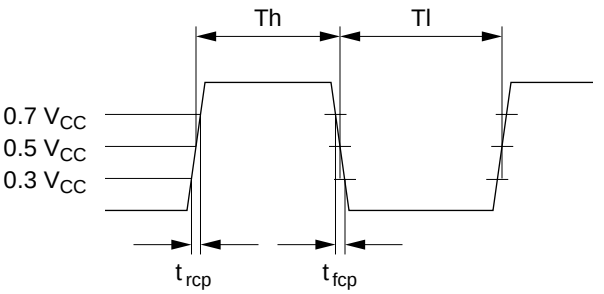


I/O Pin
Pins: DB0 –DB7
(MOS with pull-up)



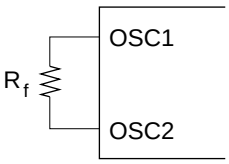
6. Applies to input pins and I/O pins, excluding the OSC1 pin.
7. Applies to I/O pins.
8. Applies to output pins.
9. Current flowing through pull–up MOSs, excluding output drive MOSs.
10. Input/output current is excluded. When input is at an intermediate level with CMOS, the excessive current flows through the input circuit to the power supply. To avoid this from happening, the input level must be fixed high or low.
11. Applies only to external clock operation.



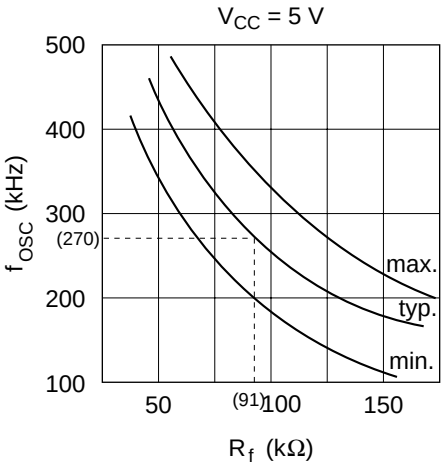


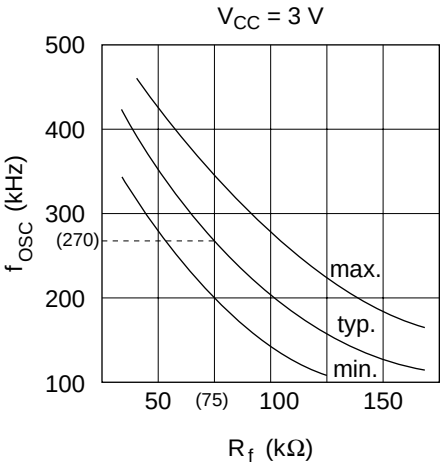
$$\text{Duty} = \frac{Th}{Th + Tl} \times 100\%$$

12. Applies only to the internal oscillator operation using oscillation resistor R_f.



R_f: 75 kΩ ± 2% (when V_{CC} = 3 V)
R_f: 91 kΩ ± 2% (when V_{CC} = 5 V)
Since the oscillation frequency varies depending on the OSC1 and OSC2 pin capacitance, the wiring length to these pins should be minimized.

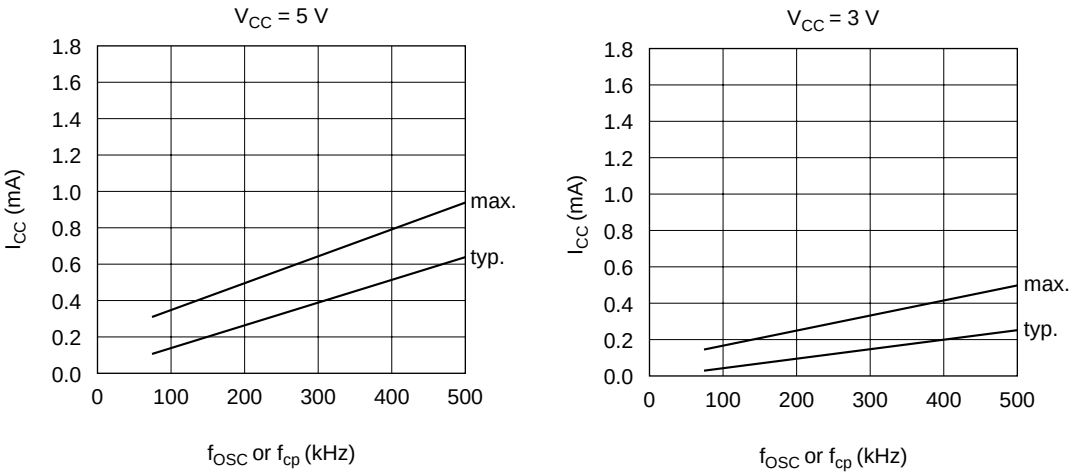




13. RCOM is the resistance between the power supply pins (V_{CC}, V1, V4, V5) and each common signal pin (COM1 to COM16).

    RSEG is the resistance between the power supply pins (V_{CC}, V2, V3, V5) and each segment signal pin (SEG1 to SEG40).

14. The following graphs show the relationship between operation frequency and current consumption.

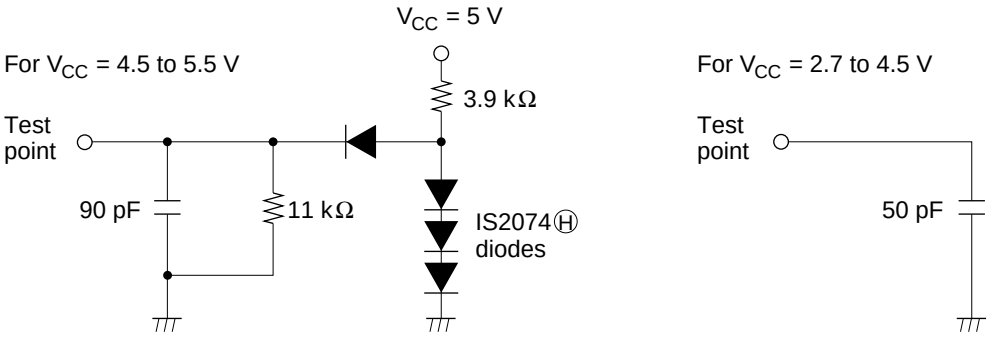


15. Applies to the OSC1 pin.

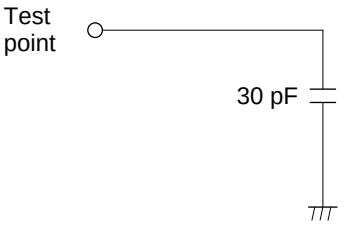
16. Each COM and SEG output voltage is within ±0.15 V of the LCD voltage (V_{CC}, V1, V2, V3, V4, V5) when there is no load.

Load Circuits

Data Bus DB0 to DB7



External Driver Control Signals: CL1, CL2, D, M



Timing Characteristics

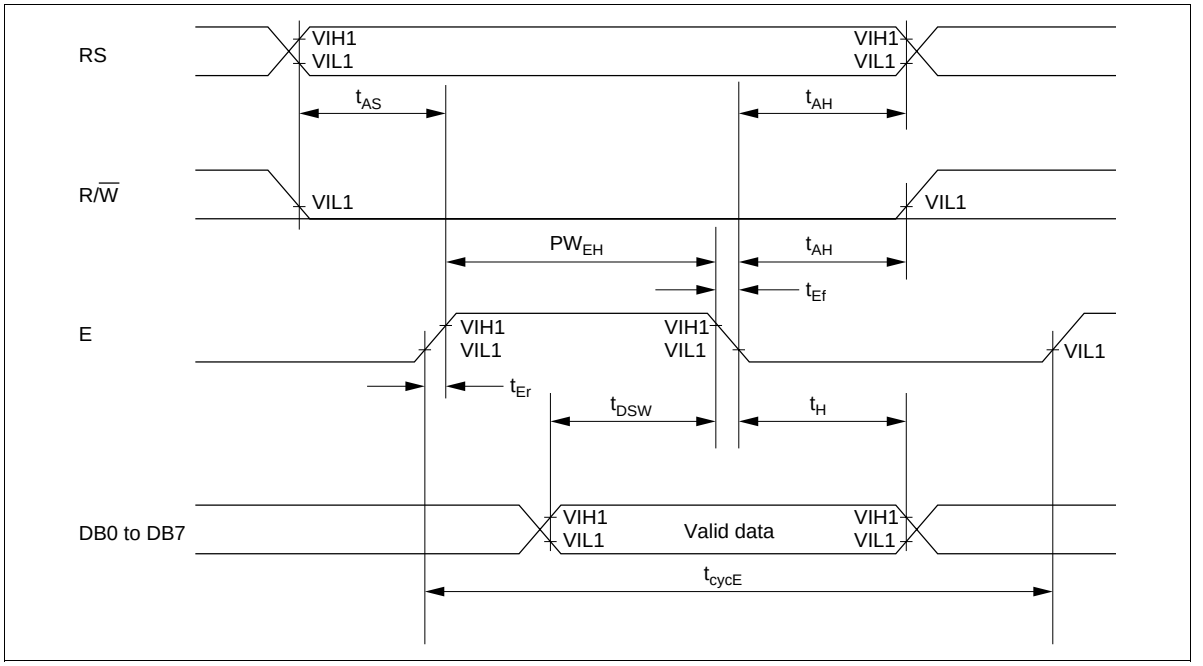
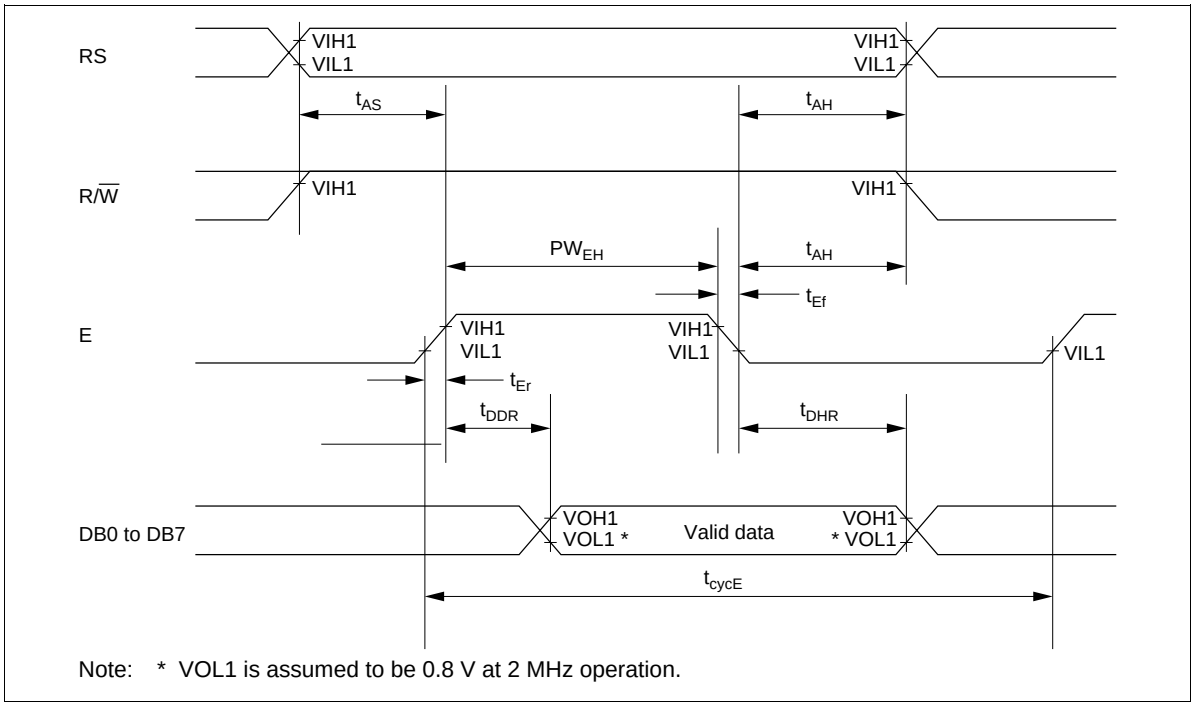


Figure 25 Write Operation



Note: * VOL1 is assumed to be 0.8 V at 2 MHz operation.

Figure 26 Read Operation

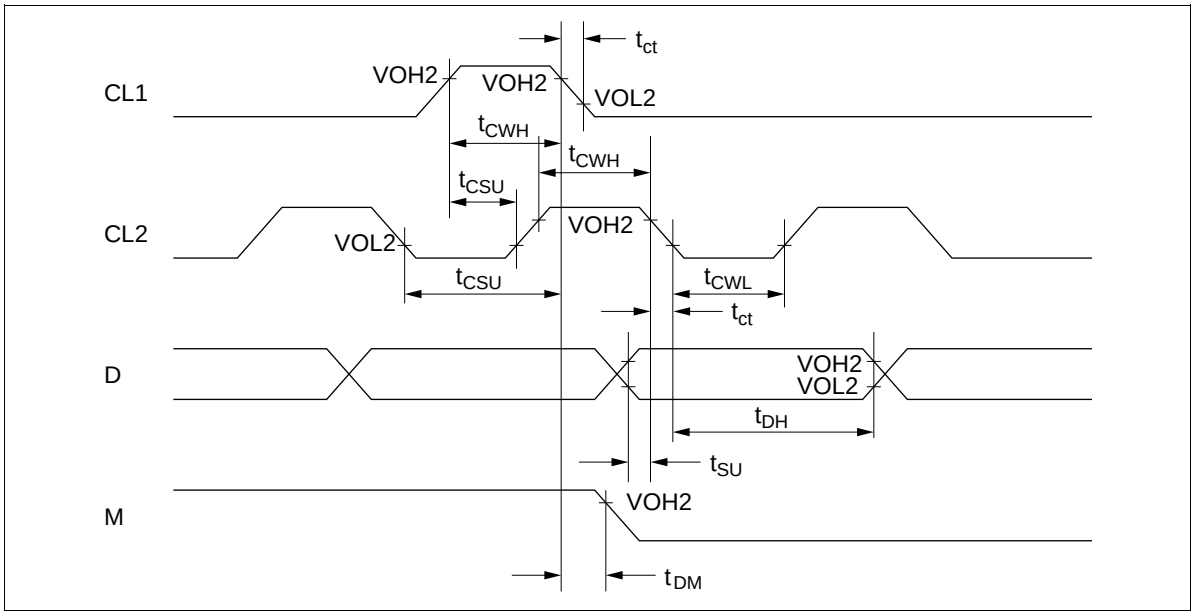
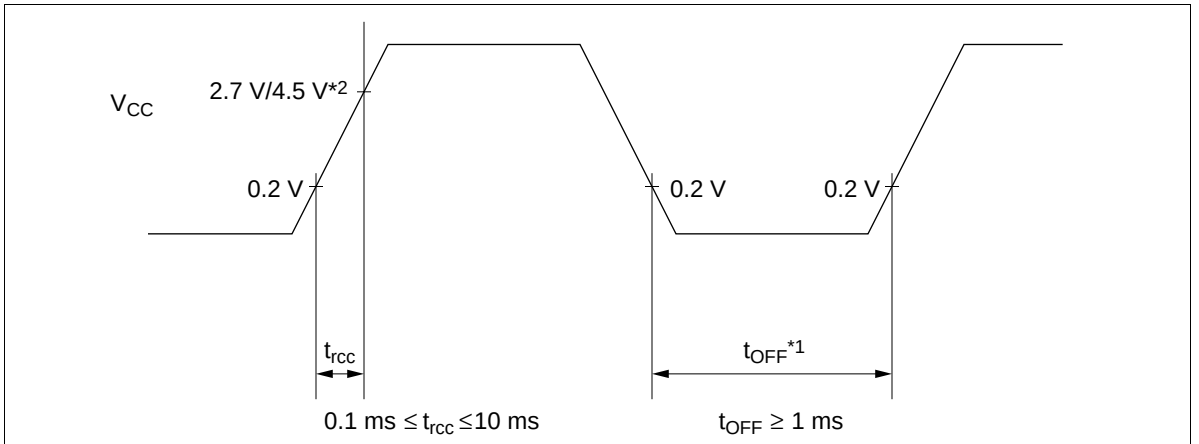


Figure 27 Interface Timing with External Driver



Notes: 1. t_{OFF} compensates for the power oscillation period caused by momentary power supply oscillations.
2. Specified at 4.5 V for 5-V operation, and at 2.7 V for 3-V operation.
3. For if 4.5 V is not reached during 5-V operation, the internal reset circuit will not operate normally.
In this case, the LSI must be initialized by software. (Refer to the Initializing by Instruction section.)

Figure 28 Internal Power Supply Reset

Cautions

1. Hitachi neither warrants nor grants licenses of any rights of Hitachi's or any third party's patent, copyright, trademark, or other intellectual property rights for information contained in this document. Hitachi bears no responsibility for problems that may arise with third party's rights, including intellectual property rights, in connection with use of the information contained in this document.
2. Products and product specifications may be subject to change without notice. Confirm that you have received the latest product standards or specifications before final design, purchase or use.
3. Hitachi makes every attempt to ensure that its products are of high quality and reliability. However, contact Hitachi's sales office before using the product in an application that demands especially high quality and reliability or where its failure or malfunction may directly threaten human life or cause risk of bodily injury, such as aerospace, aeronautics, nuclear power, combustion control, transportation, traffic, safety equipment or medical equipment for life support.
4. Design your application so that the product is used within the ranges guaranteed by Hitachi particularly for maximum rating, operating supply voltage range, heat radiation characteristics, installation conditions and other characteristics. Hitachi bears no responsibility for failure or damage when used beyond the guaranteed ranges. Even within the guaranteed ranges, consider normally foreseeable failure rates or failure modes in semiconductor devices and employ systemic measures such as fail-safes, so that the equipment incorporating Hitachi product does not cause bodily injury, fire or other consequential damage due to operation of the Hitachi product.
5. This product is not designed to be radiation resistant.
6. No one is permitted to reproduce or duplicate, in any form, the whole or part of this document without written approval from Hitachi.
7. Contact Hitachi's sales office for any questions regarding this document or Hitachi semiconductor products.

HITACHI

Hitachi, Ltd.
Semiconductor & Integrated Circuits.
Nippon Bldg., 2-6-2, Ohte-machi, Chiyoda-ku, Tokyo 100-0004, Japan
Tel: Tokyo (03) 3270-2111 Fax: (03) 3270-5109

URL
NorthAmerica : http:semiconductor.hitachi.com/
Europe : http://www.hitachi-eu.com/hel/ecg
Asia (Singapore) : http://www.has.hitachi.com.sg/grp3/sicd/index.htm
Asia (Taiwan) : http://www.hitachi.com.tw/E/Product/SICD_Frame.htm
Asia (HongKong) : http://www.hitachi.com.hk/eng/bo/grp3/index.htm
Japan : http://www.hitachi.co.jp/Sicd/indx.htm

For further information write to:

Hitachi Semiconductor (America) Inc.
179 East Tasman Drive,
San Jose,CA 95134
Tel: <1> (408) 433-1990
Fax: <1>(408) 433-0223

Hitachi Europe GmbH
Electronic components Group
Dornacher Straße 3
D-85622 Feldkirchen, Munich
Germany
Tel: <49> (89) 9 9180-0
Fax: <49> (89) 9 29 30 00

Hitachi Europe Ltd.
Electronic Components Group.
Whitebrook Park
Lower Cookham Road
Maidenhead
Berkshire SL6 8YA, United Kingdom
Tel: <44> (1628) 585000
Fax: <44> (1628) 778322

Hitachi Asia Pte. Ltd.
16 Collyer Quay #20-00
Hitachi Tower
Singapore 049318
Tel: 535-2100
Fax: 535-1533

Hitachi Asia Ltd.
Taipei Branch Office
3F, Hung Kuo Building. No.167,
Tun-Hwa North Road, Taipei (105)
Tel: <886> (2) 2718-3666
Fax: <886> (2) 2718-8180

Hitachi Asia (Hong Kong) Ltd.
Group III (Electronic Components)
7/F., North Tower, World Finance Centre,
Harbour City, Canton Road, Tsim Sha Tsui,
Kowloon, Hong Kong
Tel: <852> (2) 735 9218
Fax: <852> (2) 730 0281
Telex: 40815 HITEC HX

Copyright © Hitachi, Ltd., 1998. All rights reserved. Printed in Japan.

BDA 34873-Rev-RZ_Layout 1 21.03.03 09:14 Seite 1

VIVANCO

EDP-No.: 34873



Universal TV- DVB Controller UR Z2

		Seite/ Page
GB	Operating instructions	2 – 3
D	Bedienungsanleitung	4 – 5
F	Mode d'emploi	6 – 7
E	Manual de Instrucciones	8 – 9
NL	Gebruiksaanwijzing	10 – 11
PL	Instrukcja obsługi	12 – 13
S	Bruksanvisning	14 – 15
RUS	Руководство по эксплуатации	16 – 17

GB

Universal TV- DVB Controller UR Z2



Before you start using the device, please first read through the safety information and this operating instruction very carefully. Only then you can use all the functions safely and reliably. Please maintain the operating instruction well and pass on to possible other owners.

Intended use
Universal remotes replace original remote controls etc. of consumer electronics devices. The device is meant for sole use in private households and may not be used for commercial purposes. Any other use is not deemed as intended use and may lead to material damages or even to personal injuries.
The manufacturer does not assume any liability for damages that have occurred due to unintended use.



Battery/rechargeable battery safety
Please read these instructions very carefully. Disregarding these instructions can cause damages/injuries and possibly even an explosion of batteries/rechargeable batteries! Do not short-circuit the contacts, for example by common storage with metallic objects. Do not expose batteries/rechargeable batteries to long periods of direct sunlight or heat. Do not throw batteries/rechargeable batteries into the fire. Do not open batteries/rechargeable batteries, they do not contain any parts that need to be maintained. Only replace batteries/rechargeable batteries with the same type or with an equivalent recommended by the manufacturer. Accumulators only: For charging accumulators, do only use the charging functions integrated in the device or any other suitable chargers. Never charge batteries.

Inserting the batteries
Open the battery flap on the back of the remote control, press the tongue lightly downwards, then insert 2 alkaline AA batteries. Check that the polarity markings on the batteries and in the battery compartment are correctly lined up (+=+), then close the flap carefully. – Always use undamaged batteries.

Adapting to appliances
In order to control e.g. your TV or DVB you must adapt or set the remote control to it. There are two ways of doing so: direct control code entry or code search.

Direct control code entry using the enclosed list
Example: setting to control a Panasonic television set:
1. The attached list gives, for example, the Panasonic Code 0158. Please make sure you have selected the correct list for television sets (TV).
2. To prepare code entry, press the SET button, keep it pressed and then press the required appliance button, e.g. TV, additionally until the lamp (LED) lights constantly.
3. Then press the number buttons (e.g. 0158) in sequence.
4. Once the numbers have been entered the lamp goes out and the remote control is ready to control your appliance.
5. Finally, test the control code setting by trying to control (in this example) your television set. If it does not work at all or not correctly try a different code from the list or the code search (see below). You can terminate the mode by pressing any device button.

Code search
If the brand of your appliance is not in the list or none of the codes given fits, we recommend a code search.

Manual code search
1. Switch on the appliance to be controlled. Program is running.
2. To prepare code search, press the SET button, keep it pressed and then press the required appliance button, e.g. TV, additionally until the lamp (LED) lights constantly.
3. Then point the remote control at the appliance to be controlled.
4. Briefly press the SET button once. The lamp (LED) blinks.
5. Press the ON/OFF ⏻ (appliance on/off) button repeatedly for testing the next code, approx. every 1,5 seconds, until the appliance, e.g. your TV reacts by power off. - If your appliance has switched itself off you must switch it back on again, e.g. on the appliance or with the original remote control, to continue searching. - When your appliancereacts you can also try out other buttons without interrupting the search. If some buttons do not work properly, simply continue searching with ON/OFF.
6. When your appliance reacts correctly, press the appliance button, e.g. TV, and end the search. The lamp (LED) goes out. - The UR Z2 continues with the search at the point where you interrupted it. The code last entered remains stored. The search stops after approx. 30 seconds of inactivity.

Automatic code search
Your remote control finds the correct settings almost automatically in seconds:
1. Switch on the appliance to be controlled. A channel number, for example, appears in the display of your DVB.
2. To start the code search, press the required appliance button, e.g. DVB briefly.
3. Then press SET for about 5 seconds until the lamp (LED) starts to blink.
4. Then point the remote control at the appliance to be controlled while the remote control emits a different ON/OFF signal every second.
5. When your appliance reacts, press SET within 1-2 seconds, to end the search. – If you ended the search too late, continue to search using the Manual Code Search (see above).
6. Finally, test the control code setting by trying to control, for example, your DVB appliance. If it does not work correctly, please try the code search again. The code search then begins with the next code in the sequence. - Remember to switch your appliance, e.g. DVB, back on again if you wish to continue searching.

Code identification
You can read off the 4-digit codes you have set at any time:
1. Press the appliance button of the appliance which code you want to read off, e.g. TV.
2. Press the SET button and 1,2,3,4 regardless of the part of the code you want to read off. For example, if you press SET+1 after TV and the code 0158 (in this example) has been set, the lamp does not blink after SET+1 is released to show that the thousands 25figure is 0. All you have to do is count. After SET+2 is released the lamp blinks once to show that the hundred figure is 1.The ten figure is indicated after SET+3 is released, e.g. it blinks 5 times for the 5 in 158. The one figure is indicated after SET+4 is released, e.g. it blinks 8 times for the 8 in 0158. - *Please open the battery compartment and write down the found codes, for later use, onto the sticker placed at the innerside of the compartment door.*

Operation
You operate your appliances basically in exactly the same way as if you were using the original remote controls, except that you must first press an appliance button (TV, DVB, etc.) to control the required appliance. As the symbols on the UR Z2 could differ from those on the original remote controls you may have to try out all the buttons in order to find the functions.

Punch through
If certain control codes e.g. for controlling DVB, don´t contain volume and mute key functions, then the controller controls TV volume and mute automatically.

Specifications
Batteries: 2x AA batteries, NiMH, rechargeable can also be used
Range: max. 7 m
Not for 400 kHz equipment

Hereby Vivanco GmbH declares that UR Z2 is in conformity with the fundamental requirements and other relevant regulations of the directive 2004/108/EU.

Disposal of the device
This device may not be disposed along with household waste. Please hand it over to a collection point for electrical devices of your area or your district. Thus it will be guaranteed that the old devices are recycled properly and economically and potential environmental and health hazards are avoided.

Pb/Cd/Hg

Recycle Batteries
Rechargeable and non-rechargeable batteries shall not be disposed with household waste. You are legally obligated to return used batteries, so that a professional, environmentally safe disposition is assured. Please take them to a municipal collection point or retailer. Please only hand in batteries and accumulators that are completely discharged.

The packing should be supplied to re-use. Please do not throw it into household waste and handle it in conformity with local disposal rules.

21
PAP

Universal TV- DVB Controller UR Z2



Bevor Sie Ihr Gerät benutzen, lesen Sie bitte zuerst die Sicherheitshinweise und diese Gebrauchsanleitung aufmerksam durch. Nur so können Sie alle Funktionen sicher und zuverlässig nutzen.Bewahren Sie die Gebrauchsanleitung gut auf und geben Sie sie an mögliche Nachbesitzer weiter.

Bestimmungsgemäßer Gebrauch
Universalfernbedienungen ersetzen Originalfernbedienungen u.a. von Unterhaltungselektronikgeräten. Das Gerät ist ausschließlich zum Einsatz im privaten Bereich bestimmt und darf nicht für gewerbliche Zwecke verwendet werden. Jede andere Verwendung gilt als nicht bestimmungsgemäß und kann zu Sachschaden oder sogar zu Personenschäden führen. Der Hersteller übernimmt keine Haftung für Schäden, die durch nicht bestimmungsgemäßen Gebrauch entstanden sind.



Batterie-/Akkusicherheit
Bitte diese Hinweise sehr sorgfältig beachten. Die Missachtung dieser Hinweise kann zu Beschädigungen/Verletzungen und unter Umständen gar zur Explosion von Akkus/ Batterien führen! Schließen Sie die Kontakte nicht, z.B. durch gemeinsame Lagerung mit metallischen Gegenständen, kurz. Setzen Sie Akkus/Batterien nie längerer, direkter Sonneneinstrahlung oder Hitze aus. Werfen Sie Akkus/Batterien nicht ins Feuer. Öffnen Sie Akkus/Batterien niemals, es sind keine zu wartenden Teile enthalten. Bitte Akkus/Batterien nur gegen gleiche Typen oder vom Hersteller empfohlene gleichwertige Typen austauschen. Nur Akkus: Benutzen Sie zur Ladung von Akkus nur die im Gerät eingebaute Ladefunktion oder geeignete Ladegeräte. Batterien bitte niemals aufladen.

Batterien einlegen
Bitte öffnen Sie die Batterieklappe an der Geräterückseite, Lasche leicht nach unten drücken, und legen Sie 2 Alkali-Mignonbatterien ein. Achten Sie auf die angegebenen Polaritäten auf Batterien und im Batteriefach (+=+), und schließen Sie das Fach wieder vorsichtig. – Bitte achten Sie darauf, immer einwandfreie Batterien zu verwenden.

Anpassung an ein Gerät, z.B. einen Fernseher oder eine DVB Set Top Box (oder Satreceiver/ Kabetuner) oder ein anderes Gerät.
Um Ihr Geräte steuern zu können, müssen Sie die Fernbedienung für Ihre Geräte einstellen. Dazu gibt es zwei Möglichkeiten: die direkte Steuerkodeeingabe und die Kodesuche.

Die direkte Steuerkodeeingabe mithilfe der beigefügten Liste
Beispiel: Einstellung der Fernbedienung für ein Panasonic-Fernsehgerät:
1. Aus der beigefügten Liste entnehmen Sie z.B. den Panasonic-Code 0158. Bitte vergewissern Sie sich, dass Sie die korrekte Liste für Fernseher (TV) gewählt haben.
2. Zur Vorbereitung der Kodeeingabe drücken Sie kurz die Tasten SET (Einstellung) und dann dazu die gewünschte Gerätetaste, z.B. TV, bis das Lämpchen (LED) dauernd leuchtet.
3. Dann drücken Sie nacheinander die Zifferntasten (z.B. 0158).
4. Nach der Zifferneingabe erlischt die Lampe und die Fernbedienung ist bereit zum Steuern Ihres Gerätes.
5. Zum Abschluss testen Sie den eingestellten Steuerkode, indem Sie versuchen in diesem Fall Ihr TV-Gerät zu steuern. Funktioniert es nicht oder nicht richtig, dann versuchen Sie bitte einen weiteren Kode aus der Liste oder die Kodesuche, s.u.. Die Einstellung der Fernbedienung zur Steuerung weiterer Geräte nehmen Sie auf gleiche Weise vor. Sie beenden die Kodeingabe durch Drücken irgeneiner Gerätetaste.

Steuerkodesuche
Wenn Sie Ihre Marke in der Kodeliste nicht finden oder keinen passenden Kode, nutzen Sie bitte die Kodesuche.

Manuelle Kodesuche
1. Bitte das zusteuernde Gerät einschalten. Programm läuft.
2. Zur Vorbereitung der Kodesuche drücken Sie die Tasten SET (Einstellung), und dann dazu die gewünschte Gerätetaste, z.B. TV, bis das Lämpchen (LED) dauernd leuchtet.
3. Dann Fernbedienung auf das zu steuernde Geräte richten.
4. Einmal kurz SET-Taste drücken. Das Tastenlämpchen (LED) blinkt.
5. Wiederholt, ca. jede Sekunde, EIN/AUS ⏻ (Gerät ein/aus) Taste drücken bis Gerät, z.B. Ihr Fernsehgerät, reagiert. - *Hatte sich Ihr Gerät ausgeschaltet, so müssen Sie es, z.B. am Gerät oder mit der Originalfernbedienung zur weiteren Suche wieder einschalten. Wenn Ihr Gerät reagiert können Sie auch andere Tasten probieren, ohne die Suche zu verlassen. Wenn einige Tasten nicht richtig funktionieren, suchen Sie einfach weiter mit EIN/AUS_Drücken.*
Wenn Ihr Gerät richtig reagiert, drücken Sie die Gerätetaste, z.B. TV, und beendenden so die Suche.

Das Lämpchen (LED) geht aus. - Die UR Z2 setzt die Suche dort fort, wo Sie die Suche abgebrochen hatten. Der zuletzt eingegebene Kode bleibt gespeichert. Die Suche wird nach ca. 30 Sekunden Untätigkeit abgebrochen.

Automatische Kodesuche

In wenigen Sekunden findet Ihre Fernbedienung fast automatisch die richtigen Steuereinstellungen:

1. Bitte das zu steuernde Gerät einschalten. Z.B. erscheint eine Programmnummer in der Anzeige Ihres DVB.
2. Zum Start der Kodesuche drücken Sie kurz die gewünschte Gerätetaste, z.B. TV.
3. Dann SET-Taste mehr als 5 Sekunden lang drücken bis das Lämpchen (LED) blinkt.
4. Dann Fernbedienung auf das zu steuernde Geräte richten, während die Fernbedienung jede Sekunde ein anderes EIN/AUS Signal sendet.
5. Wenn Ihr Gerät reagiert, drücken Sie SET innerhlb von 1-2 Sekunden, um die Kodesuche zu beenden. – Wenn Sie die Suche zu spät beendet haben, dann suchen Sie bitte weiter mit der Manuellen Kodesuche s.o..
6. Zum Abschluss testen Sie den eingestellten Steuerkode, indem Sie versuchen z.B. Ihr TV-Gerät zu steuern. Funktioniert es nicht richtig, dann versuchen Sie bitte die Kodesuche noch einmal. Die Kodesuche startet dann mit dem nächstfolgenden Code. - Vergessen Sie nicht Ihr Gerät z.B. TV wenn Sie weiter suchen lassen wollen, wieder am Gerät einzuschalten.

Kodeindentifikation

Sie können die eingestellten 4-stelligen Kodes jederzeit auslesen:

1. Gerätetaste des Gerätes drücken, dessen Kode Sie auslesen möchten. Z.B. TV.
2. SET (Einstellen) drücken und dazu 1,2,3,4 drücken, je nachdem welche Stelle des Kodes Sie auslesen möchten. Z.B. wenn Sie nach TV SET+1 gedrückt haben und der Bespielkode 0158 eingestellt ist, blinkt das Lämpchen kein mal, nach dem Loslassen von SET+1, 0x um die Tausenderstelle mit 0 anzuzeigen. Sie brauchen also nur zu zählen. Mit SET+2 wird nach dem Loslassen die 100er Stelle ausgegeben, z.B. blinkt es 1x für die 1 von 0158. Und mit SET+3 wird nach dem Loslassen die 10er Stelle ausgegeben, z.B. blinkt es 5x für die 5 von 0158. Mit SET+4 wird die 1er Stelle ausgegeben, z.B. blinkt es 8x für die 8 von 0158. - *Bitte notieren Sie sich die gefundenen, genutzten Steuerkodes zur späteren Verwendung auf dem Aufkleber im Batteriefach.*

Bedienung

Sie bedienen Ihre Geräte praktisch genauso, wie Sie es von Ihren Originalfernbedienungen her kennen, nur, dass Sie zuvor eine Gerätetaste TV, DVB usw. drücken müssen, um das gewünschte Gerät steuern zu können. Da die Symbole der UR Z2 sich von denen Ihrer Originalfernbedienungen unterscheiden können, müssen Sie u.U. alle Tasten zum Finden von Funktionen ausprobieren.

Lautstärke-Punch-Through-Funktion

Wenn bestimmte Steuerkodes, z.B. für DVB, keine Lautstärke- und Stummschaltfunktionen enthalten, nutzt UR Z2 automatisch die entsprechenden Funktionen des Fernsehgerätes.

Technische Daten

Batterien: 2x Mignonzellen, AA oder NiMH Akkus
Reichweite: max. 7 m
Nicht für 400 kHz Geräte

CE

Hiermit erklärt die Vivanco GmbH, dass sich UR Z2 in Übereinstimmung mit den grundlegenden Anforderungen und den anderen relevanten Vorschriften der Richtlinie 2004/108/EG befindet.

Entsorgung des Gerätes

Dieses Gerät darf nicht mit dem Hausmüll entsorgt werden. Bitte geben Sie es bei einer Sammelstelle für Elektrogeräte Ihrer Gemeinde oder Ihres Stadtteils ab. Damit wird gewährleistet, dass die Altgeräte fachgerecht, wirtschaftlich verwertet und potentielle Umwelt - und Gesundheitsschäden vermieden werden.

Pb/Cd/Hg

Batterie- oder Akkuentsorgung

Batterien und Akkus dürfen nicht in den Hausmüll gegeben werden. Sie sind zur Rückgabe von Altbatterien gesetzlich verpflichtet, damit eine fachgerechte, umweltschonende Verwertung gewährleistet werden kann. Bitte geben Sie Altbatterien an einer kommunalen Sammelstelle oder im Handel ab. Batterien und Akkus bitte nur in entladenem Zustand abgeben.

21
PAP

Die Verpackung ist einer Wiederverwendung zuzuführen. Bitte werfen Sie sie nicht in den Hausmüll und handeln Sie entsprechend der lokalen Entsorgungsvorschriften.

F

Universal TV- DVB Controller UR Z2



Avant d'utiliser l'appareil, lisez attentivement les mesures de sécurité et le manuel d'instructions afin d'utiliser toutes les fonctions de manière sûre et fiable. Soyez attentifs au mode d'emploi et fournissez-le à d'éventuels futurs propriétaires.

Mode d'emploi adéquat
Des télécommandes universelles remplacent les télécommandes d'origine, etc des appareils électroniques grand public. Il est conçu pour un usage privé et il est interdit de s'en servir à des fins commerciales. Toute autre utilisation est considérée comme inappropriée et peut endommager des objets ou blesser des personnes. Le fabricant n'est pas responsable des dommages qui résultent d'un usage inapproprié de l'appareil.

Sécurité batterie / accu
Veuillez suivre attentivement ces indications. Le non-respect de ces indications peut provoquer des dommages/blessures et éventuellement même l'explosion d'accus/batteries! Ne court-circuitez pas les contacts, par exemple par un entreposage avec des objets métalliques. N'exposez jamais des accus/batteries pendant longtemps aux rayons directes de soleil ou à la chaleur. Ne jetez pas des accus/batteries dans le feu. N'ouvrez jamais des accus/batteries, ils ne contiennent pas d'éléments à entretenir. Veuillez échanger des accus/batteries seulement contre des batteries du même type ou des types comparables recommandés par le fabricant. Seulement pour accus: N'utilisez pour le rechargement d'accus que la fonction de recharge intégrée dans l'appareil ou des chargeurs adaptés. Ne rechargez jamais des batteries.

Mise en place des piles
Ouvrez le compartiment des piles au dos de l'appareil, en poussant légèrement vers le bas la languette, puis mettez en place 2 piles alcalines R6. Veillez respecter la polarité telle qu'elle est indiquée sur les piles et dans le compartiment des piles (+=+). Refermez ensuite le compartiment avec précaution.
– N'utilisez que des piles en parfait état.

Réglage pour un appareil p. ex. un téléviseur ou un Set Top Box DVB (ou récepteur satellite/tuner pour réception par câble) ou un autre appareil
Pour pouvoir commander vos appareils, vous devez régler la télécommande pour l'appareil en question. A cette fin, vous avez deux possibilités : entrée directe du code ou recherche du code.

Entrée directe d'un code à l'aide de la liste fournie
Exemple : réglage de la télécommande pour un téléviseur Panasonic :
1. Veuillez vous assurer d'avoir choisi la liste correcte pour téléviseurs (TV). Utilisez par ex. le code Panasonic 0158 figurant sur la liste fournie.
2. Avant de pouvoir entrer le code, appuyez d'abord brièvement sur les touches SET (Réglages), située en bas à droite, et ensuite sur la touche souhaitée de l'appareil, par ex. TV jusqu'à ce que le voyant (LED) s'allume en permanence.
3. Appuyez ensuite successivement sur les touches numériques (par ex. 0158).
4. Une fois que vous avez entré les chiffres, la LED s'éteint et la télécommande est prête à commander votre appareil.
5. Complétez l'entrée par un test du code de commande saisi, consistant à chercher à commander votre téléviseur. S'il ne se passe rien ou pas ce que vous souhaiteriez, réessayez avec un autre code de la liste ou avec la fonction de recherche de code (voir plus bas). Le réglage de la télécommande pour d'autres appareils est à effectuer de la même manière. Vous terminez la saisie du code en appuyant sur l'importe quelle touche de l'appareil.

Recherche du code de commande
Lorsque vous ne trouvez pas votre marque dans la liste de codes ou s'il n'y a aucun code adéquat, veuillez utiliser la recherche de codes.
Recherche manuelle de code
1. Allumez l'appareil que vous souhaitez commander. L'appareil en question étant déjà réglé sur une chaîne.
2. Avant de pouvoir entrer le code, appuyez d'abord brièvement sur les touches SET (Réglages), située en bas à droite, et ensuite sur la touche souhaitée de l'appareil, par
ex. TV, jusqu'à ce que le voyant (LED) s'allume en permanence.
3. Dirigez ensuite la télécommande vers l'appareil que vous souhaitez commander.
4. Appuyez une fois brièvement sur la touche SET Le voyant (LED) clignote.
5. Environ chaque seconde, appuyez à répétition sur la touche MARCHE/ARRÊT ⏻ (Appareil marche/arrêt) jusqu'à ce que l'appareil – par exemple votre téléviseur – réagisse. Si votre appareil s'était arrêté, vous devez alors le remettre en marche, par exemple sur l'appareil ou avec la télécommande d'origine, afin de poursuivre la recherche. – Si votre appareil réagit, vous pouvez aussi essayer d'autres

touches sans quitter la recherche. Si quelques touches ne fonctionnent par correctement, recherchez simplement en continuant d'appuyer sur MARCHE/ARRÊT:
6. Lorsque votre appareil réagit correctement, appuyez sur la touche de l'appareil, par ex. TV pour terminer la recherche. Le voyant (LED) s'éteindra. - La UR Z2 reprend la recherche là, où vous avez interrompu la recherche. La recherche s'arrête automatiquement si vous avez atteint la fin de la liste de codes. Dans ce cas, la télécommande clignote plusieurs fois de suite. Le dernier code sauvegardé reste mémorisé. La fonction de recherche se termine automatiquement au bout d'env. 30 secondes d'inactivité.

Recherche automatique de code
Au bout de quelques secondes, votre télécommande effectuera les bons réglages de façon presque automatique :
1. Allumez l'appareil que vous souhaitez commander. Le DVB doit par ex. afficher un numéro de programme.
2. Pour démarrer la recherche de code, appuyez un court instant sur la touche d'appareil souhaitée, p. ex. TV
3. Ensuite, appuyez plus de 5 secondes sur la touche SET jusqu'à ce que la petite lampe (diode) clignote.
4. Dirigez ensuite la télécommande vers l'appareil que vous souhaitez commander, tandis que la télécommande envoie chaque seconde un autre signal ON/OFF.
5. Lorsque votre appareil réagit, appuyez sur SET dans les 1 à 2 secondes, pour terminer la recherche de code. – Si vous avez terminé la recherche trop tard, poursuivez-la defaçon manuelle (voir plus haut Recherche manuelle de code).
6. Pour finir, testez le code de contrôle réglé, en essayant par ex. de commander votre lecteur TV. S'il ne fonctionne pas, redémarrez une nouvelle fois la fonction de recherche de code. La recherche de code démarre alors avec le code suivant. - Si vous souhaitez poursuivre la recherche, n'oubliez pas de rallumer votre appareil, par x. votre lecteur TV via l'appareil lui-même.

Identification de code
Vous avez à tout moment la possibilité de lire le code à 4 chiffres réglé :
1. Appuyez sur la touche associée à l'appareil dont vous souhaitez connaître le code, par ex. TV.
2. Appuyer la touche SET et 1, 2, 3, 4, quel que soit la partie du code que vous souhaitez lire. Par example, si vous appuyez SET + 1 après avoir défini TV et le code 0158 (dans cet exemple), la lampe ne clignote pas après que SET + 1 soient relâchés pour montrer que le chiffre des milliers est 0. Tout ce que vous avez à faire est compter. Après avoir relâché SET + 2, la lampe clignote une fois pour montrer que le chiffre des centaines est 1. Les dizaines sont indiquées après avoir relâché SET + 3, p. ex. Elle clignote 5 fois pour le 5 dans 158. Le chiffre un est indiqué après avoir relâché SET+4, p. ex. elle clignote 8 fois pour le 8 dans 0158. *Veuillez inscrire sur l'autocollant du logement à piles les codes de commande utilisés que vous avez trouvés pour une utilisation ultérieure.*

Commande
Vous commandez vos appareils pratiquement de la même façon qu'avec les télécommandes d'origine, sauf que vous devez d'abord presser une des touches pour les appareils, comme TV, DVB etc., pour pouvoir commander l'appareil souhaité. Comme les symboles sur l'UR Z2 peuvent différer de ceux des télécommandes d'origine, il vous faudra tester toutes les touches pour découvrir les fonctions de chacune.

Fonction de volume Punch-Through
Lorsque certains codes de commande, par exemple pour le DVB, ne comportent aucune fonction de volume et de coupure de son, UR Z2 utilise automatiquement les fonctions correspondantes du téléviseur.

Données techniques
Piles : 2 piles rondes, R6, UM3, AA / Portée : max. 7 m / Non adapté pour appareils de 400 kHz

Vivanco GmbH déclare que ce UR Z2 rempli toutes les conditions de base requises et respecte les autres réglementations applicables dans le cadre de la norme européenne 2004/108/EG.

Elimination de l'appareil
Cet appareil ne doit pas être jeté dans la poubelle domestique. Remettez-le à un point de collecte de matériel électrique dans votre quartier ou votre zone. De cette manière, vous vous assurez que les vieux appareils électriques sont traités de manière scientifique et professionnelle, évitant ainsi de potentiels dommages causés à la santé et à l'environnement.

Piles et accumulateurs
Pb/Cd/Hg
Les piles et accumulateurs ne doivent pas être jetés avec les ordures ménagères. Ils font l'objet d'une obligation légale de retour, afin de pouvoir garantir un recyclage selon les règles et dans le respect de l'environnement. Veuillez apporter les piles usagées dans un centre de collecte communale ou un commerce. Les piles et accumulateurs doivent être entièrement déchargés avant d'être rapportés.

21
PAP
L'emballage est recyclable. Ne le jetez pas dans la poubelle domestique, amenez-le au bon endroit pour un traitement adéquat des déchets.

E

Universal TV- DVB Controller UR Z2



Antes de utilizar el dispositivo, lea atentamente las recomendaciones de seguridad y el manual de instrucciones Solo así se podrán utilizar todas las funciones de un modo seguro y fiable. Preste atención al modo de empleo y facilíteselo a posibles futuros propietarios.

Modo de empleo adecuado
Mandos a distancia universales sustituyen a los controles remotos originales etc de los dispositivos de electrónica de consumo. El dispositivo está diseñado para un uso privado y no está permitido su uso con fines comerciales. Cualquier otro empleo se considera inapropiado y puede provocar daños a objetos o incluso a personas. El fabricante queda exento de responsabilidades por daños ocasionados que deriven de un uso inapropiado del dispositivo.



Seguridad de baterías/baterías recargables
Favor de leer con mucho cuidado estas instrucciones. Hacer caso omiso de estas instrucciones puede provocar daños/lesiones y posiblemente hasta una explosión de las baterías/baterías recargables. No provoque cortos circuitos en los contactos, por ejemplo, al almacenar con objetos metálicos comunes. No exponga las baterías/baterías recargables a períodos prolongados de luz del sol directa o calor. No tire las baterías/baterías recargables al fuego. No abra las baterías/baterías recargables, no contienen piezas que requieran mantenimiento. Remplace las baterías/baterías recargables únicamente con baterías del mismo tipo, o con un equivalente recomendado por el fabricante. Sólo acumuladores: Para cargar los acumuladores, solamente utilice la función de carga integrada en el dispositivo o cargadores adecuados. Nunca cargue las baterías.

Colocación de pilas
Abra la tapa del compartimiento de pilas situado en la parte trasera del telemando y apriete ligeramente hacia abajo la lengüeta para colocar entonces 2 AA-pilas alcalinas. Preste atención a que están correctas las polaridades indicadas en las pilas y en el compartimiento (+=+) y cierre cuidadosamente el compartimiento. – Rogamos sírvanse de emplear siempre pilas impecables..

Ajuste al un dispositivo, p.ej. un televisor o un receptor DVB Set Top Box (o receptor SAT/ sintonizador por cable) o un otro dispositivo.
Para poder controlar sus aparatos, debe ajustar su telemando a sus aparatos a mandar. A esto, hay dos posibilidades: la introducción directa de códigos de mando y la búsqueda de códigos.

Introducción directa del código de mando por medio de la lista adjuntada
Ejemplo: Ajustar el telemando a un televisor de Panasonic:
1. En la lista adjunta encuentra el código Panasonic de, p. ej., 0158. Asegurese que han selectido la lista de TV correcta.
2. Para preparar la introducción del código, presione brevemente las teclas SET (ajuste) y también la tecla TV hasta que la lámpara (LED) esté permanentemente iluminada.
3. Entonces, presione una tras otra las teclas de número (p.ej. 0158).
4. Después de haber introducido los dígitos, la lámpara en la tecla correspondiente se apaga y el telemando está listo para controlar el aparato.
5. Para terminar, debe controlar el código de mando ajustado intentando mandar su televisor. Caso que no funcione y no funcione de la manera correcta, debe introducir otro código indicado en la lista o proceder con la búsqueda de código (v.a.). Para la instalación del telemando para la control de otros dispositivos procede en misma manera. Si pulsa cualquier tecla del aparato podrá concluir la introducción del código.

Búsqueda del codigo de control
Si no encuentra su marca en la lista de códigos o si no encuentra ningún código que sea válido, utilice la búsqueda de códigos.

Búsqueda de código manual
1. Active el aparato a mandar. El programa esta puesto.
2. Para preparar la búsqueda de código, accione brevemente las teclas SET (ajuste) y entonces también una tecla de aparato p. ej. TV hasta que la lámpara (LED) de luz permanente.
3. Entonces, dirija el telemando hacia el aparato a controlar.
4. Presione una vez brevemente la tecla SET Se apaga la lámpara (LED).
5. Pulse aprox. cada segundo y de forma reiterada la tecla ON/OFF ⏻ (aparato con/desc) hasta que el aparato, por ejemplo hasta que su televisor reaccione. Caso que su aparato se haya desactivado, debe activarlo de nuevo a través o del botón del aparato mismo o el telemando original para poder seguir con la búsqueda. – Si su aparato reacciona, también puede accionar otras teclas sin tener que salir del modo de búsqueda. Caso que algunas teclas no funcionen correctamente, siga buscando mediante ON/OFF.
6. Si su aparato reacciona de la manera correcta, presione las teclas de aparato p. ej TV para terminar de esta manera el proceso de búsqueda. Se apaga la lámpara (LED). - El UR Z2 prosigue la búsqueda

donde se interrumpió. Si se ha alcanzado el final de la lista de códigos, la búsqueda finaliza automáticamente. En ese momento el mando a distancia parpadea más rápido. El último código introducido queda guardado. El proceso de búsqueda es interrumpido después de unos 30 segundos de inactividad.

Búsqueda de código automática
Dentro de pocos segundos, su telemando encuentra de manera casi automática los ajustes de control correctos:
1. Conecte el aparato que va a utilizar; aparece, por ejemplo, un número de programa en el indicador de su DVB.
2. Para iniciar la búsqueda de código, presione brevemente la tecla de aparato p. ej. TV deseado.
3. A continuación, pulse durante más de 5 segundos la tecla SET hasta que parpadee la lucecita (LED).
4. Entonces, dirija el telemando hacia el aparato a mandar mientras que el telemando emite casa segundo una señal ON/OFF diferente.
5. Si su aparato reacciona, presione SET para terminar la búsqueda de código. – Caso que haya terminado demasiado tarde la búsqueda, debe seguir buscando mediante el modo de búsqueda manual (v.a.).
6. Para terminar debe controlar el código de mando elegido, mientras utiliza su TV, por ejemplo. Si no funciona correctamente, debe repetir de nuevo la búsqueda de código. Entonces, el modo de búsqueda es iniciado a partir del código próximo. – No olvide activar su aparato, p.ej su TV, en el mismo, si desea seguir buscando código.

Identificación de código
En cualquier momento, puede lecturar los códigos de 4 dígitos:
1. Presionar la tecla de aparato de aquel aparato, cuyo código desea lecturar, p.ej. TV.
2. Pulse el botón SET y 1,2,3,4, independientemente de la parte del código que desee leer. Si pulsa por ejemplo, SET + 1 después de TV y se estableció el código 0158 (en este ejemplo), la luz no parpadeará después de la desconexión de SET + 1 para indicar que el número mil es 0. Sólo tiene que contar. Después de la desconexión de SET + 2 la luz parpadeará una vez para indicar que el número cien es 1. El número diez se muestra después de la desconexión de SET + 3, es decir, parpadeará 5 veces para el número 5 en 158. Las cifras individuales se muestran después de la desconexión de SET + 4, es decir, parpadeará 8 veces para el número 8 en 0158. *Anote el código de control que ha utilizado en la etiqueta de la tapa de las pilas para poder usarlo en un futuro.*

Manejo
El aparato se maneja prácticamente como su mando a distancia original, la única diferencia es que previamente debe pulsar una tecla del aparato de TV, DVB, etc. para poder controlar el aparato deseado. Dado que los símbolos del UR Z2 pueden diferir de los que se encuentran en su mando a distancia original, es posible que deban probarse todas las teclas para identificar cada función.

Volumen- Función Punch-Through
Cuando contiene códigos de control determinados, por ejemplo para el descodificador DVB, sin función de volumen y mudo, el UR Z2 utiliza automáticamente las funciones correspondientes del televisor.

Datos técnicos
Pilas: 2x AA-pilas, R6, UM3,
Alcance: máx. 7 m
No apropiado para aparatos de 400 kHz

Vivanco GmbH declara que este su UR Z2 cumple todos los requisitos básicos y otras regulaciones aplicables en el marco de la normativa europea 2004/108/EG.

Eliminación del dispositivo
Este dispositivo no debe ser eliminado con la basura doméstica. Entréguelo en un punto de recogida de dispositivos eléctricos en su barrio o zona. De este modo se garantiza que los dispositivos eléctricos viejos son tratados de forma científica y profesional y evitando potenciales daños a la salud y al medioambiente.

Pilas y acumuladores
Pilas y acumuladores no deben ser tirados con los residuos domésticos. Esta legalmente obligado a devolver las pilas utilizadas, para que se asegura una eliminación profesional y amigable con el ambiente. Por favor llevarlas a un punto de colecta municipal o tienda minorista. Por favor entregue las pilas y acumuladores en estado descargado solamente.

Pb/Cd/Hg

El envoltorio es de uso reciclable. No lo arroje a la basura doméstica, llévelo al lugar correspondiente para un correcto procesamiento de residuos.

21
PAP

NL

Universal TV- DVB Controller UR Z2



Alvorens het apparaat te gebruiken dient u de veiligheidsinstructies en deze gebruiksaanwijzing zorgvuldig door te lezen. Alleen zo kunt u alle functies veilig en betrouwbaar benutten. Bewaar deze gebruiksaanwijzing goed en geef hem aan een eventuele volgende bezitter door.

Gebruik volgens de bepalingen

Universele afstandsbedieningen vervangen originele afstandsbedieningen o.a. van elektronische entertainmentapparaten. Het apparaat is uitsluitend bedoeld voor privé gebruik en mag niet voor commerciële doeleinden worden gebruikt. Ieder ander gebruik geldt als in tegenstrijd met de bepalingen en kan leiden tot beschadigingen aan voorwerpen of zelfs tot persoonlijk letsel. De fabrikant stelt zich niet aansprakelijk voor schades die worden veroorzaakt door gebruik in strijd met de bepalingen.



Veiligheid van batterij/accu

Deze instructies a.u.b. zeer zorgvuldig in acht nemen. Het niet in acht nemen van deze instructies kan leiden tot beschadigingen/letsel en eventueel zelfs tot explosie van accu´s/batterijen! Sluit de contacten niet kort door bijv. het opslaan samen met metalen voorwerpen. Stel de accu´s/batterijen niet bloot aan langer, direct zonlicht of hitte. Gooi accu´s/batterijen niet in het vuur. Open accu´s/batterijen nooit – ze bevatten geen onderdelen die onderhouden moeten worden. Accu´s/batterijen uitsluitend vervangen door dezelfde types of door de fabrikant aanbevolen gelijkwaardige types. Alleen accu´s: Gebruik voor het laden van accu´s uitsluitend de in het apparaat ingebouwde laadfunctie of geschikte opladers. Batterijen nooit opladen.

Plaatsen van de batterijen

Open de batterijklep aan de achterzijde van het toestel, druk het klepje licht naar onderen en plaats 2 alkaline AA-batterijen. Let op de aangegeven polariteit op de batterijen en in het batterijvak (+=+), en sluit het vak dan weer voorzichtig. – Let erop dat u altijd onbeschadigde batterijen gebruikt.

Instellen van een apparaat

Om uw toestel te kunnen bedienen moet u eerst de afstandsbediening voor uw toestellen instellen. Daarvoor zijn er twee mogelijkheden: het direct invoeren van een bedieningscode en het zoeken van codes.

Directe invoer van bedieningscodes met behulp van de bijgevoegde lijst

Voorbeeld: Instellen van de afstandsbediening voor een Panasonic-televisietoestel:

1. Op de bijgevoegde lijst ziet u bijv. de Panasonic-code 0158. Vergewis u ervan dat u de correcte lijst voor televisie (TV) gekozen heeft.
2. Ter voorbereiding van het invoeren van de code drukt u kort op de toets SET (instellen) en daarbij ook op de b.v. TV-toets, tot het lampje (LED) constant brandt.
3. Dan drukt u achtereenvolgens de cijfertoetsen in (b.v. 0158).
4. Na het invoeren van de cijfers gaat het lampje uit en de afstandsbediening is klaar om uw toestel te bedienen.
5. Tot slot test u de ingestelde bedieningscode, door uw televisietoestel te bedienen. Als het niet werkt of niet goed werkt, probeer dan een andere code uit de lijst of het zoeken van codes, zie hieronder. De instelling van de afstandsbediening voor de bediening van andere toestellen vindt u op dezelfde manier. U kunt deze modus afsluiten door op een willekeurige toets te drukken.

Zoeken naar stuurcodes

Als u uw merk niet in de lijst met codes kunt vinden of u kunt geen passende code vinden, gebruik dan zoeken naar codes.

Handmatig zoeken van codes

1. Schakel het te bedienen toestel in zodat een programma afgespeeld wordt.
2. Ter voorbereiding op het zoeken van codes drukt u kort op de toets SET (instellen) en daarbij ook op een toets van een apparaat, bijv. TV, totdat het lampje (LED) constant brandt.
3. Richt dan de afstandsbediening op het te bedienen toestel.
4. Druk een keer kort op de toets SET. Het lampje (LED) knippert.
5. Druk ongeveer elke seconde op de AAN/UIT ⏻ (apparaat aan/uit) toets tot het apparaat, bijvoorbeeld uw televisie, reageert. Wanneer uw toestel is uitgeschakeld, dan moet u het weer inschakelen om verder te kunnen zoeken, bijv. op het toestel of met de originele afstandsbediening. - Als uw toestel reageert, dan kunt u ook andere toetsen proberen zonder het zoeken te verlaten. As een aantal toetsen niet goed werkt, dan zoekt u eenvoudig verder met AAN/UIT. Als uw toestel juist reageert, druk dan op de toets bijv. TV en stop zo met zoeken. Het lampje (LED) gaat uit. - De UR Z2 vervolgt het zoeken daar waar u

het zoeken onderbroken hebt. Als het einde van de codelijst bereikt is, stopt het zoeken automatisch. De laatst ingegeven code blijft opgeslagen. Het zoeken wordt afgebroken wanneer u ca. 30 seconden niets met het toestel doet.

Automatisch zoeken van codes

Binnen een paar seconden vindt uw afstandsbediening bijna automatisch de juiste bedieningsinstellingen:

1. Schakel het te bedienen toestel in, zodat een televisieprogramma speelt.
2. Druk om het zoeken naar de code te starten even op de gewenste apparaattoets, bijvoorbeeld TV.
3. Druk vervolgens de SET-toets ruim 5 seconden in totdat het lampje (LED) knippert.
4. Richt de afstandsbediening vervolgens op het te bedienen apparaat terwijl de afstandsbediening elke seconde een ander AAN/UITsignaal stuurt.
5. Als het apparaat reageert, drukt u binnen 1-2 seconden op SET om het zoeken naar de code af te sluiten. – Als u het zoeken te laat afsluit, zoek dan verder met handmatig zoeken naar de code, zie hierboven.
6. Ten slotte test u de ingestelde stuurcode door te proberen om bijvoorbeeld uw TV-speler te bedienen. Als dat niet goed gaat, moet u nogmaals naar de code zoeken. Het zoeken naar de code begint met de eerstvolgende code. - Vergeet niet uw toestel weer in te schakelen bijv. TV, als u verder wilt zoeken.

Code-identificatie

U kunt de ingestelde viercijferige code uitlezen:

1. Druk op de toets van het toestel waarvan u de code wilt uitlezen, bijv. op TV.
2. Druk op de SET knop en 1, 2, 3, 4, ongeacht welk deel van de code u wilt lezen. Als u bijvoorbeeld SET+ 1 na TV activeert en de code 0158 (in dit voorbeeld) is ingesteld, gaat de lamp niet knipperen na het loslaten van SET+1, om aan te geven dat het aantal duizenden 0 bedraagt. U hoeft alleen maar te tellen. Na het loslaten van SET +2 knippert het lampje eenmaal, om aan te geven dat het aantal honderden 1 is. Het aantal tientallen verschijnt na het loslaten van SET+3, dat wil zeggen het lampje knippert 5 keer voor het nummer 5 in 158. Het aantal eenheden wordt na het loslaten van SET+4 aangegeven, dat wil zeggen het lampje knippert 8 keer voor het cijfer 8 in 0158. *- Noteer de gevonden bruikbare stuurcodes op de sticker in het batterijvak zodat u ze later weer kunt gebruiken.*

Bediening

U bedient uw apparaat praktisch net zo als u van uw originele afstandsbediening gewend bent, u moet alleen eerst de toets voor het apparaat,TV, DVB enz. indrukken om het gewenste apparaat te kunnen bedienen. Omdat de symbolen van de UR Z2 anders kunnen zijn dan die van uw originele afstandsbediening, moet u mogelijk alle toetsen uitproberen om de functies te vinden.

Volume Punch-Through-functie

Als bepaalde stuurcodes, bijvoorbeeld voor een DVB, geen volume- en mutefunctie heeft, gebruikt de UR Z2 automatisch de betreffende functies van de tv.

Technische gegevens

Batterijen: 2x AA-batterijen, R6, UM3
Bereik: max. 7 m
Niet voor 400 kHz apparaten

CE

Hiermede verklaart de Vivanco GmbH dat UR Z2 zich in overeenstemming met de fundamentele eisen en de andere relevante voorschriften van de richtlijn 2004/108/EG bevindt.

Verwijdering van het apparaat

Dit apparaat mag niet met het normale huisvuil worden verwijderd. Breng het naar een speciale verzamelplaats voor elektrische apparaten in uw gemeente of woonwijk. Daarmee wordt gegarandeerd dat de niet meer bruikbare apparaten vakkundig, economisch verwerkt worden en dat potentiële schade voor milieu en gezondheid vermeden wordt.

Pb/Cd/Hg

Batterijen en accu´s

Batterijen en accu´s mogen niet met het normale huisvuil worden verwijderd. U bent wettelijk verplicht tot het teruggeven van oude batterijen, zodat een vakkundige, milieuvriendelijke verwerking kan worden gegarandeerd. Geef a.u.b. oude batterijen bij een gemeentelijke verzamelplaats of bij uw dealer af. Batterijen en accu´s uitsluitend in ontladen toestand afgeven.

21
PAP

De verpakking dient naar een recyclingplaats te worden gebracht. Doe de verpakking niet bij het huisvuil en neem de plaatselijke voorschriften m.b.t. afvalverwijdering in acht.

PL

Universal TV- DVB Controller UR Z2



Przed użyciem urządzenia należy dokładnie zapoznać się z informacjami dotyczącymi bezpieczeństwa oraz z instrukcją obsługi. Tylko w ten sposób możesz bezpiecznie i pewnie korzystać z wszystkich funkcji urządzenia. Niniejszą instrukcję obsługi należy przechowywać w pewnym miejscu, aby w razie potrzeby móc z niej skorzystać w przyszłości. Należy przekazać ją ewentualnym kolejnym właścicielom urządzenia.

Użycie zgodne z przepisami
Uniwersalne piloty zdalnego sterowania zastępują oryginalne piloty np. urządzeń RTV. Przeznaczone jest ono do użycia wyłącznie w zakresie prywatnym i nie może być używane do celów przemysłowych. Każde inne zastosowanie uznane jest za niezgodne z przepisami i może prowadzić do szkód materialnych lub nawet do szkód na osobach. Producent nie przejmuje odpowiedzialności za szkody, które powstały w wyniku użycia urządzenia niezgodnie z przepisami.



Bezpieczeństwo baterii/akumulatorów
Proszę stosować się dokładnie do niniejszych wskazówek. Niestosowanie się do niniejszych wskazówek może prowadzić do uszkodzeń/zranień i w pewnych warunkach nawet do eksplozji akumulatorków/baterii! Nie zwierać kontaktów, np. poprzez przechowywanie razem z metalowymi przedmiotami. Nigdy nie wystawiać na dłuższy okres czasu akumulatorków/baterii na promieniowanie słoneczne lub cieplne. Nie wrzucać akumulatorków/baterii do ognia. Nigdy nie otwierać akumulatorków/baterii, nie zawierają one części wymagających konserwacji. Akumulatorki/baterie wymieniać wyłącznie na tego samego typu lub zalecanego przez producenta, równoważnego typu. Tylko akumulatory: Do ładowania akumulatorów należy używać wyłącznie urządzeń z wbudowaną funkcją ładowania lub odpowiednich ładowarek. Nigdy nie ładować baterii.

Włożenie baterii
Proszę otworzyć klapkę baterii na tylnej stronie urządzenia, lekko wcisnąć w dół języczek i włożyć 2 mignonogniwowe baterie alkaliczne. Zwrócić uwagę na podane na bateriach i w przegródce na baterie biegunowości (+=+), i ponownie ostrożnie zamknąć przegródkę. – Proszę zwracać uwagę na to, aby zawsze używać nienagannych baterii.

Dopasowanie do urządzenia, np. telewizora lub DVB Set Top Box (lub odbiornika satelitarnego/ tunera kablowego) albo innego urządzenia
Aby można było sterować Waszymi urządzeniami, musicie Państwa dostroić pilota do Waszych urządzeń. W tym celu istnieją dwie możliwości: bezpośrednie wprowadzenie kodu sterującego i wyszukiwanie kodu.

Bezpośrednie wprowadzenie kodu sterującego za pomocą załączonej listy
Przykład: ustawienie pilota dla telewizora marki Panasonic:
1. Z załączonej listy wybrać np. kod Panasonic 0158.
2. Proszę upewnić się, czy została wybrana prawidłowa lista dla telewizora (TV). W celu przygotowania wprowadzenia kodu wcisnąć krótko przyciski SET (ustawienie) a potem dodatkowo przycisk np. TV do chwili aż lampka (LED) będzie świecić ciągle.
3. Następnie przyciskać po kolei przyciski z cyframi (np. 0158).
4. Po wprowadzeniu cyfr lampka gaśnie i pilot jest gotowy do sterowania Waszym urządzeniem.
5. Na zakończenie przetestować ustawiony kod sterujący, próbując sterować Waszym telewizorem. Jeżeli to nie funkcjonuje lub nie funkcjonuje właściwie, to proszę spróbować następny kod z listy lub wyszukiwanie kodów, patrz niżej. Ustawienia pilota w celu sterowania dalszymi urządzeniami należy przeprowadzić w ten sam sposób. - Wczytywanie kodu należy zakończyć przez naciśnięcie dowolnego przycisku urządzenia.

Poszukiwanie kodu sterującego
Jeśli nie znaleźli Państwo swej marki lub listy kodów lub odpowiedniego kodu, wtedy prosimy skorzystać z funkcji poszukiwania kodu.

Manualne wyszukiwanie kodów
1. Proszę włączyć urządzenie, które ma być sterowane. Program działa.
2. W celu przygotowania wyszukiwania kodów wcisnąć krótko przyciski SET (ustawienie) po prawej stronie na dole, a potem dodatkowo przycisk urządzenia np. TV do chwili aż lampka (LED) będzie ciągle świecić.
3. Następnie skierować pilota na urządzenie, które ma być sterowane.
4. Wcisnąć raz przycisk SET. Lampka (LED) miga.
5. Powtarzać co ok. 1 sekundę WŁĄCZANIE / WYŁĄCZANIE ⏻ (włączenie i wyłączanie urządzenia) aż na urządzenie zareaguje na przykład Państwa telewizor. -Gdyby Wasze urządzenie się wyłączyło, to należy je ponownie włączyć w celu dalszego wyszukiwania np. przyciskiem na urządzeniu lub za pomocą oryginalnego pilota. - Gdy urządzenie reaguje, należy wypróbować również inne przyciski bez wychodzenia z wyszukiwania. Jeśli niektóre przyciski nie funkcjonują właściwie należy dalej wyszukiwać za pomocą ON/OFF lub Play. Jeśli urządzenie reaguje prawidłowo, należy wcisnąć przycisk urządzenia TV, i w ten sposób zakończyć wyszukiwanie. Lampka (LED) gaśnie. - UR Z2 rozpoczyna w tym miejscu poszukiwanie, w którym

zostało przerwane. Jeśli osiągnięto koniec listy kodów, poszukiwanie zakończone jest automatycznie. Jednocześnie miga światełko w pilocie kilkakrotnie i szybko. Ostatnio podany kod zostanie zakodowany. Poszukiwanie zostanie po około 30 sekundach przerwy przerwane.

Automatyczne wyszukiwanie kodów
W ciągu kilku sekund Wasz pilot znajduje prawie automatycznie prawidłowe ustawienia sterowania:
1. Proszę włączyć urządzenie, które ma być sterowane, tak aby włączony był jakikolwiek program telewizyjny.
2. W celu rozpoczęcia procedury poszukiwania kodu należy krótko nacisnąć przycisk żądanego urządzenia, np. TV.
3. Następnie, przez ponad 5 sekund przytrzymać przyciśnięty przycisk SET, aż zacznie migać dioda kontrolki.
4. Następnie skierować pilota na urządzenie, które ma być sterowane, podczas gdy pilot co sekundy wysyła inny sygnał ON/OFF.
5. Jeśli urządzenie reaguje, należy wcisnąć i przytrzymać przez 1-2 sekund jakikolwiek przycisk oprócz SET, aby zakończyć wyszukiwanie kodów. – Jeśli poszukiwanie zostało zakończone za późno, proszę wyszukiwać dalej za pomocą manualnego wyszukiwania kodów, patrz powyżej.
6. Na zakończenie, proszę przetestować ustawiony kod sterujący, przez sterowanie np. urządzeniem TV. Jeśli nie funkcjonuje on prawidłowo, proszę spróbować ponownie wyszukiwania kodów. Wyszukiwanie kodów zaczyna się wtedy z następnym kolejnym kodem. – Jeśli zamierzacie Państwo dalej wyszukiwać, należy pamiętać o ponownym włączeniu urządzenie przyciskiem na urządzeniu.

Identyfikacja kodów
Ustawiony 4-cyfrowy kod możecie Państwo w każdej chwili wyczytać:
1. Wcisnąć klawisz tego urządzenia, którego kod chcecie Państwo wyczytać. Np. TV.
2. Naciśnij przycisk SET i 1, 2, 3, 4 zależnie od tego, którą cyfrę kodu chcesz odczytać. Na przykład jeśli naciśniesz SET+1, po tym jak ustawiono TV i kod 0158 (dla tego przykładu), to po puszczeniu przycisków SET+1 kontrolka nie zaświeci się, ponieważ cyfra przedstawiająca tysiące wynosi 0. Wszystko co musisz zrobić, to liczyć. Po puszczeniu przycisków SET+2 kontrolka zamiga raz, aby wskazać, że cyfra przedstawiająca setki wynosi 1. Cyfra przedstawiająca dziesiątki wskazywana jest po puszczeniu przycisków SET+3, np. miga 5 razy dla cyfry 5 w 0158. Cyfra przedstawiająca jedności wskazywana jest po puszczeniu przycisków SET+4, np. miga 8 razy dla 0158. *- Prosimy o zanotowanie na nalepce kasety bateryjnej znalezionego, używanego kodu potrzebnego do późniejszych zastosowań.*

Obsługa
Wasze urządzenia obsługujecie Państwo praktycznie tak samo, jak znacie to z oryginalnych pilotów, różnica polega jedynie na tym, że w tym przypadku należy przycisnąć przycisk TV lub DVB aby móc sterować wybranym urządzeniem. Ponieważ symbole UR Z2 mogą różnić się od tych z Waszego oryginalnego pilota, to musicie Państwo ewentualnie wypróbować wszystkie klawisze w celu znalezienia ich funkcji.

Funkcja głośności Punch-Through
Jeśli określony kod sterujący, np. DVB nie zawierają funkcji regulacji głośności oraz wyciszania, wtedy UR Z2 przejmuje automatycznie odpowiednie funkcje odbiornika telewizyjnego.

Dane techniczne
Baterie: 2x mignonogniwa, AA;UM3, R6
Zasięg: maks. 7 m
Nie dla urządzeń o 400 kHz

CE

Firma Vivanco GmbH oświadcza, iż UR Z2 jest zgodne z podstawowymi wymogami i innymi ważnymi zarządzeniami wytycznych 2004/108/EG.

Utylizacja urządzenia
Urządzenia nie należy wyrzucać wraz z innymi odpadami gospodarczymi. W przypadku konieczności utylizacji użytkownik zobowiązany jest przekazać urządzenie do wyspecjalizowanego punktu zbiórki. Prawidłowa utylizacja umożliwia uniknięcie negatywnego wpływu na środowisko i zdrowie, które może być zagrożone przez nieodpowiednie postępowanie z odpadami.

Pb/Cd/Hg

Baterie i akumulatory
Baterii i akumulatorów nie wolno utylizować wraz z odpadami komunalnymi. Konsumenci są prawnie zobowiązani do zwrotu zużytych baterii w punktach ich zbiórek w celu zapewnienia ich prawidłowego, ekologicznego recyklingu. Proszę zwracać zużyte baterie w komunalnych punktach ich zbiórek lub w sklepach. Baterie i akumulatory proszę zwracać wyłącznie rozładowane.

21
PAP

Opakowanie wielokrotnego użytku. Nie należy wyrzucać go wraz z innymi odpadami gospodarczymi. Zaleca się postępować zgodnie z lokalnymi przepisami dotyczącymi usuwania odpadów.

S

Universal TV- DVB Controller UR Z2



Innan du använder enheten, läs först säkerhetsföreskrifterna och den här bruksanvisningen noggrant. Endast på det viset kan du använda alla funktioner på ett säkert och tillförlitligt sätt. Förvara anvisningarna på ett säkert ställe och överlämna dem till eventuella efterföljande ägare.

Avsedd användning
Universella fjärrkontroller ersätter originalfjärrkontroller och andra elektroniska enheter. Enheten är avsedd att användas inom den privata sektorn och inte för kommersiellt bruk. All annan användning anses olämplig och kan även leda till skador av föremål eller personskador. Tillverkaren tar inget ansvar för skador som orsakats av felaktig användning.



Säkerhet med batterier
Följ dessa anvisningar noga. Underlåtenhget med att följa dessa anvisningar kan leda till sakskada/ personskada och under vissa omständigheter även leda till att batteriet exploderar! Låt aldrig kontakterna, t.ex. vid förvaring, kortslutas genom metall i omgivningen. Förvara aldrig batterierna någon längre tid i direkt solljus eller hetta. Kasta inte batterierna i öppen eld. Öppna aldrig ett batterie. Det finns inget underhållsbehov inuti batteriet. Byt alltid batterier till motsvarande batterier av samma tillverkare eller till batterier av tillverkaren rekommenderad typ. Ladda aldrig upp batterier som inte är återuppladdningsbara.

Inläggning av batterier
Öppna batteriluckan på baksidan av enheten, tryck tungan lätt nedåt och lägg in de båda alkaliska AAbatterierna. Se noga till att polariteten på batterierna blir densamma som på bilden i batterifacket (+=+) och stäng sedan luckan försiktigt. – Var noga med att alltid använda batterier som är i fullgott skick.

Anpassning till en apparat, t.ex. TV eller en DVB Set Top Box (eller satellitmottagare/ kabeltuner) eller en annan apparat
För att du ska kunna styra dina apparater måste du ställa in fjärrkontrollen till dina apparater, så att den passar till resp. apparat. För detta ändamål har du två möjligheter, antingen direkt inmatning av styrkod eller kodsökning.

Direktinmatning av styrkod med hjälp av den medlevererade listan
Exempel: fjärrkontrollen ska ställas in för en Panasonic TV:
1. Av den bifogade listan framgår t ex koden för Panasonic, 0158. Var vänlig se till, att du valt korrekt lista för TV:n.
2. För att förbereda kodinmatningen ska du kort trycka på knapparna SET (inställning) och sedan på önskad apparatknapp, t.ex. TV tills den lilla lampan (lysdioden) lyser permanent.
3. Tryck därefter i tur och ordning på sifferknapparna (i exemplet: 0158).
4. Efter det att du matat in siffrorna slocknar lampan och fjärrkontrollen är redo för att styra din apparat.
5. Som avslutning ska du testa den inställda styrkoden genom att försöka manövrera din TV-apparat. Fungerar det inte eller inte riktigt, så får du försöka någon annan kod ur listan eller kodsökningen, se nedan. Inställningen av fjärrkontrollen för styrning av ytterligare apparater går till på samma sätt. Avsluta kodinmatningen genom att trycka på valfri apparatknapp.

Sökning av styrkod
Om du inte hittar ditt fabrikat i kodlistan eller ingen passande kod kan du utnyttja kodsökningen.

Manuell kodsökning
1. Slå på den apparat du avser att styra, så att programmet går.
2. För att förbereda kodsökningen ska du kort trycka på knapparna SET (inställning) och sedan på en apparatknapp t.ex. TV till dess att den lilla lampan (lysdioden) lyser med fastsken.
3. Rikta sedan fjärrkontrollen mot den apparat du vill styra.
4. Tryck en gång kort på SET. Den lilla lampan på knappen (lysdioden) blinkar.
5. Tryck upprepade gånger, ungefär en gång i sekunden, på TILL/FRÅN ⏻ (apparatströmbrytaren) till dess apparaten, t ex din TV, reagerar. -Om apparaten har slagit från, så måste du åter slå på den för vidare sökning, t ex på själva apparaten eller med originalfjärrkontrollen. – Om apparaten reagerar, så kan du också prova med andra knappar, utan att avbryta sökningen. Om några knappar inte fungerar på rätt sätt, söker du helt enkelt vidare via TILL/ FRÅN. Om din apparat reagerar korrekt, ska du trycka på apparatknappen t ex TV för att därigenom avsluta sökningen. Den lilla lampan (lysdioden) slocknar. - UR Z2 fortsätter sökningen där, som du avbrutit sökningen. När kodlistans slut nåtts, slutar sökningen automatiskt. Då blinkar fjärrkontrollen flera gånger snabbt efter varandra. Den senast inmatade koden sparas. Sökningen avbryts efter ca 30 sekunder utan aktivitet.

Automatisk kodsökning
På några få sekunder finner din fjärrkontroll nästan automatiskt de rätta styrinställningarna:
1. Slå på apparaten, t.ex. visas ett programnummer på satmottagarens display.
2. För att starta kodsökningen ska du trycka kort på den aktuella apparatknappen t.ex. TV
3. Håll sedan SET-knappen intryckt längre än 5 sekunder till dess den lilla lampan (lysdioden) blinkar.
4. Rikta sedan fjärrkontrollen mot den apparat, som ska styras, under det att fjärrkontrollen varje sekund sänder en annan TILL/ FRÅN-signal.
5. När din apparat reagerar, kan du trycka på vilken knapp som helst utom SET inom 1-2 sekunder, för att avsluta kodsökningen. – Om du har avslutat sökningen alltför sent, sök då vidare med den manuella kodsökningen, se ovan.
6. Som avslutning ska du testa den inställda styrkoden, genom att du t.ex. försöker styra din TV-apparat. Om den inte fungerar korrekt, så ska du försöka med kodsökningen en gång till. Den startar då med nästföljande kod. – Glöm inte att åter slå på din apparat t ex DVB, om du vill fortsätta sökningen.

Kodidentifikation
Du kan när som helst avläsa de inställda 4-siffriga koderna:
1. Tryck på knappen för den apparat, vars kod du vill avläsa, t ex TV.
2. Tryck på SET-knappen och 1,2,3,4, oberoende av vilken del av koden du vill läsa. Om du till exempel trycker på SET+1 efter TV och koden 0158 (i detta exempel) ställs in, kommer inte lampan att lysa efter aktivering av SET+1, för att visa att tusentalssiffran är 0. Du behöver bara räkna. Efter att SET+2 aktiverats, kommer lampan att blinka en gång för att visa att hundratalssiffran är 1. Tiotalssiffran visas efter aktivering av SET+3, dvs. den blinkar 5 gånger för siffran 5 i 158. Entalssiffran visas efter aktivering av SET+4, dvs. den blinkar 8 gånger för siffran 8 i 0158. - *För senare bruk ber vi dig på dekalen i batterifacket notera de funna, använda styrkoderna.*

Manövrering
Du manövrerar dina apparater praktiskt taget precis på samma sätt som du är van med från originalfjärrkontrollerna, du måste bara först trycka på en apparatknapp, TV, DVB osv., för att kunna styra önskad apparat. Eftersom symbolerna på UR Z2 kan skilja sig från symbolerna på originalfjärrkontrollerna, så måste du i vissa fall prova dig fram med alla knappar för att hitta funktionerna.

Volym-Punch-Through-funktion
När vissa styrkoder, t ex för DVB inte omfattar några volym- och ljudbortkopplingsfunktioner utnyttjar UR Z2 automatiskt motsvarande funktioner hos TV-apparaten.

Tekniska data
Batterier: 2 x AA-batterier
Räckvidd: max 7 m
Inte för 400 kHz-utrustning

CE

Vivanco GmbH förklarar härmed att UR Z2 är i enlighet med de grundläggande kraven och andra relevanta bestämmelser i direktiv 2004/108/EG .

Skrotning av enheten
Denna enhet får inte slängas i hushållssoporna. Vänligen lämna på en uppsamlingsplats för elektriska apparater i din kommun eller i ditt grannskap. Detta säkerställer att de förbrukade instrumenten tillvaratas såväl fackmannamässigt som ekonomiskt och att potentiella faror för miljön eller hälsorosker undviks.

Pb/Cd/Hg

Batterier
Batterier får inte kastas i hushållsavfallet. De måste enligt lag återlämnas, så att de förstörs på ett miljömässigt sätt. Lämna gamla batterier till ett kommunalt uppsamlingsställe eller till återförsäljaren. Batterier får bara lämnas i urladdat tillstånd.

Förpackningen kan återanvändas. Vänligen kasta inte förpackningen i hushållssoporna och följ alla lokala skrotningsbestämmelser.

21
PAP

RUS

Universal TV- DVB Controller UR Z2



Перед началом использования Вашего прибора пожалуйста внимательно прочтите указания по технике безопасности и это руководство по эксплуатации. Только в этом случае Вы сможете уверенно и надежно пользоваться всеми его функциями. Сохраните руководство по эксплуатации и передайте возможному следующему владельцу.

Применение по назначению
Оригинальное дистанционное управление приборов бытовой электроники может заменяться универсальным дистанционным управлением и т.д.. Прибор предназначен для применения исключительно в домашних условиях и не должен использоваться в производственных целях. Любое другое применение считается применением не по назначению и может привести к материальному ущербу или телесным повреждениям. Изготовитель не несет ответственности за ущерб, возникший в результате применения прибора не по назначению.



Безопасность батареи
Соблюдайте, пожалуйста, данное руководство и будьте внимательными. Пренебрежение к данным указаниям может привести к травмам/повреждениям и при особых обстоятельствах даже к взрыву батареи. Избегайте замыкания контактов, не храните совместно с металлическими предметами. Не бросайте батарею в огонь. Никогда не открывайте батарею. Внутри не содержатся части для обслуживания. Использовать батареи только одного и того же типа или рекоммендованные от производителя аналогичные модели. Никогда не заряжайте батареи.

Вложить батареи
На задней стороне прибора открыть крышку гнезда для батарей, слегка надавить накладку вниз и вложить 2 щелочных AA-элемента. Обратите внимание на указанную полярность на батареях и в гнезде (+=+) и снова осторожно закройте гнездо. – Обратите пожалуйста внимание на то, чтобы всегда использовались только безупречные батареи.

Согласование с устройством, например, телевизором или DVB Set Top Box (или сателлитовым приёмником/ кабельным блоком настройки) или другим устройством
Чтобы управлять Вашими приборами, Вам необходимо настроить пульт для дистанционного управления устройствами. Для этого имеются две возможности: непосредственный ввод управляющего кода или поиск кода.

Непосредственный ввод управляющего кода с помощью приложенного перечня
Пример: Настройка дистанционного управления телевизора фирмы Панасоник.
1. Из приложенного перечня выберите, напр., Панасоник-код 0158. Пожалуйста, убедитесь, что вы выбрали правильный список – телевизоры (TV).
2. Для подготовки ввода кода нажмите коротко на клавиши SET (настройка), а затем дополнительно на клавишу TV, пока лампочка (СИД) начнёт светиться непрерывным светом.
3. После этого нажимайте последовательно цифровые кнопки (напр., 0158).
4. После ввода цифр погасает лампочка и устройство истанционного управления готово к управлению Вашим прибором.
5. В заключении проверьте настроенный управляющий код, напр., тем, что Вы попробуете управлять Вашим телевизором. Если это не функционирует или функционирует неправильно, то попробуйте использовать дальнейший код из перечня или поиск кода, смотри ниже. Настройка пульта для дистанционного управления другими устройствами происходит таким же образом. Вы закончите ввод кода нажатием любой приборной кнопки.

Пойск кода управления
Если Вы не найдёте Вашу марку в перечне кодов или не найдёте подходящий код, то используйте пойск кода.

Ручной поиск кода
1. Включите пожалуйста управляемый прибор. Программа выполняется.
2. Для подготовки поиска кода нажмите коротко клавиши SET (настройка), внизу справа, и затем требуемую кнопку прибора, например, TV, пока лампочка (СИД) начнет светиться непрерывно.
3. Затем направьте устройство дистанционного управления на управляемый прибор.
4. Один раз коротко нажать SET. Лампочка (СИД) мигает.
5. Повторно, приблизительно каждую секунду, нажмите ВКЛ./ОТКЛ. ⏻ (вкл./откл. прибор) до того, как Ваш прибор, например Ваш телевизор, не реагирует. Если прибор выключился, то Вам необходимо, напр., на приборе или с помощью оригинального устройства дистанционного управления снова включить его для дальнейшего поиска. – Если Ваш прибор реагирует, то Вы можете испробовать также и другие клавиши без оставления поиска. Если некоторые клавиши не функционируют правильно, то поиск следует просто продолжать с помощью ВКЛ/ОТКЛ. Если Ваш прибор реагирует правильно, то нажмите приборные напр., клавиши TV и прекратите таким образом поиск. Лампочка (LED) гаснет. - UR Z2 продолжает поиск там, где Вы прервали его.

Когда достигнут конец списка кодов, поиск автоматически прекращается. При этом ДУ несколько раз быстро мигает. Последний введенный код сохраняется. Через прибл. 30 сек. бездействия поиск прерывается.

Автоматический поиск кода
За несколько секунд Ваше дистанционное управление почти автоматически найдёт правильные управляющие настройки:
1. Включите пожалуйста управляемый прибор. Например, на индикаторе Вашего DVB появляется номер программы.
2. Для запуска пойска кода коротко нажмите желаемую приборную кнопку, например TV.
3. Затем нажмите кнопку SET более 5 секунд, пока лампочка (LED) не мигает.
4. Затем направьте устройство дистанционного управления на управляемый прибор, в то время как дистанционное управление каждую секунду посылает другой сигнал ВКЛ/ОТКЛ.
5. Если Ваш прибор реагирует, то нажмите в течение 1-2 секунд SET, чтобы закончить поиск кода.
– Если Вы слишком поздно закончили поиск, то продолжайте пожалуйста дальше искать с помощью ручного поиска кода, смотри выше. 6 В заключении проверьте настроенный управляющий код, попробовав управлять, например, Вашим TV-прибором. Если он функционирует неправильно, то попробуйте поиск кода пожалуйста ещё один раз. Поиск кода запускается в таком случае следующим кодом. – Не забудьте Ваш прибор например TV, если Вы желаете продолжить поиск, повторно включить на приборе.

Идентификация кода
Вы можете в любое время прочитать установленный четырехзначное код:
1. Нажать клавишу на приборе, на котором Вы желаете прочитать код, напр., TV.
2. Нажмите кнопку SET и 1,2,3,4 независимо от того, какую часть кода Вы хотели бы считывать. К примеру, в случае нажатия комбинации SET+1 после TV и установления кода 0158 (как в данном примере), лампочка не загорается после прекращения ввода SET+1, что показывает, что число тысячных здесь равняется 0. Далее Вам необходимо лишь считать. После ввода SET+2 лампочка мигает один раз, что означает, что число сотых равняется 1. Вводом комбинации SET+3 определяется число десятков, например, лампочка мигает 5 раз для кода 0158. Число единиц задается вводом комбинации SET+4, например, лампочка мигает 8 раз для кода 0158. *Запишите найдённые и использованные коды управления на наклейку ящика батарей для их использования в более поздный срок.*

Управление
Вы управляете практически Вашими приборами также, как Вам это знакомо с Ваших оригинальных устройств дистанционного управления. Разница заключается лишь в том, что Вам прийдётся, чтобы управлять желаемым прибором, нажать прежде всего на клавишу TV, DVB и.т.д. Так как символы устройства UR Z2 отличаются от символов Ваших оригинальных устройств дистанционного управления, возможно, что для нахождения функции Вам потребуется испробовать все клавиши.

Громкость-Punch-Through-функция
В случае, если определённые коды управления, например, DVB, не содержат функций громкости или выключения звукового сигнала, то UR Z2 автоматически использует соответствующие функции телевизора.

Технические данные
Батареи: 2x AA-элемента
Радиус действия: макс. 7 м
Не используется для приборов 400 кГц.

CE

Настоящим Vivanco GmbH заявляет, что UR Z2 соответствует основным требованиям и другим значимым предписаниям Директивы 2004/108/EC.

Утилизация прибора
Этот прибор нельзя утилизировать вместе с бытовым мусором. Пожалуйста, сдайте его в приемный пункт электроприборов Вашей общины или Вашего района города. Благодаря этому б/у приборы будут переработаны квалифицированно и рентабельно и это поможет избежать потенциальной опасности для окружающей среды и здоровья людей.

Pb/Cd/Hg

Батарея и аккумуляторы
Батареи и аккумуляторы запрещено выбрасывать в домашний мусор. Вы обязаны сдавать старые батареи с специальные точки приема или магазины. Этим вы соблюдайте правовое обязательство и сохраняйте природу. Сдавать батареи и аккумуляторы только в разряженном состоянии.

21
PAP

Упаковку необходимо утилизировать для повторного использования. Пожалуйста, не выбрасывайте ее в бытовой мусор и действуйте в соответствии с местными предписаниями по утилизации.

TV Code

A.R.Systems	0005 0059 0069 0080 0099
Accent	0001 0005 0099
Acer	0278
Adcom	0112
ADL	0218
Admiral	0014 0015 0022 0044 0065 0075
Advent	0189
Adyson	0034 0035
AEA	0005 0099
AEG	0105 0205 0243
Agashi	0034 0035 0044
AGB	0093
Agef	0014
Aiko	0001 0003 0005 0010 0027 0034 0035 0044 0063 0068 0076 0099
Aim	0005 0010 0033 0069 0074 0080 0099 0105 0117 0132 0136 0154 0170
Aiwa	0133 0135 0240 0254 0256 0258
Akai	0001 0003 0005 0010 0022 0026 0027 0033 0034 0035 0036 0044 0063 0068 0070 0076 0081 0082 0093 0095 0099 0104 0105 0115 0120 0127 0130 0138 0139 0145 0151 0154 0176 0198 0226 0228 0242 0256 0260
Akashi	0001 0186
Akiba	0005 0036 0045 0080 0099
Akira	0075 0154
Akito	0005 0099
Akura	0001 0005 0036 0044 0045 0074 0099 0126 0138 0264
Alaron	0034
Alba	0001 0004 0005 0022 0034 0036 0041 0061 0067 0068 0075 0078 0086 0099 0126 0138 0198 0254 0256 0260
Alkos	0003
All-Tel	0188 0231
Allorgan	0032 0035
Allstar	0005 0099
Ambassador	0025
Amplivision	0035 0050 0067
Amstrad	0001 0005 0025 0036 0044 0064 0068 0074 0076 0093 0099 0120 0198 0254 0264
Anam	0001 0005 0099 0121
Anam National	0005 0099 0121
Andersson	0204 0205
Anglo	0001 0044
Anitech	0001 0005 0016 0044 0045 0099
Ansonic	0001 0005 0016 0017 0022 0041 0043 0049 0067 0069 0073 0099 0126 0238 0254
Apollo	0081
Arc en Ciel	0020
Arcam	0034 0035
Ardem	0005 0085 0099 0116 0138
Arena	0005
Aristona	0005 0099
Arthur Martin	0022
ASA	0009 0014 0017 0018 0022 0057
Asberg	0005 0016 0099
Asora	0001
Astra	0005 0099
Asuka	0034 0035 0036 0044 0045 0254
ATD	0132
Atlantic	0005 0032 0034 0043 0050 0099
Atori	0001
Auchan	0022
Audiosonic	0001 0005 0020 0035 0036 0044 0055 0067 0069 0085 0099 0138 0139 0173 0234
Audioton	0035 0044 0067 0085
Audioworld	0132
Aumark	0008
Autovox	0014 0032 0035 0041 0058
AVP	0254 0256
AWA	0001 0002 0004 0005 0019 0021 0034 0035 0044 0069 0074 0099 0105 0280
Axxent	0001
Axxon	0138
B&D	0218
Baier	0189
Baird	0010 0011 0020 0028 0033 0035 0105 0211 0254 0288
Bang & Olufsen	0014 0101
Barco	0022
Baseline	0223
Basic Line	0001 0005 0022 0035 0036 0045 0056 0069 0080 0099 0126 0198
Bastide	0035
Bauer	0001 0044 0132 0136 0170
Baur	0001 0005 0027 0030 0058 0063 0092 0094 0097 0099 0197 0240
Bazin	0035
Beaumark	0026
Beko	0003 0005 0067 0075 0085 0099 0105 0138 0139 0171 0198
Belson	0209 0287
Bennett	0005 0099
BenQ	0251
Beon	0005 0075 0099
Berthen	0126
Best	0055 0067
Bestar	0005 0067 0069 0099
Bestar-Daewoo	0069
Binatone	0035
Black Diamond	0099 0108 0154 0173 0174 0198 0205
Black Panther	0016
Black Star	0041

Black Strip	0003
Blackway	0036 0045
Blaupunkt	0004 0024 0027 0030 0031 0052 0053 0080 0094 0097
Blue Sky	0005 0036 0045 0071 0080 0086 0099 0126 0138 0139 0198 0209 0254 0256 0257 0259 0279
Blue Star	0045
Bondstec	0041
Boots	0001 0035
Bosch	0050 0052
Boxlight	0148
BPL	0005 0045 0099
Brandt	0020 0046 0054 0100 0112 0288
Brandt Electronique	0046 0054
Brinkmann	0005 0075 0085 0099 0126 0271
Brionvega	0005 0014 0064 0099
Britannia	0034 0035
Brother	0044
Bruns	0014 0085
BSR	0022 0063
BTC	0036
Bush	0001 0004 0005 0022 0033 0035 0036 0044 0045 0058 0061 0063 0065 0068 0069 0086 0099 0108 0109 0124 0126 0132 0138 0162 0198 0222 0226 0228 0243 0252 0254 0256 0258 0264 0271
Canton	0036
Capsonic	0044
Carad	0005 0099 0107 0126 0198
Carena	0005 0080 0099
Carrefour	0004 0005 0009 0099
Carver	0024
Cascade	0001 0005 0099
Casio	0005 0022 0058 0099 0254
Cathay	0005 0099
CCE	0005 0035 0099
Celestial	0172 0173 0174
Centrex	0164 0176 0195
Centrum	0198
Centurion	0005 0099
Century	0014 0038 0041
CGE	0012 0013 0022 0041 0067 0075 0254
Cimline	0001 0036
Cinex	0120 0243
Citizen	0008
City	0001
Clarivox	0005 0009 0016 0075 0099
Clatronic	0001 0005 0016 0035 0036 0041 0044 0050 0067 0068 0099 0105 0120 0138 0205 0254
Clayton	0198
CMS	0034
CMS hightec	0035
Cobolt	0195
Combitech	0256
Concorde	0001
Condor	0001 0005 0016 0022 0034 0041 0044 0045 0050 0067 0073 0075 0099
Conia	0155 0173 0174 0287
Conrac	0171
Conrad	0005 0099
Contec	0001 0002 0004 0005 0021 0034 0044 0099
Continental Edison	0020 0046 0086
Cosmel	0001 0005 0055 0099
Crosley	0012 0013 0014 0022 0041
Crown	0001 0005 0033 0056 0067 0075 0085 0086 0099 0105 0122 0127 0137 0138 0139
Crown	0198 0259
CS Electronics	0034 0036 0041
CTC	0041
CTX	0251
Curtis Mathes	0008 0015 0023
Cybertron	0036
Cytronix	0233
D-Vision	0005 0099 0264
D.Boss	0005
Daewoo	0001 0005 0034 0035 0036 0069 0090 0099 0117 0125 0132 0138 0182 0188 0189 0192 0201 0253 0256 0257 0277 0285
Dainichi	0034 0036
Daitsu	0274
Dansai	0001 0003 0004 0005 0033 0034 0035 0044 0099
Dansette	0074
Dantax	0067 0085 0105 0138 0139 0256
Datsura	0033
Davis	0148
Dawa	0001 0005 0099
Daytek	0044 0127 0132 0136 0212 0280
Dayton	0212
Daytron	0001 0005 0069 0099
de Graaf	0006 0022 0033 0065 0095
DEC	0166 0186 0195
Decca	0005 0010 0035 0093 0099 0111 0201 0254 0256
Deitron	0005 0036 0069 0099
Dell	0229
Denko	0044
Denver	0005 0099 0103 0105 0208 0265
DER	0028
Desmet	0001 0005 0014 0050 0099
Diamant	0005 0099
Diamond	0001 0034 0044 0068 0127 0130 0132 0169 0173 0175 0186 0196
DiBoss	0005 0099
Digatron	0005 0099
Digihome	0204
Digiline	0005 0018 0099 0126
DigiLogic	0005 0099
Digitex	0173
Digitor	0005 0099 0187 0193

Digivision	0063
DigiX	0192
DiK	0005 0099
Dixi	0001 0005 0014 0035 0041 0099
DL	0005 0103 0164 0183 0188 0195
Domeos	0005 0126 0198
Domland	0072
Doric	0058
Dream Vision	0248
DSE	0173 0187 0193 0243
DTS	0001
Dual	0005 0022 0035 0043 0058 0059 0072 0099 0115 0198 0201 0204 0254 0271 0288
Dumont	0009 0010 0014 0016 0017 0035 0254
Dunai	0022
Durabrand	0005 0099 0238 0265
Dux	0005 0099
DVX	0195
Dynatron	0005 0099
e:max	0105
Easy Living	0226 0228
Ecco	0132 0136 0161
Edison-Minerva	0086
Eiki	0147
Elbe	0005 0022 0027 0035 0036 0038 0043 0049 0063 0064 0067 0073 0077 0093 0099 0107 0114
Elbe-Sharp	0093
Elcit	0014 0016 0022 0041 0093
Elekta	0001 0005 0044 0045 0099
Elfunk	0198 0213
ELG	0005 0099
Elin	0001 0005 0017 0018 0022 0034 0058 0063 0095 0099
Elite	0005 0036 0050 0099
Elman	0016
Elta	0001 0034 0044
Emco	0041
Emerson	0005 0009 0014 0022 0025 0026 0041 0045 0050 0051 0063 0067 0068 0085 0099 0126 0138 0254 0257
Emperor	0045
Enzer	0130 0154 0161 0186
Epson	0181 0200
Erres	0005 0099
ESC	0005 0035 0099
Estèle	0022
Etron	0000 0001 0005 0022 0099 0173
Eurofeel	0035 0044
EuroLine	0005
Euroman	0005 0034 0035 0044 0067 0099
Europa	0005 0099
Europhon	0005 0016 0022 0034 0035 0041 0093 0099
Evesham	0226 0228
Evolution	0251
Excel	0005 0099
Expert	0022 0032 0043
Exquisit	0005 0041 0099
Fagor	0005 0099
FairTec	0209
Family Life	0005 0099
Fast	0286
Fenner	0001 0069
Ferguson	0003 0005 0011 0017 0019 0020 0028 0038 0046 0054 0063 0078 0095 0099 0100 0112 0122 0198 0254 0256 0258 0288
Fidelity	0005 0022 0028 0034 0044 0063 0065 0068 0074 0092 0099 0254 0255 0256
Filsai	0035
Finlandia	0007 0010 0022 0033 0046 0057 0063 0065 0095 0288
Finlux	0005 0009 0010 0014 0016 0017 0018 0022 0035 0057 0073 0075 0081 0082 0088
Finlux	0093 0099 0105 0111 0113 0115 0138 0139 0171 0243
Firstline	0001 0005 0010 0016 0022 0033 0034 0035 0038 0041 0051 0058 0063 0069 0073 0099 0126 0138 0171 0198 0205 0209 0234 0257 0279
Fisher	0004 0007 0010 0014 0017 0021 0022 0033 0035 0058 0063 0067 0098
Flint	0005 0010 0036 0044 0080 0099 0107
FNR	0016
Force	0204
Forgestone	0028
Formenti	0005 0014 0022 0034 0050 0085 0099
Formenti-Phoenix	0034 0050
Fortress	0014 0015
Fraba	0005 0067 0099
Friac	0001 0005 0016 0067 0090 0099 0107 0123
Frontech	0001 0022 0035 0041 0044 0058 0065
Fuchsware	0164
Fujimaro	0188
Fujitsu	0001 0005 0010 0016 0022 0032 0035 0043 0059 0063 0099 0129 0185
Fujitsu General	0001 0022 0032 0035
Fujitsu Siemens	0171 0205 0226 0228 0233
Funai	0044 0074 0126 0240 0254
Furichi	0186
Futronic	0044 0132 0166 0186
Future	0005 0099
Galaxi	0005 0016 0022 0063 0099
Galaxis	0005 0016 0067 0075 0099
Galeria	0001
GBC	0001 0022 0036 0065 0069
GE	0020 0026 0045 0046 0054 0100 0112 0147 0288

Geant Casino	0022
GEC	0005 0010 0022 0035 0058 0063 0093 0099
Geloso	0001 0022 0041 0065 0069
General	0020 0046
General Electric	0046 0288
General Technic	0001
Genesis	0001 0005 0099
Genexxa	0001 0005 0022 0036 0074 0099
Gericom	0188 0192 0218 0233
Giant	0001 0035
Goldfunk	0126
Goldhand	0034
Goldline	0055
GoldStar	0000 0001 0005 0006 0020 0022 0026 0034 0035 0041 0047 0063 0065 0067 0070 0080 0099 0105 0138 0139 0259
Gooding	0086
Goodmans	0001 0003 0004 0005 0010 0035 0036 0044 0047 0054 0068 0069 0082 0086 0090 0093 0099 0100 0103 0114 0117 0125 0126 0138 0192
Goodmans	0198 0204 0226 0228 0234 0252 0256 0257 0280 0288
Gorenje	0067
GPM	0036
Gradiente	0005 0024 0099 0122
Graetz	0005 0022 0056 0063 0068 0099 0138 0205
Gran Prix	0120
Granada	0004 0005 0007 0010 0019
Granada	0022 0033 0035 0037 0054 0056 0062 0065 0081 0093 0095 0099 0100 0266 0288
Grandin	0001 0005 0022 0036 0045 0050 0069 0080 0099 0107 0126 0138 0139 0188 0192 0198 0209
Gronic	0016 0022 0035
Grundig	0001 0004 0005 0009 0026 0027 0030 0067 0078 0086 0091 0094 0097 0099 0103 0114 0136 0149 0219 0234 0256 0258 0260 0280 0288
Grunkel	0205
H&B	0171
Haaz	0136
Haier	0044 0132
Halifax	0034 0035 0044
Hammerstein	0008 0044
Hampton	0034 0035
Hanimex	0036 0256
Hanseatic	0001 0005 0014 0022 0035 0045 0049 0050 0058 0063 0067 0070 0072 0085 0090 0099 0112 0117 0125 0138 0171 0271
Hantarex	0001 0005 0016 0038 0093 0099 0177
Hantor	0005 0099
Harsper	0188
Harwa	0161 0211 0231
Harwood	0001 0005 0074 0086 0099
Hauppauge	0005 0099
HCM	0001 0005 0035 0036 0044 0045 0074 0075 0099
Hedzon	0005 0099
Hema	0001 0035
Hemmermann	0058
Hicon	0233 0282
Hifivox	0020
Higashi	0034
Highline	0005 0044 0099
Hikona	0036
Hinari	0001 0004 0005 0022 0033 0036 0044 0061 0078 0086 0099 0256
Hisawa	0036 0045 0080 0107 0138 0256
HISense	0033 0132 0183 0186 0195 0279
Hit	0014
Hitachi	0004 0005 0006 0010 0018 0019 0020 0022 0026 0029 0035 0037 0058 0062 0063 0065 0081 0082 0083 0088 0090 0092 0093 0095 0099 0102 0113 0117 0141 0150 0167 0190 0198 0199 0201 0204 0220 0239 0244 0254
Hitachi	0288
Hitachi Fujian	0005 0019 0037 0099
Hitsu	0001 0036 0080 0107
HMV	0014 0028
Höher	0138 0188 0205 0243
Home Electronics	0105
Hornyphon	0005 0099
Hoshai	0036 0045 0080
Huanyu	0034 0069
Hugoson	0194 0218
Hygashi	0034 0035
Hyper	0001 0034 0035 0041
Hypersonic	0063
Hypson	0005 0035 0038 0044 0045 0080 0085 0099 0126 0138 0139 0198 0256 0259
Hyundai	0005 0099 0132 0136 0154 0186 0188 0189 0223 0243 0285
Iberia	0005 0099
ICE	0001 0005 0034 0035 0036 0044 0068 0099
Ices	0034 0036
Ict	0005 0099 0201
iDEAL	0243
IISonic	0280
Iiyama	0194 0218
Imperial	0005 0012 0013 0022 0041 0058 0063 0067 0073 0075 0099 0114
Indiana	0005 0099
InFocus	0148 0153 0206
Ingelen	0022 0063 0086 0107 0138 0236

Ingersoll	0001
Inno Hit	0001 0005 0010 0016 0035 0036 0041 0045 0047 0093 0099 0205
Innovation	0005 0099 0271
Innowert	0188 0233
Inotech	0161 0173
Interactive	0005 0014 0020 0022 0052 0063 0067 0073 0092 0099
Interbuy	0001 0005 0041 0044 0092 0099
Interfunk	0005 0014 0020 0022 0031 0041 0052 0058 0063 0067 0092 0099
Internal	0005 0069 0090 0099 0257
International	0034
Intervision	0001 0005 0016 0022 0035 0036 0041 0044 0045 0067 0068 0070 0072 0080 0085 0086 0099 0271
Irradio	0001 0005 0036 0041 0047 0068 0099
Isukai	0005 0036 0045 0080 0099
ITC	0035 0050
ITS	0001 0005 0034 0036 0044 0045 0068 0099
ITT	0022 0028 0033 0056 0057 0058 0063 0081 0082 0095 0107
ITT Nokia	0022 0033 0056 0057 0058 0063 0065 0081 0082 0095 0105 0107 0115
ITV	0005 0041 0044 0069 0099
JDV	0264
Jean	0004
JEC	0003
Jinxing	0183
JMB	0005 0069 0078 0090 0099 0117 0256
JNC	0189
Jocel	0137
Jubilee	0099
JVC	0004 0015 0028 0036 0068 0075 0105 0121 0122 0129 0146
Kaisui	0001 0005 0034 0035 0036 0045 0080 0099
Kambrook	0070
Kamp	0034
Kapsch	0017 0022 0032 0043 0063
Karcher	0005 0016 0022 0044 0045 0067 0099 0105 0107 0138 0162 0243
Kathrein	0005 0099
Kawa	0068
Kawasho	0034
KB Aristocrat	0022
Kendo	0005 0016 0022 0041 0064 0065 0067 0070 0073 0085 0092 0099 0107 0120 0198 0238 0271
Kennedy	0022 0032 0077
Kennex	0198
Kenwood	0018
KIC	0035
Kingsley	0034
Kiota	0000
Kiss	0168
Kiton	0005 0099 0126
Kneissel	0005 0038 0043 0049 0064 0067 0069 0073 0077 0090 0099 0107 0256
Kobra	0233 0282
Kolster	0005 0016 0036 0041 0058 0099
Konichi	0001
Konka	0005 0036 0068 0075 0099 0103 0105 0118 0138 0155 0163 0287
Kontakt	0086
Korpel	0005 0099
Korting	0014 0050 0067
Kosmos	0005 0099
Kotron	0044 0074 0252
Koyoda	0001
Kraking	0038
KTV	0035
Kuba	0022 0058
Kuba Electronic	0022 0058
Kyoshu	0044 0074 0075
Kyoto	0022 0034 0035
L&S Electronic	0138 0188
LaSAT	0067 0085
Lavis	0005 0022 0198
Leader	0001
Lecson	0005 0099
Legend	0001 0055
Lemair	0073
Lenco	0001 0005 0017 0022 0069 0099 0103 0142 0198
Lenoir	0001
Lesa	0041
Levis Austria	0005 0099
Lexsor	0211
Leyco	0005 0010 0044 0099
LG	0000 0001 0005 0020 0022 0026 0034 0035 0041 0047 0063 0067 0070 0099 0105 0132 0138 0139 0177 0202 0203 0209 0245 0259 0273 0276 0281
Liesenk & Tter	0005 0052 0099
Liesenkotter	0005 0052 0053 0099
Lifetec	0001 0005 0036 0044 0069 0092 0099 0123 0126 0129
Lifetec	0138 0198 0201 0226 0228 0271
Lloyd's	0000 0001
Lodos	0198 0204
Loewe	0005 0014 0049 0064 0067 0092 0093 0099 0116 0123 0165
Logik	0000 0001 0002 0008 0028 0044 0068 0132 0161 0192 0198 0218
Logix	0126 0271
Luker	0264

Luma	0001 0005 0022 0032 0043 0064 0065 0069 0073 0099
Lumatron	0005 0011 0022 0032 0035 0043 0044 0063 0064 0065 0099
Lux May	0001 0005 0099
Luxor	0022 0029 0033 0035 0047 0057 0058 0062 0063 0065 0081 0082 0088 0095 0115 0198 0205
LXI	0026
Madison	0005 0099
Magnadyne	0014 0016 0022 0041 0093
Magnafon	0011 0016 0034 0093
Magnavox	0004 0005 0099 0164
Magnum	0005 0099 0120 0138 0139 0232
Mandor	0044
Manesth	0003 0005 0022 0035 0044 0050 0065 0099
Manhattan	0005 0022 0099 0126 0162 0189 0198 0274
Maqma	0233
Marantz	0005 0074 0099
Marelli	0014
Mark	0001 0005 0034 0035 0069 0099 0138 0139
Master's	0005 0090
Masuda	0001 0005 0035 0036 0044 0068 0099
Matsui	0001 0002 0003 0004 0005 0007 0010 0022 0025 0027 0030 0033 0035 0054 0058 0061 0065 0068 0076 0078 0080 0086 0093 0099 0113 0138 0192 0198 0256 0258
Matsushita	0121
Matsuviama	0103
Maxam	0044
Maxdorf	0044 0132 0136 0161
Maxim	0243 0264
MCE	0001
Mediator	0005 0099
Medion	0005 0040 0092 0099 0126 0132 0138 0171 0192 0198 0201 0226 0228 0232 0238 0243 0252 0256 0258 0271
Megas	0107
Megatron	0026
MEI	0005 0099 0198
Melectronic	0001 0005 0017 0018 0020 0022 0027 0030 0034 0035 0041 0046 0057 0058 0063 0069 0073 0082 0088 0092 0099 0117 0125 0138 0288
Memorex	0001 0008 0026 0121
Memphis	0001 0010 0055
Mercury	0000 0001 0005 0008 0044 0099 0132 0136
Merritt	0022 0063 0095
Metz	0005 0014 0027 0030 0066 0071 0079 0094 0097 0099 0103 0126 0152 0198 0205 0284
MGA	0005 0017 0026 0036 0099
Micromaxx	0005 0099 0114 0126 0171 0198
Microstar	0171
MicroTEK	0044 0132 0136 0173 0186
Mikomi	0150 0204 0205
Minato	0005 0099
Minerva	0009 0019 0027 0030 0086 0093 0094 0097
Minoka	0005 0074 0099
Mirror	0252
Mistral Electronics	0028
Mitsubishi	0004 0005 0014 0015 0019 0026 0028 0033 0092 0094 0099 0198 0259
Mitsuri General	0022
Mivar	0034 0035 0047 0048 0049 0067 0093 0106
Monaco	0001
Morgan's	0005 0099
Motorola	0015
MTC	0008 0022 0034 0058 0063 0067 0092
MTlogic	0138 0234
Multibroadcast	0028
Multistandard	0016
Multitec	0005 0085 0099 0126 0198 0243 0264
Multitech	0001 0005 0016 0034 0035 0041 0044 0058 0065 0067 0085 0099
Murphy	0010 0017 0022 0034
Musikland	0005 0036 0041 0099
MyCom	0026 0280
Myryad	0005 0099
NAD	0022 0023 0026 0063
Naiko	0005 0099 0105 0264
Nakimura	0005 0069 0099
Naonis	0022 0065
Narita	0264
NAT	0266
National	0266
NEC	0001 0002 0004 0005 0024 0035 0040 0050 0069 0080 0090 0099 0103 0122 0125 0134 0135 0158 0207 0248 0275
Neckermann	0005 0014 0022 0027 0031 0035 0041 0050 0052 0058 0065 0067 0073 0075 0092 0097 0099 0197 0240
NEI	0005 0022 0055 0068 0099 0198
Neovia	0188 0189 0280
Nesco	0041 0256
Netsat	0005 0099
Network	0055
Neufunk	0001 0005 0036 0045 0099 0107 0138
New Tech	0001 0005 0035 0099 0288
New World	0036
Nfren	0168
Nicamagic	0034

Nikkai	0001 0003 0004 0005 0010 0034 0035 0036 0044 0055 0099
Nikkei	0138
Nikko	0026
Nishi	0084
Nobliko	0009 0016 0034
Nogamatic	0020
Nokia	0020 0022 0033 0050 0056 0057 0058 0063 0069 0081 0082 0088 0095 0105 0107 0115
Nordic	0035
Nordmende	0005 0020 0022 0030 0046 0054 0099 0100 0138 0195 0226 0228 0288
Norfolk	0022
Normerel	0005 0099
Novak	0005 0099
Novatronic	0005 0018 0069 0099
NU-TEC	0005 0080 0099 0132 0173 0174
Nurnberg	0022 0063
Oceanic	0022 0033 0063 0081 0082 0095
Odeon	0044
Okano	0001 0005 0010 0044 0067 0099
Olidata	0280
Omega	0044
Omni	0132 0164 0176 0195
Onn	0204
Onwa	0016 0036 0068 0076 0104
Opera	0005 0099
Optimus	0023 0121
Optonica	0015
Orbit	0005 0099
Orcom	0283
Orion	0001 0002 0005 0016 0025 0044 0050 0051 0061 0074 0078 0093 0099 0123 0138 0192 0211 0256 0258 0263
Orline	0005 0036 0099
Ormond	0005 0099 0126 0198
Orsowe	0093
Osaki	0005 0010 0035 0036 0044 0061 0069 0074 0099
Osio	0005 0047 0099
Oso	0036
Osume	0004 0005 0010 0021 0036 0099
Otto Versand	0001 0004 0005 0015 0020
Otto Versand	0027 0030 0035 0041 0045 0050 0058 0063 0092 0093 0094 0097 0099 0197 0240 0266 0271 0288
Pace	0256
Pacific	0005 0078 0099 0138 0198 0201
Pael	0034
Palladium	0005 0014 0022 0027 0031 0035 0041 0052 0058 0065 0067 0073 0075 0097 0099 0114 0123 0138 0197 0201 0240 0271
Palsonic	0000 0005 0035 0036 0044 0070 0074 0075 0099 0132 0161 0162 0163 0211 0231
Panama	0001 0005 0034 0035 0041
Panama	0044 0099
Panashiba	0000
Panasonic	0000 0005 0008 0019 0022 0026 0063 0066 0093 0095 0099 0121 0158 0184 0185 0235 0246 0266
Panavision	0005 0073 0099
Papouw	0005 0099
Pathe Cinema	0022 0034 0038 0049 0050 0058 0067
Pathe Marconi	0020
Pausa	0001
Perdio	0005 0010 0022 0034 0045 0050 0099
Perfekt	0005 0099
Philco	0005 0012 0013 0014 0022 0041 0067 0075 0099 0271
Philex	0022 0028
Philharmonic	0035
Philips	0001 0005 0014 0027 0028 0031 0038 0063 0069 0097 0099 0142 0143 0159 0224 0251 0288
Phocus	0138 0232 0234
Phoenix	0005 0010 0014 0022 0034 0050 0067 0085 0099
Phonola	0005 0014 0028 0034 0099
Pilot	0005 0099 0136 0137
Pioneer	0002 0005 0020 0022 0023 0024 0046 0063 0067 0085 0092 0099 0128 0157 0272 0288
Pionier	0067 0085
Plantron	0001 0005 0044 0099
Playsonic	0005 0035 0056 0138 0139
Plus	0158
Policom	0009 0012 0013 0014 0016 0020 0022 0032 0038 0063 0094
Polytron	0045 0131
Poppy	0001
Portland	0069 0257
Powerpoint	0005 0086 0099 0132
Prandoni-Prince	0041 0063 0065 0093
Precision	0035
Premier	0001 0044
President	0186
Prima	0001 0044 0074 0231
Princess	0132
Prinston	0198
Prinz	0010 0029 0058 0063
Profex	0001 0022 0063 0065
Profi	0001
Profitronic	0005 0016 0099

Proline	0005 0010 0051 0073 0099 0111 0112 0114 0117 0280
Prosonic	0005 0034 0035 0067 0068 0069 0099 0126 0138
Protech	0001 0005 0016 0022 0035 0041 0044 0045 0055 0058 0075 0085 0099 0126 0198
Proton	0026
Provision	0005 0090 0099 0138 0198
PVision	0189 0209
Pye	0005 0014 0058 0069 0074 0097 0099
Pymi	0001
Quadral	0036
Quasar	0001 0041 0121 0188
Quelle	0001 0002 0004 0005 0009 0012 0013 0017 0018 0020 0021 0027 0030 0031 0035 0041 0044 0047 0052 0053 0057 0058 0063 0092 0094 0097 0099 0126 0197 0198 0240
Questa	0004
R-Line	0005 0022 0099
Radialva	0005 0020 0022 0036 0046 0055 0099
Radiola	0005 0035 0099
Radiomarelli	0005 0014 0041 0093 0099
RadioShack	0005 0026 0099
Radiotone	0001 0005 0044 0067 0074 0075 0099 0120 0126 0198
Rank	0009
Rank Arena	0004 0021 0104 0154
RBM	0009
RCA	0015 0054 0100 0112 0154 0175 0288
Realistic	0026
Recor	0005 0075 0099
Rectiligne	0005 0099
Rediffusion	0004 0057 0063 0095
Redstar	0005 0099
Reflex	0005 0099 0126 0198
Relisys	0188 0189 0190 0191 0212 0214 0215 0216 0217 0230 0233
Reoc	0117 0138 0257
Revox	0005 0067 0099
Rex	0022 0032 0041 0043 0044 0065 0073
RFT	0005 0010 0014 0044 0049 0067 0085 0099
Rhapsody	0034
Ricoh	0005 0099
Rinex	0044 0075 0132 0136 0161
Roadstar	0001 0005 0036 0044 0045 0075 0099 0126 0138 0198 0208 0236 0252 0258
Robotron	0014
Rodex	0005 0099
Rover	0004 0190
Rowa	0001 0005 0034 0035 0044 0099 0103 0132 0136 0137
Royal	0075 0105 0175
Royal Lux	0054 0067 0074
Rukopir	0005 0099
S-Media	0218
Saba	0014 0020 0022 0046 0054 0063 0089 0093 0095 0100 0112 0121 0138 0288
Saccs	0038
Sagem	0045 0080 0107 0110 0178
Saisho	0001 0002 0022 0025 0035 0044 0069 0093 0117
Saivod	0005 0099 0126 0137 0198 0205 0264
Sakaï	0022
Sakyno	0080
Salora	0022 0029 0033 0047 0056 0058 0062 0063 0065 0082 0093 0095 0111 0115
Salsa	0054
Sambers	0016 0093
Samsung	0001 0005 0008 0010 0015 0022 0026 0033 0034 0035 0044 0047 0067 0084 0099 0103 0110 0112 0119 0221 0227 0271
Sandra	0034 0035
Sanela	0038
Sansui	0005 0068 0080 0099 0104 0123 0136 0144 0145 0154 0176 0187 0193
Santon	0001
Sanyo	0001 0002 0004 0005 0007 0010 0017 0019 0021 0022 0024 0033 0034 0035 0049 0056 0067 0085 0098 0099 0134 0142 0147 0198 0213
Save	0005 0099
Saville	0256
SBR	0005 0028 0099
Schaub Lorenz	0058 0063 0069 0085 0095 0105 0138 0209
Schneider	0005 0009 0022 0035 0036 0041 0043 0058 0059 0063 0068 0072 0099 0120 0126 0138 0198 0201 0256 0257 0258 0264 0271 0288
Scotland	0022
Scott	0026 0208
Sears	0026
Seaway	0117
Seelver	0005 0099 0198
SEG	0001 0004 0005 0016 0022 0034 0035 0036 0041 0044 0058 0064 0086 0099 0117 0126 0138 0198 0205 0236 0238 0257 0286
SEI	0005 0014 0016 0022 0025 0032 0058 0093 0099 0197 0240
Sei-Sinudyne	0005 0014 0016 0032 0093 0099 0197 0240
Seleco	0004 0022 0032 0043 0044 0057 0064 0065 0068 0073 0077
Sencora	0001
Sentra	0001 0003 0036 0058

Serino	0015 0034 0080 0107
Shanghai	0183
Sharp	0004 0015 0021 0028 0031 0042 0087 0093 0122 0157 0205 0210 0237
Shintoshi	0005 0099
Shivaki	0005 0026 0069 0078 0099
Show	0001 0005 0010 0044 0075 0132 0136
Siarem	0014 0016 0022 0093
Siemens	0005 0021 0027 0030 0031 0052 0053 0063 0094 0097 0099
Siera	0005 0099 0103
Siesta	0067
Silva	0005 0034 0063 0099 0120
Silva Schneider	0243
Silver	0004 0005 0022 0063 0080 0099 0139
SilverCrest	0005 0099 0198
Singer	0001 0005 0014 0016 0022 0041 0054 0077 0099 0242
Sinotec	0075 0105 0132 0161
Sinudyne	0005 0014 0016 0022 0025 0032 0058 0063 0093 0099 0105 0197 0240 0256
Skantic	0062
Sky	0005 0026 0045 0099 0192 0283
Skymaster	0018
Skysonic	0130 0154
Sliding	0168 0188 0192
SLX	0126
Smaragd	0086
Soemtron	0188 0233
Sogera	0050
Solavox	0005 0010 0022 0063 0095 0099
Sonawa	0036
Sonber	0016
Soniko	0005 0099
Soniq	0233 0282
Sonitron	0033 0035 0056 0067
Sonneclair	0005 0099
Sonoko	0001 0005 0035 0044 0045 0099
Sonolor	0022 0033 0045 0063 0095 0197 0240
Sontec	0001 0005 0067 0099
Sony	0002 0004 0005 0012 0015 0016 0021 0024 0060 0099 0180 0197 0240 0247 0249
Sound & Vision	0005 0016 0036 0069 0099
Soundesign	0026
Soundwave	0005 0050 0075 0099 0139
Soyea	0161
Spectra	0001
Ssangyong	0001
Staksonic	0001
Standard	0001 0005 0035 0036 0050 0069 0099 0198
Starlite	0001 0005 0022 0044 0074 0099
Stenway	0036 0045
Stern	0022 0032 0043 0044 0065 0073 0077
Stevison	0243 0264
Strato	0001 0005 0044 0099
Strong	0005 0099 0198 0204 0205
Stylandia	0035
Sunic Line	0005 0099
Sunkai	0005 0036 0051 0061 0080
Sunkai	0086 0099 0107 0188
Sunstar	0001 0005 0044 0068 0099
Sunwatt	0080
Sunwood	0001 0005 0099
Superla	0034 0035 0093
Supersonic	0001 0033 0044 0080 0099 0132 0170
SuperTech	0001 0005 0034 0036 0099
Supervision	0044 0132
Supra	0001 0005 0026 0069
Susumu	0036 0046 0054
Sutron	0001
SVA	0044 0080 0132 0170
Svasa	0033 0044 0132 0136 0170
Swissline	0041
Swisstec	0188 0192 0283
Sydney	0034 0035
Symphonic	0254
Sysline	0005 0099
Sytong	0034
T+A	0079
Tandberg	0020 0022 0055 0063 0064 0066 0073
Tandy	0010 0015 0022 0035 0036 0041
Tashiko	0004 0022 0024 0034 0035 0065
Tatung	0002 0005 0010 0035 0093 0099 0111 0113 0209 0226 0228 0243 0251 0256
TCM	0138 0171 0232
Teac	0001 0005 0024 0026 0035 0044 0045 0058 0074 0075 0080 0092 0099 0126 0132 0136 0137 0138 0142 0156 0198 0204 0238 0250 0257 0262
TEC	0001 0005 0022 0035 0041 0043 0054 0055 0063 0099
Tech Line	0005 0099 0126 0205 0238
Tech Lux	0208
Techica	0036
Technema	0050
Technica	0264
Technics	0121
TechniSat	0005 0022 0099 0123 0274
Technisson	0138 0232
Technol Ace	0034 0044 0069 0117 0132
Technosonic	0005 0090 0099 0192
Techwood	0205
Tecnimagen	0099
Tedelex	0001 0005 0033 0035 0075 0099 0103 0105 0132 0136 0195 0242
Teiron	0001

Tek	0001 0005 0132 0173
Teknika	0008
Tele System	0189
Teleavia	0020 0046 0288
Telecor	0005 0022 0035 0036 0043 0072 0099
Telefunken	0005 0011 0012 0013 0020 0046 0054 0057 0085 0089 0099 0100 0103 0112 0132 0137 0138 0155 0164 0172 0173 0174 0195 0196 0222 0243 0283 0288
Telefusion	0005 0099
Telegazi	0005 0022 0036 0043 0044 0099
Telemeister	0005 0050 0099
Telesonic	0005 0099
Telestar	0001 0005 0074 0099
Teletech	0001 0005 0041 0055 0099 0126 0198
Teleton	0004 0022 0032 0035 0043 0058 0065
Televideon	0022 0034 0050
Teleview	0005 0099
Telexa	0016
Tempest	0001 0005 0044 0099
Tennessee	0005 0099
Tensai	0001 0005 0017 0018 0022 0035 0036 0041 0050 0068 0069 0070 0099 0139 0198
Tenson	0001 0050
Tesla	0005
Tevion	0005 0099 0120 0126 0138 0171 0188 0198 0201 0226 0228 0232 0233 0243 0271 0287
Texet	0001 0034 0035 0036 0069
Thomas	0254
Thomson	0005 0020 0034 0046 0050 0054 0058 0099 0100 0112 0288
Thorn	0002 0003 0004 0005 0007 0009 0010 0011 0012 0013 0017 0019 0020 0028 0037 0038 0054 0062 0063 0069
Thorn	0090 0092 0094 0096 0099 0197 0240 0288
Thorn-Ferguson	0003 0011 0017 0019 0020 0028 0038 0054 0063 0090
Tiny	0211 0231
Tiny Digital	0124
TMK	0025 0026
Tobishi	0036
Tokai	0001 0005 0010 0022 0035 0055 0069 0099 0126 0198
Tokaido	0198
Tokyo	0003 0034
Tomashi	0036 0045
Tongtel	0103 0164
Top Show	0001 0044 0075 0132 0136 0161
Topline	0126 0198
Toshiba	0001 0003 0004 0005 0008 0009 0016 0020 0027 0030 0035 0039 0091 0096 0110 0121 0132 0138 0140 0148 0179 0205 0206 0232 0241 0243 0248 0256 0258 0260 0261 0273
Towada	0016 0035 0058
Toyoda	0001 0044 0068
Trakton	0001 0035 0044
TRANS-continents	0005 0035 0085 0099 0111 0126 0188 0198
Transonic	0001 0005 0044 0075 0080 0092 0099 0103 0132 0137
Transtec	0034
Triad	0001 0005 0036 0099
Trical	0021
Trident	0035 0093
Tristar	0028 0036 0044
Triumph	0005 0025 0057 0093 0099
Tsoschi	0045
TVTEXT 95	0099
Uher	0005 0032 0043 0050 0067 0069 0075 0082 0085 0099
Ultravox	0005 0014 0016 0022 0034 0041 0069 0099
Unic Line	0005 0058 0080 0081 0099
United	0005 0099 0103 0138 0139 0198 0256 0258 0264
Universal	0005 0099 0138
Universum	0001 0002 0004 0005 0009 0012 0013 0017 0018 0020 0021 0022 0024 0025 0030 0031 0035 0041 0044 0047 0052 0053 0055 0057 0058 0063 0064 0067 0073 0075 0081 0082 0088 0092 0094 0097 0099 0110 0115 0126 0197 0198 0204 0205 0236 0238 0240 0286
Univox	0005 0014 0022 0038 0055 0099
Utax	0022
V7 Videoseven	0026 0192 0218 0250 0280
Vestel	0005 0022 0035 0099 0126 0198 0205
Vexa	0001 0005 0099
Victor	0122
Videologic	0034 0036
Videologique	0034 0035 0036
Videosat	0041
VideoSystem	0005 0099
Videotechnic	0034 0035 0050 0069
Videoton	0022 0062
Vidikron	0016 0147
Vidtech	0004 0026
Viewpia	0189
ViewSonic	0250
Viper	0055
Visiola	0034
Vision	0005 0035 0044 0050 0099
Vistar	0032 0063
Vortec	0005 0099

Voxson	0005 0014 0022 0026 0065 0075 0099
Waltham	0005 0020 0022 0035 0046 0062 0075 0078 0099 0126 0198
Watson	0001 0005 0022 0036 0050 0052 0072 0099 0126 0198 0226 0228 0238 0256 0271 0280
Watt Radio	0016 0022 0034 0058
Wega	0004 0005 0014 0099
Wegavox	0001 0005 0099
Welltech	0138
Weltblick	0005 0035 0050 0099
Weltstar	0198
Weston	0005 0099
Wharfedale	0005 0099 0132 0136 0186 0187 0195 0196 0243 0265 0271
White Westinghouse	0005 0034 0050 0055 0099 0257
Windsor	0126 0198
Windstar	0045 0055
Windy Sam	0099
Wintel	0138
World-of-Vision	0188 0190 0192 0194 0218 0232 0233
Worldview	0007 0080
X-View	0209
Xenius	0117
XLogic	0132 0186
Xoro	0211 0218 0222
Xrypton	0005 0099
Yamaha	0121 0167 0244
Yamishi	0005 0035 0036 0045 0080 0099
Yokan	0005 0099
Yoko	0001 0005 0034 0035 0036 0041 0044 0056 0067 0099
Yorx	0036
Yoshita	0175
Zanela	0038
Zanussi	0032 0035 0065
Zenith	0257
Zenor	0056
ZX	0256

DBV Code

@sat	0702
@Sky	0713
ABsat	0625 0627 0709 0856
ADB	0623 0690 0719 07260746 0815 0829 0857
Adcom	0608
ADT	0619
AGS	0625
Akai	0608
Alba	0627 0697 0756 0779 0856 0883
Allsat	0608 0646 0648
Allsonic	0851
Alltech	0627 0856
Allvision	0682
Alpha	0608
Amitronica	0627 0856
Ampere	0603
Amstrad	0603 0615 0627 06360639 0642 0661 0668 08560814
Anglo	0627 0856
Ankaro	0627 0693 0851 0856 1279
AntSat	0646 0653
Apollo	0756
Apro	0782
Arcon	0603 0635 0648 0651 0672 0693
Armstrong	0608
Arnion	0702
ASA	0613
Asat	0608
ASCI	0601 0713 0738
ASLF	0627 0856
AssCom	0637
Astacom	0625
Aston	0605 0691 0858
Astra	0627 0856
Astratec	0878
Astro	0604 0607 0608 0612 0624 0655 0656 0661
Atlantic Telephone	0712
Audioline	0730
Auna	0815 0829
Aurora	0623 0641 0712 0732
Austar	0617 0623 0639 0641 0667 0669 0690 0802 0851
Axiel	0625
Axil	0736 0779 1659
Axis	0635 0660 0851
B@ytronic	0613 0722
Bauckhage	0711
Beko	0756
Bentley Walker	0646
Belgacom TV	0830
Best	0851
Big Sat	0736
Birmingham cable Communications	0831
Black Diamond	0697
Blaupunkt	0607
Blue Sky	0627 0856
Blue Star	0693
Boca	0603 0627 0682 0718 0743 0856
Boston	0603 0625
Brainwave	0624 0676 0782
British Sky Broadcasting	0636 0668
British Telecom	0832
Broco	0627 0856
BSkyB	0636 0668 0833 0870
BT	0625 0701 0834 0800
BT Satalite	0835
BT Vision	0836
Bubu Sat	0627 0856
Bush	0616 0623 0697 0740 0744 0782 0868 0878 0883
Canal Digital	0606 0637 0649 0739 0770 0887
Canal+	0612 0637 0887
Canal Plus	0803
CanalSatellite	0612 0637 0714 0887
CCE	0615
Centrex	0751
CGV	0723 0759
Cherokee	0625 0709
Chesley	0751
Chess	0601 0627 0654 0713 0772 0856
Chili	0876
CityCom	0613 0618 0633 0651 0669
CityCom	0682 0752
Clatronic	0723
Clemens Kamphus	0635
Colombia	0603
Columbia	0603
Comag	0603 0682 0718 0722 0723 0762 0866
Comcrypt	0803
Condor	0851 0874
Conia	0873
Connexions	0851
Conrad	0603 0620 0851
Contec	0852
Continental Edison	0873
Coship	0736
Crown	0697
Cryptovision	0756
Cyrus	0608
D-box	0629 0662 0837
Daewoo	0627 0660 0701 0856 0878
Darty box	0838
Delfa	0639
Deltasat	0651
DGTEC	0684 0864 0867
Digatron	0700
Digenius	0613 0665
DigiFusion	0763 0775 0868
Digihome	0697
Diginet	0751
Digipro	0659
DigiQuest	0639 0702 0736
DigiSat	0682
Digisky	0736
Digital+	0887

DigitAlb	0751
DigitAll World	0681
DigiTurk	0652
DiPro	0719 0726 0751 0865
Discovery	0625 0834
Distratel	0672 0696 0875
Distrisat	0608
DMT	0651
DNT	0608
Dream Multimedia	0683 0733 0749 0774 0777 0778
DSE	0863
DStv	0623 0641
Dune	0851
Durabrand	0697
Echostar	0606 0622 0627 0637 0671 0709 0721 0726 0739 0745 0856
Einhell	0603 0627
Elap	0621 0625 0627 0759 0768 0856 0858
Electron	0693
Elsat	0627 0856
Elta	0608 0779
Emme Esse	0729 0851
eMTech	0676
Energy Sistem	0867
Engel	0627 0646 0687 0856
EP Sat	0756
Erastar	0766
Eurieult	0642
Eurocrypt	0756
EuroLine	0687
Europa	0608 0639
Europhon	0603 0613
Eurosat	0759
Eurosky	0132 0299 0601 0603 0613 0850 0851
Eurostar	0633
Eutelsat	0627 0856
Eycos	0693
Fast	0791
Fenner	0627 0851 0856
Ferguson	0626 0693 0698 0756 0878
Filmnet	0803
Finepass	0741 0765
Finlandia	0756
Finlux	0620 0756
Flair Mate	0627 0856
FMD	0601 0687 0723 0736
Force	0657 0670
Fortec Star	0646 0653 0729
Foxtel	0615 0617 0628 0641 0666
Foxtel	0667 0669 0717 0756 0814
Fracarro	0602
France Satallite/TV	0839
France Telecom	0804 0806 0838
Free	0840
Freecom	0607
Freesat	0621 0642
Freebox	0817 0840
FTEmaximal	0627 0639 0753 0851 0856
Fuba	0607 0613 0620 0665 0687 0729 0850 0851 0852 0882
Fugionkyo	0602 0659
Galaxis	0635 0637 0639 0641 0657 0660 0669 0754 0851
Gardiner	0633
Garnet	0651
GbSAT	0676
Gecco	0692 0722
General Instrument	0800 0802
General Satellite	0669
GF	0635 0648
Globo	0687 0713 0722 0730 0743 0772
GOD Digital	0608
Gold Box	0612 0637
Gold Vision	0646 0867
Golden Channel	0812
Golden Interstar	0696
GoldStar	0852
Goodmans	0697 0756 0764
Gradiente	0857
Granada	0756
Grundig	0607 0615 0630 0636 0637 0641 0697 0698 0710
H&B	0751
Handan	0770
Hanseatic	0655 0656
Hänsel & Gretel	0603
Hauppauge	0700 0782
HB	0676
HDT	0644 0664
HE@D	0693
Hewlett Packard	0793
Hills	0682
Hirschmann	0602 0607 0613 0620 0625 0642 0654 0660 0661 0682 0722 0851
Hitachi	0697 0756
HNE	0603 0682
Homecast	0676 0785 0874
HomeChoice	0820 0823
HOT	0810 0812
Humax	0639 0669 0680 0728 0760 0776 0780 0781 0783 0828 0841 0878 0884 0888
Huth	0603 0646 0651 0663
HyperVision	0827
Hyundai	0644 0651 0664
iCan	0719
ID Digital	0639 0669
ILLUSION sat	0754 0867
Imperial	0608 0612 0655 0656 0713 0716 0730 0757 0782 0860
Ingelen	0642
International	0603
Interstar	0646 0659 0676
iotronic	0603
Irradio	0711
ITT Nokia	0620 0629 0756
Jadeworld	0623
Jaeger	0713
Jepssen	0676
Jerrold	0800 0802
JOK	0625
K-SAT	0627 0856

Kabel Deutschland	0842
Kabelvision	0800
Kamm	0627 0856
Kansalaisboksi	0751
Kaon	0702
KaTelco	0660
Kathrein	0601 0607 0608 0610 0618 0624 0625 0627 0633 0650 0678 0709 0725 0758 0759 0852 0854 0855 0856
Kennex	0602
Kenwood	0637
Key West	0603
Kiton	0601
Klap	0625
Kocmoc TB	0712
Kongque	0702
Koscom	0635 0648
Kosmos	0853
Kreiling	0601 0610 0624 0738
Kreiselmeyer	0607
Kross	0873
Kyostar	0603
L&S Electronic	0603 0648 0713 0851
Labgear	0701
LaSAT	0603 0613 0851
Legend	0876
Legend Digital	0876
Lemon	0738
Lenco	0627 0851
Lennox	0851
Lenoxx	0768
Lexus	0608
LG	0651 0724
Lifesat	0603 0613 0627 0648 0851 0856
Lifetec	0621
Listo	0772
Lodos	0697
Logik	0697
Logix	0646 0651
Lorenzen	0603 0613 0640 0665 0700
Lupus	0851
LorenzSat	0766
Luxor	0615 0620
Luxorit	0786
M vision	0754
Macab	0637 0806
Madritel	0623 0815
Manata	0603 0625 0627 0856
Manhattan	0625 0635 0646 0653 0756
Marantz	0608
Mascom	0853
Maspro	0607 0627 0630 0856
Master's	0852
Matav	0810
Matsui	0607 0625 0878
Maximum	0651 0713
MDS	0680
Mediabox	0612 0637
Mediacom	0673
MediaSat	0612 0637
Mediastar	0667
Medion	0603 0613 0627 0648 0651 0682 0713 0722 0772 0851 0856
Medison	0627 0856
Mega	0608
Melectronic	0633
Melita	0800
Metronic	0603 0627 0633 0672 0677 0693 0694 0695 0696 0856 0875 0863
Metz	0607
Mico	0883
Micro	0627 0700 0856
Micro electronic	0627 0856
Micro Technology	0627 0856
Micromaxx	0613 0851
Micronik	0643
Microsoft	0890 0793
Microstar	0651
Microtec	0627 0856
Mitsubishi	0756
MNet	0801 0803
Morgan's	0603 0608 0627 0721
Motorola	0638 0802 0818
Mr Zap	0813
Mr. Zapp	0843
Muller	0873
Multichoice	0623 0641 0712 0732 0801 0803
Mx Onda	0779
Myryad	0608
Mysat	0627 0856
NEC	0769
Neotion	0713
Netgem	0708
Neuf TV	0708
Neuf Telecom	0708 0843
Neuhaus	0627 0635 0856
Neuling	0682 0743
Neusat	0621 0627 0635 0693 0711 0856
Neveling	0665
Nevir	0779
NextWave	0646 0663 0752
Nikko	0608 0627 0629 0856
Noda Electronic	0875
Nokia	0620 0629 0631 0637 0647 0662 0679 0706 0756 0787 0819
Noos	0806 0824
Nordmende	0756 0768
Nova	0641
NTL	0808
Oak	0801
OctalTV	0700 0730
Okano	0853
Opentel	0682 0722
Optex	0601 0619 0627 0635 0648 0694 0696 0768 0772 0856
Optus	0612 0637 0641 0663 0802 0808
Orbis	0713 0722
Orbitech	0601 0608 0612 0635 0655 0656 0716 0850

Origo	0617
OSAT	0615
Ouralis	0672
P/Sat	0682
Pace	0608 0609 0617 0625 0628 0632 0636 0637 0640 0668 0709 0717 0727 0756 0771 0808 0821 0857 0886
Pacific	0697 0863
Pacific Satellite	0635
Packard Bell	0660
Packsat	0625
Palcom	0613 0621 0665 0711 0721
Panasat	0641
Panasonic	0636 0658 0703 0707 0720 0756 0792
Panda	0607 0635 0756
Patriot	0603 0625
peeKTON	0736
Philips	0600 0604 0607 0608 0612 0625 0630 0633 0637 0652 0662 0730 0756 0782 0804 0806 0891
Phoenix	0692
Phonotrend	0639 0646 0671 0741
Pilotime	0714
Pinnacle Systems	0795
Pino	0713
Pioneer	0612 0614 0616 0637 0685 0705 0843
Polytron	0852
Pixx	0619
Polytron	0852
Portland	0701
Power Sky	0693
Preisner	0603 0657 0661 0718
Premier	0612
Premiere	0610 0612 0624 0629 0637 0656 0660 0662 0669 0757 0835 0859
Primacom	0660
Profile	0625
Proline	0779
Promax	0756
Prosat	0667
PVP Stereo Visual	0800
Pyxis	0635 0694
QNS	0720
Quadral	0625 0709 0851
Quelle	0613
Radiola	0608
Radix	0642 0661 0852 0862
RCA	0698
Rebox	0676
Red Star	0851
Regal	0687
Rex	0619
RFT	0608
Roadstar	0627 0637 0856
Rollmaster	0723
Rover	0627 0851 0856
Rownsonic	0759
S-ZWO	0674
SAB	0687 0702
Saba	0625
Sagem	0634 0662 0806 0813 0824
Samsung	0637 0639 0646 0673 0685 0686 0699 0737 0761 0784 0808 0826 0844 0874 0889
Sansui	0631
Sat Control	0702
Sat Cruiser	0663
Sat Team	0627 0856
SAT+	0721
Satec	0627 0635 0856
Satelco	0851
Satplus	0656
Satstation	0653
Schaub Lorenz	0676
Schneider	0625 0673 0687 0791
Schwaiger	0603 0618 0619 0621 0639
Schwaiger	0651 0653 0660 0713 0722 0736 0743 0852 0862
Scientific Atlanta	0805
SCS	0613
Sedea Electronique	0602 0603 0659 0693 0696 0772
SEG	0601 0651 0687 0772 0851
Septimo	0672
Serino	0622
Servimat	0768
ServiSat	0627 0687
Sherwood	0768
Siemens	0607 0713 0730 0835 0869
Silva	0613
SilverCrest	0866
Skantin	0627
Skardin	0621
SKR	0627 0856
SKT	0735
Sky	0626 0636 0638 0645 0668 0845 0857
Sky cable	0846
Sky digital	0646 0849
SKY Italia	0788 0789 0886
Sky XL	0687 0722
SKY+	0668
Skymaster	0621 0627 0651 0654 0671 0713 0721 0768 0856
Skymax	0608 0723
Skypex	0613
skyplus	0682 0713 0722 0736
Skyvision	0839
SkySat	0601 0627 0856
SL	0603 0743 0782
SM Electronic	0621 0627 0651 0654 0671 0856
Smart	0603 0613 0627 0642 0657 0661 0682 0692 0723 0856
Soniq	0759
Sony	0611 0612 0636 0637 0755 0756
SR	0603
Star Sat	0603 0654
Starland	0627 0856
Stream	0788 0789

Strong	0602 0634 0637 0641 0659 0664 0689 0697 0702 0719 0721 0726 0731 0742 0772 0839 0865 0879 1782
Sunny	0702
Sunny Sound	0851
Sunsat	0627 0856
Sunstar	0603 0623 0851
Supermax	0663 0696
Systec	0603 0713
Tantec	0756
Tarbs	0680
Tatung	0756
TBoston	0687 0736 0779
Teac	0680 0681 0708
Tecatel	0671
TechniSat	0601 0608 0612 0639 0655 0656 0708 0716 0756 0757 0850 0859 0860
Technomate	0696 0767
Technosat	0663 0673
Technosonic	0782
Technotrend	0730
Technowelt	0603
Techsan	0646
Techwood	0601 0697 0772
tekComm	0646
Tele Danmark	0807
Tele System	0687 0721 0768 0882
Tele System Electronic	0768
Tele+1	0803
Teleciel	0648
TeleClub	0719
Telecom	0627 0856
Telefonica	0726 0834
Telefunken	0625
Teleka	0630 0635 0850 0853
Telenet	0726
Telepiu	0803
Telestar	0601 0608 0612 0655 0656 0687 0713 0716 0757 0772 0782
Teletech	0601
Televes	0603 0619 0676 0702 0713 0721 0756 0877
Telewest	0808
Telewire	0682
Tempo	0663
Tesla	0772
Tevebox	0786
Tevel	0810
Tevion	0627 0713 0721 0759 0770 0773 0782 0856
Thomson	0612 0625 0626 0627 0634 0636 0637 0649 0668 0698 0747 0748 0756 0790 0812 0822 0856 0865 0870
Thorn	0756
Tiny	0782
Tioko	0603 0852
Tividi	0730
Tokai	0608
Tonna	0621 0627 0635 0756 0768 0856
Topfield	0673 0674 0675 0750 0834 0881 0885
Toshiba	0697 0734
TPS	0634 0688 0704 0861
Tri-Vision	0816
Triax	0601 0603 0608 0627 0637 0655 0661 0681 0687 0698 0701 0723 0743 0766 0768 0772 0856 0877
Trio	0651
TT-micro	0730
Turnsat	0627 0856
Twinner	0603 0627 0768 0856 0877
UEC	0641 0666 0717
Uniden	0635
Unimax	0751
Unisat	0603 0608
United	0687
United Cable	0800
Univers	0753
Universum	0601 0607 0613 0655
UPC	0822
Van Hunen	0613 0665
Variosat	0607
Vega	0851
Ventana	0608
Vestel	0601 0687
VH Sat	0613 0665
Viasat	0634 0647 0709
Virgin Media	0808
Visionic	0602 0659 0693
VisionNet	0754
Visiopass	0804 0806
Visiosat	0601 0605 0619 0625 0627 0723 0736 0856 0858 0876 0877
Vivid	0666
VTech	0633
Wavelength	0682 0723
Wewa	0756
Wharfedale	0697
Winbox	0882
Wisi	0603 0607 0613 0635 0715 0756
Wisplus	0689
Worldsat	0601 0625 0676 0687 0726 0742 0856 0865
Xanadu	0619
XMS	0651
Xrypton	0851
XSat	0627 0636 0676 0709 0856
Xtreme	0702
Yamada	0876
YES	0857
Yess	0751
Zehnder	0601 0618 0619 0633 0651 0682 0687 0713 0722 0723
Zehnder	0851 0852
Zenith	0638
Zeta Technology	0608
Zinwell	0667 0669
Zodiac	0882

BDA 34873-Rev-RZ_Layout 1 21.03.13 09:14 Seite 34

Vivanco International

Austria
Vivanco Austria GmbH
Triesterstrasse 10/3
AT - 2351 Wr. Neudorf
Phone: (+43) 2236 893 7400
Fax: (+43) 2236 893 7408
office@vivanco.at
www.vivanco.at

Benelux
Vivanco (Benelux) B.V.
Galvanibaan 8
NL – 3439 MG Nieuwegein
Phone: (+31) 306 007 050
Fax: (+31) 306 007 055
info@vivanco.nl
www.vivanco.nl

Croatia
FENIKS-KOMPJUTORI d.o.o.
Maksimirska 132
HR - 10 000 Zagreb
Phone: (+385) 1 6395 277
Fax: (+385) 1 6395 282
ana@feniks-kompjutori.hr
www.feniks-kompjutori.hr

Czech Republic
DSI Czech s.r.o.
Pernerova 35 a
CZ - 18600 Prag 8
Phone: (+420) 221 897 111
Fax: (+420) 221 897 171
vivanco@di.cz

Denmark
Cenor Nordic
Box 10045
SE - 18110 Lindingö
Phone: (+46) 8 446 0590
Fax: (+46) 8 446 0595
info@cenor.dk
www.cenor.dk

Estonia
Nordic Digital AS
Tööstuse tee 6, Törvandi
EST - 61715 Ülenurme / Tartumaa
Phone: (+372) 733 770 0
Fax: (+372) 733 772 5
info@nordic-digital.ee
www.nordic-digital.ee

Finland
AV-Komponentti OY
Koronakatu 1a
FI - 02210 Espoo
Phone: (+358) 9 8678 020
Fax (+358) 9 8678 0250
info@avkomponentti.fi
www.avkomponentti.fi

France
Vivanco France S.A.S.
40 Avenue de Lingenfeld
FR – 77220 Torcy
Phone: (+33) 164 731139
Fax: (+33) 164 617032
hotline@vivanco.fr
www.vivanco.fr

Germany
Vivanco GmbH
Ewige Weide 15
D-22926 Ahrensburg
Tel. +49 (0) 4102 - 231 247
hotline@vivanco.de
www.vivanco.de

Greece
DIGITECH
Pithagora 1 & Iroon
Politechniou 227 str.
GR - 41221 Larissa
Phone: (+30) 2410 284800
Fax (+30) 2410 579092
info@digitech-net.com

Hungary
Sky-Log Magyarország Kft
Almáskert u. 4
HU - 2220 Vecsés
Phone: (+36) 29 555 060
Fax: (+36) 29 555 068
nobl@sky-log.hu

Iceland
Nyherji hf
Borgartún 37
IS - 105 Reykjavik
Phone: (+354) 559 7700
Fax: (+354) 559 7799
nyherji@nyherji.is

Italy
Cellular Italia S.p.a.
Via Lambrakis, 1/A
42122 Reggio Emilia (Italy)
Phone: +39 0522 33 40 02
Fax: +39 0522 33 40 01
www.cellularline.com

Lithunia
Elektronikos Pasaulis
Sausio 13-osios 4-1
LT - 04343 Vilnius
Phone: (+370) 278 867 8
Fax: (+370) 278 867 8
info@elektronikospasaulis.lt
www.elektronikospasaulis.lt

Poland
Vivanco Poland Sp.z.o.o.
Ul. Sosnkowskiego 1
PL – 02-495 Warszawa
Phone: (+48) 22 723 0745
Fax: (+48) 22 723 0750
info@vivanco.pl
www.vivanco.pl

Portugal
Esotérico,
Consultores de Som, Lda.
Praceta Alves Redol, No. 2
Franqueiro-Loures
PT - 2670 Loures
Phone: (+351) 219 839 550
Fax: (+351) 219 839 559
info@esoterico.pt
www.esoterico.pt

Serbia
DESK d.o.o.
Bul. despota Stefana 110
RS - 11 000 Belgrad
Phone: (+381) 113 292 140
Fax: (+381) 113 292 750
vivanco@desk.rs

Slovakia
DSI Slovakia s.r.o.
Juzna trieda 117
SK - 040 01 Kosice
Phone: (+421) 55 6770 666
Fax: (+421) 55 6770 662
vivanco@dsi.sk

Slovenia
BIG BANG d.o.o
– SKUPINA MERKUR
Smartinska 152
SLO - 1000 Ljubljana
Phone: (+38) 61 309 3700
Fax: (+38) 61 309 3760
E-mail:info@bigbang.si
www.bigbang.si /

Spain
Vivanco Accesorios S.A.
Antic Poligono Industrial
Güttermann Cami Vell
de Sant Celoni S/N
E - 08460
Santa Maria Palautordera
Phone: (+34) - 902 027 724
Fax: (+34) - 93 848 16 67
vivanco@vivanco.es
www.vivanco.es

Sweden
Cenor Nordic AB
Box 10045
SE - 18110 Lidingö
Phone: (+46) 8 446 0590
Fax: (+46) 8 446 0595
info@cenor.se

Switzerland
Vivanco Suisse AG
Alte Landstr. 15
CH - 8804 Au ZH
Phone: (+41) 44 782 6800
Fax: (+41) 44 782 6810
spoc@vivanco.ch
www.vivanco.ch

Turkey
KALE Elektronik Dis Ticaret A.S.
Besyol Mah.
E-5 Karayolu Cad. No. 29
TR-34295 Kucukcemece/
Istanbul
Phone: (+90) 212 693 02 02
www.goldmaster.com.tr

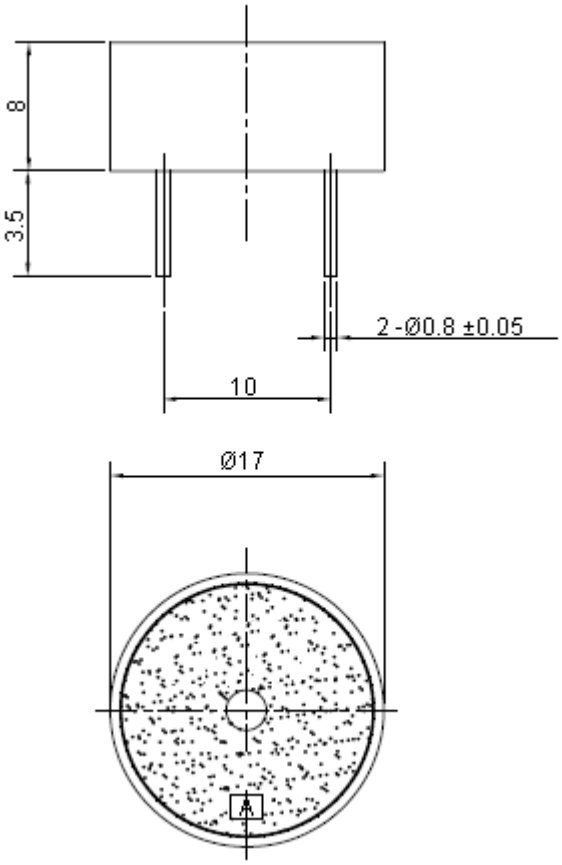
United Kingdom
Vivanco (UK) Ltd.
Luton Enterprise Park
Sundon Park Road
GB - Luton LU3 3GU
Phone: (+44) 1582 578 720
Fax: (+44) 1582 578 730
consumer@vivanco.co.uk

BDA 34873-Rev-RZ_Layout 1 21.03.13 09:14 Seite 36

Vivanco GmbH
Ewige Weide 15
D-22926 Ahrensburg/Germany
Fax +49(0)41 02 / 231 - 160
www.vivanco.com

IT 06/13

Piezo Transducer



Dimensions : Millimetres

Specifications:

Rated Voltage	: 3 Vp-p Square Wave
Maximum Allowable Voltage	: 25 Vp-p Square Wave
Maximum Rated Current	: 2 mA at 2 KHz / 3 Vp-p Square Wave
Minimum Sound Pressure Level	: 70 dB at 2 KHz / 3 Vp-p Square Wave / 10 cm
Resonant Frequency	: 2 ±0.5 KHz
Capacitance at 100 Hz	: 25,000 pF ±30%
Material / Colour	: PC
Operating Temperature	: -20°C to +80°C
Store Temperature	: -30°C to +100°C
Weight	: 2 g

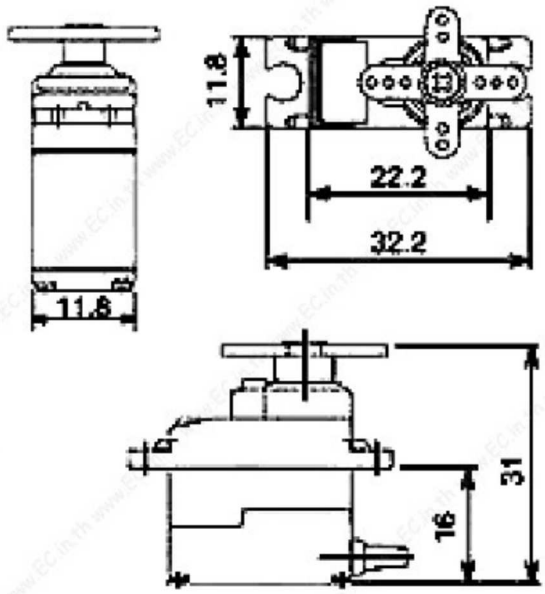
Part Number Table

Description	Part Number
Piezo Transducer	MCKPT-G1712A-3921

Important Notice : This data sheet and its contents (the "Information") belong to the members of the Premier Farnell group of companies (the "Group") or are licensed to it. No licence is granted for the use of it other than for information purposes in connection with the products to which it relates. No licence of any intellectual property rights is granted. The Information is subject to change without notice and replaces all data sheets previously supplied. The Information supplied is believed to be accurate but the Group assumes no responsibility for its accuracy or completeness, any error in or omission from it or for any use made of it. Users of this data sheet should check for themselves the Information and the suitability of the products for their purpose and not make any assumptions based on information included or omitted. Liability for loss or damage resulting from any reliance on the Information or use of it (including liability resulting from negligence or where the Group was aware of the possibility of such loss or damage arising) is excluded. This will not operate to limit or restrict the Group's liability for death or personal injury resulting from its negligence. Multicomp is the registered trademark of the Group. © Premier Farnell plc 2012.

SG90 9 g Micro Servo





Tiny and lightweight with high output power. Servo can rotate approximately 180 degrees (90 in each direction), and works just like the standard kinds but *smaller*. You can use any servo code, hardware or library to control these servos. Good for beginners who want to make stuff move without building a motor controller with feedback & gear box, especially since it will fit in small places. It comes with a 3 horns (arms) and hardware.

Specifications

- Weight: 9 g
- Dimension: 22.2 x 11.8 x 31 mm approx.
- Stall torque: 1.8 kgf·cm
- Operating speed: 0.1 s/60 degree
- Operating voltage: 4.8 V (~5V)
- Dead band width: 10 µs
- Temperature range: 0 ºC – 55 ºC

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is all the way to the left. ms pulse) is all the way to the right, ""-90" (~1ms pulse) is all the way to the left.

TowerPro SG90 - Micro Servo



Basic Information

Modulation: Analog

Torque: **4.8V:** 25.0 oz-in (1.80 kg-cm)

Speed: **4.8V:** 0.10 sec/60°

Weight: 0.32 oz (9.0 g)

Dimensions:

Length: 0.91 in (23.1 mm)

Width: 0.48 in (12.2 mm)

Height:1.14 in (29.0 mm)

Motor Type: 3-pole
Gear Type: Plastic
Rotation/Support: Bushing

Additional Specifications

Rotational Range: 180°
Pulse Cycle: ca. 20 ms
Pulse Width: 500-2400 µs

T-Pro Mini Servo SG-90 9G Servo



The **TP SG90** is similar in size and weight to the Hitec HS-55, and is a good choice for most park flyers and helicopters. Hobbyists from around the world has used the SG90 on famous planes like GWS Slow Stick, E-Flite Airplanes, Great Planes, Thunder Tiger, Align, EDF jets and more. If you are looking for a servo that won't break your arm or leg, this is the perfect choice for you!

The **TP SG90** servo weighs 0.32 ounces (9.0 grams). Total weight with wire and connector is 0.37 ounces (10.6 grams).

The **TP SG90** has the universal "S" type connector that fits most receivers, including Futaba, JR, GWS, Cirrus, Blue Bird, Blue Arrow, Corona, Berg and Hitec.

The wire colors are **Red = Battery(+)** **Brown = Battery(-)** **Orange = Signal**

TP SG90 Specifications:

Dimensions (L x W x H) = 0.86 x 0.45 x 1.0 inch (22.0 x 11.5 x 27 mm)

Weight = 0.32 ounces (9 grams)

Weight with wire and connecter = 0.37 ounce (10.6 grams)

Stall Torque at 4.8 volts = 16.7 oz/in (1.2 kg/cm)

Operating Voltage = 4.0 to 7.2 volts

Operating Speed at 4.8 volts (no load) = 0.12 sec/ 60 degrees

Connector Wire Length = 9.75 inches (248 mm)

Universal "S" type connector fits most receivers

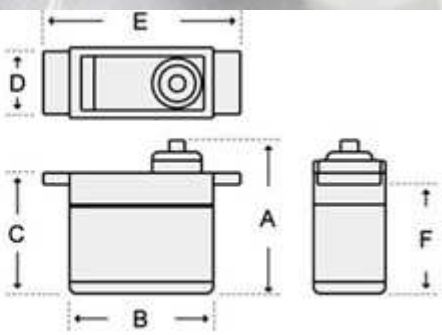
9g Tower Pro Servo

Tower Pro 9g servo
These Micro strong and made to last, great for planes. The servos include 3 servo arms.
Specs:

- DIMENSION: 26mm*13mm*24mm
- WEIGHT: 9G
- OPERATING SPEED: 0.12sec/60degree(4.8V);0.11sec/60degree(6V)
- STALL TORQUE: 1.2kg/cm or 17oz-in. (4.8V) 1.6kg/cm or 22oz-in.(6.0V)
- OPERATING VOLTAGE: 4.8V~6.0V
- FEATURE: 3 pole wire, all nylon gear, connector wire length: 15cm



Usage Type :	
Voltage (V) :	4.8,6.0
Speed@4.8V (sec/60 :	0.12
Torque@4.8V (kg-cm) :	1.2
Speed@6.0V (sec/60 :	0.11
Torque@6.0V (kg-cm) :	1.6
Weight (g) :	9
Dimension B (mm) :	26
Dimension C (mm) :	24
Dimension D (mm) :	13





The TG9e boasts the same performance as other servo's 10x the price with a .10sec travel time and up to 1.5kg in torque and an ultra narrow dead bandwidth!

The TG9e performance is on par with the famous HXT900, however the TG9e isn't as resistant to crashes or over-loading.

Please always ensure your control surfaces are bind free.

Spec.

Dimension: 23x12.2x29mm

Torque: 1.5kg/cm (4.8V)

Operating speed: 0.10sec/60 degree 0.09sec/60 degree(6.0V)

Operating voltage: 4.8V

Temperature range: 0-55C

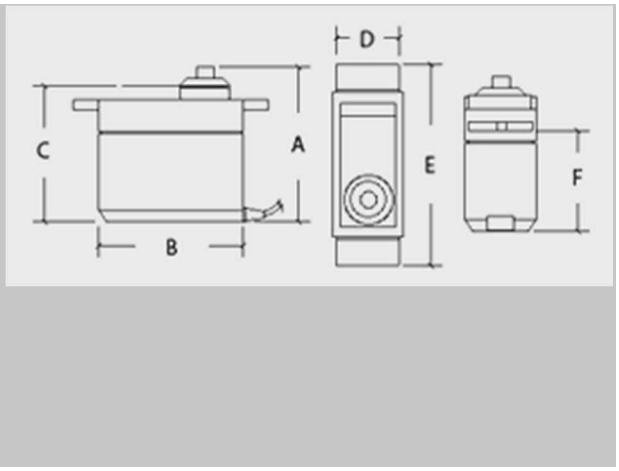
Dead band-width: 7us

Lead Length: 260mm

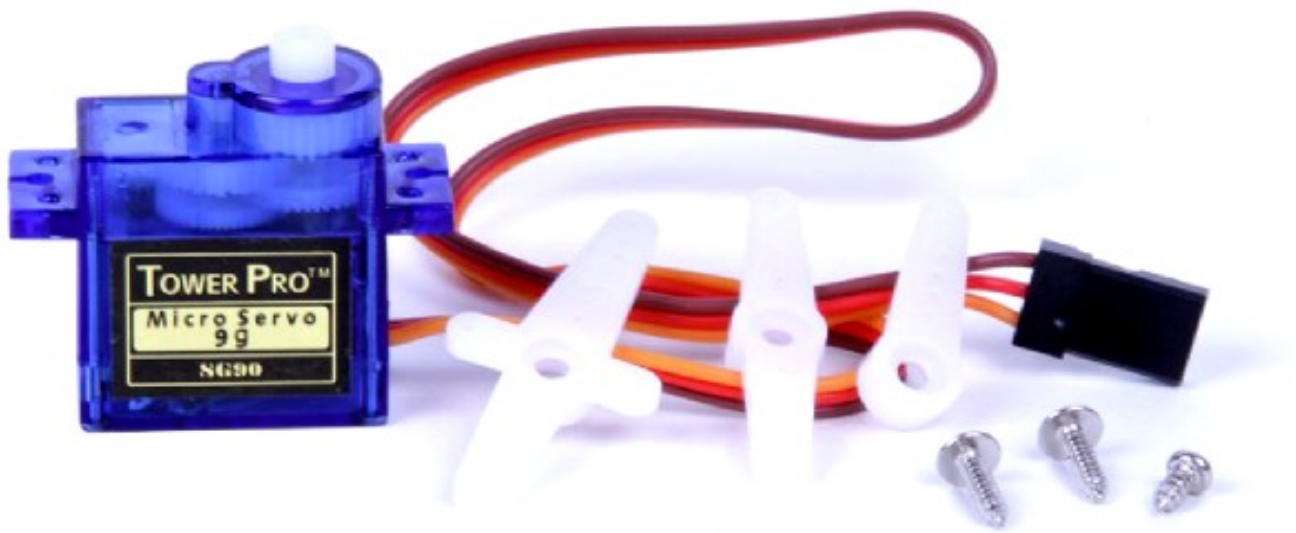
Important.
The TG9e does not have the same strength and longevity as the HXT900.

Please ensure your control surfaces are bind-free and always check servos after a hard landing or crash!

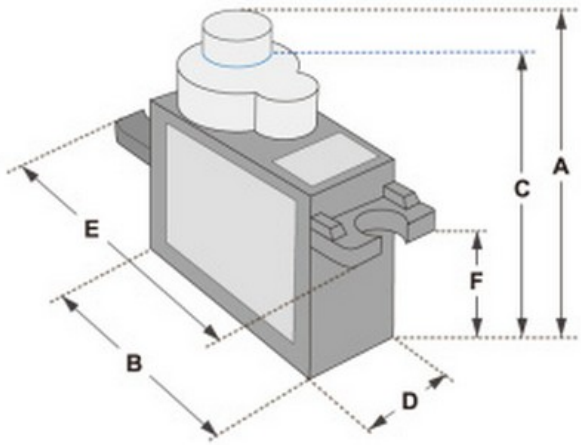
Weight (g)	9
Torque (kg)	1.5
Speed(Sec/60deg)	0.09
A(mm)	30
B(mm)	23
C(mm)	27
D(mm)	12
E(mm)	33
F(mm)	16



SERVO MOTOR SG90 DATA SHEET

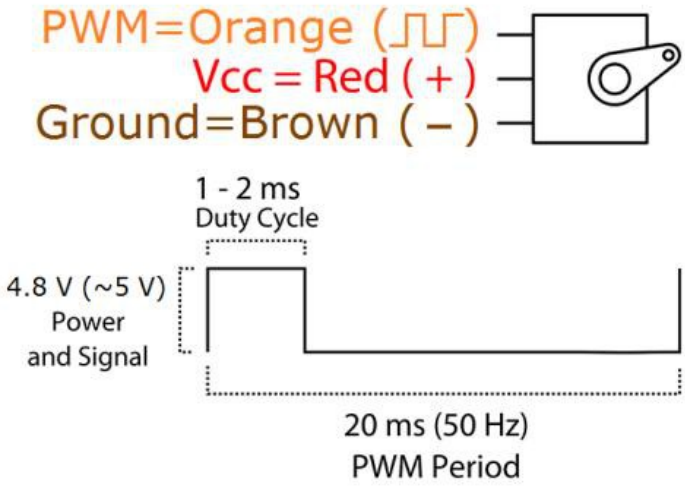


Tiny and lightweight with high output power. Servo can rotate approximately 180 degrees (90 in each direction), and works just like the standard kinds but smaller. You can use any servo code, hardware or library to control these servos. Good for beginners who want to make stuff move without building a motor controller with feedback & gear box, especially since it will fit in small places. It comes with a 3 horns (arms) and hardware.



Dimensions & Specifications
A (mm) : 32
B (mm) : 23
C (mm) : 28.5
D (mm) : 12
E (mm) : 32
F (mm) : 19.5
Speed (sec) : 0.1
Torque (kg-cm) : 2.5
Weight (g) : 14.7
Voltage : 4.8 - 6

Position "0" (1.5 ms pulse) is middle, "90" (~2ms pulse) is middle, is all the way to the right, "-90" (~1ms pulse) is all the way to the left.



main.asm

```
; file main.asm target ATmega128L - 4 MHz - STK300 (KAN-MEG-DEV)
;
; Top-level program. Owns shared SRAM variables, reset/init, main loop and three modes.
;
; Operating modes (variable "mode"):
; 0 = PARK : servo stays at rest, buzzer beeps (parktronic sound)
; 1 = RADAR : servo sweeps left<->right, buzzer beeps (radar sound)
; 2 = MANUAL : VOL+/VOL-/UP/DOWN change speed or direction of the rotation,
; make a small rotation (clockwise/counter clock.)
;
; IR t■l■comande (ir_rc5.asm):
; VOL+, VOL-, UP, DOWN -> manual mode
; GUIDE -> change modes (PARK->RADAR->PARK, MANUAL->PARK)
; POWER -> power on/off
; MUTE -> buzzer on/off


.include "macros.asm"
.include "definitions.asm"


; main passes between beeps for different distance
.equ BUZZ_REPEAT_VFAST = 1 ; < 10 cm : ~1 pass (~50 ms)
.equ BUZZ_REPEAT_FAST = 4 ; 10-24 cm : ~4 passes (~200 ms)
.equ BUZZ_REPEAT_SLOW = 12 ; 25-49 cm : ~12 passes (~600 ms)

; glitch filter. Sudden large changes in distance are considered as glitches
.equ DIST_JUMP_LIMIT = 20

; radar sweep timing
; Target: 4.5 seconds per full turn (30 steps). Each step = SWEEP_STEP_MS +
; HC_measure (~3 ms with echo, ~45 ms on timeout) + lcd (~2 ms).
; With an echo the total is ~153 ms; on timeout ~195 ms. The variation
; comes from HC_measure and is acceptable for a visual sweep.
.equ SWEEP_STEPS = 15
.equ SWEEP_STEP_MS = 148

.equ PWR_ON = 0
.equ PWR_OFF = 1
.equ PWR_STARTUP = 2

; delays
.equ LOOP_DELAY_MS = 50
.equ POWEROFF_DELAY_MS = 200
.equ STARTUP_DELAY_MS = 1500


.dseg
.org 0x0200
mode: .byte 1 ; 0=PARK, 1=RADAR, 2=MANUAL
servo_level: .byte 1 ; 0=centre, 1..5=right, 255..251=left
last_cmd: .byte 1 ; last received IR command
power_state: .byte 1 ; 0=on, 1=off, 2=turning_on
buzzer_on: .byte 1 ; 1=buzzer enabled, 0=muted
deb_counter: .byte 1 ; IR debounce: INT7 disabled while > 0
step_timer: .byte 1 ; UP/DOWN kick countdown (main loop decrements)
distance_cm: .byte 1 ; raw measured distance
dist_prev: .byte 1 ; previous raw distance (for the glitch filter)
dist_display: .byte 1 ; filtered distance (used for LCD and beeps)
buzz_counter: .byte 1 ; non-blocking beep cadence counter
tone_fn: .byte 2 ; word address of the active tone routine (Park_tone or Radar_tone)


.cseg
.org 0x0000
```

```
    rjmp reset

.org 0x0012
    rjmp INT7_handler ; defined in ir_rc5.asm

; peripheriques
.include "lcd.asm"
.include "printf.asm"
.include "HC-SR04.asm"
.include "servo.asm"
.include "ir_rc5.asm"


reset:
    LDSP RAMEND
    rcall LCD_init
    rcall HC_init
    rcall Servo_init

    ldi r16, 0
    sts mode, r16 ; set to parking by default
    sts servo_level, r16
    sts power_state, r16 ; set to power on
    sts deb_counter, r16
    sts step_timer, r16
    sts distance_cm, r16
    sts dist_prev, r16
    sts dist_display, r16
    sts buzz_counter, r16
    ldi r16, 1
    sts buzzer_on, r16 ; buzzer enabled by default
    ldi r16, 0xFF
    sts last_cmd, r16

    ; default - park tone, switched to radar mode - radar tone
    ldi r16, low(Park_tone)
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16

    ldi r16, 0
    rcall Servo_set_level ; set servo at centre

    ; enable INT7 on the falling edge (button pressed)
    ldi r16, (1<<ISC71)|(0<<ISC70)
    sts EICRB, r16
    ldi r16, (1<<INT7)
    out EIMSK, r16
    sei

    rjmp main


main:
    ; When step_timer counts down to 0, stop the rotation
    lds r16, step_timer
    cpi r16, 0
    breq main_no_step
    dec r16
    sts step_timer, r16
    cpi r16, 0
    brne main_no_step
    ldi r16, 0
    sts servo_level, r16

main_no_step:
    ; countdown to enable remote again if the button was pushed
    rcall deb_tick
```

```
    ; power on/off logic
    lds r16, power_state
    cpi r16, PWR_OFF
    brne PC+2
    rjmp main_power_off
    cpi r16, PWR_STARTUP
    brne PC+2
    rjmp main_power_on_seq

    ; mode RADAR
    lds r16, mode
    cpi r16, 1
    brne PC+2
    rjmp main_radar

    ; mode PARK/MANUAL
    lds r16, servo_level
    rcall Servo_set_level

    ; measure distance
    rcall measure_and_filter

    rcall buzz_tick

main_lcd:
    rcall lcd_show
    WAIT_MS LOOP_DELAY_MS
    rjmp main

; displays POWER OFF, buttons disabled, no sound, no rotation
main_power_off:
    ldi r16, 0
    rcall Servo_set_level
    cbi PORTE, SPEAKER

    rcall LCD_home
    PRINTF LCD
    .db "POWER OFF ", 0
    ldi r18, 0x40
    rcall LCD_pos
    PRINTF LCD
    .db " ", 0

    WAIT_MS POWEROFF_DELAY_MS
    rjmp main

; power was justed turned on, wait for STARTUP_DELAY_MS for display
main_power_on_seq:
    rcall LCD_home
    PRINTF LCD
    .db "POWER ON ", 0
    ldi r18, 0x40
    rcall LCD_pos
    PRINTF LCD
    .db " Starting... ", 0

    WAIT_MS STARTUP_DELAY_MS

    ; re-initialise state (same defaults as reset, without the one-time setup)
    ldi r16, 0
    sts mode, r16
    sts servo_level, r16
    sts power_state, r16
    sts step_timer, r16
    sts distance_cm, r16
    sts dist_prev, r16
    sts dist_display, r16
```

```
    sts buzz_counter, r16
    ldi r16, 1
    sts buzzer_on, r16
    ldi r16, low(Park_tone)
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16
    ldi r16, 0
    rcall Servo_set_level

    rjmp main


; mode RADAR logic
main_radar:
    ; switch the tone of the buzzer
    ldi r16, low(Radar_tone)
    sts tone_fn, r16
    ldi r16, high(Radar_tone)
    sts tone_fn+1, r16


    rcall Servo_level_half ; set direction to right
    ldi r18, SWEEP_STEPS

main_radar_fwd:
    push r18

    ; same as PARK logic
    rcall measure_and_filter
    rcall buzz_tick
    rcall lcd_show
    rcall deb_tick
    rcall sweep_dwell ; wait on the current posision for SWEEP_STEP_MS ms

    pop r18

    lds r16, mode
    cpi r16, 1
    brne main_radar_exit
    dec r18
    brne main_radar_fwd

    rcall Servo_level_mhalf ; change the direction
    ldi r18, SWEEP_STEPS

main_radar_bwd:
    push r18

    rcall measure_and_filter
    rcall buzz_tick
    rcall lcd_show
    rcall deb_tick
    rcall sweep_dwell ; wait on the current posision for SWEEP_STEP_MS ms

    pop r18

    lds r16, mode
    cpi r16, 1
    brne main_radar_exit
    dec r18
    brne main_radar_bwd

    rjmp main_radar

main_radar_exit:
    ; restore the state of the system to PARK/MANUAL mode
    ldi r16, low(Park_tone)
```

```
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16
    ldi r16, 0
    sts servo_level, r16
    rcall Servo_set_level
    rjmp main_lcd


; delay for each small sweep step ( basically speed adjusting )
sweep_dwell:
    WAIT_MS SWEEP_STEP_MS
    ret

; sound (beeping) logic
buzz_tick:

    ; muted
    lds r16, buzzer_on
    cpi r16, 0
    breq bt_silent


    lds r16, buzz_counter
    cpi r16, 0
    brne bt_wait

    ; checks if beeping is needed (if distance <= 50 cm)
    lds r21, dist_display
    cpi r21, DIST_NONE
    breq bt_silent ; no echo -> silence
    cpi r21, DIST_FAR
    brsh bt_silent ; >= 50 cm -> silence

    ; play one beep with tone corresponding to the mode
    lds zl, tone_fn
    lds zh, tone_fn+1
    icall

    ; reload rhythme counter based on current distance.
    lds r21, dist_display
    cpi r21, DIST_NEAR
    brsh bt_check25
    ldi r16, BUZZ_REPEAT_VFAST ; < 10 cm
    rjmp bt_set

bt_check25:
    cpi r21, DIST_MID
    brsh bt_slow
    ldi r16, BUZZ_REPEAT_FAST ; 10-24 cm
    rjmp bt_set

bt_slow:
    ldi r16, BUZZ_REPEAT_SLOW ; 25-49 cm

bt_set:
    sts buzz_counter, r16
    ret

bt_wait:
    dec r16
    sts buzz_counter, r16
    ret

bt_silent:
    cbi PORTE, SPEAKER
    ldi r16, 0
    sts buzz_counter, r16
```

```
    ret


; read ultrasonic sensor logic
measure_and_filter:
    push r16
    push r17

    rcall HC_measure ; r21 = distance in cm
    sts distance_cm, r21

    ; r16 = previous sample then remember the new raw sample.
    lds r16, dist_prev
    sts dist_prev, r21

    ; check if the previous or current distance is available
    cpi r21, DIST_NONE
    breq mf_accept
    cpi r16, DIST_NONE
    breq mf_accept

    ; r17 = |new - prev|
    mov r17, r21
    sub r17, r16
    brpl mf_pos
    neg r17

mf_pos:
    cpi r17, DIST_JUMP_LIMIT
    brlo mf_accept ; small change -> accept
    ; large jump -> treat as a glitch, keep the previous value.
    mov r21, r16

mf_accept:
    sts dist_display, r21

    pop r17
    pop r16
    ret


; line 1: mode + speed level
; line 2: distance in cm or --- when no echo
lcd_show:
    push r20
    push r18

    ; line 1
    rcall LCD_home
    lds r16, mode

    cpi r16, 2
    brne lcd_chk_radar

    ; MANUAL: show the servo speed level
    lds r20, servo_level
    PRINTF LCD
    .db "Manual s=", FDEC|FSIGN, 20, " ", 0
    rjmp lcd_line2

lcd_chk_radar:
    cpi r16, 1
    brne lcd_park

    PRINTF LCD
    .db "Radar s= 0 ", 0
    rjmp lcd_line2
```

```
lcd_park:
    PRINTF LCD
    .db "Park s= 0 ", 0

lcd_line2:
    ; line 2
    ldi r18, 0x40
    rcall LCD_pos

    lds r20, dist_display
    cpi r20, DIST_NONE
    brne lcd_dist_ok

    PRINTF LCD
    .db "Dist: --- cm ", 0
    rjmp lcd_end

lcd_dist_ok:
    PRINTF LCD
    .db "Dist: ", FDEC|FDIG3, 20, " cm ", 0

lcd_end:
    pop r18
    pop r20
    ret
```

ir_rc5.asm

```
; === IR RC-5 Receiver Driver ===
; file ir_rc5.asm target ATmega128L-4MHz-STK300
; IR RC-5 Receiver logic

.equ T1 = 1778 ; RC-5 bit period in microseconds

; IR command codes
.equ CMD_VOLUP = 0xE0 ; VOL+
.equ CMD_VOLDN = 0xE1 ; VOL-
.equ CMD_UP = 0xC0 ; UP
.equ CMD_DOWN = 0xC1 ; DOWN
.equ CMD_GUIDE = 0xC2 ; GUIDE
.equ CMD_POWER = 0xEC ; POWER
.equ CMD_MUTE = 0xED ; MUTE

.equ STEP_KICK_PASSES = 12


; deb_tick - debounce tick, call once per main loop iteration
; decrements deb_counter; re-enables INT7 when it reaches 0.
deb_tick:
    lds r16, deb_counter
    cpi r16, 0
    breq deb_tick_done
    dec r16
    sts deb_counter, r16
    cpi r16, 0
    brne deb_tick_done
    ldi r16, (1<<INT7)
    out EIMSK, r16
deb_tick_done:
    ret


; INT7_handler - IR interrupt service routine
; Decodes one RC-5 command sent
INT7_handler:
    push r16
    push r17
    push r18
    push r20
    push r22
    push r23
    push r24
    push r3
    in r16, SREG
    push r16

    ; decode 14-bit RC-5 frame
    ; wait T1/4 to align the first sample to the middle of the start bit,
    ; then sample one bit every T1 us. Each bit is shifted into b1:b0.
    CLR2 b1, b0
    ldi b2, 14
    WAIT_US (T1/4)

INT7_loop:
    P2C PINE, IR ; sample IR pin -> Carry
    ROL2 b1, b0 ; shift Carry into b1:b0 from LSB
    WAIT_US (T1-4) ; wait one bit period (minus loop overhead)
    DJNZ b2, INT7_loop ; repeat for all 14 bits

    sts last_cmd, b0 ; save command byte (low 8 bits of frame)

    cpi b0, CMD_POWER
    brne PC+2
```

```
    rjmp INT7_power

    ; all other commands are ignored while powered off
    lds r16, power_state
    cpi r16, 0
    brne PC+2
    rjmp INT7_dispatch
    rjmp INT7_end

INT7_dispatch:
    cpi b0, CMD_MUTE
    brne PC+2
    rjmp INT7_mute
    cpi b0, CMD_VOLUP
    brne PC+2
    rjmp INT7_vol_up
    cpi b0, CMD_VOLDN
    brne PC+2
    rjmp INT7_vol_dn
    cpi b0, CMD_UP
    brne PC+2
    rjmp INT7_step_fwd
    cpi b0, CMD_DOWN
    brne PC+2
    rjmp INT7_step_bwd
    cpi b0, CMD_GUIDE
    brne PC+2
    rjmp INT7_guide
    rjmp INT7_end


; disables INT7 with deb_counter=5 to avoid bounce/repeat.
INT7_power:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 5
    sts deb_counter, r16

    lds r16, power_state
    cpi r16, 0
    breq INT7_power_off

    ldi r16, 2
    sts power_state, r16
    rjmp INT7_end
INT7_power_off:

    ldi r16, 1
    sts power_state, r16
    rjmp INT7_end

; mute
INT7_mute:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16

    lds r16, buzzer_on
    cpi r16, 1
    breq INT7_mute_off
    ldi r16, 1
    sts buzzer_on, r16
    rjmp INT7_end
INT7_mute_off:
    ldi r16, 0
    sts buzzer_on, r16
    rjmp INT7_end
```

```
; vol+ = increase/decrease speed (depends on the direction of the rotation)
INT7_vol_up:
    ldi r16, 2
    sts mode, r16
    ldi r16, 0
    sts step_timer, r16
    lds r17, servo_level
    cpi r17, 251
    brlo vup_not_neg
    inc r17
    rjmp vup_apply
vup_not_neg:
    cpi r17, 5
    brlo PC+2
    rjmp INT7_end
    inc r17
vup_apply:
    sts servo_level, r17
    mov r16, r17
    rcall Servo_set_level_raw
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16
    rjmp INT7_end

; vol- = decrease/increase speed (depends on the direction of the rotation)
INT7_vol_dn:
    ldi r16, 2
    sts mode, r16
    ldi r16, 0
    sts step_timer, r16
    lds r17, servo_level
    cpi r17, 6
    brsh vdn_not_pos
    cpi r17, 0
    breq vdn_from_zero
    dec r17
    rjmp vdn_apply
vdn_from_zero:
    ldi r17, 255
    rjmp vdn_apply
vdn_not_pos:
    cpi r17, 251
    brne PC+2
    rjmp INT7_end
    dec r17
vdn_apply:
    sts servo_level, r17
    mov r16, r17
    rcall Servo_set_level_raw
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16
    rjmp INT7_end

; small rotation in counter-clockwise direction
INT7_step_fwd:
    ldi r16, 2
    sts mode, r16
    ldi r17, 1
    sts servo_level, r17
    ldi r17, STEP_KICK_PASSES
    sts step_timer, r17
    rjmp INT7_end
```

```
; small rotation in clockwise direction
INT7_step_bwd:
    ldi r16, 2
    sts mode, r16
    ldi r17, 255
    sts servo_level, r17
    ldi r17, STEP_KICK_PASSES
    sts step_timer, r17
    rjmp INT7_end


; change modes MANUAL -> PARK, RADAR -> PARK, PARK -> RADAR
INT7_guide:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 5
    sts deb_counter, r16

    lds r16, mode
    cpi r16, 2 ; MANUAL
    breq guide_to_park
    cpi r16, 1 ; RADAR
    breq guide_to_park
    ; PARK -> RADAR
    ldi r16, 1
    sts mode, r16
    rjmp INT7_end
guide_to_park:
    ldi r16, 0
    sts mode, r16
    sts servo_level, r16
    sts step_timer, r16
    rcall Servo_set_level_raw
    rjmp INT7_end

INT7_end:
    pop r16
    out SREG, r16
    pop r3
    pop r24
    pop r23
    pop r22
    pop r20
    pop r18
    pop r17
    pop r16
    reti
```

HC-SR04.asm

```
; file HC-SR04.asm target ATmega128L - 4 MHz - STK300
; HC-SR04 ultrasonic distance sensor logic + beeper logic
;
;DISTANCE TIMING (why the divisor is 33)
; Speed of sound = 343 m/s, so a round trip of 1 cm takes 58.14 us.
; At 4 MHz one CPU cycle is 0.25 us.
; The echo-counting loop takes 7 cycles per pass = 1.75 us.
; Therefore 1 cm corresponds to 58.14 / 1.75 = 33.22 loop passes.
; Dividing the raw loop count by 33 gives exact results across the full range.

; PINs
.equ TRIG = 0 ; PD0
.equ ECHO = 1 ; PD1

; distances
.equ DIST_NEAR = 10 ; below this: very fast beeping
.equ DIST_MID = 25 ; below this: fast beeping
.equ DIST_FAR = 50 ; below this: slow beeping; above: silence
.equ DIST_NONE = 0xFF ; no echo / out of range

; count to centimeter divisor
; The HC_count loop is 7 cycles (1.75 us per pass):
; inc r21 (1) + brne taken (2) + sbis skip-1-word (2) + rjmp HC_count (2) = 7
; Round trip for 1 cm at 343 m/s = 58.14 us.
; Passes per cm = 58.14 / 1.75 = 33.22 -> divisor 33.
.equ CM_DIVISOR = 33


HC_init:
    sbi DDRD, TRIG ; TRIG is an output
    cbi DDRD, ECHO ; ECHO is an input
    cbi PORTD, TRIG ; TRIG idle low
    sbi DDRE, SPEAKER ; SPEAKER is an output
    cbi PORTE, SPEAKER ; buzzer off
    ret


; perform 1 distance measurement
HC_measure:
    push r20
    push r22
    push r23
    push r24
    push r25

    ; emit 10 us (40 cycles) trigger pulse
    ; required by the sensor
    sbi PORTD, TRIG
    ldi r25, 10
HC_trig_delay: ; waits 40 cycles (10 us)
    nop
    nop
    dec r25
    brne HC_trig_delay
    cbi PORTD, TRIG

    ; wait for echo to go high with ~30 ms timeout
    ; 120000 cycles = 30 ms
    ldi r22, 240
    ldi r23, 250
HC_wait_high:
    sbic PIND, ECHO
    rjmp HC_echo_started
    dec r23
    brne HC_wait_high
```

```
    ldi r23, 250
    dec r22
    brne HC_wait_high
    rjmp HC_timeout ; timeout if echo never rises

HC_echo_started:
    ; measure how long echo stays high
    clr r20
    clr r21
HC_count:
    inc r21 ; 1 cycle
    brne HC_count2 ; 1/2 cycles
    inc r20 ; carry into high byte when r21 wraps
HC_count2:
    sbis PIND, ECHO ; keep counting for high echo
    rjmp HC_done ; echo finished
    rjmp HC_count

HC_done:
    ; r20:r21 = raw loop count. Convert to centimetres.
    ldi r22, CM_DIVISOR
    rcall DIV16 ; r20:r21 = count / CM_DIVISOR

    ; fit the result to <= 254 for 0xff to be reserved for timeout
    tst r20
    brne HC_overflow ; high byte set -> result too large
    cpi r21, DIST_NONE
    brne HC_finish
HC_overflow:
    ldi r21, 0xFE ; 254 is far but still valid
    rjmp HC_finish

HC_timeout:
    ldi r21, DIST_NONE ; 0xFF = no echo / out of range
    clr r20

HC_finish:
    pop r25
    pop r24
    pop r23
    pop r22
    pop r20
    ret

; unsigned 16-bit / 8-bit division
DIV16:
    clr r23
    ldi r24, 16
DIV16_loop:
    lsl r21
    rol r20
    rol r23
    cp r23, r22
    brlo DIV16_skip
    sub r23, r22
    inc r21
DIV16_skip:
    dec r24
    brne DIV16_loop
    ret


; park_tone - 1 kHz beep
; 60 square-wave periods of 1 ms (0.5 ms high + 0.5 ms low)
Park_tone:
    push r24
    push r25
    ldi r24, 60
```

```
Park_tone_loop:
    sbi PORTE, SPEAKER
    WAIT_US 500
    cbi PORTE, SPEAKER
    WAIT_US 500
    dec r24
    brne Park_tone_loop
    pop r25
    pop r24
    ret


Radar_tone:
    push r24
    push r25
    push r27
    ldi r24, 20
    ldi r25, 200
Radar_tone_loop:
    sbi PORTE, SPEAKER
    mov r27, r25
Radar_delay1:
    nop
    dec r27
    brne Radar_delay1
    cbi PORTE, SPEAKER
    mov r27, r25
Radar_delay2:
    nop
    dec r27
    brne Radar_delay2
    dec r25
    dec r24
    brne Radar_tone_loop
    pop r27
    pop r25
    pop r24
    ret
```

servo.asm

```
; file servo.asm target ATmega128L - 4 MHz - STK300
; SG-90 servo-motor logic


; Timer3 OC3A = PE3
.equ SERVO_PIN = 3

; pulse widths (OCR3A counts) for each level
; the PWM period (ICR3) is 10000 counts; these values are the servo
; pulse lengths that produce each steering position.
.equ OCR_CENTRE = 760
.equ OCR_R1 = 765
.equ OCR_R2 = 770
.equ OCR_R3 = 775
.equ OCR_R4 = 780
.equ OCR_R5 = 785
.equ OCR_L1 = 750
.equ OCR_L2 = 745
.equ OCR_L3 = 740
.equ OCR_L4 = 735
.equ OCR_L5 = 725

; sweep end-points used by the radar mode
.equ OCR_SWEEP_R = 775
.equ OCR_SWEEP_L = 735

.equ SERVO_PERIOD = 10000 ; Timer3 TOP (ICR3) = PWM period


; load a 16-bit OCR value into r18:r17 (high:low) and return.
.macro SET_OCR ; @0 = OCR constant
    ldi r17, low(@0)
    ldi r18, high(@0)
    ret
    .endmacro


Servo_init:

    ; setup Timer3 to generate a PWM signal on the servo pin
    sbi DDRE, SERVO_PIN
    ldi r16, (1<<COM3A1)|(1<<WGM31)
    sts TCCR3A, r16
    ldi r16, (1<<WGM33)|(1<<WGM32)|(1<<CS31)
    sts TCCR3B, r16

    ldi r16, high(SERVO_PERIOD)
    sts ICR3H, r16
    ldi r16, low(SERVO_PERIOD)
    sts ICR3L, r16

    cli
    ldi r17, high(OCR_CENTRE)
    sts OCR3AH, r17
    ldi r17, low(OCR_CENTRE)
    sts OCR3AL, r17
    sei
    ret

; sets speed based on OCR value
Servo_set_level:
    push r17
    push r18
    rcall Servo_get_ocr ; r18:r17 = OCR value for this level
    cli
```

```
    sts OCR3AH, r18
    sts OCR3AL, r17
    sei
    pop r18
    pop r17
    ret

; servo_set_level without disabling interruptions
Servo_set_level_raw:
    push r17
    push r18
    rcall Servo_get_ocr
    sts OCR3AH, r18
    sts OCR3AL, r17
    pop r18
    pop r17
    ret

; sets OCR value based on the level
Servo_get_ocr:
    cpi r16, 0
    brne SG_1
    SET_OCR OCR_CENTRE
SG_1:
    cpi r16, 1
    brne SG_2
    SET_OCR OCR_R1
SG_2:
    cpi r16, 2
    brne SG_3
    SET_OCR OCR_R2
SG_3:
    cpi r16, 3
    brne SG_4
    SET_OCR OCR_R3
SG_4:
    cpi r16, 4
    brne SG_5
    SET_OCR OCR_R4
SG_5:
    cpi r16, 5
    brne SG_m1
    SET_OCR OCR_R5
SG_m1:
    cpi r16, 255
    brne SG_m2
    SET_OCR OCR_L1
SG_m2:
    cpi r16, 254
    brne SG_m3
    SET_OCR OCR_L2
SG_m3:
    cpi r16, 253
    brne SG_m4
    SET_OCR OCR_L3
SG_m4:
    cpi r16, 252
    brne SG_m5
    SET_OCR OCR_L4
SG_m5:
    cpi r16, 251
    brne SG_default
    SET_OCR OCR_L5
SG_default:
    SET_OCR OCR_CENTRE ; unknown level -> centre

; set servo to rotate right continuously
Servo_level_half:
```

```
    cli
    ldi r17, high(OCR_SWEEP_R)
    sts OCR3AH, r17
    ldi r17, low(OCR_SWEEP_R)
    sts OCR3AL, r17
    sei
    ret


; set servo to rotate left continuously
Servo_level_mhalf:
    cli
    ldi r17, high(OCR_SWEEP_L)
    sts OCR3AH, r17
    ldi r17, low(OCR_SWEEP_L)
    sts OCR3AL, r17
    sei
    ret
```

lcd.asm

```
; file lcd.asm target ATmega128L-4MHz-STK300
; purpose LCD HD44780U library
; ATmega 128 and Atmel Studio 7.0 compliant

; === definitions ===
.equ LCD_IR = 0x8000 ; address LCD instruction reg
.equ LCD_DR = 0xc000 ; address LCD data register

; === subroutines ===
LCD_wr_ir:
; in w (byte to write to LCD IR)
    lds u, LCD_IR ; read IR to check busy flag (bit7)
    JB1 u,7,LCD_wr_ir ; Jump if Bit=1 (still busy)
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_IR, w ; store w in IR
    ret

lcd_4us:
    rcall lcd_2us ; recursive call
lcd_2us:
    nop ; rcall(3) + nop(1) + ret(4) = 8 cycles (2us)
    ret

LCD:
LCD_putc:
    JK a0,CR,LCD_cr ; Jump if a0=CR
    JK a0,LF,LCD_lf ; Jump if a0=LF
LCD_wr_dr:
; in a0 (byte to write to LCD DR)
    lds w, LCD_IR ; read IR to check busy flag (bit7)
    JB1 w,7,LCD_wr_dr ; Jump if Bit=1 (still busy)
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_DR, a0 ; store a0 in DR
    ret

LCD_clear: JW LCD_wr_ir, 0b00000001 ; clear display
LCD_home: JW LCD_wr_ir, 0b00000010 ; return home
LCD_cursor_left: JW LCD_wr_ir, 0b00010000 ; move cursor to left
LCD_cursor_right: JW LCD_wr_ir, 0b00010100 ; move cursor to right
LCD_display_left: JW LCD_wr_ir, 0b00011000 ; shifts display to left
LCD_display_right: JW LCD_wr_ir, 0b00011100 ; shifts display to right
LCD_blink_on: JW LCD_wr_ir, 0b00001101 ; Display=1,Cursor=0,Blink=1
LCD_blink_off: JW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0
LCD_cursor_on: JW LCD_wr_ir, 0b00001110 ; Display=1,Cursor=1,Blink=0
LCD_cursor_off: JW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0

LCD_init:
    in w,MCUCR ; enable access to ext. SRAM
    sbr w,(1<<SRE)+(1<<SRW10)
    out MCUCR,w
    CW LCD_wr_ir, 0b00000001 ; clear display
    CW LCD_wr_ir, 0b00000110 ; entry mode set (Inc=1, Shift=0)
    CW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0
    CW LCD_wr_ir, 0b00111000 ; 8bits=1, 2lines=1, 5x8dots=0
    ret

LCD_pos:
; in a0 = position (0x00..0x0f first line, 0x40..0x4f second line)
    mov w,a0
    ori w,0b10000000
    rjmp LCD_wr_ir

LCD_cr:
; moving the cursor to the beginning of the line (carriage return)
    lds w, LCD_IR ; read IR to check busy flag (bit7)
```

```
    JB1 w,7,LCD_cr ; Jump if Bit=1 (still busy)
    andi w,0b01000000 ; keep bit6 (begin of line 1/2)
    ori w,0b10000000 ; write address command
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_IR,w ; store in IR
    ret

LCD_lf:
; moving the cursor to the beginning of the line 2 (line feed)
    push a0 ; safeguard a0
    ldi a0,$40 ; load position $40 (begin of line 2)
    rcall LCD_pos ; set cursor position
    pop a0 ; restore a0
    ret
```

printf.asm

```
; file printf.asm target ATmega128L-4MHz-STK300
; purpose library, formatted output generation
; author (c) R.Holzer (adapted MICRO210/EE208 A.Schmid)
; v2019.02 20180821 AxS supports SRAM input from 0x0260
; through 0x02ff that should be reserved

; === description ===
;
; The program "printf" interprets and prints formatted strings.
; The special formatting characters regognized are:
;
; FDEC decimal number
; FHEX hexadecimal number
; FBIN binary number
; FFRAC fixed fraction number
; FCHAR single ASCII character
; FSTR zero-terminated ASCII string
;
; The special formatting characters are distinguished from normal
; ASCII characters by having their bit7 set to 1.
;
; Signification of bit fields:
;
; b bytes 1..4 b bytes 2
; s sign 0(unsigned), 1(signed) 1
; i integer digits
; e base 2,,36 5
; dp dec. point 0..32 5
; $if i=integer digits, 0=all digits, 1..15 digits
; f=fraction digits, 0=no fraction, 1..15 digits
;
; Formatting characters must be followed by an SRAM address (0..ff)
; that determines the origin of variables that must be printed (if any)
; FBIN, sram
; FHEX, sram
; FDEC, sram
; FCHAR,sram
; FSTR, sram
;
; The address 'sram' is a 1-byte constant. It addresses
; 0..1f registers r0..r31,
; 20..3f i/o ports, (need to be addressed with an offset of $20)
; 0x0260..0x02ff SRAM
; Variables can be located into register and I/0s, and can also
; be stored into data SRAM at locations 0x0200 through 0x02ff. Any
; sram address higher than 0x0060 is assumed to be at (0x0260+address)
; from automatic address detection in _printf_formatted: and subsequent
; assignment to xh; xl keeps its value. Consequently, variables that are
; to be stored into SRAM and further printed by fprint must reside at
; 0x0200 up to 0x02ff, and must be addressed using a label. Usage: see
; file string1.asm, for example.

; The FFRAC formatting character must be followed by
; ONE sram address and
; TWO more formatting characters
; FFRAC,sram,dp,$if

; dp decimal point position, 0=right, 32=left
; $if format i.f, i=integer digits, f=fraction digits

; The special formatting characters use the following coding
;
; FDEC 11bb'iiis i=0 all digits, i=1-7 digits
; FBIN 101i'iiis i=0 8 digits, i=1-7 digits
; FHEX 1001'iiis i=0 8 digits, i=1-7 digits
```

```
; FFRAC 1000'1bbs
; FCHAR 1000'0100
; FSTR 1000'0101
; FREP 1000'0110
; FFUNC 1000'0111
; 1000'0010
; 1000'0011
; FESC 1000'0000

; examples
; formatting string printing
; "a=",FDEC,a,0 1-byte variable a, unsigned decimal
; "a=",FDEC2,a,0 2-byte variable a (a1,a0), unsigend
; "a=",FDEC|FSIGN,a,0 1-byte variable 1, signed decimal
; "n=",FBIN,PIND+$20,0 i/o port, binary, notice offset of $20
; "f=",FFRAC4|FSIGN,a,16,$88,0 4-byte signed fixed-point fraction
; dec.point at 16, 8 int.digits, 8 frac.digits
; "f=",FFRAC2,a,16,$18,0 2-byte unsigned fixed-point fraction
; dec.point at 16, 1 int.digits, 8 frac.digits
; "a=",FDEC|FDIG5|FSIGN,a,0 1-byte variable, 5-digit, decimal, signed
; "a=",FDEC|FDIG5,a,0 1-byte variable, 5-digit, decimal, unsigned

; === registers modified ===
; e0,e1 used to transmit address of putc routine
; zh,zl used as pointer to prog-memory

; === constants ==============================================

.equ FDEC = 0b11000000 ; 1-byte variable
.equ FDEC2 = 0b11010000 ; 2-byte variable
.equ FDEC3 = 0b11100000 ; 3-byte variable
.equ FDEC4 = 0b11110000 ; 4-byte variable

.equ FBIN = 0b10100000
.equ FHEX = 0b10010100 ; 1-byte variable
.equ FHEX2 = 0b10011000 ; 2-byte variable
.equ FHEX3 = 0b10011100 ; 3-byte variable
.equ FHEX4 = 0b10010000 ; 4-byte variable

.equ FFRAC = 0b10001000 ; 1-byte variable
.equ FFRAC2 = 0b10001010 ; 2-byte variable
.equ FFRAC3 = 0b10001100 ; 3-byte variable
.equ FFRAC4 = 0b10001110 ; 4-byte variable

.equ FCHAR = 0b10000100
.equ FSTR = 0b10000101

.equ FSIGN = 0b00000001

.equ FDIG1 = 1<<1
.equ FDIG2 = 2<<1
.equ FDIG3 = 3<<1
.equ FDIG4 = 4<<1
.equ FDIG5 = 5<<1
.equ FDIG6 = 6<<1
.equ FDIG7 = 7<<1

; ===macro ============================================================

.macro PRINTF ; putc function (UART, LCD...)
    ldi w, low(@0) ; address of "putc" in e1:d0
    mov e0,w
    ldi w,high(@0)
    mov e1,w
    rcall _printf
    .endmacro

; mod y,z
```

```
; === routines ================================================

_printf:
    POPZ ; z points to begin of "string"
    MUL2Z ; multiply Z by two, (word ptr -> byte ptr)
    PUSHX

_printf_read:
    lpm ; places prog_mem(Z) into r0 (=c)
    adiw zl,1 ; increment pointer Z
    tst r0 ; test for ZERO (=end of string)
    breq _printf_end ; char=0 indicates end of ascii string
    brmi _printf_formatted ; bit7=1 indicates formatting character
    mov w,r0
    rcall _putw ; display the character
    rjmp _printf_read ; read next character in the string

_printf_end:
    adiw zl,1 ; point to the next character
    DIV2Z ; divide by 2 (byte ptr -> word ptr)
    POPX
    ijmp ; return to instruction after "string"

_printf_formatted:

; FDEC 11bb'iiis
; FBIN 101i'iiis
; FHEX 1001'iiis
; FFRAC 1000'1bbs
; FCHAR 1000'0100
; FSTR 1000'0101

    bst r0,0 ; store sign in T
    mov w,r0 ; store formatting character in w
    lpm
    mov xl,r0 ; load x-pointer with SRAM address
    cpi xl,0x60
    brlo rio_space
dataram_space: ; variable originates from SRAM memory
    ldi xh,0x02 ;>addresses are limited to 0x0260 through 0x02ff
    rjmp space_detect_end ;>that enables automatic detection of the origin
rio_space: ; variable originates from reg or I/O space
    clr xh ; clear high-byte, addresses are 0x0000 through 0x003f (0x005f)
space_detect_end:
     adiw zl,1 ; increment pointer Z

; JB1 w,6,_putdec
; JB1 w,5,_putbin
; JB1 w,4,_puthex
; JB1 w,3,_putfrac
    JK w,FCHAR,_putchar
    JK w,FSTR ,_putstr
    rjmp _putnum

    rjmp _printf_read

; === putc (put character) ================================
; in w character to put
; e1,e0 address of output routine (UART, LCD putc)
_putw:
    PUSH3 a0,zh,zl
    MOV3 a0,zh,zl, w,e1,e0
    icall ; indirect call to "putc"
    POP3 a0,zh,zl
    ret
```

```
; === putchar (put character) ============================
; in x pointer to character to put
_putchar:
    ld w,x
    rcall _putw
    rjmp _printf_read

; === putstr (put string) ================================
; in x pointer to ascii string
; b3,b2 address of output routine (UART, LCD putc)
_putstr:
    ld w,x+
    tst w
    brne PC+2
    rjmp _printf_read
    rcall _putw
    rjmp _putstr

; === putnum (dec/bin/hex/frac) ==========================
; in x pointer to SRAM variable to print
; r0 formatting character

_putnum:
    PUSH4 a3,a2,a1,a0 ; safeguard a
    PUSH4 b3,b2,b1,b0 ; safeguard b
    LDX4 a3,a2,a1,a0 ; load operand to print into a

; FDEC 11bb'iiis
; FBIN 101i'iiis
; FHEX 1001'iiis
; FRACT 1000'1bbs

    JB1 w,6,_putdec
    JB1 w,5,_putbin
    JB1 w,4,_puthex
    JB1 w,3,_putfrac

; FDEC 11bb'iiis
_putdec:
    ldi b0,10 ; b0 = base (10)

    mov b1,w
    lsr b1
    andi b1,0b111
    swap b1 ; b1 = format 0iii'0000 (integer digits)
    ldi b2,0 ; b2 = dec. point position = 0 (right)

    mov b3,w
    swap b3
    andi b3,0b11
    inc b3 ; b3 = number of bytes (1..4)
    rjmp _getnum ; get number of digits (iii)

; FBIN 101i'iiis addr
_putbin:
    ldi b0,2 ; b0 = base (2)
    ldi b3,4 ; b3 = number of bytes (4)
    rjmp _getdig ; get number of digits (iii)

; FHEX 1001'iiis addr
_puthex:
    ldi b0,16 ; b0 = base (16)
    ldi b3,4 ; b3 = number of bytes (4)
    rjmp _getdig

_getdig:
    mov b1,w
    lsr b1
```

```
    andi b1,0b111
    brne PC+2
    ldi b1,8 ; if b1=0 then 8-digits
    swap b1 ; b1 = format 0iii'0000 (integer digits)
    ldi b2, 0 ; b2 = dec. point position = 0 (right)
    rjmp _getnum

; FFRAC 1000'1bbs addr 00dd'dddd, iiii'ffff

_putfrac:
    ldi b0,10 ; base=10
    lpm
    mov b2,r0 ; load dec.point position
    adiw zl,1 ; increment char pointer
    lpm
    mov b1,r0 ; load ii.ff format
    adiw zl,1 ; increment char pointer

    mov b3,w
    asr b3
    andi b3,0b11
    inc b3 ; b3 = number of bytes (1..4)

    rjmp _getnum

_getnum:
; in a 4-byte variable
; b3 number of bytes (1..4)
; T sign, 0=unsigned, 1=signed

    JK b3,4,_printf_4b
    JK b3,3,_printf_3b
    JK b3,2,_printf_2b

_printf_1b: ; sign extension
    clr a1
    brtc PC+3 ; T=1 sign extension
    sbrc a0,7
    ldi a1,0xff
_printf_2b:
    clr a2
    brtc PC+3 ; T=1 sign extension
    sbrc a1,7
    ldi a2,0xff
_printf_3b:
    clr a3
    brtc PC+3 ; T=1 sign extension
    sbrc a2,7
    ldi a3,0xff
_printf_4b:

    rcall _ftoa ; float to ascii
    POP4 b3,b2,b1,b0 ; restore b
    POP4 a3,a2,a1,a0 ; restore a

    rjmp _printf_read

; ================================================
; func ftoa
; converts a fixed-point fractional number to an ascii string
; author (c) Raphael Holzer
;
; in a3-a0 variable to print
; b0 base, 2 to 36, but usually decimal (10)
; b1 number of digits to print ii.ff
; b2 position of the decimal point (0=right, 32=left)
; T sign (T=0 unsiged, T=1 signed)
```

```
_ftoa:
    push d0
    PUSH4 c3,c2,c1,c0 ; c = fraction part, a = integer part
    CLR4 c3,c2,c1,c0 ; clear fraction part

    brtc _ftoa_plus ; if T=0 then unsigned
    clt
    tst a3 ; if MSb(a)=1 then a=-a
    brpl _ftoa_plus
    set ; T=1 (minus)
    tst b1
    breq PC+2 ; if b1=0 the print ALL digits
    subi b1,0x10 ; decrease int digits
    NEG4 a3,a2,a1,a0 ; negate a
_ftoa_plus:
    tst b2 ; b0=0 (only integer part)
    breq _ftoa_int
_ftoa_shift:
    ASR4 a3,a2,a1,a0 ; a = integer part
    ROR4 c3,c2,c1,c0 ; c = fraction part
    DJNZ b2,_ftoa_shift
_ftoa_int:
    push b1 ; ii.ff (ii=int digits)
    swap b1
    andi b1,0x0f

    ldi w,'.' ; push decimal point
    push w
_ftoa_int1:
    rcall _div41 ; int=int/10
    mov w,d0 ; d=reminder
    rcall _hex2asc
    push w ; push rem(int/10)
    TST4 a3,a2,a1,a0 ; (int/10)=?
    breq _ftoa_space ; (int/10)=0 then finished
    tst b1
    breq _ftoa_int1 ; if b1=0 then print ALL int-digits
    DJNZ b1,_ftoa_int1
    rjmp _ftoa_sign
_ftoa_space:
    tst b1 ; if b1=0 then print ALL int-digits
    breq _ftoa_sign
    dec b1
    breq _ftoa_sign
    ldi w,' ' ; write spaces
    rcall _putw
    rjmp _ftoa_space
_ftoa_sign:
    brtc PC+3 ; if T=1 then write 'minus'
    ldi w,'-'
    rcall _putw
_ftoa_int3:
    pop w
    cpi w,'.'
    breq PC+3
    rcall _putw
    rjmp _ftoa_int3

    pop b1 ; ii.ff (ff=frac digits)
    andi b1,0x0f
    tst b1
    breq _ftoa_end
_ftoa_point:
    rcall _putw ; write decimal point
    MOV4 a3,a2,a1,a0, c3,c2,c1,c0
_ftoa_frac:
    rcall _mul41 ; d.frac=10*frac
    mov w,d0
```

```
    rcall _hex2asc
    rcall _putw
    DJNZ b1,_ftoa_frac
_ftoa_end:
    POP4 c3,c2,c1,c0
    pop d0
    ret

; === hexadecimal to ascii ===
; in w
_hex2asc:
    cpi w,10
    brsh PC+3
    addi w,'0'
    ret
    addi w,('a'-10)
    ret

; === multiply 4byte*1byte ===
; funct mul41
; multiplies a3-a0 (4-byte) by b0 (1-byte)
; author (c) Raphael Holzer, EPFL
;
; in a3..a0 multiplicand (argument to multiply)
; b0 multiplier
; out a3..a0 result
; d0 result MSB (byte 4)
;
_mul41: clr d0 ; clear byte4 of result
    ldi w,32 ; load bit counter
__m41: clc ; clear carry
    sbrc a0,0 ; skip addition if LSB=0
    add d0,b0 ; add b to MSB of a
    ROR5 d0,a3,a2,a1,a0 ; shift-right c, LSB (of b) into carry
    DJNZ w,__m41 ; Decrement and Jump if bit-count Not Zero
    ret

; === divide 4byte/1byte ===
; func div41
; in a0..a3 divident (argument to divide)
; b0 divider
; out a0..a3 result
; d0 reminder
;
_div41: clr d0 ; d will contain the remainder
    ldi w,32 ; load bit counter
__d41: ROL5 d0,a3,a2,a1,a0 ; shift carry into result c
    sub d0, b0 ; subtract b from remainder
    brcc PC+2
    add d0, b0 ; restore if remainder became negative
    DJNZ w,__d41 ; Decrement and Jump if bit-count Not Zero
    ROL4 a3,a2,a1,a0 ; last shift (carry into result c)
    COM4 a3,a2,a1,a0 ; complement result
    ret
```

definitions.asm

```
;=====================================================================
; definitions.asm - register aliases, ASCII codes, board pin map
;=====================================================================
;
; WHAT THIS FILE DOES
; Central place for symbolic names used across the whole project:
; - named working registers (char, u, w, the multi-byte a/b/c/d
; register groups, and the X/Y/Z pointer aliases)
; - ASCII control codes
; - STK-300 / module pin assignments (speaker, IR, encoder, servo,
; I2C, sensors, etc.)
; It defines no code, only .def/.equ/.set. Included first so every
; other file can use these names.
;
; Target: ATmega128L-4MHz-STK300
;=====================================================================

; === definitions ===
.nolist ; do not include in listing
.set clock = 4000000

.def char = r0 ; character (ASCII)
.def _sreg = r1 ; saves the status during interrupts
.def _u = r2 ; saves working reg u during interrupt
.def u = r3 ; scratch register (macros, routines)

.def e0 = r4 ; temporary reg for PRINTF
.def e1 = r5

.equ c = 8
.def c0 = r8 ; 8-byte register c
.def c1 = r9
.def c2 = r10
.def c3 = r11

.equ d = 12 ; 4-byte register d (overlapping with c)
.def d0 = r12
.def d1 = r13
.def d2 = r14
.def d3 = r15

.def w = r16 ; working register for macros
.def _w = r17 ; working register for interrupts

.equ a = 18
.def a0 = r18 ; 4-byte register a
.def a1 = r19
.def a2 = r20
.def a3 = r21

.equ b = 22
.def b0 = r22 ; 4-byte register b
.def b1 = r23
.def b2 = r24
.def b3 = r25

.equ px = 26 ; pointer x
.equ py = 28 ; pointer y
.equ pz = 30 ; pointer z

; === ASCII codes
.equ BEL =0x07 ; bell
.equ HT =0x09 ; horizontal tab
.equ TAB =0x09 ; tab
.equ LF =0x0a ; line feed
```

```
.equ VT =0x0b ; vertical tab
.equ FF =0x0c ; form feed
.equ CR =0x0d ; carriage return
.equ SPACE =0x20 ; space code
.equ DEL =0x7f ; delete
.equ BS =0x08 ; back space

; === STK-300 ===
.equ LED = PORTB ; LEDs on STK-300
.equ BUTTON = PIND ; buttons on the STK-300

; === module M2 (encoder/speaker/IR remote) ===
.equ SPEAKER = 2 ; piezo speaker
.equ ENCOD_A = 4 ; angular encoder A
.equ ENCOD_B = 5 ; angular encoder B
.equ ENCOD_I = 6 ; angular encoder button
.equ IR = 7 ; IR module for PCM remote control system

; === module M5 (I2C/1Wire) ===
.equ SCL = 0 ; I2C serial clock
.equ SDA = 1 ; I2C serial data
.equ DQ = 5 ; Dallas 1Wire
               ; master transmitter status codes, Table 88
.equ I2CMT_START = 0x08 ; start
.equ I2CMT_REPSTART = 0x10 ; repeated start
.equ I2CMT_SLA_ACK= 0x18 ; slave ack
.equ I2CMT_SLA_NOACK = 0x20 ; slave no ack
.equ I2CMT_DATA_ACK = 0x28 ; data write, ack
.equ I2CMT_DATA_NOACK = 0x30 ; data write, no ack
               ; master receiver status codes, Table 89
.equ I2CMR_SLA_ACK = 0x40 ; slave address ack
.equ I2CMR_SLA_NACK = 0x48 ; slave address no ack
.equ I2CMR_DATA_ACK = 0x50 ; master data ack
.equ I2CMR_DATA_NACK= 0x58 ; master data no ack

; === module M4 (Keyboard/Sharp/Servo) ===
.equ KB_CLK = 0 ; PC-AT keyboard clock line
.equ KB_DAT = 1 ; PC-AT keyboard data line
.equ GP2_CLK = 2 ; Sharp GP2D02 distance measuring sensor
.equ GP2_DAT = 3 ; Sharp GP2D02 distance measuring sensor
.equ GP2_AVAL = 3; Shart GP2Y0A21 distance measuring sensor
.equ SERVO1 = 4 ; Futaba position servo

; === module M3 (potentiometer/BNC) ===
.equ POT = 0 ; potentiometer
.equ BNC1 = 2 ; BNC input
.equ BNC2 = 4 ; BNC input
.list
```

macros.asm

```
; file: macros.asm target ATmega128L-4MHz-STK300
; purpose library, general-purpose macros
; author (c) R.Holzer (adapted MICRO210/EE208 A.Schmid)
; v2019.01 20180820 AxS

; ==============
; pointers
; ==============

; --- loading an immediate into a pointer XYZ,SP ---
.macro LDIX ; sram
    ldi xl, low(@0)
    ldi xh,high(@0)
    .endmacro
.macro LDIY ; sram
    ldi yl, low(@0)
    ldi yh,high(@0)
    .endmacro
.macro LDIZ ; sram
    ldi zl, low(@0)
    ldi zh,high(@0)

    .endmacro
.macro LDZD ; sram, reg ; sram+reg -> Z
    mov zl,@1
    clr zh
    subi zl, low(-@0)
    sbci zh,high(-@0)
    .endmacro
.macro LDSP ; sram
    ldi r16, low(@0)
    out spl,r16
    ldi r16,high(@0)
    out sph,r16
    .endmacro

; --- load/store SRAM addr into pointer XYZ ---
.macro LDSX ; sram
    lds xl,@0
    lds xh,@0+1
    .endmacro
.macro LDSY ; sram
    lds yl,@0
    lds yh,@0+1
    .endmacro
.macro LDSZ ; sram
    lds zl,@0
    lds zh,@0+1
    .endmacro
.macro STSX ; sram
    sts @0, xl
    sts @0+1,xh
    .endmacro
.macro STSY ; sram
    sts @0, yl
    sts @0+1,yh
    .endmacro
.macro STSZ ; sram
    sts @0, zl
    sts @0+1,zh
    .endmacro

; --- push/pop pointer XYZ ---
.macro PUSHX ; push X
    push xl
```

```
    push xh
    .endmacro
.macro POPX ; pop X
    pop xh
    pop xl
    .endmacro

.macro PUSHY ; push Y
    push yl
    push yh
    .endmacro
.macro POPY ; pop Y
    pop yh
    pop yl
    .endmacro

.macro PUSHZ ; push Z
    push zl
    push zh
    .endmacro
.macro POPZ ; pop Z
    pop zh
    pop zl
    .endmacro

; --- multiply/divide Z ---
.macro MUL2Z ; multiply Z by 2
    lsl zl
    rol zh
    .endmacro
.macro DIV2Z ; divide Z by 2
    lsr zh
    ror zl
    .endmacro

; --- add register to pointer XYZ ---
.macro ADDX ;reg ; x <- y+reg
    add xl,@0
    brcc PC+2
    subi xh,-1 ; add carry
    .endmacro
.macro ADDY ;reg ; y <- y+reg
    add yl,@0
    brcc PC+2
    subi yh,-1 ; add carry
    .endmacro
.macro ADDZ ;reg ; z <- z+reg
    add zl,@0
    brcc PC+2
    subi zh,-1 ; add carry
    .endmacro

; ===================
; miscellaneous
; ===================

; --- output/store (regular I/O space) immediate value ---
.macro OUTI ; port,k output immediate value to port
    ldi w,@1
    out @0,w
    .endmacro

; --- output/store (extended I/O space) immediate value ---
.macro OUTEI ; port,k output immediate value to port
    ldi w,@1
    sts @0,w
    .endmacro
```

```
; --- add immediate value ---
.macro ADDI
    subi @0,-@1
    .endmacro
.macro ADCI
    sbci @0,-@1
    .endmacro

; --- inc/dec with range limitation ---
.macro INC_LIM ; reg,limit
    cpi @0,@1
    brlo PC+3
    ldi @0,@1
    rjmp PC+2
    inc @0
    .endmacro

.macro DEC_LIM ; reg,limit
    cpi @0,@1
    breq PC+5
    brlo PC+3
    dec @0
    rjmp PC+2
    ldi @0,@1
    .endmacro

; --- inc/dec with cyclic range ---
.macro INC_CYC ; reg,low,high
    cpi @0,@2
    brsh _low ; reg>=high then reg=low
    cpi @0,@1
    brlo _low ; reg< low then reg=low
    inc @0
    rjmp _done
_low: ldi @0,@1
_done:
    .endmacro

.macro DEC_CYC ; reg,low,high
    cpi @0,@1
    breq _high ; reg=low then reg=high
    brlo _high ; reg<low then reg=high
    dec @0
    cpi @0,@2
    brsh _high ; reg>=high then high
    rjmp _done
_high: ldi @0,@2
_done:
    .endmacro

.macro INCDEC ;port,b1,b2,reg,low,high
    sbic @0,@1
    rjmp PC+6

    cpi @3,@5
    brlo PC+3
    ldi @3,@4
    rjmp PC+2
    inc @3

    sbic @0,@2
    rjmp PC+7

    cpi @3,@4
    breq PC+5
    brlo PC+3
    dec @3
    rjmp PC+2
```

```
    ldi @3,@5
    .endmacro

; --- wait loops ---
; wait 10...196608 cycles
.macro WAIT_C ; k
    ldi w, low((@0-7)/3)
    mov u,w ; u=LSB
    ldi w,high((@0-7)/3)+1 ; w=MSB
    dec u
    brne PC-1
    dec u
    dec w
    brne PC-4
    .endmacro

; wait micro-seconds (us)
; us = x*3*1000'000/clock) ==> x=us*clock/3000'000
.macro WAIT_US ; k
    ldi w, low((clock/1000*@0/3000)-1)
    mov u,w
    ldi w,high((clock/1000*@0/3000)-1)+1 ; set up: 3 cyles
    dec u
    brne PC-1 ; inner loop: 3 cycles
    dec u ; adjustment for outer loop
    dec w
    brne PC-4
    .endmacro

; wait mili-seconds (ms)
.macro WAIT_MS ; k
    ldi w, low(@0)
    mov u,w ; u = LSB
    ldi w,high(@0)+1 ; w = MSB
wait_ms:
    push w ; wait 1000 usec
    push u
    ldi w, low((clock/3000)-5)
    mov u,w
    ldi w,high((clock/3000)-5)+1
    dec u
    brne PC-1 ; inner loop: 3 cycles
    dec u ; adjustment for outer loop
    dec w
    brne PC-4
    pop u
    pop w

    dec u
    brne wait_ms
    dec w
    brne wait_ms
    .endmacro

; --- conditional jumps/calls ---
.macro JC0 ; jump if carry=0
    brcs PC+2
    rjmp @0
    .endmacro
.macro JC1 ; jump if carry=1
    brcc PC+2
    rjmp @0
    .endmacro

.macro JK ; reg,k,addr ; jump if reg=k
    cpi @0,@1
    breq @2
    .endmacro
```

```
.macro _JK ; reg,k,addr ; jump if reg=k
    cpi @0,@1
    brne PC+2
    rjmp @2
    .endmacro
.macro JNK ; reg,k,addr ; jump if not(reg=k)
    cpi @0,@1
    brne @2
    .endmacro

.macro CK ; reg,k,addr ; call if reg=k
    cpi @0,@1
    brne PC+2
    rcall @2
    .endmacro
.macro CNK ; reg,k,addr ; call if not(reg=k)
    cpi @0,@1
    breq PC+2
    rcall @2
    .endmacro

.macro JSK ; sram,k,addr ; jump if sram=k
    lds w,@0
    cpi w,@1
    breq @2
    .endmacro
.macro JSNK ; sram,k,addr ; jump if not(sram=k)
    lds w,@0
    cpi w,@1
    brne @2
    .endmacro

; --- loops ---
.macro DJNZ ; reg,addr ; decr and jump if not zero
    dec @0
    brne @1
    .endmacro
.macro DJNK ; reg,k,addr ; decr and jump if not k
    dec @0
    cpi @0,@1
    brne @2
    .endmacro

.macro IJNZ ; reg,addr ; inc and jump if not zero
    inc @0
    brne @1
    .endmacro
.macro IJNK ; reg,k,addr ; inc and jump if not k
    inc @0
    cpi @0,@1
    brne @2
    .endmacro
.macro _IJNK ; reg,k,addr ; inc and jump if not k
    inc @0
    ldi w,@1
    cp @0,w
    brne @2
    .endmacro

.macro ISJNK ; sram,k,addr ; inc sram and jump if not k
    lds w,@0
    inc w
    sts @0,w
    cpi w,@1
    brne @2
    .endmacro
.macro _ISJNK ; sram,k,addr ; inc sram and jump if not k
    lds w,@0
```

```
    inc w
    sts @0,w
    cpi w,@1
    breq PC+2
    rjmp @2
    .endmacro

.macro DSJNK ; sram,k,addr ; dec sram and jump if not k
    lds w,@0
    dec w
    sts @0,w
    cpi w,@1
    brne @2
    .endmacro

; --- table lookup ---
.macro LOOKUP ;reg, index,tbl
    push ZL
    push ZH
    mov zl,@1 ; move index into z
    clr zh
    subi zl, low(-2*@2) ; add base address of table
    sbci zh,high(-2*@2)
    lpm ; load program memory (into r0)
    mov @0,r0
    pop ZH
    pop ZL
    .endmacro

.macro LOOKUP2 ;r1,r0, index,tbl
    mov zl,@2 ; move index into z
    clr zh
    lsl zl ; multiply by 2
    rol zh
    subi zl, low(-2*@3) ; add base address of table
    sbci zh,high(-2*@3)
    lpm ; get LSB byte
    mov w,r0 ; temporary store LSB in w
    adiw zl,1 ; increment Z
    lpm ; get MSB byte
    mov @0,r0 ; mov MSB to res1
    mov @1,w ; mov LSB to res0
    .endmacro

.macro LOOKUP4 ;r3,r2,r1,r0, index,tbl
    mov zl,@4 ; move index into z
    clr zh
    lsl zl ; multiply by 2
    rol zh
    lsl zl ; multiply by 2
    rol zh
    subi zl, low(-2*@5) ; add base address of table
    sbci zh,high(-2*@5)
    lpm
    mov @1,r0 ; load high word LSB
    adiw zl,1
    lpm
    mov @0,r0 ; load high word MSB
    adiw zl,1
    lpm
    mov @3,r0 ; load low word LSB
    adiw zl,1
    lpm
    mov @2,r0 ; load low word MSB
    .endmacro

.macro LOOKDOWN ;reg,index,tbl
    ldi ZL, low(2*@2) ; load table address
```

```
    ldi ZH,high(2*@2)
    clr @1
loop: lpm
    cp r0,@0
    breq found
    inc @1
    adiw ZL,1
    tst r0
    breq notfound
    rjmp loop
notfound:
    ldi @1,-1
found:
    .endmacro

; --- branch table ---
.macro C_TBL ; reg,tbl
    ldi ZL, low(2*@1)
    ldi ZH,high(2*@1)
    lsl @0
    add ZL,@0
    brcc PC+2
    inc ZH
    lpm
    push r0
    lpm
    mov zh,r0
    pop zl
    icall
    .endmacro
.macro J_TBL ; reg,tbl
    ldi ZL, low(2*@1)
    ldi ZH,high(2*@1)
    lsl @0
    add ZL,@0
    brcc PC+2
    inc ZH
    lpm
    push r0
    lpm
    mov zh,r0
    pop zl
    ijmp
    .endmacro

.macro BRANCH ; reg ; branching using the stack
    ldi w, low(tbl)
    add w,@0
    push w
    ldi w,high(tbl)
    brcc PC+2
    inc w
    push w
    ret
tbl:
    .endmacro

; --- multiply/division ---
.macro DIV2 ; reg
    lsr @0
    .endmacro
.macro DIV4 ; reg
    lsr @0
    lsr @0
    .endmacro
.macro DIV8 ; reg
    lsr @0
    lsr @0
```

```
    lsr @0
    .endmacro

.macro MUL2 ; reg
    lsl @0
    .endmacro
.macro MUL4 ; reg
    lsl @0
    lsl @0
    .endmacro
.macro MUL8 ; reg
    lsl @0
    lsl @0
    lsl @0
    .endmacro

; ====================================
; extending existing instructios
; ====================================

; --- immediate ops with r0..r15 ---
.macro _ADDI
    ldi w,@1
    add @0,w
    .endmacro
.macro _ADCI
    ldi w,@1
    adc @0,w
    .endmacro
.macro _SUBI
    ldi w,@1
    sub @0,w
    .endmacro
.macro _SBCI
    ldi w,@1
    sbc @0,w
    .endmacro
.macro _ANDI
    ldi w,@1
    and @0,w
    .endmacro
.macro _ORI
    ldi w,@1
    or @0,w
    .endmacro
.macro _EORI
    ldi w,@1
    eor @0,w
    .endmacro
.macro _SBR
    ldi w,@1
    or @0,w
    .endmacro
.macro _CBR
    ldi w,~@1
    and @0,w
    .endmacro
.macro _CPI
    ldi w,@1
    cp @0,w
    .endmacro
.macro _LDI
    ldi w,@1
    mov @0,w
    .endmacro

; --- bit access for port p32..p63 ---
.macro _SBI
```

```
    in w,@0
    ori w,1<<@1
    out @0,w
    .endmacro
.macro _CBI
    in w,@0
    andi w,~(1<<@1)
    out @0,w
    .endmacro

; --- extending branch distance to +/-2k ---
.macro _BREQ
    brne PC+2
    rjmp @0
    .endmacro
.macro _BRNE
    breq PC+2
    rjmp @0
    .endmacro
.macro _BRCS
    brcc PC+2
    rjmp @0
    .endmacro
.macro _BRCC
    brcs PC+2
    rjmp @0
    .endmacro
.macro _BRSH
    brlo PC+2
    rjmp @0
    .endmacro
.macro _BRLO
    brsh PC+2
    rjmp @0
    .endmacro
.macro _BRMI
    brpl PC+2
    rjmp @0
    .endmacro
.macro _BRPL
    brmi PC+2
    rjmp @0
    .endmacro
.macro _BRGE
    brlt PC+2
    rjmp @0
    .endmacro
.macro _BRLT
    brge PC+2
    rjmp @0
    .endmacro
.macro _BRHS
    brhc PC+2
    rjmp @0
    .endmacro
.macro _BRHC
    brhs PC+2
    rjmp @0
    .endmacro
.macro _BRTS
    brtc PC+2
    rjmp @0
    .endmacro
.macro _BRTC
    brts PC+2
    rjmp @0
    .endmacro
.macro _BRVS
```

```
    brvc PC+2
    rjmp @0
    .endmacro
.macro _BRVC
    brvs PC+2
    rjmp @0
    .endmacro
.macro _BRIE
    brid PC+2
    rjmp @0
    .endmacro
.macro _BRID
    brie PC+2
    rjmp @0
    .endmacro

; ====================
; bit operations
; ====================

; --- moving bits ---
.macro MOVB ; reg1,b1, reg2,b2 ; reg1,bit1 <- reg2,bit2
    bst @2,@3
    bld @0,@1
    .endmacro
.macro OUTB ; port1,b1, reg2,b2 ; port1,bit1 <- reg2,bit2
    sbrs @2,@3
    cbi @0,@1
    sbrc @2,@3
    sbi @0,@1
    .endmacro
.macro INB ; reg1,b1, port2,b2 ; reg1,bit1 <- port2,bit2
    sbis @2,@3
    cbr @0,1<<@1
    sbic @2,@3
    sbr @0,1<<@1
    .endmacro

.macro Z2C ; zero to carry
    sec
    breq PC+2 ; (Z=1)
    clc
    .endmacro
.macro Z2INVC ; zero to inverse carry
    sec
    brne PC+2 ; (Z=0)
    clc
    .endmacro

.macro C2Z ; carry to zero
    sez
    brcs PC+2 ; (C=1)
    clz
    .endmacro

.macro B2C ; reg,b ; bit to carry
    sbrc @0,@1
    sec
    sbrs @0,@1
    clc
    .endmacro
.macro C2B ; reg,b ; carry to bit
    brcc PC+2
    sbr @0,(1<<@1)
    brcs PC+2
    cbr @0,(1<<@1)
    .endmacro
.macro P2C ; port,b ; port to carry
```

```
    sbic @0,@1
    sec
    sbis @0,@1
    clc
    .endmacro
.macro C2P ; port,b ; carry to port
    brcc PC+2
    sbi @0,@1
    brcs PC+2
    cbi @0,@1
    .endmacro

; --- inverting bits ---
.macro INVB ; reg,bit ; inverse reg,bit
    ldi w,(1<<@1)
    eor @0,w
    .endmacro
.macro INVP ; port,bit ; inverse port,bit
    sbis @0,@1
    rjmp PC+3
    cbi @0,@1
    rjmp PC+2
    sbi @0,@1
    .endmacro
.macro INVC ; inverse carry
    brcs PC+3
    sec
    rjmp PC+2
    clc
    .endmacro

; --- setting a single bit ---
.macro SETBIT ; reg(0..7)
; in reg (0..7)
; out reg with bit (0..7) set to 1.
; 0=00000001
; 1=00000010
; ...
; 7=10000000
    mov w,@0
    clr @0
    inc @0
    andi w,0b111
    breq PC+4
    lsl @0
    dec w
    brne PC-2
    .endmacro

; --- logical operations with masks ---
.macro MOVMSK ; reg1,reg2,mask ; reg1 <- reg2 (mask)
    ldi w,~@2
    and @0,w
    ldi w,@2
    and @1,w
    or @0,@1
    .endmacro
.macro ANDMSK ; reg1,reg2,mask ; reg1 <- ret 1 AND reg2 (mask)
    mov w,@1
    ori w,~@2
    and @0,w
    .endmacro
.macro ORMSK ; reg1,reg2,mask ; reg1 <- ret 1 AND reg2 (mask)
    mov w,@1
    andi w,@2
    or @0,w
    .endmacro
```

```
; --- logical operations on bits ---
.macro ANDB ; r1,b1, r2,b2, r3,b3 ; reg1,b1 <- reg2,b2 AND reg3,b3
    set
    sbrs @4,@5
    clt
    sbrs @2,@3
    clt
    bld @0,@1
    .endmacro
.macro ORB ; r1,b1, r2,b2, r3,b3 ; reg1.b1 <- reg2.b2 OR reg3.b3
    clt
    sbrc @4,@5
    set
    sbrc @2,@3
    set
    bld @0,@1
    .endmacro
.macro EORB ; r1,b1, r2,b2, r3,b3 ; reg1.b1 <- reg2.b2 XOR reg3.b3
    sbrc @4,@5
    rjmp f1
f0: bst @2,@3
    rjmp PC+4
f1: set
    sbrc @0,@1
    clt
    bld @0,@0
    .endmacro

; --- operations based on register bits ---
.macro FB0 ; reg,bit ; bit=0
    cbr @0,1<<@1
    .endmacro
.macro FB1 ; reg,bit ; bit=1
    sbr @0,1<<@1
    .endmacro
.macro _FB0 ; reg,bit ; bit=0
    ldi w,~(1<<@1)
    and @0,w
    .endmacro
.macro _FB1 ; reg,bit ; bit=1
    ldi w,1<<@1
    or @0,w
    .endmacro
.macro SB0 ; reg,bit,addr ; skip if bit=0
    sbrc @0,@1
    .endmacro
.macro SB1 ; reg,bit,addr ; skip if bit=1
    sbrs @0,@1
    .endmacro
.macro JB0 ; reg,bit,addr ; jump if bit=0
    sbrs @0,@1
    rjmp @2
    .endmacro
.macro JB1 ; reg,bit,addr ; jump if bit=1
    sbrc @0,@1
    rjmp @2
    .endmacro
.macro CB0 ; reg,bit,addr ; call if bit=0
    sbrs @0,@1
    rcall @2
    .endmacro
.macro CB1 ; reg,bit,addr ; call if bit=1
    sbrc @0,@1
    rcall @2
    .endmacro
.macro WB0 ; reg,bit ; wait if bit=0
    sbrs @0,@1
    rjmp PC-1
```

```
    .endmacro
.macro WB1 ; reg,bit ; wait if bit=1
    sbrc @0,@1
    rjmp PC-1
    .endmacro
.macro RB0 ; reg,bit ; return if bit=0
    sbrs @0,@1
    ret
    .endmacro
.macro RB1 ; reg,bit ; return if bit=1
    sbrc @0,@1
    ret
    .endmacro

; wait if bit=0 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WB0T ; reg,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbrs @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; wait if bit=1 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WB1T ; reg,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbrc @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; --- operations based on port bits ---
.macro P0 ; port,bit ; port=0
    cbi @0,@1
    .endmacro
.macro P1 ; port,bit ; port=1
    sbi @0,@1
    .endmacro
.macro SP0 ; port,bit ; skip if port=0
    sbic @0,@1
    .endmacro
.macro SP1 ; port,bit ; skip if port=1
    sbis @0,@1
    .endmacro
.macro JP0 ; port,bit,addr ; jump if port=0
    sbis @0,@1
    rjmp @2
    .endmacro
.macro JP1 ; port,bit,addr ; jump if port=1
    sbic @0,@1
    rjmp @2
    .endmacro
.macro CP0 ; port,bit,addr ; call if port=0
    sbis @0,@1
    rcall @2
    .endmacro
.macro CP1 ; port,bit,addr ; call if port=1
    sbic @0,@1
    rcall @2
    .endmacro
.macro WP0 ; port,bit ; wait if port=0
    sbis @0,@1
    rjmp PC-1
    .endmacro
.macro WP1 ; port,bit ; wait if port=1
```

```
    sbic @0,@1
    rjmp PC-1
    .endmacro
.macro RP0 ; port,bit ; return if port=0
    sbis @0,@1
    ret
    .endmacro
.macro RP1 ; port,bit ; return if port=1
    sbic @0,@1
    ret
    .endmacro

; wait if port=0 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WP0T ; port,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbis @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; wait if port=1 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WP1T ; port,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbic @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; ============================
; multi-byte operations
; ============================

.macro SWAP4 ; swap 2 variables
    mov w ,@0
    mov @0,@4
    mov @4,w
    mov w ,@1
    mov @1,@5
    mov @5,w
    mov w ,@2
    mov @2,@6
    mov @6,w
    mov w ,@3
    mov @3,@7
    mov @7,w
    .endmacro
.macro SWAP3
    mov w ,@0
    mov @0,@3
    mov @3,w
    mov w ,@1
    mov @1,@4
    mov @4,w
    mov w ,@2
    mov @2,@5
    mov @5,w
    .endmacro
.macro SWAP2
    mov w ,@0
    mov @0,@2
    mov @2,w
    mov w ,@1
    mov @1,@3
    mov @3,w
```

```
    .endmacro
.macro SWAP1
    mov w ,@0
    mov @0,@1
    mov @1,w
    .endmacro

.macro LDX4 ;r..r0 ; load from (x+)
    ld @3,x+
    ld @2,x+
    ld @1,x+
    ld @0,x+
    .endmacro
.macro LDX3 ;r..r0
    ld @2,x+
    ld @1,x+
    ld @0,x+
    .endmacro
.macro LDX2 ;r..r0
    ld @1,x+
    ld @0,x+
    .endmacro

.macro LDY4 ;r..r0 ; load from (y+)
    ld @3,y+
    ld @2,y+
    ld @1,y+
    ld @0,y+
    .endmacro
.macro LDY3 ;r..r0
    ld @2,y+
    ld @1,y+
    ld @0,y+
    .endmacro
.macro LDY2 ;r..r0
    ld @1,y+
    ld @0,y+
    .endmacro

.macro LDZ4 ;r..r0 ; load from (z+)
    ld @3,z+
    ld @2,z+
    ld @1,z+
    ld @0,z+
    .endmacro
.macro LDZ3 ;r..r0
    ld @2,z+
    ld @1,z+
    ld @0,z+
    .endmacro
.macro LDZ2 ;r..r0
    ld @1,z+
    ld @0,z+
    .endmacro

.macro STX4 ;r..r0 ; store to (x+)
    st x+,@3
    st x+,@2
    st x+,@1
    st x+,@0
    .endmacro
.macro STX3 ;r..r0
    st x+,@2
    st x+,@1
    st x+,@0
    .endmacro
.macro STX2 ;r..r0
    st x+,@1
```

```
    st x+,@0
    .endmacro

.macro STY4 ;r..r0 ; store to (y+)
    st y+,@3
    st y+,@2
    st y+,@1
    st y+,@0
    .endmacro
.macro STY3 ;r..r0
    st y+,@2
    st y+,@1
    st y+,@0
    .endmacro
.macro STY2 ;r..r0
    st y+,@1
    st y+,@0
    .endmacro

.macro STZ4 ;r..r0 ; store to (z+)
    st z+,@3
    st z+,@2
    st z+,@1
    st z+,@0
    .endmacro
.macro STZ3 ;r..r0
    st z+,@2
    st z+,@1
    st z+,@0
    .endmacro
.macro STZ2 ;r..r0
    st z+,@1
    st z+,@0
    .endmacro

.macro STI4 ;addr,k ; store immediate
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    ldi w,byte3(@1)
    sts @0+2,w
    ldi w,byte4(@1)
    sts @0+3,w
    .endmacro
.macro STI3 ;addr,k
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    ldi w,byte3(@1)
    sts @0+2,w
    .endmacro
.macro STI2 ;addr,k
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    .endmacro
.macro STI ;addr,k
    ldi w,@1
    sts @0,w
    .endmacro

.macro INC4 ; increment
    ldi w,0xff
    sub @3,w
    sbc @2,w
```

```
    sbc @1,w
    sbc @0,w
    .endmacro
.macro INC3
    ldi w,0xff
    sub @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro INC2
    ldi w,0xff
    sub @1,w
    sbc @0,w
    .endmacro

.macro DEC4 ; decrement
    ldi w,0xff
    add @3,w
    adc @2,w
    adc @1,w
    adc @0,w
    .endmacro
.macro DEC3
    ldi w,0xff
    add @2,w
    adc @1,w
    adc @0,w
    .endmacro
.macro DEC2
    ldi w,0xff
    add @1,w
    adc @0,w
    .endmacro

.macro CLR9 ; clear (also clears the carry)
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    clr @7
    clr @8
    .endmacro
.macro CLR8
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    clr @7
    .endmacro
.macro CLR7
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    .endmacro
.macro CLR6
    sub @0,@0
    clr @1
    clr @2
```

```
    clr @3
    clr @4
    clr @5
    .endmacro
.macro CLR5
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    .endmacro
.macro CLR4
    sub @0,@0
    clr @1
    clr @2
    clr @3
    .endmacro
.macro CLR3
    sub @0,@0
    clr @1
    clr @2
    .endmacro
.macro CLR2
    sub @0,@0
    clr @1
    .endmacro

.macro COM4 ; one's complement
    com @0
    com @1
    com @2
    com @3
    .endmacro
.macro COM3
    com @0
    com @1
    com @2
    .endmacro
.macro COM2
    com @0
    com @1
    .endmacro

.macro NEG4 ; negation (two's complement)
    com @0
    com @1
    com @2
    com @3
    ldi w,0xff
    sub @3,w
    sbc @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro NEG3
    com @0
    com @1
    com @2
    ldi w,0xff
    sub @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro NEG2
    com @0
    com @1
    ldi w,0xff
    sub @1,w
```

```
    sbc @0,w
    .endmacro

.macro LDI4 ; r..r0, k ; load immediate
    ldi @3, low(@4)
    ldi @2, high(@4)
    ldi @1,byte3(@4)
    ldi @0,byte4(@4)
    .endmacro
.macro LDI3
    ldi @2, low(@3)
    ldi @1, high(@3)
    ldi @0,byte3(@3)
    .endmacro
.macro LDI2
    ldi @1, low(@2)
    ldi @0, high(@2)
    .endmacro

.macro LDS4 ; load direct from SRAM
    lds @3,@4
    lds @2,@4+1
    lds @1,@4+2
    lds @0,@4+3
    .endmacro
.macro LDS3
    lds @2,@3
    lds @1,@3+1
    lds @0,@3+2
    .endmacro
.macro LDS2
    lds @1,@2
    lds @0,@2+1
    .endmacro

.macro STS4 ; store direct to SRAM
    sts @0+0,@4
    sts @0+1,@3
    sts @0+2,@2
    sts @0+3,@1
    .endmacro
.macro STS3
    sts @0+0,@3
    sts @0+1,@2
    sts @0+2,@1
    .endmacro
.macro STS2
    sts @0+0,@2
    sts @0+1,@1
    .endmacro

.macro STDZ4 ; d, r3,r2,r1,r0
    std z+@0+0,@4
    std z+@0+1,@3
    std z+@0+2,@2
    std z+@0+3,@1
    .endmacro
.macro STDZ3 ; d, r2,r1,r0
    std z+@0+0,@3
    std z+@0+1,@2
    std z+@0+2,@1
    .endmacro
.macro STDZ2 ; d, r1,r0
    std z+@0+0,@2
    std z+@0+1,@1
    .endmacro

.macro LPM4 ; load program memory
```

```
    lpm
    mov @3,r0
    adiw zl,1
    lpm
    mov @2,r0
    adiw zl,1
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM3
    lpm
    mov @2,r0
    adiw zl,1
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM2
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro

.macro MOV4 ; move between registers
    mov @3,@7
    mov @2,@6
    mov @1,@5
    mov @0,@4
    .endmacro
.macro MOV3
    mov @2,@5
    mov @1,@4
    mov @0,@3
    .endmacro
.macro MOV2
    mov @1,@3
    mov @0,@2
    .endmacro

.macro ADD4 ; add
    add @3,@7
    adc @2,@6
    adc @1,@5
    adc @0,@4
    .endmacro
.macro ADD3
    add @2,@5
    adc @1,@4
    adc @0,@3
    .endmacro
.macro ADD2
    add @1,@3
    adc @0,@2
```

```
    .endmacro

.macro SUB4 ; subtract
    sub @3,@7
    sbc @2,@6
    sbc @1,@5
    sbc @0,@4
    .endmacro
.macro SUB3
    sub @2,@5
    sbc @1,@4
    sbc @0,@3
    .endmacro
.macro SUB2
    sub @1,@3
    sbc @0,@2
    .endmacro

.macro CP4 ; compare
    cp @3,@7
    cpc @2,@6
    cpc @1,@5
    cpc @0,@4
    .endmacro
.macro CP3
    cp @2,@5
    cpc @1,@4
    cpc @0,@3
    .endmacro
.macro CP2
    cp @1,@3
    cpc @0,@2
    .endmacro

.macro TST4 ; test
    clr w
    cp @3,w
    cpc @2,w
    cpc @1,w
    cpc @0,w
    .endmacro
.macro TST3
    clr w
    cp @2,w
    cpc @1,w
    cpc @0,w
    .endmacro
.macro TST2
    clr w
    cp @1,w
    cpc @0,w
    .endmacro

.macro ADDI4 ; add immediate
    subi @3, low(-@4)
    sbci @2, high(-@4)
    sbci @1,byte3(-@4)
    sbci @0,byte4(-@4)
    .endmacro
.macro ADDI3
    subi @2, low(-@3)
    sbci @1, high(-@3)
    sbci @0,byte3(-@3)
    .endmacro
.macro ADDI2
    subi @1, low(-@2)
    sbci @0, high(-@2)
    .endmacro
```

```
.macro SUBI4 ; subtract immediate
    subi @3, low(@4)
    sbci @2, high(@4)
    sbci @1,byte3(@4)
    sbci @0,byte4(@4)
    .endmacro
.macro SUBI3
    subi @2, low(@3)
    sbci @1, high(@3)
    sbci @0,byte3(@3)
    .endmacro
.macro SUBI2
    subi @1, low(@2)
    sbci @0, high(@2)
    .endmacro

.macro LSL5 ; logical shift left
    lsl @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro LSL4
    lsl @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro LSL3
    lsl @2
    rol @1
    rol @0
    .endmacro
.macro LSL2
    lsl @1
    rol @0
    .endmacro

.macro LSR4 ; logical shift right
    lsr @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro LSR3
    lsr @0
    ror @1
    ror @2
    .endmacro
.macro LSR2
    lsr @0
    ror @1
    .endmacro

.macro ASR4 ; arithmetic shift right
    asr @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro ASR3
    asr @0
    ror @1
    ror @2
    .endmacro
.macro ASR2
```

```
    asr @0
    ror @1
    .endmacro

.macro ROL8 ; rotate left through carry
    rol @7
    rol @6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL7
    rol @6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL2
    rol @1
    rol @0
    .endmacro

.macro ROR8 ; rotate right through carry
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    ror @5
    ror @6
    ror @7
    .endmacro
.macro ROR7
    ror @0
    ror @1
    ror @2
```

```
    ror @3
    ror @4
    ror @5
    ror @6
    .endmacro
.macro ROR6
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    ror @5
    .endmacro
.macro ROR5
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    .endmacro
.macro ROR4
    ror @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro ROR3
    ror @0
    ror @1
    ror @2
    .endmacro
.macro ROR2
    ror @0
    ror @1
    .endmacro

.macro PUSH2
    push @0
    push @1
    .endmacro
.macro POP2
    pop @1
    pop @0
    .endmacro

.macro PUSH3
    push @0
    push @1
    push @2
    .endmacro
.macro POP3
    pop @2
    pop @1
    pop @0
    .endmacro

.macro PUSH4
    push @0
    push @1
    push @2
    push @3
    .endmacro
.macro POP4
    pop @3
    pop @2
    pop @1
    pop @0
    .endmacro
```

```
.macro PUSH5
    push @0
    push @1
    push @2
    push @3
    push @4
    .endmacro
.macro POP5
    pop @4
    pop @3
    pop @2
    pop @1
    pop @0
    .endmacro

; --- SRAM operations ---
.macro INCS4 ; sram ; increment SRAM 4-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
    brne end
    lds w,@0+2
    inc w
    sts @0+2,w
    brne end
    lds w,@0+3
    inc w
    sts @0+3,w
end:
    .endmacro

.macro INCS3 ; sram ; increment SRAM 3-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
    brne end
    lds w,@0+2
    inc w
    sts @0+2,w
end:
    .endmacro

.macro INCS2 ; sram ; increment SRAM 2-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
end:
    .endmacro

.macro INCS ; sram ; increment SRAM 1-byte variable
    lds w,@0
    inc w
    sts @0,w
    .endmacro
```

```
.macro DECS4 ; sram ; decrement SRAM 4-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    lds u,@0+2
    sbc u,w
    sts @0+2,u
    lds u,@0+3
    sbc u,w
    sts @0+3,u
    .endmacro
.macro DECS3 ; sram ; decrement SRAM 3-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    lds u,@0+2
    sbc u,w
    sts @0+2,u
    .endmacro
.macro DECS2 ; sram ; decrement SRAM 2-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    .endmacro
.macro DECS ; sram ; decrement
    lds w,@0
    dec w
    sts @0,w
    .endmacro

.macro MOVS4 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
    lds w,@1+2
    sts @0+2,w
    lds w,@3+1
    sts @0+3,w
    .endmacro
.macro MOVS3 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
    lds w,@1+2
    sts @0+2,w
    .endmacro
.macro MOVS2 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
```

```
    .endmacro
.macro MOVS ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    .endmacro

.macro SEXT ; reg1,reg0 ; sign extend
    clr @0
    sbrc @1,7
    dec @0
    .endmacro

; =======================================
; Jump/Call with constant arguments
; =======================================

; --- calls with arguments a,b,XYZ ---
.macro CX ; subroutine,x
    ldi xl, low(@1)
    ldi xh,high(@1)
    rcall @0
    .endmacro
.macro CXY ; subroutine,x,y
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi yl, low(@2)
    ldi yh,high(@2)
    rcall @0
    .endmacro
.macro CXZ ; subroutine,x,z
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi zl, low(@2)
    ldi zh,high(@2)
    rcall @0
    .endmacro
.macro CXYZ ; subroutine,x,y,z
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi yl, low(@2)
    ldi yh,high(@2)
    ldi zl, low(@3)
    ldi zh,high(@3)
    rcall @0
    .endmacro
.macro CW ; subroutine,w
    ldi w, @1
    rcall @0
    .endmacro
.macro CA ; subroutine,a
    ldi a0, @1
    rcall @0
    .endmacro
.macro CAB ; subroutine,a,b
    ldi a0, @1
    ldi b0, @2
    rcall @0
    .endmacro

; --- jump with arguments w,a,b ---
.macro JW ; subroutine,w
    ldi w, @1
    rjmp @0
    .endmacro
.macro JA ; subroutine,a
    ldi a0, @1
    rjmp @0
    .endmacro
```

```
.macro JAB ; subroutine,a,b
    ldi a0, @1
    ldi b0, @2
    rjmp @0
    .endmacro
.list
```

main.asm

```
; file main.asm target ATmega128L - 4 MHz - STK300 (KAN-MEG-DEV)
;
; Top-level program. Owns shared SRAM variables, reset/init, main loop and three modes.
;
; Operating modes (variable "mode"):
; 0 = PARK : servo stays at rest, buzzer beeps (parktronic sound)
; 1 = RADAR : servo sweeps left<->right, buzzer beeps (radar sound)
; 2 = MANUAL : VOL+/VOL-/UP/DOWN change speed or direction of the rotation,
; make a small rotation (clockwise/counter clock.)
;
; IR t■l■comande (ir_rc5.asm):
; VOL+, VOL-, UP, DOWN -> manual mode
; GUIDE -> change modes (PARK->RADAR->PARK, MANUAL->PARK)
; POWER -> power on/off
; MUTE -> buzzer on/off


.include "macros.asm"
.include "definitions.asm"


; main passes between beeps for different distance
.equ BUZZ_REPEAT_VFAST = 1 ; < 10 cm : ~1 pass (~50 ms)
.equ BUZZ_REPEAT_FAST = 4 ; 10-24 cm : ~4 passes (~200 ms)
.equ BUZZ_REPEAT_SLOW = 12 ; 25-49 cm : ~12 passes (~600 ms)

; glitch filter. Sudden large changes in distance are considered as glitches
.equ DIST_JUMP_LIMIT = 20

; radar sweep timing
; Target: 4.5 seconds per full turn (30 steps). Each step = SWEEP_STEP_MS +
; HC_measure (~3 ms with echo, ~45 ms on timeout) + lcd (~2 ms).
; With an echo the total is ~153 ms; on timeout ~195 ms. The variation
; comes from HC_measure and is acceptable for a visual sweep.
.equ SWEEP_STEPS = 15
.equ SWEEP_STEP_MS = 148

.equ PWR_ON = 0
.equ PWR_OFF = 1
.equ PWR_STARTUP = 2

; delays
.equ LOOP_DELAY_MS = 50
.equ POWEROFF_DELAY_MS = 200
.equ STARTUP_DELAY_MS = 1500


.dseg
.org 0x0200
mode: .byte 1 ; 0=PARK, 1=RADAR, 2=MANUAL
servo_level: .byte 1 ; 0=centre, 1..5=right, 255..251=left
last_cmd: .byte 1 ; last received IR command
power_state: .byte 1 ; 0=on, 1=off, 2=turning_on
buzzer_on: .byte 1 ; 1=buzzer enabled, 0=muted
deb_counter: .byte 1 ; IR debounce: INT7 disabled while > 0
step_timer: .byte 1 ; UP/DOWN kick countdown (main loop decrements)
distance_cm: .byte 1 ; raw measured distance
dist_prev: .byte 1 ; previous raw distance (for the glitch filter)
dist_display: .byte 1 ; filtered distance (used for LCD and beeps)
buzz_counter: .byte 1 ; non-blocking beep cadence counter
tone_fn: .byte 2 ; word address of the active tone routine (Park_tone or Radar_tone)


.cseg
.org 0x0000
```

```
    rjmp reset

.org 0x0012
    rjmp INT7_handler ; defined in ir_rc5.asm

; peripheriques
.include "lcd.asm"
.include "printf.asm"
.include "HC-SR04.asm"
.include "servo.asm"
.include "ir_rc5.asm"


reset:
    LDSP RAMEND
    rcall LCD_init
    rcall HC_init
    rcall Servo_init

    ldi r16, 0
    sts mode, r16 ; set to parking by default
    sts servo_level, r16
    sts power_state, r16 ; set to power on
    sts deb_counter, r16
    sts step_timer, r16
    sts distance_cm, r16
    sts dist_prev, r16
    sts dist_display, r16
    sts buzz_counter, r16
    ldi r16, 1
    sts buzzer_on, r16 ; buzzer enabled by default
    ldi r16, 0xFF
    sts last_cmd, r16

    ; default - park tone, switched to radar mode - radar tone
    ldi r16, low(Park_tone)
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16

    ldi r16, 0
    rcall Servo_set_level ; set servo at centre

    ; enable INT7 on the falling edge (button pressed)
    ldi r16, (1<<ISC71)|(0<<ISC70)
    sts EICRB, r16
    ldi r16, (1<<INT7)
    out EIMSK, r16
    sei

    rjmp main


main:
    ; When step_timer counts down to 0, stop the rotation
    lds r16, step_timer
    cpi r16, 0
    breq main_no_step
    dec r16
    sts step_timer, r16
    cpi r16, 0
    brne main_no_step
    ldi r16, 0
    sts servo_level, r16

main_no_step:
    ; countdown to enable remote again if the button was pushed
    rcall deb_tick
```

```
    ; power on/off logic
    lds r16, power_state
    cpi r16, PWR_OFF
    brne PC+2
    rjmp main_power_off
    cpi r16, PWR_STARTUP
    brne PC+2
    rjmp main_power_on_seq

    ; mode RADAR
    lds r16, mode
    cpi r16, 1
    brne PC+2
    rjmp main_radar

    ; mode PARK/MANUAL
    lds r16, servo_level
    rcall Servo_set_level

    ; measure distance
    rcall measure_and_filter

    rcall buzz_tick

main_lcd:
    rcall lcd_show
    WAIT_MS LOOP_DELAY_MS
    rjmp main

; displays POWER OFF, buttons disabled, no sound, no rotation
main_power_off:
    ldi r16, 0
    rcall Servo_set_level
    cbi PORTE, SPEAKER

    rcall LCD_home
    PRINTF LCD
    .db "POWER OFF ", 0
    ldi r18, 0x40
    rcall LCD_pos
    PRINTF LCD
    .db " ", 0

    WAIT_MS POWEROFF_DELAY_MS
    rjmp main

; power was justed turned on, wait for STARTUP_DELAY_MS for display
main_power_on_seq:
    rcall LCD_home
    PRINTF LCD
    .db "POWER ON ", 0
    ldi r18, 0x40
    rcall LCD_pos
    PRINTF LCD
    .db " Starting... ", 0

    WAIT_MS STARTUP_DELAY_MS

    ; re-initialise state (same defaults as reset, without the one-time setup)
    ldi r16, 0
    sts mode, r16
    sts servo_level, r16
    sts power_state, r16
    sts step_timer, r16
    sts distance_cm, r16
    sts dist_prev, r16
    sts dist_display, r16
```

```
    sts buzz_counter, r16
    ldi r16, 1
    sts buzzer_on, r16
    ldi r16, low(Park_tone)
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16
    ldi r16, 0
    rcall Servo_set_level

    rjmp main


; mode RADAR logic
main_radar:
    ; switch the tone of the buzzer
    ldi r16, low(Radar_tone)
    sts tone_fn, r16
    ldi r16, high(Radar_tone)
    sts tone_fn+1, r16


    rcall Servo_level_half ; set direction to right
    ldi r18, SWEEP_STEPS

main_radar_fwd:
    push r18

    ; same as PARK logic
    rcall measure_and_filter
    rcall buzz_tick
    rcall lcd_show
    rcall deb_tick
    rcall sweep_dwell ; wait on the current posision for SWEEP_STEP_MS ms

    pop r18

    lds r16, mode
    cpi r16, 1
    brne main_radar_exit
    dec r18
    brne main_radar_fwd

    rcall Servo_level_mhalf ; change the direction
    ldi r18, SWEEP_STEPS

main_radar_bwd:
    push r18

    rcall measure_and_filter
    rcall buzz_tick
    rcall lcd_show
    rcall deb_tick
    rcall sweep_dwell ; wait on the current posision for SWEEP_STEP_MS ms

    pop r18

    lds r16, mode
    cpi r16, 1
    brne main_radar_exit
    dec r18
    brne main_radar_bwd

    rjmp main_radar

main_radar_exit:
    ; restore the state of the system to PARK/MANUAL mode
    ldi r16, low(Park_tone)
```

```
    sts tone_fn, r16
    ldi r16, high(Park_tone)
    sts tone_fn+1, r16
    ldi r16, 0
    sts servo_level, r16
    rcall Servo_set_level
    rjmp main_lcd


; delay for each small sweep step ( basically speed adjusting )
sweep_dwell:
    WAIT_MS SWEEP_STEP_MS
    ret

; sound (beeping) logic
buzz_tick:

    ; muted
    lds r16, buzzer_on
    cpi r16, 0
    breq bt_silent


    lds r16, buzz_counter
    cpi r16, 0
    brne bt_wait

    ; checks if beeping is needed (if distance <= 50 cm)
    lds r21, dist_display
    cpi r21, DIST_NONE
    breq bt_silent ; no echo -> silence
    cpi r21, DIST_FAR
    brsh bt_silent ; >= 50 cm -> silence

    ; play one beep with tone corresponding to the mode
    lds zl, tone_fn
    lds zh, tone_fn+1
    icall

    ; reload rhythme counter based on current distance.
    lds r21, dist_display
    cpi r21, DIST_NEAR
    brsh bt_check25
    ldi r16, BUZZ_REPEAT_VFAST ; < 10 cm
    rjmp bt_set

bt_check25:
    cpi r21, DIST_MID
    brsh bt_slow
    ldi r16, BUZZ_REPEAT_FAST ; 10-24 cm
    rjmp bt_set

bt_slow:
    ldi r16, BUZZ_REPEAT_SLOW ; 25-49 cm

bt_set:
    sts buzz_counter, r16
    ret

bt_wait:
    dec r16
    sts buzz_counter, r16
    ret

bt_silent:
    cbi PORTE, SPEAKER
    ldi r16, 0
    sts buzz_counter, r16
```

```
    ret


; read ultrasonic sensor logic
measure_and_filter:
    push r16
    push r17

    rcall HC_measure ; r21 = distance in cm
    sts distance_cm, r21

    ; r16 = previous sample then remember the new raw sample.
    lds r16, dist_prev
    sts dist_prev, r21

    ; check if the previous or current distance is available
    cpi r21, DIST_NONE
    breq mf_accept
    cpi r16, DIST_NONE
    breq mf_accept

    ; r17 = |new - prev|
    mov r17, r21
    sub r17, r16
    brpl mf_pos
    neg r17

mf_pos:
    cpi r17, DIST_JUMP_LIMIT
    brlo mf_accept ; small change -> accept
    ; large jump -> treat as a glitch, keep the previous value.
    mov r21, r16

mf_accept:
    sts dist_display, r21

    pop r17
    pop r16
    ret


; line 1: mode + speed level
; line 2: distance in cm or --- when no echo
lcd_show:
    push r20
    push r18

    ; line 1
    rcall LCD_home
    lds r16, mode

    cpi r16, 2
    brne lcd_chk_radar

    ; MANUAL: show the servo speed level
    lds r20, servo_level
    PRINTF LCD
    .db "Manual s=", FDEC|FSIGN, 20, " ", 0
    rjmp lcd_line2

lcd_chk_radar:
    cpi r16, 1
    brne lcd_park

    PRINTF LCD
    .db "Radar s= 0 ", 0
    rjmp lcd_line2
```

```
lcd_park:
    PRINTF LCD
    .db "Park s= 0 ", 0

lcd_line2:
    ; line 2
    ldi r18, 0x40
    rcall LCD_pos

    lds r20, dist_display
    cpi r20, DIST_NONE
    brne lcd_dist_ok

    PRINTF LCD
    .db "Dist: --- cm ", 0
    rjmp lcd_end

lcd_dist_ok:
    PRINTF LCD
    .db "Dist: ", FDEC|FDIG3, 20, " cm ", 0

lcd_end:
    pop r18
    pop r20
    ret
```

ir_rc5.asm

```
; === IR RC-5 Receiver Driver ===
; file ir_rc5.asm target ATmega128L-4MHz-STK300
; IR RC-5 Receiver logic

.equ T1 = 1778 ; RC-5 bit period in microseconds

; IR command codes
.equ CMD_VOLUP = 0xE0 ; VOL+
.equ CMD_VOLDN = 0xE1 ; VOL-
.equ CMD_UP = 0xC0 ; UP
.equ CMD_DOWN = 0xC1 ; DOWN
.equ CMD_GUIDE = 0xC2 ; GUIDE
.equ CMD_POWER = 0xEC ; POWER
.equ CMD_MUTE = 0xED ; MUTE

.equ STEP_KICK_PASSES = 12


; deb_tick - debounce tick, call once per main loop iteration
; decrements deb_counter; re-enables INT7 when it reaches 0.
deb_tick:
    lds r16, deb_counter
    cpi r16, 0
    breq deb_tick_done
    dec r16
    sts deb_counter, r16
    cpi r16, 0
    brne deb_tick_done
    ldi r16, (1<<INT7)
    out EIMSK, r16
deb_tick_done:
    ret


; INT7_handler - IR interrupt service routine
; Decodes one RC-5 command sent
INT7_handler:
    push r16
    push r17
    push r18
    push r20
    push r22
    push r23
    push r24
    push r3
    in r16, SREG
    push r16

    ; decode 14-bit RC-5 frame
    ; wait T1/4 to align the first sample to the middle of the start bit,
    ; then sample one bit every T1 us. Each bit is shifted into b1:b0.
    CLR2 b1, b0
    ldi b2, 14
    WAIT_US (T1/4)

INT7_loop:
    P2C PINE, IR ; sample IR pin -> Carry
    ROL2 b1, b0 ; shift Carry into b1:b0 from LSB
    WAIT_US (T1-4) ; wait one bit period (minus loop overhead)
    DJNZ b2, INT7_loop ; repeat for all 14 bits

    sts last_cmd, b0 ; save command byte (low 8 bits of frame)

    cpi b0, CMD_POWER
    brne PC+2
```

```
    rjmp INT7_power

    ; all other commands are ignored while powered off
    lds r16, power_state
    cpi r16, 0
    brne PC+2
    rjmp INT7_dispatch
    rjmp INT7_end

INT7_dispatch:
    cpi b0, CMD_MUTE
    brne PC+2
    rjmp INT7_mute
    cpi b0, CMD_VOLUP
    brne PC+2
    rjmp INT7_vol_up
    cpi b0, CMD_VOLDN
    brne PC+2
    rjmp INT7_vol_dn
    cpi b0, CMD_UP
    brne PC+2
    rjmp INT7_step_fwd
    cpi b0, CMD_DOWN
    brne PC+2
    rjmp INT7_step_bwd
    cpi b0, CMD_GUIDE
    brne PC+2
    rjmp INT7_guide
    rjmp INT7_end


; disables INT7 with deb_counter=5 to avoid bounce/repeat.
INT7_power:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 5
    sts deb_counter, r16

    lds r16, power_state
    cpi r16, 0
    breq INT7_power_off

    ldi r16, 2
    sts power_state, r16
    rjmp INT7_end
INT7_power_off:

    ldi r16, 1
    sts power_state, r16
    rjmp INT7_end

; mute
INT7_mute:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16

    lds r16, buzzer_on
    cpi r16, 1
    breq INT7_mute_off
    ldi r16, 1
    sts buzzer_on, r16
    rjmp INT7_end
INT7_mute_off:
    ldi r16, 0
    sts buzzer_on, r16
    rjmp INT7_end
```

```
; vol+ = increase/decrease speed (depends on the direction of the rotation)
INT7_vol_up:
    ldi r16, 2
    sts mode, r16
    ldi r16, 0
    sts step_timer, r16
    lds r17, servo_level
    cpi r17, 251
    brlo vup_not_neg
    inc r17
    rjmp vup_apply
vup_not_neg:
    cpi r17, 5
    brlo PC+2
    rjmp INT7_end
    inc r17
vup_apply:
    sts servo_level, r17
    mov r16, r17
    rcall Servo_set_level_raw
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16
    rjmp INT7_end

; vol- = decrease/increase speed (depends on the direction of the rotation)
INT7_vol_dn:
    ldi r16, 2
    sts mode, r16
    ldi r16, 0
    sts step_timer, r16
    lds r17, servo_level
    cpi r17, 6
    brsh vdn_not_pos
    cpi r17, 0
    breq vdn_from_zero
    dec r17
    rjmp vdn_apply
vdn_from_zero:
    ldi r17, 255
    rjmp vdn_apply
vdn_not_pos:
    cpi r17, 251
    brne PC+2
    rjmp INT7_end
    dec r17
vdn_apply:
    sts servo_level, r17
    mov r16, r17
    rcall Servo_set_level_raw
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 3
    sts deb_counter, r16
    rjmp INT7_end

; small rotation in counter-clockwise direction
INT7_step_fwd:
    ldi r16, 2
    sts mode, r16
    ldi r17, 1
    sts servo_level, r17
    ldi r17, STEP_KICK_PASSES
    sts step_timer, r17
    rjmp INT7_end
```

```
; small rotation in clockwise direction
INT7_step_bwd:
    ldi r16, 2
    sts mode, r16
    ldi r17, 255
    sts servo_level, r17
    ldi r17, STEP_KICK_PASSES
    sts step_timer, r17
    rjmp INT7_end


; change modes MANUAL -> PARK, RADAR -> PARK, PARK -> RADAR
INT7_guide:
    ldi r16, 0
    out EIMSK, r16
    ldi r16, 5
    sts deb_counter, r16

    lds r16, mode
    cpi r16, 2 ; MANUAL
    breq guide_to_park
    cpi r16, 1 ; RADAR
    breq guide_to_park
    ; PARK -> RADAR
    ldi r16, 1
    sts mode, r16
    rjmp INT7_end
guide_to_park:
    ldi r16, 0
    sts mode, r16
    sts servo_level, r16
    sts step_timer, r16
    rcall Servo_set_level_raw
    rjmp INT7_end

INT7_end:
    pop r16
    out SREG, r16
    pop r3
    pop r24
    pop r23
    pop r22
    pop r20
    pop r18
    pop r17
    pop r16
    reti
```

HC-SR04.asm

```
; file HC-SR04.asm target ATmega128L - 4 MHz - STK300
; HC-SR04 ultrasonic distance sensor logic + beeper logic
;
;DISTANCE TIMING (why the divisor is 33)
; Speed of sound = 343 m/s, so a round trip of 1 cm takes 58.14 us.
; At 4 MHz one CPU cycle is 0.25 us.
; The echo-counting loop takes 7 cycles per pass = 1.75 us.
; Therefore 1 cm corresponds to 58.14 / 1.75 = 33.22 loop passes.
; Dividing the raw loop count by 33 gives exact results across the full range.

; PINs
.equ TRIG = 0 ; PD0
.equ ECHO = 1 ; PD1

; distances
.equ DIST_NEAR = 10 ; below this: very fast beeping
.equ DIST_MID = 25 ; below this: fast beeping
.equ DIST_FAR = 50 ; below this: slow beeping; above: silence
.equ DIST_NONE = 0xFF ; no echo / out of range

; count to centimeter divisor
; The HC_count loop is 7 cycles (1.75 us per pass):
; inc r21 (1) + brne taken (2) + sbis skip-1-word (2) + rjmp HC_count (2) = 7
; Round trip for 1 cm at 343 m/s = 58.14 us.
; Passes per cm = 58.14 / 1.75 = 33.22 -> divisor 33.
.equ CM_DIVISOR = 33


HC_init:
    sbi DDRD, TRIG ; TRIG is an output
    cbi DDRD, ECHO ; ECHO is an input
    cbi PORTD, TRIG ; TRIG idle low
    sbi DDRE, SPEAKER ; SPEAKER is an output
    cbi PORTE, SPEAKER ; buzzer off
    ret


; perform 1 distance measurement
HC_measure:
    push r20
    push r22
    push r23
    push r24
    push r25

    ; emit 10 us (40 cycles) trigger pulse
    ; required by the sensor
    sbi PORTD, TRIG
    ldi r25, 10
HC_trig_delay: ; waits 40 cycles (10 us)
    nop
    nop
    dec r25
    brne HC_trig_delay
    cbi PORTD, TRIG

    ; wait for echo to go high with ~30 ms timeout
    ; 120000 cycles = 30 ms
    ldi r22, 240
    ldi r23, 250
HC_wait_high:
    sbic PIND, ECHO
    rjmp HC_echo_started
    dec r23
    brne HC_wait_high
```

```
    ldi r23, 250
    dec r22
    brne HC_wait_high
    rjmp HC_timeout ; timeout if echo never rises

HC_echo_started:
    ; measure how long echo stays high
    clr r20
    clr r21
HC_count:
    inc r21 ; 1 cycle
    brne HC_count2 ; 1/2 cycles
    inc r20 ; carry into high byte when r21 wraps
HC_count2:
    sbis PIND, ECHO ; keep counting for high echo
    rjmp HC_done ; echo finished
    rjmp HC_count

HC_done:
    ; r20:r21 = raw loop count. Convert to centimetres.
    ldi r22, CM_DIVISOR
    rcall DIV16 ; r20:r21 = count / CM_DIVISOR

    ; fit the result to <= 254 for 0xff to be reserved for timeout
    tst r20
    brne HC_overflow ; high byte set -> result too large
    cpi r21, DIST_NONE
    brne HC_finish
HC_overflow:
    ldi r21, 0xFE ; 254 is far but still valid
    rjmp HC_finish

HC_timeout:
    ldi r21, DIST_NONE ; 0xFF = no echo / out of range
    clr r20

HC_finish:
    pop r25
    pop r24
    pop r23
    pop r22
    pop r20
    ret

; unsigned 16-bit / 8-bit division
DIV16:
    clr r23
    ldi r24, 16
DIV16_loop:
    lsl r21
    rol r20
    rol r23
    cp r23, r22
    brlo DIV16_skip
    sub r23, r22
    inc r21
DIV16_skip:
    dec r24
    brne DIV16_loop
    ret


; park_tone - 1 kHz beep
; 60 square-wave periods of 1 ms (0.5 ms high + 0.5 ms low)
Park_tone:
    push r24
    push r25
    ldi r24, 60
```

```
Park_tone_loop:
    sbi PORTE, SPEAKER
    WAIT_US 500
    cbi PORTE, SPEAKER
    WAIT_US 500
    dec r24
    brne Park_tone_loop
    pop r25
    pop r24
    ret


Radar_tone:
    push r24
    push r25
    push r27
    ldi r24, 20
    ldi r25, 200
Radar_tone_loop:
    sbi PORTE, SPEAKER
    mov r27, r25
Radar_delay1:
    nop
    dec r27
    brne Radar_delay1
    cbi PORTE, SPEAKER
    mov r27, r25
Radar_delay2:
    nop
    dec r27
    brne Radar_delay2
    dec r25
    dec r24
    brne Radar_tone_loop
    pop r27
    pop r25
    pop r24
    ret
```

servo.asm

```
; file servo.asm target ATmega128L - 4 MHz - STK300
; SG-90 servo-motor logic


; Timer3 OC3A = PE3
.equ SERVO_PIN = 3

; pulse widths (OCR3A counts) for each level
; the PWM period (ICR3) is 10000 counts; these values are the servo
; pulse lengths that produce each steering position.
.equ OCR_CENTRE = 760
.equ OCR_R1 = 765
.equ OCR_R2 = 770
.equ OCR_R3 = 775
.equ OCR_R4 = 780
.equ OCR_R5 = 785
.equ OCR_L1 = 750
.equ OCR_L2 = 745
.equ OCR_L3 = 740
.equ OCR_L4 = 735
.equ OCR_L5 = 725

; sweep end-points used by the radar mode
.equ OCR_SWEEP_R = 775
.equ OCR_SWEEP_L = 735

.equ SERVO_PERIOD = 10000 ; Timer3 TOP (ICR3) = PWM period


; load a 16-bit OCR value into r18:r17 (high:low) and return.
.macro SET_OCR ; @0 = OCR constant
    ldi r17, low(@0)
    ldi r18, high(@0)
    ret
    .endmacro


Servo_init:

    ; setup Timer3 to generate a PWM signal on the servo pin
    sbi DDRE, SERVO_PIN
    ldi r16, (1<<COM3A1)|(1<<WGM31)
    sts TCCR3A, r16
    ldi r16, (1<<WGM33)|(1<<WGM32)|(1<<CS31)
    sts TCCR3B, r16

    ldi r16, high(SERVO_PERIOD)
    sts ICR3H, r16
    ldi r16, low(SERVO_PERIOD)
    sts ICR3L, r16

    cli
    ldi r17, high(OCR_CENTRE)
    sts OCR3AH, r17
    ldi r17, low(OCR_CENTRE)
    sts OCR3AL, r17
    sei
    ret

; sets speed based on OCR value
Servo_set_level:
    push r17
    push r18
    rcall Servo_get_ocr ; r18:r17 = OCR value for this level
    cli
```

```
    sts OCR3AH, r18
    sts OCR3AL, r17
    sei
    pop r18
    pop r17
    ret

; servo_set_level without disabling interruptions
Servo_set_level_raw:
    push r17
    push r18
    rcall Servo_get_ocr
    sts OCR3AH, r18
    sts OCR3AL, r17
    pop r18
    pop r17
    ret

; sets OCR value based on the level
Servo_get_ocr:
    cpi r16, 0
    brne SG_1
    SET_OCR OCR_CENTRE
SG_1:
    cpi r16, 1
    brne SG_2
    SET_OCR OCR_R1
SG_2:
    cpi r16, 2
    brne SG_3
    SET_OCR OCR_R2
SG_3:
    cpi r16, 3
    brne SG_4
    SET_OCR OCR_R3
SG_4:
    cpi r16, 4
    brne SG_5
    SET_OCR OCR_R4
SG_5:
    cpi r16, 5
    brne SG_m1
    SET_OCR OCR_R5
SG_m1:
    cpi r16, 255
    brne SG_m2
    SET_OCR OCR_L1
SG_m2:
    cpi r16, 254
    brne SG_m3
    SET_OCR OCR_L2
SG_m3:
    cpi r16, 253
    brne SG_m4
    SET_OCR OCR_L3
SG_m4:
    cpi r16, 252
    brne SG_m5
    SET_OCR OCR_L4
SG_m5:
    cpi r16, 251
    brne SG_default
    SET_OCR OCR_L5
SG_default:
    SET_OCR OCR_CENTRE ; unknown level -> centre

; set servo to rotate right continuously
Servo_level_half:
```

```
    cli
    ldi r17, high(OCR_SWEEP_R)
    sts OCR3AH, r17
    ldi r17, low(OCR_SWEEP_R)
    sts OCR3AL, r17
    sei
    ret


; set servo to rotate left continuously
Servo_level_mhalf:
    cli
    ldi r17, high(OCR_SWEEP_L)
    sts OCR3AH, r17
    ldi r17, low(OCR_SWEEP_L)
    sts OCR3AL, r17
    sei
    ret
```

lcd.asm

```
; file lcd.asm target ATmega128L-4MHz-STK300
; purpose LCD HD44780U library
; ATmega 128 and Atmel Studio 7.0 compliant

; === definitions ===
.equ LCD_IR = 0x8000 ; address LCD instruction reg
.equ LCD_DR = 0xc000 ; address LCD data register

; === subroutines ===
LCD_wr_ir:
; in w (byte to write to LCD IR)
    lds u, LCD_IR ; read IR to check busy flag (bit7)
    JB1 u,7,LCD_wr_ir ; Jump if Bit=1 (still busy)
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_IR, w ; store w in IR
    ret

lcd_4us:
    rcall lcd_2us ; recursive call
lcd_2us:
    nop ; rcall(3) + nop(1) + ret(4) = 8 cycles (2us)
    ret

LCD:
LCD_putc:
    JK a0,CR,LCD_cr ; Jump if a0=CR
    JK a0,LF,LCD_lf ; Jump if a0=LF
LCD_wr_dr:
; in a0 (byte to write to LCD DR)
    lds w, LCD_IR ; read IR to check busy flag (bit7)
    JB1 w,7,LCD_wr_dr ; Jump if Bit=1 (still busy)
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_DR, a0 ; store a0 in DR
    ret

LCD_clear: JW LCD_wr_ir, 0b00000001 ; clear display
LCD_home: JW LCD_wr_ir, 0b00000010 ; return home
LCD_cursor_left: JW LCD_wr_ir, 0b00010000 ; move cursor to left
LCD_cursor_right: JW LCD_wr_ir, 0b00010100 ; move cursor to right
LCD_display_left: JW LCD_wr_ir, 0b00011000 ; shifts display to left
LCD_display_right: JW LCD_wr_ir, 0b00011100 ; shifts display to right
LCD_blink_on: JW LCD_wr_ir, 0b00001101 ; Display=1,Cursor=0,Blink=1
LCD_blink_off: JW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0
LCD_cursor_on: JW LCD_wr_ir, 0b00001110 ; Display=1,Cursor=1,Blink=0
LCD_cursor_off: JW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0

LCD_init:
    in w,MCUCR ; enable access to ext. SRAM
    sbr w,(1<<SRE)+(1<<SRW10)
    out MCUCR,w
    CW LCD_wr_ir, 0b00000001 ; clear display
    CW LCD_wr_ir, 0b00000110 ; entry mode set (Inc=1, Shift=0)
    CW LCD_wr_ir, 0b00001100 ; Display=1,Cursor=0,Blink=0
    CW LCD_wr_ir, 0b00111000 ; 8bits=1, 2lines=1, 5x8dots=0
    ret

LCD_pos:
; in a0 = position (0x00..0x0f first line, 0x40..0x4f second line)
    mov w,a0
    ori w,0b10000000
    rjmp LCD_wr_ir

LCD_cr:
; moving the cursor to the beginning of the line (carriage return)
    lds w, LCD_IR ; read IR to check busy flag (bit7)
```

```
    JB1 w,7,LCD_cr ; Jump if Bit=1 (still busy)
    andi w,0b01000000 ; keep bit6 (begin of line 1/2)
    ori w,0b10000000 ; write address command
    rcall lcd_4us ; delay to increment DRAM addr counter
    sts LCD_IR,w ; store in IR
    ret

LCD_lf:
; moving the cursor to the beginning of the line 2 (line feed)
    push a0 ; safeguard a0
    ldi a0,$40 ; load position $40 (begin of line 2)
    rcall LCD_pos ; set cursor position
    pop a0 ; restore a0
    ret
```

printf.asm

```
; file printf.asm target ATmega128L-4MHz-STK300
; purpose library, formatted output generation
; author (c) R.Holzer (adapted MICRO210/EE208 A.Schmid)
; v2019.02 20180821 AxS supports SRAM input from 0x0260
; through 0x02ff that should be reserved

; === description ===
;
; The program "printf" interprets and prints formatted strings.
; The special formatting characters regognized are:
;
; FDEC decimal number
; FHEX hexadecimal number
; FBIN binary number
; FFRAC fixed fraction number
; FCHAR single ASCII character
; FSTR zero-terminated ASCII string
;
; The special formatting characters are distinguished from normal
; ASCII characters by having their bit7 set to 1.
;
; Signification of bit fields:
;
; b bytes 1..4 b bytes 2
; s sign 0(unsigned), 1(signed) 1
; i integer digits
; e base 2,,36 5
; dp dec. point 0..32 5
; $if i=integer digits, 0=all digits, 1..15 digits
; f=fraction digits, 0=no fraction, 1..15 digits
;
; Formatting characters must be followed by an SRAM address (0..ff)
; that determines the origin of variables that must be printed (if any)
; FBIN, sram
; FHEX, sram
; FDEC, sram
; FCHAR,sram
; FSTR, sram
;
; The address 'sram' is a 1-byte constant. It addresses
; 0..1f registers r0..r31,
; 20..3f i/o ports, (need to be addressed with an offset of $20)
; 0x0260..0x02ff SRAM
; Variables can be located into register and I/0s, and can also
; be stored into data SRAM at locations 0x0200 through 0x02ff. Any
; sram address higher than 0x0060 is assumed to be at (0x0260+address)
; from automatic address detection in _printf_formatted: and subsequent
; assignment to xh; xl keeps its value. Consequently, variables that are
; to be stored into SRAM and further printed by fprint must reside at
; 0x0200 up to 0x02ff, and must be addressed using a label. Usage: see
; file string1.asm, for example.

; The FFRAC formatting character must be followed by
; ONE sram address and
; TWO more formatting characters
; FFRAC,sram,dp,$if

; dp decimal point position, 0=right, 32=left
; $if format i.f, i=integer digits, f=fraction digits

; The special formatting characters use the following coding
;
; FDEC 11bb'iiis i=0 all digits, i=1-7 digits
; FBIN 101i'iiis i=0 8 digits, i=1-7 digits
; FHEX 1001'iiis i=0 8 digits, i=1-7 digits
```

```
; FFRAC 1000'1bbs
; FCHAR 1000'0100
; FSTR 1000'0101
; FREP 1000'0110
; FFUNC 1000'0111
; 1000'0010
; 1000'0011
; FESC 1000'0000

; examples
; formatting string printing
; "a=",FDEC,a,0 1-byte variable a, unsigned decimal
; "a=",FDEC2,a,0 2-byte variable a (a1,a0), unsigend
; "a=",FDEC|FSIGN,a,0 1-byte variable 1, signed decimal
; "n=",FBIN,PIND+$20,0 i/o port, binary, notice offset of $20
; "f=",FFRAC4|FSIGN,a,16,$88,0 4-byte signed fixed-point fraction
; dec.point at 16, 8 int.digits, 8 frac.digits
; "f=",FFRAC2,a,16,$18,0 2-byte unsigned fixed-point fraction
; dec.point at 16, 1 int.digits, 8 frac.digits
; "a=",FDEC|FDIG5|FSIGN,a,0 1-byte variable, 5-digit, decimal, signed
; "a=",FDEC|FDIG5,a,0 1-byte variable, 5-digit, decimal, unsigned

; === registers modified ===
; e0,e1 used to transmit address of putc routine
; zh,zl used as pointer to prog-memory

; === constants ==============================================

.equ FDEC = 0b11000000 ; 1-byte variable
.equ FDEC2 = 0b11010000 ; 2-byte variable
.equ FDEC3 = 0b11100000 ; 3-byte variable
.equ FDEC4 = 0b11110000 ; 4-byte variable

.equ FBIN = 0b10100000
.equ FHEX = 0b10010100 ; 1-byte variable
.equ FHEX2 = 0b10011000 ; 2-byte variable
.equ FHEX3 = 0b10011100 ; 3-byte variable
.equ FHEX4 = 0b10010000 ; 4-byte variable

.equ FFRAC = 0b10001000 ; 1-byte variable
.equ FFRAC2 = 0b10001010 ; 2-byte variable
.equ FFRAC3 = 0b10001100 ; 3-byte variable
.equ FFRAC4 = 0b10001110 ; 4-byte variable

.equ FCHAR = 0b10000100
.equ FSTR = 0b10000101

.equ FSIGN = 0b00000001

.equ FDIG1 = 1<<1
.equ FDIG2 = 2<<1
.equ FDIG3 = 3<<1
.equ FDIG4 = 4<<1
.equ FDIG5 = 5<<1
.equ FDIG6 = 6<<1
.equ FDIG7 = 7<<1

; ===macro ============================================================

.macro PRINTF ; putc function (UART, LCD...)
    ldi w, low(@0) ; address of "putc" in e1:d0
    mov e0,w
    ldi w,high(@0)
    mov e1,w
    rcall _printf
    .endmacro

; mod y,z
```

```
; === routines ================================================

_printf:
    POPZ ; z points to begin of "string"
    MUL2Z ; multiply Z by two, (word ptr -> byte ptr)
    PUSHX

_printf_read:
    lpm ; places prog_mem(Z) into r0 (=c)
    adiw zl,1 ; increment pointer Z
    tst r0 ; test for ZERO (=end of string)
    breq _printf_end ; char=0 indicates end of ascii string
    brmi _printf_formatted ; bit7=1 indicates formatting character
    mov w,r0
    rcall _putw ; display the character
    rjmp _printf_read ; read next character in the string

_printf_end:
    adiw zl,1 ; point to the next character
    DIV2Z ; divide by 2 (byte ptr -> word ptr)
    POPX
    ijmp ; return to instruction after "string"

_printf_formatted:

; FDEC 11bb'iiis
; FBIN 101i'iiis
; FHEX 1001'iiis
; FFRAC 1000'1bbs
; FCHAR 1000'0100
; FSTR 1000'0101

    bst r0,0 ; store sign in T
    mov w,r0 ; store formatting character in w
    lpm
    mov xl,r0 ; load x-pointer with SRAM address
    cpi xl,0x60
    brlo rio_space
dataram_space: ; variable originates from SRAM memory
    ldi xh,0x02 ;>addresses are limited to 0x0260 through 0x02ff
    rjmp space_detect_end ;>that enables automatic detection of the origin
rio_space: ; variable originates from reg or I/O space
    clr xh ; clear high-byte, addresses are 0x0000 through 0x003f (0x005f)
space_detect_end:
     adiw zl,1 ; increment pointer Z

; JB1 w,6,_putdec
; JB1 w,5,_putbin
; JB1 w,4,_puthex
; JB1 w,3,_putfrac
    JK w,FCHAR,_putchar
    JK w,FSTR ,_putstr
    rjmp _putnum

    rjmp _printf_read

; === putc (put character) ================================
; in w character to put
; e1,e0 address of output routine (UART, LCD putc)
_putw:
    PUSH3 a0,zh,zl
    MOV3 a0,zh,zl, w,e1,e0
    icall ; indirect call to "putc"
    POP3 a0,zh,zl
    ret
```

```
; === putchar (put character) ============================
; in x pointer to character to put
_putchar:
    ld w,x
    rcall _putw
    rjmp _printf_read

; === putstr (put string) ================================
; in x pointer to ascii string
; b3,b2 address of output routine (UART, LCD putc)
_putstr:
    ld w,x+
    tst w
    brne PC+2
    rjmp _printf_read
    rcall _putw
    rjmp _putstr

; === putnum (dec/bin/hex/frac) ==========================
; in x pointer to SRAM variable to print
; r0 formatting character

_putnum:
    PUSH4 a3,a2,a1,a0 ; safeguard a
    PUSH4 b3,b2,b1,b0 ; safeguard b
    LDX4 a3,a2,a1,a0 ; load operand to print into a

; FDEC 11bb'iiis
; FBIN 101i'iiis
; FHEX 1001'iiis
; FRACT 1000'1bbs

    JB1 w,6,_putdec
    JB1 w,5,_putbin
    JB1 w,4,_puthex
    JB1 w,3,_putfrac

; FDEC 11bb'iiis
_putdec:
    ldi b0,10 ; b0 = base (10)

    mov b1,w
    lsr b1
    andi b1,0b111
    swap b1 ; b1 = format 0iii'0000 (integer digits)
    ldi b2,0 ; b2 = dec. point position = 0 (right)

    mov b3,w
    swap b3
    andi b3,0b11
    inc b3 ; b3 = number of bytes (1..4)
    rjmp _getnum ; get number of digits (iii)

; FBIN 101i'iiis addr
_putbin:
    ldi b0,2 ; b0 = base (2)
    ldi b3,4 ; b3 = number of bytes (4)
    rjmp _getdig ; get number of digits (iii)

; FHEX 1001'iiis addr
_puthex:
    ldi b0,16 ; b0 = base (16)
    ldi b3,4 ; b3 = number of bytes (4)
    rjmp _getdig

_getdig:
    mov b1,w
    lsr b1
```

```
    andi b1,0b111
    brne PC+2
    ldi b1,8 ; if b1=0 then 8-digits
    swap b1 ; b1 = format 0iii'0000 (integer digits)
    ldi b2, 0 ; b2 = dec. point position = 0 (right)
    rjmp _getnum

; FFRAC 1000'1bbs addr 00dd'dddd, iiii'ffff

_putfrac:
    ldi b0,10 ; base=10
    lpm
    mov b2,r0 ; load dec.point position
    adiw zl,1 ; increment char pointer
    lpm
    mov b1,r0 ; load ii.ff format
    adiw zl,1 ; increment char pointer

    mov b3,w
    asr b3
    andi b3,0b11
    inc b3 ; b3 = number of bytes (1..4)

    rjmp _getnum

_getnum:
; in a 4-byte variable
; b3 number of bytes (1..4)
; T sign, 0=unsigned, 1=signed

    JK b3,4,_printf_4b
    JK b3,3,_printf_3b
    JK b3,2,_printf_2b

_printf_1b: ; sign extension
    clr a1
    brtc PC+3 ; T=1 sign extension
    sbrc a0,7
    ldi a1,0xff
_printf_2b:
    clr a2
    brtc PC+3 ; T=1 sign extension
    sbrc a1,7
    ldi a2,0xff
_printf_3b:
    clr a3
    brtc PC+3 ; T=1 sign extension
    sbrc a2,7
    ldi a3,0xff
_printf_4b:

    rcall _ftoa ; float to ascii
    POP4 b3,b2,b1,b0 ; restore b
    POP4 a3,a2,a1,a0 ; restore a

    rjmp _printf_read

; ================================================
; func ftoa
; converts a fixed-point fractional number to an ascii string
; author (c) Raphael Holzer
;
; in a3-a0 variable to print
; b0 base, 2 to 36, but usually decimal (10)
; b1 number of digits to print ii.ff
; b2 position of the decimal point (0=right, 32=left)
; T sign (T=0 unsiged, T=1 signed)
```

```
_ftoa:
    push d0
    PUSH4 c3,c2,c1,c0 ; c = fraction part, a = integer part
    CLR4 c3,c2,c1,c0 ; clear fraction part

    brtc _ftoa_plus ; if T=0 then unsigned
    clt
    tst a3 ; if MSb(a)=1 then a=-a
    brpl _ftoa_plus
    set ; T=1 (minus)
    tst b1
    breq PC+2 ; if b1=0 the print ALL digits
    subi b1,0x10 ; decrease int digits
    NEG4 a3,a2,a1,a0 ; negate a
_ftoa_plus:
    tst b2 ; b0=0 (only integer part)
    breq _ftoa_int
_ftoa_shift:
    ASR4 a3,a2,a1,a0 ; a = integer part
    ROR4 c3,c2,c1,c0 ; c = fraction part
    DJNZ b2,_ftoa_shift
_ftoa_int:
    push b1 ; ii.ff (ii=int digits)
    swap b1
    andi b1,0x0f

    ldi w,'.' ; push decimal point
    push w
_ftoa_int1:
    rcall _div41 ; int=int/10
    mov w,d0 ; d=reminder
    rcall _hex2asc
    push w ; push rem(int/10)
    TST4 a3,a2,a1,a0 ; (int/10)=?
    breq _ftoa_space ; (int/10)=0 then finished
    tst b1
    breq _ftoa_int1 ; if b1=0 then print ALL int-digits
    DJNZ b1,_ftoa_int1
    rjmp _ftoa_sign
_ftoa_space:
    tst b1 ; if b1=0 then print ALL int-digits
    breq _ftoa_sign
    dec b1
    breq _ftoa_sign
    ldi w,' ' ; write spaces
    rcall _putw
    rjmp _ftoa_space
_ftoa_sign:
    brtc PC+3 ; if T=1 then write 'minus'
    ldi w,'-'
    rcall _putw
_ftoa_int3:
    pop w
    cpi w,'.'
    breq PC+3
    rcall _putw
    rjmp _ftoa_int3

    pop b1 ; ii.ff (ff=frac digits)
    andi b1,0x0f
    tst b1
    breq _ftoa_end
_ftoa_point:
    rcall _putw ; write decimal point
    MOV4 a3,a2,a1,a0, c3,c2,c1,c0
_ftoa_frac:
    rcall _mul41 ; d.frac=10*frac
    mov w,d0
```

```
    rcall _hex2asc
    rcall _putw
    DJNZ b1,_ftoa_frac
_ftoa_end:
    POP4 c3,c2,c1,c0
    pop d0
    ret

; === hexadecimal to ascii ===
; in w
_hex2asc:
    cpi w,10
    brsh PC+3
    addi w,'0'
    ret
    addi w,('a'-10)
    ret

; === multiply 4byte*1byte ===
; funct mul41
; multiplies a3-a0 (4-byte) by b0 (1-byte)
; author (c) Raphael Holzer, EPFL
;
; in a3..a0 multiplicand (argument to multiply)
; b0 multiplier
; out a3..a0 result
; d0 result MSB (byte 4)
;
_mul41: clr d0 ; clear byte4 of result
    ldi w,32 ; load bit counter
__m41: clc ; clear carry
    sbrc a0,0 ; skip addition if LSB=0
    add d0,b0 ; add b to MSB of a
    ROR5 d0,a3,a2,a1,a0 ; shift-right c, LSB (of b) into carry
    DJNZ w,__m41 ; Decrement and Jump if bit-count Not Zero
    ret

; === divide 4byte/1byte ===
; func div41
; in a0..a3 divident (argument to divide)
; b0 divider
; out a0..a3 result
; d0 reminder
;
_div41: clr d0 ; d will contain the remainder
    ldi w,32 ; load bit counter
__d41: ROL5 d0,a3,a2,a1,a0 ; shift carry into result c
    sub d0, b0 ; subtract b from remainder
    brcc PC+2
    add d0, b0 ; restore if remainder became negative
    DJNZ w,__d41 ; Decrement and Jump if bit-count Not Zero
    ROL4 a3,a2,a1,a0 ; last shift (carry into result c)
    COM4 a3,a2,a1,a0 ; complement result
    ret
```

definitions.asm

```
;=====================================================================
; definitions.asm - register aliases, ASCII codes, board pin map
;=====================================================================
;
; WHAT THIS FILE DOES
; Central place for symbolic names used across the whole project:
; - named working registers (char, u, w, the multi-byte a/b/c/d
; register groups, and the X/Y/Z pointer aliases)
; - ASCII control codes
; - STK-300 / module pin assignments (speaker, IR, encoder, servo,
; I2C, sensors, etc.)
; It defines no code, only .def/.equ/.set. Included first so every
; other file can use these names.
;
; Target: ATmega128L-4MHz-STK300
;=====================================================================

; === definitions ===
.nolist ; do not include in listing
.set clock = 4000000

.def char = r0 ; character (ASCII)
.def _sreg = r1 ; saves the status during interrupts
.def _u = r2 ; saves working reg u during interrupt
.def u = r3 ; scratch register (macros, routines)

.def e0 = r4 ; temporary reg for PRINTF
.def e1 = r5

.equ c = 8
.def c0 = r8 ; 8-byte register c
.def c1 = r9
.def c2 = r10
.def c3 = r11

.equ d = 12 ; 4-byte register d (overlapping with c)
.def d0 = r12
.def d1 = r13
.def d2 = r14
.def d3 = r15

.def w = r16 ; working register for macros
.def _w = r17 ; working register for interrupts

.equ a = 18
.def a0 = r18 ; 4-byte register a
.def a1 = r19
.def a2 = r20
.def a3 = r21

.equ b = 22
.def b0 = r22 ; 4-byte register b
.def b1 = r23
.def b2 = r24
.def b3 = r25

.equ px = 26 ; pointer x
.equ py = 28 ; pointer y
.equ pz = 30 ; pointer z

; === ASCII codes
.equ BEL =0x07 ; bell
.equ HT =0x09 ; horizontal tab
.equ TAB =0x09 ; tab
.equ LF =0x0a ; line feed
```

```
.equ VT =0x0b ; vertical tab
.equ FF =0x0c ; form feed
.equ CR =0x0d ; carriage return
.equ SPACE =0x20 ; space code
.equ DEL =0x7f ; delete
.equ BS =0x08 ; back space

; === STK-300 ===
.equ LED = PORTB ; LEDs on STK-300
.equ BUTTON = PIND ; buttons on the STK-300

; === module M2 (encoder/speaker/IR remote) ===
.equ SPEAKER = 2 ; piezo speaker
.equ ENCOD_A = 4 ; angular encoder A
.equ ENCOD_B = 5 ; angular encoder B
.equ ENCOD_I = 6 ; angular encoder button
.equ IR = 7 ; IR module for PCM remote control system

; === module M5 (I2C/1Wire) ===
.equ SCL = 0 ; I2C serial clock
.equ SDA = 1 ; I2C serial data
.equ DQ = 5 ; Dallas 1Wire
               ; master transmitter status codes, Table 88
.equ I2CMT_START = 0x08 ; start
.equ I2CMT_REPSTART = 0x10 ; repeated start
.equ I2CMT_SLA_ACK= 0x18 ; slave ack
.equ I2CMT_SLA_NOACK = 0x20 ; slave no ack
.equ I2CMT_DATA_ACK = 0x28 ; data write, ack
.equ I2CMT_DATA_NOACK = 0x30 ; data write, no ack
               ; master receiver status codes, Table 89
.equ I2CMR_SLA_ACK = 0x40 ; slave address ack
.equ I2CMR_SLA_NACK = 0x48 ; slave address no ack
.equ I2CMR_DATA_ACK = 0x50 ; master data ack
.equ I2CMR_DATA_NACK= 0x58 ; master data no ack

; === module M4 (Keyboard/Sharp/Servo) ===
.equ KB_CLK = 0 ; PC-AT keyboard clock line
.equ KB_DAT = 1 ; PC-AT keyboard data line
.equ GP2_CLK = 2 ; Sharp GP2D02 distance measuring sensor
.equ GP2_DAT = 3 ; Sharp GP2D02 distance measuring sensor
.equ GP2_AVAL = 3; Shart GP2Y0A21 distance measuring sensor
.equ SERVO1 = 4 ; Futaba position servo

; === module M3 (potentiometer/BNC) ===
.equ POT = 0 ; potentiometer
.equ BNC1 = 2 ; BNC input
.equ BNC2 = 4 ; BNC input
.list
```

macros.asm

```
; file: macros.asm target ATmega128L-4MHz-STK300
; purpose library, general-purpose macros
; author (c) R.Holzer (adapted MICRO210/EE208 A.Schmid)
; v2019.01 20180820 AxS

; ==============
; pointers
; ==============

; --- loading an immediate into a pointer XYZ,SP ---
.macro LDIX ; sram
    ldi xl, low(@0)
    ldi xh,high(@0)
    .endmacro
.macro LDIY ; sram
    ldi yl, low(@0)
    ldi yh,high(@0)
    .endmacro
.macro LDIZ ; sram
    ldi zl, low(@0)
    ldi zh,high(@0)

    .endmacro
.macro LDZD ; sram, reg ; sram+reg -> Z
    mov zl,@1
    clr zh
    subi zl, low(-@0)
    sbci zh,high(-@0)
    .endmacro
.macro LDSP ; sram
    ldi r16, low(@0)
    out spl,r16
    ldi r16,high(@0)
    out sph,r16
    .endmacro

; --- load/store SRAM addr into pointer XYZ ---
.macro LDSX ; sram
    lds xl,@0
    lds xh,@0+1
    .endmacro
.macro LDSY ; sram
    lds yl,@0
    lds yh,@0+1
    .endmacro
.macro LDSZ ; sram
    lds zl,@0
    lds zh,@0+1
    .endmacro
.macro STSX ; sram
    sts @0, xl
    sts @0+1,xh
    .endmacro
.macro STSY ; sram
    sts @0, yl
    sts @0+1,yh
    .endmacro
.macro STSZ ; sram
    sts @0, zl
    sts @0+1,zh
    .endmacro

; --- push/pop pointer XYZ ---
.macro PUSHX ; push X
    push xl
```

```
    push xh
    .endmacro
.macro POPX ; pop X
    pop xh
    pop xl
    .endmacro

.macro PUSHY ; push Y
    push yl
    push yh
    .endmacro
.macro POPY ; pop Y
    pop yh
    pop yl
    .endmacro

.macro PUSHZ ; push Z
    push zl
    push zh
    .endmacro
.macro POPZ ; pop Z
    pop zh
    pop zl
    .endmacro

; --- multiply/divide Z ---
.macro MUL2Z ; multiply Z by 2
    lsl zl
    rol zh
    .endmacro
.macro DIV2Z ; divide Z by 2
    lsr zh
    ror zl
    .endmacro

; --- add register to pointer XYZ ---
.macro ADDX ;reg ; x <- y+reg
    add xl,@0
    brcc PC+2
    subi xh,-1 ; add carry
    .endmacro
.macro ADDY ;reg ; y <- y+reg
    add yl,@0
    brcc PC+2
    subi yh,-1 ; add carry
    .endmacro
.macro ADDZ ;reg ; z <- z+reg
    add zl,@0
    brcc PC+2
    subi zh,-1 ; add carry
    .endmacro

; ===================
; miscellaneous
; ===================

; --- output/store (regular I/O space) immediate value ---
.macro OUTI ; port,k output immediate value to port
    ldi w,@1
    out @0,w
    .endmacro

; --- output/store (extended I/O space) immediate value ---
.macro OUTEI ; port,k output immediate value to port
    ldi w,@1
    sts @0,w
    .endmacro
```

```
; --- add immediate value ---
.macro ADDI
    subi @0,-@1
    .endmacro
.macro ADCI
    sbci @0,-@1
    .endmacro

; --- inc/dec with range limitation ---
.macro INC_LIM ; reg,limit
    cpi @0,@1
    brlo PC+3
    ldi @0,@1
    rjmp PC+2
    inc @0
    .endmacro

.macro DEC_LIM ; reg,limit
    cpi @0,@1
    breq PC+5
    brlo PC+3
    dec @0
    rjmp PC+2
    ldi @0,@1
    .endmacro

; --- inc/dec with cyclic range ---
.macro INC_CYC ; reg,low,high
    cpi @0,@2
    brsh _low ; reg>=high then reg=low
    cpi @0,@1
    brlo _low ; reg< low then reg=low
    inc @0
    rjmp _done
_low: ldi @0,@1
_done:
    .endmacro

.macro DEC_CYC ; reg,low,high
    cpi @0,@1
    breq _high ; reg=low then reg=high
    brlo _high ; reg<low then reg=high
    dec @0
    cpi @0,@2
    brsh _high ; reg>=high then high
    rjmp _done
_high: ldi @0,@2
_done:
    .endmacro

.macro INCDEC ;port,b1,b2,reg,low,high
    sbic @0,@1
    rjmp PC+6

    cpi @3,@5
    brlo PC+3
    ldi @3,@4
    rjmp PC+2
    inc @3

    sbic @0,@2
    rjmp PC+7

    cpi @3,@4
    breq PC+5
    brlo PC+3
    dec @3
    rjmp PC+2
```

```
    ldi @3,@5
    .endmacro

; --- wait loops ---
; wait 10...196608 cycles
.macro WAIT_C ; k
    ldi w, low((@0-7)/3)
    mov u,w ; u=LSB
    ldi w,high((@0-7)/3)+1 ; w=MSB
    dec u
    brne PC-1
    dec u
    dec w
    brne PC-4
    .endmacro

; wait micro-seconds (us)
; us = x*3*1000'000/clock) ==> x=us*clock/3000'000
.macro WAIT_US ; k
    ldi w, low((clock/1000*@0/3000)-1)
    mov u,w
    ldi w,high((clock/1000*@0/3000)-1)+1 ; set up: 3 cyles
    dec u
    brne PC-1 ; inner loop: 3 cycles
    dec u ; adjustment for outer loop
    dec w
    brne PC-4
    .endmacro

; wait mili-seconds (ms)
.macro WAIT_MS ; k
    ldi w, low(@0)
    mov u,w ; u = LSB
    ldi w,high(@0)+1 ; w = MSB
wait_ms:
    push w ; wait 1000 usec
    push u
    ldi w, low((clock/3000)-5)
    mov u,w
    ldi w,high((clock/3000)-5)+1
    dec u
    brne PC-1 ; inner loop: 3 cycles
    dec u ; adjustment for outer loop
    dec w
    brne PC-4
    pop u
    pop w

    dec u
    brne wait_ms
    dec w
    brne wait_ms
    .endmacro

; --- conditional jumps/calls ---
.macro JC0 ; jump if carry=0
    brcs PC+2
    rjmp @0
    .endmacro
.macro JC1 ; jump if carry=1
    brcc PC+2
    rjmp @0
    .endmacro

.macro JK ; reg,k,addr ; jump if reg=k
    cpi @0,@1
    breq @2
    .endmacro
```

```
.macro _JK ; reg,k,addr ; jump if reg=k
    cpi @0,@1
    brne PC+2
    rjmp @2
    .endmacro
.macro JNK ; reg,k,addr ; jump if not(reg=k)
    cpi @0,@1
    brne @2
    .endmacro

.macro CK ; reg,k,addr ; call if reg=k
    cpi @0,@1
    brne PC+2
    rcall @2
    .endmacro
.macro CNK ; reg,k,addr ; call if not(reg=k)
    cpi @0,@1
    breq PC+2
    rcall @2
    .endmacro

.macro JSK ; sram,k,addr ; jump if sram=k
    lds w,@0
    cpi w,@1
    breq @2
    .endmacro
.macro JSNK ; sram,k,addr ; jump if not(sram=k)
    lds w,@0
    cpi w,@1
    brne @2
    .endmacro

; --- loops ---
.macro DJNZ ; reg,addr ; decr and jump if not zero
    dec @0
    brne @1
    .endmacro
.macro DJNK ; reg,k,addr ; decr and jump if not k
    dec @0
    cpi @0,@1
    brne @2
    .endmacro

.macro IJNZ ; reg,addr ; inc and jump if not zero
    inc @0
    brne @1
    .endmacro
.macro IJNK ; reg,k,addr ; inc and jump if not k
    inc @0
    cpi @0,@1
    brne @2
    .endmacro
.macro _IJNK ; reg,k,addr ; inc and jump if not k
    inc @0
    ldi w,@1
    cp @0,w
    brne @2
    .endmacro

.macro ISJNK ; sram,k,addr ; inc sram and jump if not k
    lds w,@0
    inc w
    sts @0,w
    cpi w,@1
    brne @2
    .endmacro
.macro _ISJNK ; sram,k,addr ; inc sram and jump if not k
    lds w,@0
```

```
    inc w
    sts @0,w
    cpi w,@1
    breq PC+2
    rjmp @2
    .endmacro

.macro DSJNK ; sram,k,addr ; dec sram and jump if not k
    lds w,@0
    dec w
    sts @0,w
    cpi w,@1
    brne @2
    .endmacro

; --- table lookup ---
.macro LOOKUP ;reg, index,tbl
    push ZL
    push ZH
    mov zl,@1 ; move index into z
    clr zh
    subi zl, low(-2*@2) ; add base address of table
    sbci zh,high(-2*@2)
    lpm ; load program memory (into r0)
    mov @0,r0
    pop ZH
    pop ZL
    .endmacro

.macro LOOKUP2 ;r1,r0, index,tbl
    mov zl,@2 ; move index into z
    clr zh
    lsl zl ; multiply by 2
    rol zh
    subi zl, low(-2*@3) ; add base address of table
    sbci zh,high(-2*@3)
    lpm ; get LSB byte
    mov w,r0 ; temporary store LSB in w
    adiw zl,1 ; increment Z
    lpm ; get MSB byte
    mov @0,r0 ; mov MSB to res1
    mov @1,w ; mov LSB to res0
    .endmacro

.macro LOOKUP4 ;r3,r2,r1,r0, index,tbl
    mov zl,@4 ; move index into z
    clr zh
    lsl zl ; multiply by 2
    rol zh
    lsl zl ; multiply by 2
    rol zh
    subi zl, low(-2*@5) ; add base address of table
    sbci zh,high(-2*@5)
    lpm
    mov @1,r0 ; load high word LSB
    adiw zl,1
    lpm
    mov @0,r0 ; load high word MSB
    adiw zl,1
    lpm
    mov @3,r0 ; load low word LSB
    adiw zl,1
    lpm
    mov @2,r0 ; load low word MSB
    .endmacro

.macro LOOKDOWN ;reg,index,tbl
    ldi ZL, low(2*@2) ; load table address
```

```
    ldi ZH,high(2*@2)
    clr @1
loop: lpm
    cp r0,@0
    breq found
    inc @1
    adiw ZL,1
    tst r0
    breq notfound
    rjmp loop
notfound:
    ldi @1,-1
found:
    .endmacro

; --- branch table ---
.macro C_TBL ; reg,tbl
    ldi ZL, low(2*@1)
    ldi ZH,high(2*@1)
    lsl @0
    add ZL,@0
    brcc PC+2
    inc ZH
    lpm
    push r0
    lpm
    mov zh,r0
    pop zl
    icall
    .endmacro
.macro J_TBL ; reg,tbl
    ldi ZL, low(2*@1)
    ldi ZH,high(2*@1)
    lsl @0
    add ZL,@0
    brcc PC+2
    inc ZH
    lpm
    push r0
    lpm
    mov zh,r0
    pop zl
    ijmp
    .endmacro

.macro BRANCH ; reg ; branching using the stack
    ldi w, low(tbl)
    add w,@0
    push w
    ldi w,high(tbl)
    brcc PC+2
    inc w
    push w
    ret
tbl:
    .endmacro

; --- multiply/division ---
.macro DIV2 ; reg
    lsr @0
    .endmacro
.macro DIV4 ; reg
    lsr @0
    lsr @0
    .endmacro
.macro DIV8 ; reg
    lsr @0
    lsr @0
```

```
    lsr @0
    .endmacro

.macro MUL2 ; reg
    lsl @0
    .endmacro
.macro MUL4 ; reg
    lsl @0
    lsl @0
    .endmacro
.macro MUL8 ; reg
    lsl @0
    lsl @0
    lsl @0
    .endmacro

; ====================================
; extending existing instructios
; ====================================

; --- immediate ops with r0..r15 ---
.macro _ADDI
    ldi w,@1
    add @0,w
    .endmacro
.macro _ADCI
    ldi w,@1
    adc @0,w
    .endmacro
.macro _SUBI
    ldi w,@1
    sub @0,w
    .endmacro
.macro _SBCI
    ldi w,@1
    sbc @0,w
    .endmacro
.macro _ANDI
    ldi w,@1
    and @0,w
    .endmacro
.macro _ORI
    ldi w,@1
    or @0,w
    .endmacro
.macro _EORI
    ldi w,@1
    eor @0,w
    .endmacro
.macro _SBR
    ldi w,@1
    or @0,w
    .endmacro
.macro _CBR
    ldi w,~@1
    and @0,w
    .endmacro
.macro _CPI
    ldi w,@1
    cp @0,w
    .endmacro
.macro _LDI
    ldi w,@1
    mov @0,w
    .endmacro

; --- bit access for port p32..p63 ---
.macro _SBI
```

```
    in w,@0
    ori w,1<<@1
    out @0,w
    .endmacro
.macro _CBI
    in w,@0
    andi w,~(1<<@1)
    out @0,w
    .endmacro

; --- extending branch distance to +/-2k ---
.macro _BREQ
    brne PC+2
    rjmp @0
    .endmacro
.macro _BRNE
    breq PC+2
    rjmp @0
    .endmacro
.macro _BRCS
    brcc PC+2
    rjmp @0
    .endmacro
.macro _BRCC
    brcs PC+2
    rjmp @0
    .endmacro
.macro _BRSH
    brlo PC+2
    rjmp @0
    .endmacro
.macro _BRLO
    brsh PC+2
    rjmp @0
    .endmacro
.macro _BRMI
    brpl PC+2
    rjmp @0
    .endmacro
.macro _BRPL
    brmi PC+2
    rjmp @0
    .endmacro
.macro _BRGE
    brlt PC+2
    rjmp @0
    .endmacro
.macro _BRLT
    brge PC+2
    rjmp @0
    .endmacro
.macro _BRHS
    brhc PC+2
    rjmp @0
    .endmacro
.macro _BRHC
    brhs PC+2
    rjmp @0
    .endmacro
.macro _BRTS
    brtc PC+2
    rjmp @0
    .endmacro
.macro _BRTC
    brts PC+2
    rjmp @0
    .endmacro
.macro _BRVS
```

```
    brvc PC+2
    rjmp @0
    .endmacro
.macro _BRVC
    brvs PC+2
    rjmp @0
    .endmacro
.macro _BRIE
    brid PC+2
    rjmp @0
    .endmacro
.macro _BRID
    brie PC+2
    rjmp @0
    .endmacro

; ====================
; bit operations
; ====================

; --- moving bits ---
.macro MOVB ; reg1,b1, reg2,b2 ; reg1,bit1 <- reg2,bit2
    bst @2,@3
    bld @0,@1
    .endmacro
.macro OUTB ; port1,b1, reg2,b2 ; port1,bit1 <- reg2,bit2
    sbrs @2,@3
    cbi @0,@1
    sbrc @2,@3
    sbi @0,@1
    .endmacro
.macro INB ; reg1,b1, port2,b2 ; reg1,bit1 <- port2,bit2
    sbis @2,@3
    cbr @0,1<<@1
    sbic @2,@3
    sbr @0,1<<@1
    .endmacro

.macro Z2C ; zero to carry
    sec
    breq PC+2 ; (Z=1)
    clc
    .endmacro
.macro Z2INVC ; zero to inverse carry
    sec
    brne PC+2 ; (Z=0)
    clc
    .endmacro

.macro C2Z ; carry to zero
    sez
    brcs PC+2 ; (C=1)
    clz
    .endmacro

.macro B2C ; reg,b ; bit to carry
    sbrc @0,@1
    sec
    sbrs @0,@1
    clc
    .endmacro
.macro C2B ; reg,b ; carry to bit
    brcc PC+2
    sbr @0,(1<<@1)
    brcs PC+2
    cbr @0,(1<<@1)
    .endmacro
.macro P2C ; port,b ; port to carry
```

```
    sbic @0,@1
    sec
    sbis @0,@1
    clc
    .endmacro
.macro C2P ; port,b ; carry to port
    brcc PC+2
    sbi @0,@1
    brcs PC+2
    cbi @0,@1
    .endmacro

; --- inverting bits ---
.macro INVB ; reg,bit ; inverse reg,bit
    ldi w,(1<<@1)
    eor @0,w
    .endmacro
.macro INVP ; port,bit ; inverse port,bit
    sbis @0,@1
    rjmp PC+3
    cbi @0,@1
    rjmp PC+2
    sbi @0,@1
    .endmacro
.macro INVC ; inverse carry
    brcs PC+3
    sec
    rjmp PC+2
    clc
    .endmacro

; --- setting a single bit ---
.macro SETBIT ; reg(0..7)
; in reg (0..7)
; out reg with bit (0..7) set to 1.
; 0=00000001
; 1=00000010
; ...
; 7=10000000
    mov w,@0
    clr @0
    inc @0
    andi w,0b111
    breq PC+4
    lsl @0
    dec w
    brne PC-2
    .endmacro

; --- logical operations with masks ---
.macro MOVMSK ; reg1,reg2,mask ; reg1 <- reg2 (mask)
    ldi w,~@2
    and @0,w
    ldi w,@2
    and @1,w
    or @0,@1
    .endmacro
.macro ANDMSK ; reg1,reg2,mask ; reg1 <- ret 1 AND reg2 (mask)
    mov w,@1
    ori w,~@2
    and @0,w
    .endmacro
.macro ORMSK ; reg1,reg2,mask ; reg1 <- ret 1 AND reg2 (mask)
    mov w,@1
    andi w,@2
    or @0,w
    .endmacro
```

```
; --- logical operations on bits ---
.macro ANDB ; r1,b1, r2,b2, r3,b3 ; reg1,b1 <- reg2,b2 AND reg3,b3
    set
    sbrs @4,@5
    clt
    sbrs @2,@3
    clt
    bld @0,@1
    .endmacro
.macro ORB ; r1,b1, r2,b2, r3,b3 ; reg1.b1 <- reg2.b2 OR reg3.b3
    clt
    sbrc @4,@5
    set
    sbrc @2,@3
    set
    bld @0,@1
    .endmacro
.macro EORB ; r1,b1, r2,b2, r3,b3 ; reg1.b1 <- reg2.b2 XOR reg3.b3
    sbrc @4,@5
    rjmp f1
f0: bst @2,@3
    rjmp PC+4
f1: set
    sbrc @0,@1
    clt
    bld @0,@0
    .endmacro

; --- operations based on register bits ---
.macro FB0 ; reg,bit ; bit=0
    cbr @0,1<<@1
    .endmacro
.macro FB1 ; reg,bit ; bit=1
    sbr @0,1<<@1
    .endmacro
.macro _FB0 ; reg,bit ; bit=0
    ldi w,~(1<<@1)
    and @0,w
    .endmacro
.macro _FB1 ; reg,bit ; bit=1
    ldi w,1<<@1
    or @0,w
    .endmacro
.macro SB0 ; reg,bit,addr ; skip if bit=0
    sbrc @0,@1
    .endmacro
.macro SB1 ; reg,bit,addr ; skip if bit=1
    sbrs @0,@1
    .endmacro
.macro JB0 ; reg,bit,addr ; jump if bit=0
    sbrs @0,@1
    rjmp @2
    .endmacro
.macro JB1 ; reg,bit,addr ; jump if bit=1
    sbrc @0,@1
    rjmp @2
    .endmacro
.macro CB0 ; reg,bit,addr ; call if bit=0
    sbrs @0,@1
    rcall @2
    .endmacro
.macro CB1 ; reg,bit,addr ; call if bit=1
    sbrc @0,@1
    rcall @2
    .endmacro
.macro WB0 ; reg,bit ; wait if bit=0
    sbrs @0,@1
    rjmp PC-1
```

```
    .endmacro
.macro WB1 ; reg,bit ; wait if bit=1
    sbrc @0,@1
    rjmp PC-1
    .endmacro
.macro RB0 ; reg,bit ; return if bit=0
    sbrs @0,@1
    ret
    .endmacro
.macro RB1 ; reg,bit ; return if bit=1
    sbrc @0,@1
    ret
    .endmacro

; wait if bit=0 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WB0T ; reg,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbrs @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; wait if bit=1 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WB1T ; reg,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbrc @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; --- operations based on port bits ---
.macro P0 ; port,bit ; port=0
    cbi @0,@1
    .endmacro
.macro P1 ; port,bit ; port=1
    sbi @0,@1
    .endmacro
.macro SP0 ; port,bit ; skip if port=0
    sbic @0,@1
    .endmacro
.macro SP1 ; port,bit ; skip if port=1
    sbis @0,@1
    .endmacro
.macro JP0 ; port,bit,addr ; jump if port=0
    sbis @0,@1
    rjmp @2
    .endmacro
.macro JP1 ; port,bit,addr ; jump if port=1
    sbic @0,@1
    rjmp @2
    .endmacro
.macro CP0 ; port,bit,addr ; call if port=0
    sbis @0,@1
    rcall @2
    .endmacro
.macro CP1 ; port,bit,addr ; call if port=1
    sbic @0,@1
    rcall @2
    .endmacro
.macro WP0 ; port,bit ; wait if port=0
    sbis @0,@1
    rjmp PC-1
    .endmacro
.macro WP1 ; port,bit ; wait if port=1
```

```
    sbic @0,@1
    rjmp PC-1
    .endmacro
.macro RP0 ; port,bit ; return if port=0
    sbis @0,@1
    ret
    .endmacro
.macro RP1 ; port,bit ; return if port=1
    sbic @0,@1
    ret
    .endmacro

; wait if port=0 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WP0T ; port,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbis @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; wait if port=1 with timeout
; if timeout (in units of 5 cyc) then jump to addr
.macro WP1T ; port,bit,timeout,addr
    ldi w,@2+1
    dec w ; 1 cyc
    breq @3 ; 1 cyc
    sbic @0,@1 ; 1 cyc
    rjmp PC-3 ; 2 cyc = 5 cycles
    .endmacro

; ============================
; multi-byte operations
; ============================

.macro SWAP4 ; swap 2 variables
    mov w ,@0
    mov @0,@4
    mov @4,w
    mov w ,@1
    mov @1,@5
    mov @5,w
    mov w ,@2
    mov @2,@6
    mov @6,w
    mov w ,@3
    mov @3,@7
    mov @7,w
    .endmacro
.macro SWAP3
    mov w ,@0
    mov @0,@3
    mov @3,w
    mov w ,@1
    mov @1,@4
    mov @4,w
    mov w ,@2
    mov @2,@5
    mov @5,w
    .endmacro
.macro SWAP2
    mov w ,@0
    mov @0,@2
    mov @2,w
    mov w ,@1
    mov @1,@3
    mov @3,w
```

```
    .endmacro
.macro SWAP1
    mov w ,@0
    mov @0,@1
    mov @1,w
    .endmacro

.macro LDX4 ;r..r0 ; load from (x+)
    ld @3,x+
    ld @2,x+
    ld @1,x+
    ld @0,x+
    .endmacro
.macro LDX3 ;r..r0
    ld @2,x+
    ld @1,x+
    ld @0,x+
    .endmacro
.macro LDX2 ;r..r0
    ld @1,x+
    ld @0,x+
    .endmacro

.macro LDY4 ;r..r0 ; load from (y+)
    ld @3,y+
    ld @2,y+
    ld @1,y+
    ld @0,y+
    .endmacro
.macro LDY3 ;r..r0
    ld @2,y+
    ld @1,y+
    ld @0,y+
    .endmacro
.macro LDY2 ;r..r0
    ld @1,y+
    ld @0,y+
    .endmacro

.macro LDZ4 ;r..r0 ; load from (z+)
    ld @3,z+
    ld @2,z+
    ld @1,z+
    ld @0,z+
    .endmacro
.macro LDZ3 ;r..r0
    ld @2,z+
    ld @1,z+
    ld @0,z+
    .endmacro
.macro LDZ2 ;r..r0
    ld @1,z+
    ld @0,z+
    .endmacro

.macro STX4 ;r..r0 ; store to (x+)
    st x+,@3
    st x+,@2
    st x+,@1
    st x+,@0
    .endmacro
.macro STX3 ;r..r0
    st x+,@2
    st x+,@1
    st x+,@0
    .endmacro
.macro STX2 ;r..r0
    st x+,@1
```

```
    st x+,@0
    .endmacro

.macro STY4 ;r..r0 ; store to (y+)
    st y+,@3
    st y+,@2
    st y+,@1
    st y+,@0
    .endmacro
.macro STY3 ;r..r0
    st y+,@2
    st y+,@1
    st y+,@0
    .endmacro
.macro STY2 ;r..r0
    st y+,@1
    st y+,@0
    .endmacro

.macro STZ4 ;r..r0 ; store to (z+)
    st z+,@3
    st z+,@2
    st z+,@1
    st z+,@0
    .endmacro
.macro STZ3 ;r..r0
    st z+,@2
    st z+,@1
    st z+,@0
    .endmacro
.macro STZ2 ;r..r0
    st z+,@1
    st z+,@0
    .endmacro

.macro STI4 ;addr,k ; store immediate
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    ldi w,byte3(@1)
    sts @0+2,w
    ldi w,byte4(@1)
    sts @0+3,w
    .endmacro
.macro STI3 ;addr,k
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    ldi w,byte3(@1)
    sts @0+2,w
    .endmacro
.macro STI2 ;addr,k
    ldi w, low(@1)
    sts @0+0,w
    ldi w, high(@1)
    sts @0+1,w
    .endmacro
.macro STI ;addr,k
    ldi w,@1
    sts @0,w
    .endmacro

.macro INC4 ; increment
    ldi w,0xff
    sub @3,w
    sbc @2,w
```

```
    sbc @1,w
    sbc @0,w
    .endmacro
.macro INC3
    ldi w,0xff
    sub @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro INC2
    ldi w,0xff
    sub @1,w
    sbc @0,w
    .endmacro

.macro DEC4 ; decrement
    ldi w,0xff
    add @3,w
    adc @2,w
    adc @1,w
    adc @0,w
    .endmacro
.macro DEC3
    ldi w,0xff
    add @2,w
    adc @1,w
    adc @0,w
    .endmacro
.macro DEC2
    ldi w,0xff
    add @1,w
    adc @0,w
    .endmacro

.macro CLR9 ; clear (also clears the carry)
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    clr @7
    clr @8
    .endmacro
.macro CLR8
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    clr @7
    .endmacro
.macro CLR7
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    clr @5
    clr @6
    .endmacro
.macro CLR6
    sub @0,@0
    clr @1
    clr @2
```

```
    clr @3
    clr @4
    clr @5
    .endmacro
.macro CLR5
    sub @0,@0
    clr @1
    clr @2
    clr @3
    clr @4
    .endmacro
.macro CLR4
    sub @0,@0
    clr @1
    clr @2
    clr @3
    .endmacro
.macro CLR3
    sub @0,@0
    clr @1
    clr @2
    .endmacro
.macro CLR2
    sub @0,@0
    clr @1
    .endmacro

.macro COM4 ; one's complement
    com @0
    com @1
    com @2
    com @3
    .endmacro
.macro COM3
    com @0
    com @1
    com @2
    .endmacro
.macro COM2
    com @0
    com @1
    .endmacro

.macro NEG4 ; negation (two's complement)
    com @0
    com @1
    com @2
    com @3
    ldi w,0xff
    sub @3,w
    sbc @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro NEG3
    com @0
    com @1
    com @2
    ldi w,0xff
    sub @2,w
    sbc @1,w
    sbc @0,w
    .endmacro
.macro NEG2
    com @0
    com @1
    ldi w,0xff
    sub @1,w
```

```
    sbc @0,w
    .endmacro

.macro LDI4 ; r..r0, k ; load immediate
    ldi @3, low(@4)
    ldi @2, high(@4)
    ldi @1,byte3(@4)
    ldi @0,byte4(@4)
    .endmacro
.macro LDI3
    ldi @2, low(@3)
    ldi @1, high(@3)
    ldi @0,byte3(@3)
    .endmacro
.macro LDI2
    ldi @1, low(@2)
    ldi @0, high(@2)
    .endmacro

.macro LDS4 ; load direct from SRAM
    lds @3,@4
    lds @2,@4+1
    lds @1,@4+2
    lds @0,@4+3
    .endmacro
.macro LDS3
    lds @2,@3
    lds @1,@3+1
    lds @0,@3+2
    .endmacro
.macro LDS2
    lds @1,@2
    lds @0,@2+1
    .endmacro

.macro STS4 ; store direct to SRAM
    sts @0+0,@4
    sts @0+1,@3
    sts @0+2,@2
    sts @0+3,@1
    .endmacro
.macro STS3
    sts @0+0,@3
    sts @0+1,@2
    sts @0+2,@1
    .endmacro
.macro STS2
    sts @0+0,@2
    sts @0+1,@1
    .endmacro

.macro STDZ4 ; d, r3,r2,r1,r0
    std z+@0+0,@4
    std z+@0+1,@3
    std z+@0+2,@2
    std z+@0+3,@1
    .endmacro
.macro STDZ3 ; d, r2,r1,r0
    std z+@0+0,@3
    std z+@0+1,@2
    std z+@0+2,@1
    .endmacro
.macro STDZ2 ; d, r1,r0
    std z+@0+0,@2
    std z+@0+1,@1
    .endmacro

.macro LPM4 ; load program memory
```

```
    lpm
    mov @3,r0
    adiw zl,1
    lpm
    mov @2,r0
    adiw zl,1
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM3
    lpm
    mov @2,r0
    adiw zl,1
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM2
    lpm
    mov @1,r0
    adiw zl,1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro
.macro LPM1
    lpm
    mov @0,r0
    adiw zl,1
    .endmacro

.macro MOV4 ; move between registers
    mov @3,@7
    mov @2,@6
    mov @1,@5
    mov @0,@4
    .endmacro
.macro MOV3
    mov @2,@5
    mov @1,@4
    mov @0,@3
    .endmacro
.macro MOV2
    mov @1,@3
    mov @0,@2
    .endmacro

.macro ADD4 ; add
    add @3,@7
    adc @2,@6
    adc @1,@5
    adc @0,@4
    .endmacro
.macro ADD3
    add @2,@5
    adc @1,@4
    adc @0,@3
    .endmacro
.macro ADD2
    add @1,@3
    adc @0,@2
```

```
    .endmacro

.macro SUB4 ; subtract
    sub @3,@7
    sbc @2,@6
    sbc @1,@5
    sbc @0,@4
    .endmacro
.macro SUB3
    sub @2,@5
    sbc @1,@4
    sbc @0,@3
    .endmacro
.macro SUB2
    sub @1,@3
    sbc @0,@2
    .endmacro

.macro CP4 ; compare
    cp @3,@7
    cpc @2,@6
    cpc @1,@5
    cpc @0,@4
    .endmacro
.macro CP3
    cp @2,@5
    cpc @1,@4
    cpc @0,@3
    .endmacro
.macro CP2
    cp @1,@3
    cpc @0,@2
    .endmacro

.macro TST4 ; test
    clr w
    cp @3,w
    cpc @2,w
    cpc @1,w
    cpc @0,w
    .endmacro
.macro TST3
    clr w
    cp @2,w
    cpc @1,w
    cpc @0,w
    .endmacro
.macro TST2
    clr w
    cp @1,w
    cpc @0,w
    .endmacro

.macro ADDI4 ; add immediate
    subi @3, low(-@4)
    sbci @2, high(-@4)
    sbci @1,byte3(-@4)
    sbci @0,byte4(-@4)
    .endmacro
.macro ADDI3
    subi @2, low(-@3)
    sbci @1, high(-@3)
    sbci @0,byte3(-@3)
    .endmacro
.macro ADDI2
    subi @1, low(-@2)
    sbci @0, high(-@2)
    .endmacro
```

```
.macro SUBI4 ; subtract immediate
    subi @3, low(@4)
    sbci @2, high(@4)
    sbci @1,byte3(@4)
    sbci @0,byte4(@4)
    .endmacro
.macro SUBI3
    subi @2, low(@3)
    sbci @1, high(@3)
    sbci @0,byte3(@3)
    .endmacro
.macro SUBI2
    subi @1, low(@2)
    sbci @0, high(@2)
    .endmacro

.macro LSL5 ; logical shift left
    lsl @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro LSL4
    lsl @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro LSL3
    lsl @2
    rol @1
    rol @0
    .endmacro
.macro LSL2
    lsl @1
    rol @0
    .endmacro

.macro LSR4 ; logical shift right
    lsr @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro LSR3
    lsr @0
    ror @1
    ror @2
    .endmacro
.macro LSR2
    lsr @0
    ror @1
    .endmacro

.macro ASR4 ; arithmetic shift right
    asr @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro ASR3
    asr @0
    ror @1
    ror @2
    .endmacro
.macro ASR2
```

```
    asr @0
    ror @1
    .endmacro

.macro ROL8 ; rotate left through carry
    rol @7
    rol @6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL7
    rol @6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL6
    rol @5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL5
    rol @4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL4
    rol @3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL3
    rol @2
    rol @1
    rol @0
    .endmacro
.macro ROL2
    rol @1
    rol @0
    .endmacro

.macro ROR8 ; rotate right through carry
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    ror @5
    ror @6
    ror @7
    .endmacro
.macro ROR7
    ror @0
    ror @1
    ror @2
```

```
    ror @3
    ror @4
    ror @5
    ror @6
    .endmacro
.macro ROR6
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    ror @5
    .endmacro
.macro ROR5
    ror @0
    ror @1
    ror @2
    ror @3
    ror @4
    .endmacro
.macro ROR4
    ror @0
    ror @1
    ror @2
    ror @3
    .endmacro
.macro ROR3
    ror @0
    ror @1
    ror @2
    .endmacro
.macro ROR2
    ror @0
    ror @1
    .endmacro

.macro PUSH2
    push @0
    push @1
    .endmacro
.macro POP2
    pop @1
    pop @0
    .endmacro

.macro PUSH3
    push @0
    push @1
    push @2
    .endmacro
.macro POP3
    pop @2
    pop @1
    pop @0
    .endmacro

.macro PUSH4
    push @0
    push @1
    push @2
    push @3
    .endmacro
.macro POP4
    pop @3
    pop @2
    pop @1
    pop @0
    .endmacro
```

```
.macro PUSH5
    push @0
    push @1
    push @2
    push @3
    push @4
    .endmacro
.macro POP5
    pop @4
    pop @3
    pop @2
    pop @1
    pop @0
    .endmacro

; --- SRAM operations ---
.macro INCS4 ; sram ; increment SRAM 4-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
    brne end
    lds w,@0+2
    inc w
    sts @0+2,w
    brne end
    lds w,@0+3
    inc w
    sts @0+3,w
end:
    .endmacro

.macro INCS3 ; sram ; increment SRAM 3-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
    brne end
    lds w,@0+2
    inc w
    sts @0+2,w
end:
    .endmacro

.macro INCS2 ; sram ; increment SRAM 2-byte variable
    lds w,@0
    inc w
    sts @0,w
    brne end
    lds w,@0+1
    inc w
    sts @0+1,w
end:
    .endmacro

.macro INCS ; sram ; increment SRAM 1-byte variable
    lds w,@0
    inc w
    sts @0,w
    .endmacro
```

```
.macro DECS4 ; sram ; decrement SRAM 4-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    lds u,@0+2
    sbc u,w
    sts @0+2,u
    lds u,@0+3
    sbc u,w
    sts @0+3,u
    .endmacro
.macro DECS3 ; sram ; decrement SRAM 3-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    lds u,@0+2
    sbc u,w
    sts @0+2,u
    .endmacro
.macro DECS2 ; sram ; decrement SRAM 2-byte variable
    ldi w,1
    lds u,@0
    sub u,w
    sts @0,u
    clr w
    lds u,@0+1
    sbc u,w
    sts @0+1,u
    .endmacro
.macro DECS ; sram ; decrement
    lds w,@0
    dec w
    sts @0,w
    .endmacro

.macro MOVS4 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
    lds w,@1+2
    sts @0+2,w
    lds w,@3+1
    sts @0+3,w
    .endmacro
.macro MOVS3 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
    lds w,@1+2
    sts @0+2,w
    .endmacro
.macro MOVS2 ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    lds w,@1+1
    sts @0+1,w
```

```
    .endmacro
.macro MOVS ; addr0,addr1 ; [addr0] <-- [addr1]
    lds w,@1
    sts @0,w
    .endmacro

.macro SEXT ; reg1,reg0 ; sign extend
    clr @0
    sbrc @1,7
    dec @0
    .endmacro

; =======================================
; Jump/Call with constant arguments
; =======================================

; --- calls with arguments a,b,XYZ ---
.macro CX ; subroutine,x
    ldi xl, low(@1)
    ldi xh,high(@1)
    rcall @0
    .endmacro
.macro CXY ; subroutine,x,y
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi yl, low(@2)
    ldi yh,high(@2)
    rcall @0
    .endmacro
.macro CXZ ; subroutine,x,z
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi zl, low(@2)
    ldi zh,high(@2)
    rcall @0
    .endmacro
.macro CXYZ ; subroutine,x,y,z
    ldi xl, low(@1)
    ldi xh,high(@1)
    ldi yl, low(@2)
    ldi yh,high(@2)
    ldi zl, low(@3)
    ldi zh,high(@3)
    rcall @0
    .endmacro
.macro CW ; subroutine,w
    ldi w, @1
    rcall @0
    .endmacro
.macro CA ; subroutine,a
    ldi a0, @1
    rcall @0
    .endmacro
.macro CAB ; subroutine,a,b
    ldi a0, @1
    ldi b0, @2
    rcall @0
    .endmacro

; --- jump with arguments w,a,b ---
.macro JW ; subroutine,w
    ldi w, @1
    rjmp @0
    .endmacro
.macro JA ; subroutine,a
    ldi a0, @1
    rjmp @0
    .endmacro
```

```
.macro JAB ; subroutine,a,b
    ldi a0, @1
    ldi b0, @2
    rjmp @0
    .endmacro
.list
```